

Wszystkie_zadania_z_notatkami

Lab 1

Zadanie 1

Sprawdź, czy suma dwóch liczb zmiennoprzecinkowych (zarówno w pojedynczej jak i podwójnej precyzji) jest zawsze równa oczekiwanej matematycznie wartości. Wyjaśnij, dlaczego odpowiedź może być błędna przy bezpośrednim porównaniu liczb zmiennoprzecinkowych.

PYTHON

```
import numpy as np

def suma_pojedyncza_precyzja(a, b):
    a32 = np.float32(a)
    b32 = np.float32(b)
    return np.float32(a32 + b32)

poj = suma_pojedyncza_precyzja(0.1, 0.2)

print(poj)

def suma_podwojna_precyzja(a: float, b: float) -> float:
    return a + b

pod = suma_podwojna_precyzja(0.1, 0.2)

print(pod)

print("Czy równe?: ", poj == pod)
```

```
0.3
0.30000000000000004
Czy równe?: False
```

dlaczego odpowiedź może być błędna przy bezpośrednim porównaniu liczb zmiennoprzecinkowych

Komputery przechowują liczby zmiennoprzecinkowe w formacie binarnym (standard IEEE 754).

Niektóre liczby dziesiętne, takie jak:

- 0.1
- 0.2

- 0.3

nie mają dokładnej reprezentacji w systemie binarnym, podobnie jak $1/3$ nie ma skończonego zapisu w systemie dziesiętnym.

W efekcie:

- 0.1 jest zapisywane jako liczba bardzo bliska 0.1
- 0.2 również
- ich suma daje 0.30000000000000004 zamiast dokładnie 0.3

To jest **błąd reprezentacji (rounding error)**.

Dlaczego bezpośrednio porównanie jest błędne?

Porównanie:

```
suma == 0.3
```

sprawdza **dokładną równość bitową**, a nie matematyczną równość w sensie „wystarczająco blisko”.

Z powodu minimalnych błędów zaokrągleń wynik może się różnić o bardzo małą wartość (np. $5e-17$), więc porównanie zwróci **False**.

Poprawny sposób porównywania

Zamiast `==` należy używać porównania z tolerancją:

```
import math math.isclose(suma, 0.3)
```

lub dla NumPy:

```
np.isclose(suma32, np.float32(0.3))
```

Można też ręcznie sprawdzić:

```
abs(suma - 0.3) < 1e-9
```

Wniosek

Suma dwóch liczb zmiennoprzecinkowych **nie zawsze** jest dokładnie równa wartości matematycznej.

Bezpośrednie porównanie (`==`) może dawać błędne wyniki.

Należy stosować porównania z tolerancją (`isclose`).

Zadanie 2

Sprawdź, co się stanie, gdy dodasz bardzo dużą liczbę do bardzo małej, a następnie odejmiesz dużą liczbę od wyniku.

```
a = 10000000000000000
b = 0.0000000000000001

wynik = (a + b) - a

print(wynik)
```

PYTHON

0.0

Matematycznie

Mamy:

$a = 10000000000000000$
 $b = 0.0000000000000001$

czyli:

$a = 10^{16}$
 $b = 10^{-15}$

Matematycznie:

$(10^{16} + 10^{-15}) - 10^{16} = 10^{-15}$

Powinniśmy dostać:

0.0000000000000001

Co dzieje się w praktyce?

Wynik będzie:

0.0

Dlaczego?

1. Ograniczona precyzja float64

Liczby zmiennoprzecinkowe typu `float` w Pythonie mają:

- 64 bity

- około 15–16 cyfr znaczących

Liczba:

10000000000000000

ma już **16 cyfr znaczących**.

To oznacza, że:

- cała dostępna precyzja jest zużyta na zapis tej dużej liczby,
- nie ma już miejsca na zapis bardzo małej zmiany rzędu 10^{-15} .

2. Co dzieje się przy dodawaniu?

Komputer próbuje policzyć:

10000000000000000 + 0.0000000000000001

Ale różnica rzędów wielkości wynosi:

10^{16} vs 10^{-15}

To różnica **31 rzędów wielkości**.

Mała liczba jest tak niewyobrażalnie mała względem dużej, że:

jej wpływ mieści się poza zakresem precyzji **mantysy**

Mantysa to część liczby zmiennoprzecinkowej, która przechowuje jej **cyfry znaczące**, czyli właściwą „treść” liczby.

W efekcie:

$$a + b \approx a$$

czyli w pamięci:

$$10000000000000000 + 0.0000000000000001 \\ = 10000000000000000$$

3. Odejmowanie

Skoro w pamięci mamy:

$$(a + b) = a$$

to:

$$(a + b) - a = 0$$

Co to pokazuje?

- Bardzo małe liczby mogą całkowicie „zniknąć” przy dodawaniu do bardzo dużych.
- Operacje na liczbach zmiennoprzecinkowych nie są dokładne przy dużych różnicach skali.
- To przykład **utraty precyzji** wynikającej z ograniczonej liczby cyfr znaczących.

Kluczowa intuicja!

To tak, jakbyś próbował dodać:

10 000 000 000 000 000,00

- 0,000000000000000001

Przy dokładności do grosza ta zmiana po prostu nie istnieje.

Zadanie 3

Wyświetl liczbę zmiennoprzecinkową jako float i double z precyzją do 20 miejsc po przecinku.

PYTHON

```
import numpy as np

liczba = 0.1

# float - pojedyncza precyzja
liczba_f = np.float32(liczba)

# double - podwójna precyzja
liczba_d = float(liczba)

print("float: ", format(liczba_f, ".20f"))
print("double: ", format(liczba_d, ".20f"))
```

```
float:  0.10000000149011611938
double: 0.10000000000000000555
```

- **float32** ma mniejszą precyzję - szybciej pojawia się błąd.
 - **float64** jest dokładniejszy, ale też nie przechowuje 0.1 idealnie.
 - 20 miejsc po przecinku pokazuje rzeczywistą reprezentację w pamięci.
-

Zadanie 4

Oblicz $0,3 \cdot 3 + 0,1$ i porównaj wynik z jego wartościami zaokrąglonymi do dołu i do góry (floor i ceil).

```
import math

wynik = 0.3 * 3 + 0.1

print("Wynik zwykły: ", wynik)
print("Wynik zaokrąglony w dół: ", math.floor(wynik))
print("Wynik zaokrąglony w górę: ", math.ceil(wynik))
```

PYTHON

```
Wynik zwykły:  0.9999999999999999
Wynik zaokrąglony w dół:  0
Wynik zaokrąglony w górę:  1
```

Wynik nie jest równy dokładnie 1 z powodu błędu reprezentacji binarnej liczb 0.3 i 0.1 w standardzie IEEE 754.

Zadanie 5

Oblicz różnicę między 1,00000001 i 1,00000000 oraz między 1,00000002 i 1,00000001. Wyjaśnij, dlaczego wyniki mogą się różnić od teoretycznej różnicy.

```
wynik1 = 1.00000001 - 1.00000000
wynik2 = 1.00000002 - 1.00000001

print("wynik1: ", wynik1)
print("wynik2: ", wynik2)
```

PYTHON

```
wynik1:  1.00000000005838672e-07
wynik2:  9.999999983634211e-08
```

Dlaczego wyniki różnią się od teoretycznej różnicy?

Dzieje się tak z powodu **błędów reprezentacji liczb zmiennoprzecinkowych w systemie binarnym (IEEE 754)**.

Liczby nie są przechowywane dokładnie

Liczby:

1.0000001

1.0000002

nie mają idealnej reprezentacji binarnej. W pamięci są zapisane jako **najbliższe możliwe przybliżenia**.

Czyli komputer faktycznie odejmuje:

$(1.0000001 + \text{mały_błąd1}) - (1.0000000 + \text{mały_błąd2})$

Odejmowanie liczb bliskich sobie

Tutaj odejmujemy liczby prawie równe:

$1.0000001 - 1.0000000$

To powoduje zjawisko zwane:

utratą cyfr znaczących (catastrophic cancellation)

Podczas odejmowania:

- duże wspólne cyfry się „kasują”,
- pozostaje bardzo mała różnica,
- a względny wpływ błędu zaokrąglenia rośnie.

Dlatego wyniki:

$1.0000000005838672e-07$

$9.999999983634211e-08$

różnią się minimalnie od siebie i od idealnego $1e-7$.

Podsumowanie

To są liczby bardzo bliskie $1e-7$, więc obliczenia są poprawne - różnice wynikają z reprezentacji zmiennoprzecinkowej.

Wyniki różnią się od teoretycznej wartości, ponieważ liczby zmiennoprzecinkowe nie są przechowywane dokładnie w systemie binarnym. Podczas odejmowania liczb bardzo bliskich sobie dochodzi do utraty cyfr znaczących, co powoduje, że niewielkie błędy reprezentacji stają się widoczne w wyniku.

Zadanie 6

Podziel liczbę 1,0 przez 0,0 i liczbę 0,0 przez 0,0. Sprawdź, co zwrócą te operacje.

```
wynik1 = 1.0 / 0.0
wynik2 = 0.0 / 0.0

print("wynik dzielenia pierwszego: ", wynik1)
print("wynik dzielenia drugiego: ", wynik2)
```

```
Traceback (most recent call last):
  File "zad6.py", line 1, in <module>
    wynik1 = 1.0 / 0.0
             ~~~~^~~~~
ZeroDivisionError: float division by zero
-----
Traceback (most recent call last):
  File "zad6.py", line 2, in <module>
    wynik2 = 0.0 / 0.0
             ~~~~^~~~~
ZeroDivisionError: float division by zero
```

Mimo że liczby zmiennoprzecinkowe są zgodne ze standardem IEEE 754, Python **celowo zgłasza wyjątek**, aby zapobiec dalszym obliczeniom na niepoprawnych wartościach.

W Pythonie dzielenie przez zero dla liczb zmiennoprzecinkowych powoduje zgłoszenie wyjątku `ZeroDivisionError`. Jednak zgodnie ze standardem IEEE 754 operacja $1.0 / 0.0$ powinna zwrócić nieskończoność (∞), a $0.0 / 0.0$ wartość NaN (Not a Number). W bibliotekach takich jak NumPy zachowanie jest zgodne z IEEE 754 i zamiast wyjątku zwracane są wartości `inf` oraz `nan`.

Co mówi standard IEEE 754?

W czystej arytmetyce IEEE 754 (np. w NumPy):

Operacja	Wynik
<code>1.0 / 0.0</code>	<code>+∞</code> (infinity)
<code>-1.0 / 0.0</code>	<code>-∞</code>
<code>0.0 / 0.0</code>	<code>NaN</code> (Not a Number)

Jak to sprawdzić w NumPy?

```
import numpy as np

print(np.float64(1.0) / np.float64(0.0))
print(np.float64(0.0) / np.float64(0.0))
```

Wtedy otrzymasz:

```
inf  
nan
```

(plus ostrzeżenie RuntimeWarning zamiast wyjątku)

Różnica

- ♦ Python (czysty `float`) → zgłasza `ZeroDivisionError`
- ♦ NumPy → zwraca `inf` lub `nan` zgodnie z IEEE 754

Zadanie 7

Oblicz maszynowy epsilon dla typów float i double i porównaj wyniki. Wyjaśnij, czym jest maszynowy epsilon i jak wpływa na dokładność obliczeń komputerowych.

Maszynowy epsilon - definicja

Maszynowy epsilon (ε_{mach}) to najmniejsza dodatnia liczba ε , taka że:

$$1 + \varepsilon > 1$$

w arytmetyce zmiennoprzecinkowej danego typu.

Oznacza to najmniejszą wartość, którą komputer jest w stanie „zauważyć” przy dodaniu do 1.

Wyznacza on granicę precyzji reprezentacji liczb w komputerze.

Obliczenie maszynowego epsilon w Pythonie

Maszynowy epsilon można wyznaczyć algorytmicznie, zmniejszając wartość ε tak długo, aż suma $1 + \varepsilon$ przestanie być większa od 1.

```
import numpy as np

def epsilon_float32():
    eps = np.float32(1.0)
    while np.float32(1.0) + eps / np.float32(2.0) > np.float32(1.0):
        eps = eps / np.float32(2.0)
    return eps

def epsilon_float64():
    eps = 1.0
    while 1.0 + eps / 2.0 > 1.0:
        eps = eps / 2.0
    return eps

eps32 = epsilon_float32()
eps64 = epsilon_float64()

print("Epsilon float32:", eps32)
print("Epsilon float64:", eps64)
```

```
Epsilon float32: 1.1920929e-07
Epsilon float64: 2.220446049250313e-16
```

Typ	ε_{mach}
float32	$\approx 1.19 \cdot 10^{-7}$
float64	$\approx 2.22 \cdot 10^{-16}$

Porównując wartości:

$$\frac{1.19 \cdot 10^{-7}}{2.22 \cdot 10^{-16}} \approx 10^9$$

Oznacza to, że typ **float64** jest około miliard razy dokładniejszy niż **float32**.

Wpływ maszynowego epsilon na dokładność obliczeń

1. Ogranicza liczbę cyfr znaczących

- float32 → około 7 cyfr znaczących
- float64 → około 15–16 cyfr znaczących

2. Określa maksymalny względny błąd pojedynczej operacji

Każda operacja arytmetyczna wprowadza błąd rzędu ε_{mach} .

3. Wpływa na stabilność obliczeń

- Przy odejmowaniu liczb bardzo bliskich sobie może dojść do utraty cyfr znaczących.
- Przy dużej liczbie operacji błędy mogą się kumulować.

4. Wyznacza granicę rozróżnialności liczb

Jeśli dodamy do 1 liczbę mniejszą niż ε_{mach} , wynik nadal będzie równy 1, ponieważ zmiana jest poniżej granicy precyzji.

Wniosek

Maszynowy epsilon:

- dla **float32** wynosi około $1.19 \cdot 10^{-7}$
- dla **float64** wynosi około $2.22 \cdot 10^{-16}$

Im mniejszy epsilon, tym większa dokładność obliczeń komputerowych.

Dlatego typ double (float64) zapewnia znacznie większą precyzję niż float (float32).

Zadanie 8

Sumuj liczbę 0,0001 w pętli 1.000.000 razy i porównaj wynik z wynikiem uzyskanym przez mnożenie 1.000.000 przez 0,0001. Wyjaśnij, dlaczego mogą wystąpić różnice.

PYTHON

```
# liczba iteracji
n = 1_000_000
wartosc = 0.0001

# Sumowanie w pętli
suma_petla = 0.0
for _ in range(n):
    suma_petla += wartosc

# Mnożenie
suma_mnozenie = n * wartosc

# Różnica
roznica = suma_petla - suma_mnozenie

print("Wynik sumowania w pętli:", suma_petla)
print("Wynik mnożenia:", suma_mnozenie)
print("Różnica:", roznica)
```

Wynik sumowania w pętli: 100.00000000219612

Wynik mnożenia: 100.0

Różnica: 2.1961170659778873e-09

Dlaczego występuje różnica?

1. 0.0001 nie ma dokładnej reprezentacji binarnej

Liczba `0.0001` nie jest zapisywana idealnie w systemie binarnym — jest przechowywana jako przybliżenie.

2. Akumulacja błędów

W pętli wykonujemy milion operacji dodawania:

```
suma = suma + przybliżona_wartość
```

Każde dodanie wprowadza bardzo mały błąd zaokrąglenia.

Po 1 000 000 iteracjach te małe błędy się sumują.

To zjawisko nazywamy:

akumulacją błędów numerycznych

3. Dlaczego mnożenie daje dokładniejszy wynik?

Mnożenie wykonuje:

```
n * wartosc
```

czyli jedną operację zamiast miliona, więc błąd pojawia się tylko raz, a nie milion razy.

Wniosek

- Sumowanie wielu małych liczb może prowadzić do akumulacji błędów.
- Wynik obliczony w pętli może różnić się od wyniku mnożenia.
- To pokazuje, że operacje zmiennoprzecinkowe nie są dokładne matematycznie.
- Im więcej operacji, tym większe ryzyko narastania błędów.

Zadanie 9

Oblicz sumę odwrotności liczb od 1 do 1.000.000 w kolejności rosnącej i malejącej. Porównaj wyniki.


```

n = 1_000_000

# 1 Sumowanie w kolejności rosnącej (od 1 do 1_000_000)
suma_rosnaco = 0.0
for i in range(1, n + 1):
    suma_rosnaco += 1.0 / i

# 2 Sumowanie w kolejności malejącej (od 1_000_000 do 1)
suma_malejaco = 0.0
for i in range(n, 0, -1):
    suma_malejaco += 1.0 / i

# Różnica
roznica = suma_rosnaco - suma_malejaco

print("Suma rosnąco:", suma_rosnaco)
print("Suma malejąco:", suma_malejaco)
print("Różnica:", roznica)

```

Matematycznie obie sumy powinny być identyczne:

$$\sum_{i=1}^{1,000,000} \frac{1}{i}$$

Jest to tzw. **milionowy wyraz szeregu harmonicznego**.

W praktyce otrzymujemy:

```

Suma rosnąco: 14.392726722864989
Suma malejąco: 14.392726722865772
Różnica: -7.833733661755105e-13

```

Dlaczego wyniki się różnią?

1. Ograniczona precyzja liczb zmiennoprzecinkowych

Liczby typu `float` mają ograniczoną liczbę cyfr znaczących (około 15–16).

2. Akumulacja błędów zaokrągleń

Każde dodawanie wprowadza bardzo mały błąd.

Ponieważ wykonujemy **milion operacji**, błędy się kumulują.

3. Znaczenie kolejności dodawania

To kluczowe.

♦ Sumowanie rosnąco (1 → 1 000 000)

Na początku dodajemy duże liczby (1, 1/2, 1/3...).

Na końcu bardzo małe liczby (np. 1/1 000 000).

Małe składniki dodawane do dużej sumy mogą zostać częściowo „zgubione” z powodu ograniczonej precyzji.

♦ Sumowanie malejąco (1 000 000 → 1)

Najpierw dodajemy bardzo małe liczby.

Suma jest jeszcze mała, więc precyzja względna jest lepsza.

Dopiero później dodawane są większe składniki.

→ Ta metoda daje zwykle **dokładniejszy wynik**.

Wniosek

- Matematycznie kolejność sumowania nie ma znaczenia.
- W arytmetyce zmiennoprzecinkowej ma znaczenie.
- Sumowanie od najmniejszych do największych wartości jest numerycznie stabilniejsze.
- Różnice wynikają z ograniczonej precyzji i akumulacji błędów zaokrągleń.

Zadanie 10

Niech

$$f(x) = \sqrt{x^2 + 1} - 1$$

oraz

$$g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}.$$

Łatwo zauważyć, że $g = f$. Oblicz i porównaj wartości funkcji g i f dla:

$$x = 8^{-1}, 8^{-2}, 8^{-3}, \dots$$

Łatwo zauważyć, że algebraicznie:

$$g(x) = f(x)$$

ponieważ:

$$\sqrt{x^2 + 1} - 1 = \frac{(\sqrt{x^2 + 1} - 1)(\sqrt{x^2 + 1} + 1)}{\sqrt{x^2 + 1} + 1} = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

```
import math

def f(x):
    return math.sqrt(x**2 + 1) - 1

def g(x):
    return x**2 / (math.sqrt(x**2 + 1) + 1)

print(f"{'x':>12} {'f(x)':>20} {'g(x)':>20} {'różnica':>20}")

for k in range(1, 11):
    x = 8**(-k)
    fx = f(x)
    gx = g(x)
    print(f"{'x':12.5e} {'fx':20.15e} {'gx':20.15e} {'(fx-gx)':20.15e}")
```

x	f(x)	g(x)	różnica
1.25000e-01	7.782218537318641e-03	7.782218537318706e-03	-6.505213034913027e-17
1.56250e-02	1.220628628286757e-04	1.220628628287590e-04	-8.328027937404281e-17
1.95312e-03	1.907346813823096e-06	1.907346813826566e-06	-3.469446951953614e-18
2.44141e-04	2.980232194360610e-08	2.980232194360612e-08	-1.323488980084844e-23
3.05176e-05	4.656612873077393e-10	4.656612871993190e-10	1.084202172485504e-19
3.81470e-06	7.275957614183426e-12	7.275957614156956e-12	2.646977960169689e-23
4.76837e-07	1.136868377216160e-13	1.136868377216096e-13	6.462348535570529e-27
5.96046e-08	1.776356839400250e-15	1.776356839400249e-15	1.577721810442024e-30
7.45058e-09	0.000000000000000e+00	2.775557561562891e-17	-2.775557561562891e-17
9.31323e-10	0.000000000000000e+00	4.336808689942018e-19	-4.336808689942018e-19

Co się stanie?

Dla większych wartości x obie funkcje dadzą niemal identyczne wyniki.

Jednak gdy x staje się bardzo małe (np. 8^{-8} , 8^{-9} , 8^{-10}):

- wartości f(x) zaczynają tracić dokładność,
- g(x) pozostaje stabilne numerycznie.

Dlaczego?

Dla bardzo małych x:

$$\sqrt{x^2 + 1} \approx 1$$

W funkcji:

$$f(x) = \sqrt{x^2 + 1} - 1$$

odejmujemy dwie prawie równe liczby:

$$1.000000000000... - 1$$

Powoduje to zjawisko:

katastrofalnej utraty cyfr znaczących (catastrophic cancellation)

W wyniku tracimy znaczną część dokładności.

Natomiast funkcja:

$$g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

nie zawiera odejmowania prawie równych liczb, więc jest znacznie stabilniejsza numerycznie.

Wniosek

- Algebraicznie: ($f(x) = g(x)$)
- Numerycznie: dla małych x funkcja $g(x)$ daje dokładniejsze wyniki.
- Powód: w $f(x)$ występuje utrata cyfr znaczących.
- Jest to klasyczny przykład niestabilności numerycznej.

Dla bardzo małych x funkcja $g(x)$ powinna być używana zamiast $f(x)$.

Lab 2

Zadanie 1

Napisz funkcję, która wygeneruje tablicę liczb zmiennoprzecinkowych pojedynczej precyzji reprezentujących elementy ciągu postaci:

$$S_n = \sum_{k=0}^{n-1} a_k = \sum_{k=0}^{n-1} \frac{1}{(k \bmod m + 1)(k \bmod m + 2)},$$

gdzie n i m są potęgami liczby 2 oraz $n > m$.

Jak rozumieć to zadanie?

Nie liczymy tu jeszcze całej sumy S_n , tylko najpierw generujemy **same elementy ciągu**:

$$a_k = \frac{1}{(k \bmod m + 1)(k \bmod m + 2)}$$

Czyli dla każdego k :

1. liczysz resztę z dzielenia $k \bmod m$,
2. dodajesz 1 i 2,
3. mnożysz te dwie liczby,
4. bierzesz odwrotność.

Ponieważ zadanie mówi o **pojedynczej precyzji**, każdy element zapisujemy jako `float32`.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz pustą tablicę.

Krok 2

Dla każdego k od `0` do `n-1` liczysz wartość:

$$a_k = \frac{1}{(k \bmod m + 1)(k \bmod m + 2)}$$

Krok 3

Zapisujesz wynik jako `np.float32`, czyli liczbę pojedynczej precyzji.

Krok 4

Dodajesz element do tablicy.

Kod

PYTHON

```
import numpy as np # importujemy bibliotekę numpy i nadajemy jej skrót np

def generuj_tablice(n, m): # funkcja generuje tablicę n elementów według podanego wzoru
    tablica = [] # tworzymy pustą listę, do której będziemy dodawać kolejne elementy ciągu

    for k in range(n): # pętla wykonuje się n razy, dla k od 0 do n-1
        licznik = 1 # ustawiamy licznik ułamka na 1
        mianownik = ((k % m) + 1) * ((k % m) + 2) # liczymy mianownik według wzoru z użyciem reszty z dzielenia k przez m

        element = licznik / mianownik # obliczamy wartość elementu ciągu jako licznik podzielony przez mianownik

        tablica.append(np.float32(element)) # dodajemy element do tablicy jako liczbę typu float32

    return tablica # zwracamy gotową tablicę z elementami ciągu

n = 64 # liczba elementów, które chcemy wygenerować
m = 16 # wartość używana w działaniu modulo, czyli okres powtarzania mianownika

tablica = generuj_tablice(n, m) # wywołujemy funkcję i zapisujemy wynik do zmiennej tablica

print("Elementy ciągu:") # wypisujemy napis informacyjny
for i, wartosc in enumerate(tablica): # przechodzimy po elementach tablicy razem z ich indeksami
    print("a_", i, "=", wartosc) # wypisujemy indeks elementu oraz jego wartość
```

Co otrzymujemy?

Otrzymujemy tablicę elementów ciągu $(a_0, a_1, a_2, \dots, a_{n-1})$, zapisanych w pojedynczej precyzji.

Zadanie 2

Napisz funkcję sumującą elementy tablicy z zadania 1. Sprawdź dokładność otrzymanej sumy.

Jak rozumieć to zadanie?

Tutaj liczymy już zwykłą sumę wszystkich elementów tablicy:

$$S_n = a_0 + a_1 + a_2 + \dots + a_{n-1}$$

Jest to najprostszy sposób sumowania:

- zaczynasz od zera,
- dodajesz kolejne elementy jeden po drugim.

Jak rozumieć schemat implementacji?

Krok 1

Generujesz tablicę elementów.

Krok 2

Tworzysz zmienną `suma` równą `0.0` w typie `float32`.

Krok 3

W pętli dodajesz do niej kolejne elementy tablicy.

Krok 4

Na końcu zwracasz wynik.

Kod

PYTHON

```
import numpy as np # importujemy bibliotekę numpy i nadajemy jej skrót np

def generuj_tablice(n, m): # funkcja generuje tablicę n elementów według podanego wzoru
    tablica = [] # tworzymy pustą listę, do której będziemy dodawać elementy

    for k in range(n): # pętla wykonuje się n razy, dla k od 0 do n-1
        licznik = 1 # ustawiamy licznik ułamka na 1
        mianownik = ((k % m) + 1) * ((k % m) + 2) # liczymy mianownik według wzoru z użyciem reszty z dzielenia k przez m

        a = licznik / mianownik # obliczamy wartość elementu ciągu
        tablica.append(np.float32(a)) # dodajemy element do tablicy jako liczbę typu float32

    return tablica # zwracamy gotową tablicę

def sumuj_tablice(tablica): # funkcja sumuje elementy tablicy
    suma = np.float32(0.0) # tworzymy zmienną suma typu float32 i ustawiamy ją na 0.0

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        suma = np.float32(suma + element) # dodajemy element do sumy i zapisujemy wynik jako float32

    return suma # zwracamy obliczoną sumę

n = 64 # liczba elementów tablicy
m = 16 # wartość używana w modulo, czyli okres powtarzania wartości

tablica = generuj_tablice(n, m) # generujemy tablicę elementów i zapisujemy ją do zmiennej tablica

suma = sumuj_tablice(tablica) # obliczamy sumę elementów tablicy

print("Elementy tablicy:") # wypisujemy napis informacyjny
for i, wartosc in enumerate(tablica): # przechodzimy po elementach tablicy razem z ich indeksami
    print("a_", i, "=", wartosc) # wypisujemy indeks elementu i jego wartość

print("\nSuma elementów:", suma) # wypisujemy sumę elementów tablicy
```

Jak sprawdzić dokładność?

Dokładną wartość sumy można porównać z:

$$\frac{n}{m+1}$$

W tym zadaniu:

$$\frac{64}{17}$$

To jest wartość odniesienia, z którą porównujemy wynik obliczony numerycznie.

Zadanie 3

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Gilla–Møllera. Sprawdź dokładność otrzymanej sumy.

Co to jest algorytm Gilla–Møllera?

To metoda dokładniejszego sumowania niż zwykłe dodawanie.

W zwykłym sumowaniu część informacji może się gubić przez zaokrąglenia. Algorytm Gilla–Møllera próbuje zachować tę „zgubioną” część w dodatkowej zmiennej, czyli poprawce.

Jak rozumieć schemat implementacji?

Krok 1

Zaczynasz od:

- `suma = 0`
- `poprawka = 0`

Krok 2

Dla każdego elementu liczysz:

$$t = suma + element$$

Krok 3

Obliczasz, jaka część została utracona podczas dodawania, i dopisujesz ją do poprawki.

Krok 4

Na końcu zwracasz:

$$suma + poprawka$$

```
import numpy as np # importujemy bibliotekę numpy i nadajemy jej skrót np

def generuj_tablice(n, m): # funkcja generuje tablicę n elementów według podanego wzoru
    tablica = [] # tworzymy pustą listę, do której będą dodawane elementy

    for k in range(n): # pętla wykonuje się n razy, dla k od 0 do n-1
        licznik = 1 # ustawiamy licznik ułamka na 1
        mianownik = ((k % m) + 1) * ((k % m) + 2) # obliczamy mianownik zależny od reszty z dzielenia k przez m

        a = licznik / mianownik # obliczamy wartość elementu ciągu
        tablica.append(np.float32(a)) # dodajemy element do tablicy jako liczbę typu float32

    return tablica # zwracamy gotową tablicę

def sumuj_tablice(tablica): # funkcja sumuje elementy tablicy metodą z poprawką
    suma = np.float32(0.0) # tworzymy zmienną suma typu float32 i ustawiamy ją na 0
    poprawka = np.float32(0.0) # tworzymy zmienną poprawka, która będzie przechowywać utracone części wyniku

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        t = np.float32(suma + element) # dodajemy aktualny element do sumy i zapisujemy wynik jako float32
        poprawka = np.float32(poprawka + (element - (t-suma))) # obliczamy część elementu utraconą przez zaokrąglenie i dodajemy ją do poprawki
        suma = t # aktualizujemy sumę

    return np.float32(suma + poprawka) # zwracamy sumę powiększoną o poprawkę

n = 64 # liczba elementów tablicy
m = 16 # wartość używana w modulo, czyli okres powtarzania mianownika

tablica = generuj_tablice(n, m) # generujemy tablicę elementów

suma = sumuj_tablice(tablica) # sumujemy elementy tablicy metodą z poprawką

print("Suma:", suma) # wypisujemy obliczoną sumę
```

Co daje ta metoda?

Wynik zwykle jest dokładniejszy niż przy zwykłym sumowaniu, ponieważ zmniejsza wpływ błędów zaokrągleń.

Zadanie 4

Napisz funkcję sumującą elementy tablicy z zadania 1 z wykorzystaniem algorytmu Kahana. Sprawdź dokładność otrzymanej sumy. Porównaj wyniki wszystkich omówionych metod sumowania.

Co to jest algorytm Kahana?

Algorytm Kahana to inna metoda dokładniejszego sumowania.

Tak samo jak w metodzie Gilla–Møllera, chodzi o to, żeby kontrolować błąd zaokrąglania.

Tutaj używa się zmiennej `c`, która przechowuje utraconą część sumy.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz:

- `suma = 0`
- `c = 0`

Krok 2

Dla każdego elementu liczysz:

$$y = element - c$$

czyli najpierw korygujesz element o wcześniej utracony błąd.

Krok 3

Dodajesz poprawioną wartość do sumy:

$$t = suma + y$$

Krok 4

Wyznaczasz nowy błąd:

$$c = (t - suma) - y$$

Krok 5

Ustawiasz nową sumę.

```
import numpy as np # importujemy bibliotekę numpy, żeby używać typu np.float32

def generuj_tablice(n, m): # funkcja generuje tablicę n elementów według danego wzoru
    tablica = [] # tworzymy pustą listę na elementy ciągu

    for k in range(n): # przechodzimy po kolejnych indeksach od 0 do n-1
        licznik = 1 # licznik ułamka jest równy 1
        mianownik = ((k % m) + 1) * ((k % m) + 2) # obliczamy mianownik na podstawie reszty z dzielenia k przez m

        a = licznik / mianownik # obliczamy wartość elementu ciągu
        tablica.append(np.float32(a)) # dodajemy element do tablicy jako liczbę typu float32

    return tablica # zwracamy wygenerowaną tablicę

def sumuj_tablice(tablica): # funkcja wykonuje zwykłe, klasyczne sumowanie elementów tablicy
    suma = np.float32(0.0) # ustawiamy początkową sumę na 0.0 jako float32

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        suma = np.float32(suma + element) # dodajemy element do sumy i zapisujemy wynik jako float32

    return suma # zwracamy obliczoną sumę

def sumuj_tablice_Møller(tablica): # funkcja sumuje elementy metodą Møllera, czyli z poprawką błędu
    suma = np.float32(0.0) # ustawiamy początkową sumę na 0.0 jako float32
    poprawka = np.float32(0.0) # zmienna poprawka przechowuje utracone części wyniku

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        t = np.float32(suma + element) # obliczamy tymczasową nową sumę
        poprawka = np.float32(poprawka + (element - (t-suma))) # dodajemy do poprawki część utraconą przez zaokrąglenie
        suma = t # aktualizujemy sumę

    return np.float32(suma + poprawka) # zwracamy sumę powiększoną o poprawkę

def sumuj_tablice_Kahan(tablica): # funkcja sumuje elementy metodą Kahana, czyli z kompensacją błędu
    suma = np.float32(0.0) # ustawiamy początkową sumę na 0.0 jako float32
    c = np.float32(0.0) # zmienna c przechowuje błąd kompensacji

    for element in tablica: # przechodzimy po każdym elemencie tablicy
```

```

        y = np.float32(element - c) # odejmujemy wcześniejszy błąd kompensacji od
aktualnego elementu
        t = np.float32(suma + y) # dodajemy skorygowany element do sumy
        c = np.float32((t-suma) - y) # obliczamy nowy błąd zaokrąglenia
        suma = t # aktualizujemy sumę

    return np.float32(suma) # zwracamy obliczoną sumę

n = 64 # liczba elementów tablicy
m = 16 # wartość używana w modulo, czyli okres powtarzania elementów

tablica = generuj_tablice(n, m) # generujemy tablicę elementów

suma = sumuj_tablice(tablica) # liczymy sumę zwykłą metodą
suma_m = sumuj_tablice_Møllera(tablica) # liczymy sumę metodą Møllera
suma_k = sumuj_tablice_Kahana(tablica) # liczymy sumę metodą Kahana

print("Suma zwykła:", suma) # wypisujemy wynik zwykłego sumowania
print("Suma Møllera:", suma_m) # wypisujemy wynik sumowania metodą Møllera
print("Suma Kahana:", suma_k) # wypisujemy wynik sumowania metodą Kahana

print("Sprawdzenie dokładności:", n/(m+1)) # wypisujemy wartość dokładną, z którą
można porównać wyniki

```

Jak porównać wyniki?

Porównujesz:

- zwykłą sumę,
- sumę metodą Gilla–Møllera,
- sumę metodą Kahana,

z wartością teoretyczną:

$$\frac{n}{m+1}$$

Im bliżej tej wartości, tym metoda jest dokładniejsza.

Wniosek do zadania 4

Zwykle:

- zwykłe sumowanie daje największy błąd,
 - metoda Gilla–Møllera daje dokładniejszy wynik,
 - metoda Kahana także poprawia dokładność i często daje najlepszy wynik.
-

Zadanie 5

Przeprowadź analogiczne działania dla danych w podwójnej precyzji.

Co się zmienia?

W poprzednich zadaniach używaliśmy liczb pojedynczej precyzji (`float32`).

Teraz robimy to samo, ale dla liczb podwójnej precyzji, czyli `float` / `float64`.

Podwójna precyzja daje większą dokładność, bo liczba jest przechowywana na większej liczbie bitów.

Jak rozumieć schemat implementacji?

Wszystko działa tak samo jak wcześniej:

- generujesz elementy ciągu,
- sumujesz je trzema metodami,
- porównujesz wyniki.

Różnica polega tylko na tym, że zamiast `np.float32` używasz zwykłego `float`.

```
def generuj_tablice(n, m): # funkcja generuje tablicę n elementów według danego
wzoru
    tablica = [] # tworzymy pustą listę, do której będą dodawane elementy

    for k in range(n): # pętla wykonuje się n razy, dla k od 0 do n-1
        licznik = 1 # ustawiamy licznik ułamka na 1
        mianownik = ((k % m) + 1) * ((k % m) + 2) # obliczamy mianownik z użyciem
reszty z dzielenia k przez m

        a = licznik / mianownik # obliczamy wartość elementu ciągu
        tablica.append(float(a)) # dodajemy element do tablicy jako typ float

    return tablica # zwracamy gotową tablicę

def sumuj_tablice(tablica): # funkcja sumuje elementy tablicy zwykłą metodą
suma = float(0.0) # ustawiamy początkową sumę na 0.0 jako float

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        suma = float(suma + element) # dodajemy aktualny element do sumy i
zapisujemy wynik jako float

    return suma # zwracamy obliczoną sumę

def sumuj_tablice_Møller(tablica): # funkcja sumuje elementy tablicy metodą
Møllera
    suma = float(0.0) # ustawiamy początkową sumę na 0.0 jako float
    poprawka = float(0.0) # tworzymy zmienną poprawka, która przechowuje utracone
części sumy

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        t = float(suma + element) # obliczamy tymczasową nową sumę
        poprawka = float(poprawka + (element - (t-suma))) # dodajemy do poprawki
część utraconą przez zaokrąglenie
        suma = t # aktualizujemy sumę

    return float(suma + poprawka) # zwracamy sumę powiększoną o poprawkę

def sumuj_tablice_Kahan(tablica): # funkcja sumuje elementy tablicy metodą Kahana
suma = float(0.0) # ustawiamy początkową sumę na 0.0 jako float
c = float(0.0) # tworzymy zmienną c, która przechowuje błąd kompensacji

    for element in tablica: # przechodzimy po każdym elemencie tablicy
        y = float(element - c) # odejmujemy wcześniejszy błąd kompensacji od
aktualnego elementu
        t = float(suma + y) # dodajemy skorygowany element do sumy
        c = float((t-suma) - y) # obliczamy nowy błąd zaokrąglenia
        suma = t # aktualizujemy sumę
```

```

    return float(suma) # zwracamy obliczoną sumę

n = 64 # liczba elementów tablicy
m = 16 # wartość używana w operacji modulo, czyli okres powtarzania mianownika

tablica = generuj_tablice(n, m) # generujemy tablicę elementów

suma = sumuj_tablice(tablica) # liczymy sumę zwykłą metodą
suma_m = sumuj_tablice_Møllera(tablica) # liczymy sumę metodą Møllera
suma_k = sumuj_tablice_Kahan(tablica) # liczymy sumę metodą Kahana

print("Suma zwykła:", suma) # wypisujemy wynik zwykłego sumowania
print("Suma Møllera:", suma_m) # wypisujemy wynik sumowania metodą Møllera
print("Suma Kahan:", suma_k) # wypisujemy wynik sumowania metodą Kahana

print("Sprawdzenie dokładności:", n/(m+1)) # wypisujemy wartość dokładną, z którą
porównujemy wyniki

```

Jak porównać wyniki z poprzednimi zadaniami?

Porównujesz wyniki z pojedynczej i podwójnej precyzji.

Zwykle:

- w podwójnej precyzji błędy są mniejsze,
- wszystkie metody dają wyniki bliższe wartości teoretycznej,
- metoda zwykła też działa lepiej niż w `float32`, ale nadal może być słabsza od Kahana i Gilla–Møllera.

Wniosek końcowy

W laboratorium porównano różne sposoby sumowania elementów ciągu liczb zmiennoprzecinkowych.

- Zwykłe sumowanie jest najprostsze, ale najbardziej narażone na błędy zaokrągleń.
- Algorytm Gilla–Møllera poprawia dokładność przez przechowywanie poprawki błędu.
- Algorytm Kahana również kompensuje błąd i zwykle daje bardzo dokładne wyniki.
- W podwójnej precyzji wszystkie obliczenia są dokładniejsze niż w pojedynczej precyzji.

Otrzymane wyniki potwierdzają, że sposób sumowania i rodzaj precyzji mają istotny wpływ na dokładność obliczeń numerycznych.

Lab 3

Zadanie 1. Napisz program, który obliczy normę: euklidesową, Manhattan, maximum dla n -wymiarowego wektora.

```
import math # importujemy moduł math, ponieważ używamy funkcji math.sqrt()

def normy_wektora(wektor): # funkcja liczy trzy normy wektora: euklidesową,
    Manhattan i maksimum

    # norma euklidesowa # norma euklidesowa to pierwiastek z sumy kwadratów
    elementów wektora
    suma_kwadratów = 0 # tworzymy zmienną, w której będziemy przechowywać sumę
    kwadratów
    for x in wektor: # przechodzimy po każdym elemencie wektora
        suma_kwadratów += x**2 # dodajemy kwadrat aktualnego elementu do sumy
    norma_euklidesowa = math.sqrt(suma_kwadratów) # obliczamy pierwiastek z sumy
    kwadratów

    # norma Manhattan # norma Manhattan to suma wartości bezwzględnych elementów
    wektora
    norma_manhattan = 0 # tworzymy zmienną, w której będziemy przechowywać sumę
    wartości bezwzględnych
    for x in wektor: # przechodzimy po każdym elemencie wektora
        norma_manhattan += abs(x) # dodajemy wartość bezwzględną aktualnego
    elementu

    # norma maximum # norma maksimum to największa wartość bezwzględna elementu
    wektora
    # norma_max = max(abs(x) for x in wektor) # krótsza wersja z użyciem funkcji
    max(), ale tutaj jej nie używamy
    norma_max = 0 # tworzymy zmienną przechowującą największą znalezioną wartość
    bezwzględną

    for x in wektor: # przechodzimy po każdym elemencie wektora
        wartosc_bezwzględna = abs(x) # liczymy wartość bezwzględną aktualnego
    elementu

    if wartosc_bezwzględna > norma_max: # sprawdzamy, czy aktualna wartość
    bezwzględna jest większa od dotychczasowego maksimum
        norma_max = wartosc_bezwzględna # jeśli tak, aktualizujemy maksimum

    return norma_euklidesowa, norma_manhattan, norma_max # zwracamy trzy obliczone
    normy

wektor = [3, 4, 5] # tworzymy przykładowy wektor

euklidesowa, manhattan, maksimum = normy_wektora(wektor) # wywołujemy funkcję i
zapisujemy trzy wyniki do osobnych zmiennych

print("Norma euklidesowa:", euklidesowa) # wypisujemy normę euklidesową
print("Norma Manhattan:", manhattan) # wypisujemy normę Manhattan
print("Norma maximum:", maksimum) # wypisujemy normę maksimum
```

Dla wektora $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ najczęściej używane normy:

- norma euklidesowa (2): $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- norma Manhattan (1): $\|x\|_1 = \sum_{i=1}^n |x_i|$
- norma maksimum (∞): $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$

Uwaga praktyczna: do porównań liczbowych często używa się $\|\cdot\|_\infty$ (najszybsza i „najostrzejsza” w sensie błędu maksymalnego).

1. Norma euklidesowa

Wzór

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Jak to czytać po ludzku?

- bierzesz wszystkie liczby z wektora,
- każdą podnosisz do kwadratu,
- wszystko dodajesz,
- z wyniku wyciągasz pierwiastek.

Przykład

Dla:

$$x = (2, -1, 4)$$

liczymy:

$$\|x\|_2 = \sqrt{2^2 + (-1)^2 + 4^2}$$

czyli:

$$\sqrt{4 + 1 + 16} = \sqrt{21}$$

To jest zwykła „szkolna długość” wektora.

2. Norma Manhattan

Wzór

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

Jak to czytać po ludzku?

- bierzesz wszystkie liczby z wektora,
- zamieniasz je na dodatnie (wartość bezwzględna),
- dodajesz je wszystkie.

Przykład

Dla:

$$x = (2, -1, 4)$$

liczymy:

$$\|x\|_1 = |2| + |-1| + |4|$$

czyli:

$$2 + 1 + 4 = 7$$

To się nazywa Manhattan, bo przypomina chodzenie po ulicach miasta: tylko poziomo i pionowo, bez skrótów na ukos.

3. Norma maksimum

Wzór

$$\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

Jak to czytać po ludzku?

- bierzesz wszystkie liczby z wektora,
- robisz z nich wartości dodatnie,
- wybierasz największą.

Przykład

Dla:

$$x = (2, -1, 4)$$

liczymy:

$$|2| = 2, \quad |-1| = 1, \quad |4| = 4$$

największa z nich to:

$$4$$

więc:

$$\|x\|_{\infty} = 4$$

Zadanie 2. Napisz program, który będzie wyliczał odległość pomiędzy dwoma punktami przestrzeni dwuwymiarowej w metrykach: euklidesowej, Manhattan, rzece i kolejowej.

```

import math # importujemy moduł math, ponieważ używamy pierwiastka math.sqrt() i
potęgowania math.pow()

def odleglosci(P, Q): # funkcja liczy różne odległości między punktami P i Q
    p1, p2 = P # rozpakowujemy współrzędne punktu P, czyli P = (p1, p2)
    q1, q2 = Q # rozpakowujemy współrzędne punktu Q, czyli Q = (q1, q2)

    # metryka euklidesowa # zwykła odległość między punktami w linii prostej
    euklidesowa = math.sqrt(math.pow((q1 - p1), 2) + math.pow((q2 - p2), 2)) #
    liczymy pierwiastek z sumy kwadratów różnic współrzędnych

    #metryka Manhattan # odległość liczona jak poruszanie się po kratce, czyli
    poziomo i pionowo
    manhattan = abs(p1 - q1) + abs(p2 - q2) # dodajemy wartości bezwzględne
    różnic współrzędnych

    #metryka rzeka # odległość w metryce rzeka
    if p1 == q1: # sprawdzamy, czy punkty leżą na tej samej pionowej prostej
        rzeka = abs(p2 - q2) # jeśli tak, odległość to tylko różnica drugich
współrzędnych
    else: # jeśli punkty nie leżą na tej samej pionowej prostej
        rzeka = abs(p1 - q1) + abs(p2) + abs(q2) # liczymy drogę przez rzekę,
czyli dojdęcie do osi, przejście wzdłuż osi i odejście od osi

    # metryka kolejowa # odległość w metryce kolejowej, gdzie centrum jest punktem
(0, 0)
    det = p1 * q2 - p2 * q1 # liczymy wyznacznik, który sprawdza, czy punkty i
początek układu są współliniowe
    if det == 0: # jeśli wyznacznik jest równy 0, punkty leżą na jednej prostej
przechodzącej przez początek układu
        kolejowa = math.sqrt(math.pow((q1 - p1), 2) + math.pow((q2 - p2), 2)) #
wtedy odległość kolejowa jest taka sama jak euklidesowa między P i Q
    else: # jeśli punkty nie leżą na jednej prostej z początkiem układu
        kolejowa = math.sqrt(math.pow((0 - p1), 2) + math.pow((0 - p2), 2)) +
math.sqrt(math.pow((0 - q1), 2) + math.pow((0 - q2), 2)) # liczymy drogę z P do
centrum i z centrum do Q

    return euklidesowa, manhattan, rzeka, kolejowa # zwracamy wszystkie obliczone
odległości

punktP = (2, 3) # tworzymy punkt P o współrzędnych (2, 3)
punktQ = (5, 7) # tworzymy punkt Q o współrzędnych (5, 7)

euklidesowa, manhattan, rzeka, kolejowa = odleglosci(punktP, punktQ) # wywołujemy
funkcję i zapisujemy wyniki do osobnych zmiennych

print("Norma euklidesowa:", euklidesowa) # wypisujemy odległość euklidesową
print("Norma Manhattan:", manhattan) # wypisujemy odległość Manhattan
print("Norma rzeka:", rzeka) # wypisujemy odległość w metryce rzeka
print("Norma kolejowa/centrum:", kolejowa) # wypisujemy odległość w metryce
kolejowej

```

Dla punktów $p = (x_1, x_2)$ oraz $q = (y_1, y_2)$:

- **Euklidesowa:** $d_2(p, q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$
- **Manhattan:** $d_1(p, q) = |x_1 - y_1| + |x_2 - y_2|$
- **Rzeka** (rzeka = oś Ox):

$$d_R(p, q) = \begin{cases} |x_2 - y_2|, & x_1 = y_1, \\ |x_1 - y_1| + |x_2| + |y_2|, & x_1 \neq y_1 \end{cases}$$

- **Kolejowa / centrum** ("jedź do węzła" $(0, 0)$, jeśli trzeba):

$$d_C(p, q) = \begin{cases} \|p - q\|_2, & p \text{ i } q \text{ na jednej prostej przez } (0, 0), \\ \|p\|_2 + \|q\|_2, & \text{w przeciwnym razie.} \end{cases}$$

Dla punktów:

$$p = (x_1, x_2)$$

oraz

$$q = (y_1, y_2)$$

chcemy policzyć odległość między nimi na kilka różnych sposobów.

1. Metryka euklidesowa

Wzór

$$d_2(p, q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Jak to czytać po ludzku?

- bierzesz różnicę pierwszych współrzędnych,
- bierzesz różnicę drugich współrzędnych,
- obie różnice podnosisz do kwadratu,
- dodajesz je,
- z wyniku wyciągasz pierwiastek.

To jest zwykła „szkolna” odległość między dwoma punktami.

Przykład

Dla:

$$p = (2, 3), \quad q = (5, 7)$$

liczymy:

$$d_2(p, q) = \sqrt{(2 - 5)^2 + (3 - 7)^2}$$

czyli:

$$\sqrt{(-3)^2 + (-4)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

2. Metryka Manhattan

Wzór

$$d_1(p, q) = |x_1 - y_1| + |x_2 - y_2|$$

Jak to czytać po ludzku?

- bierzesz różnicę pierwszych współrzędnych,
- bierzesz różnicę drugich współrzędnych,
- zamieniasz je na wartości dodatnie,
- dodajesz je.

Ta metryka przypomina poruszanie się po ulicach miasta: tylko poziomo i pionowo, bez skrótów na ukos.

Przykład

Dla:

$$p = (2, 3), \quad q = (5, 7)$$

liczymy:

$$d_1(p, q) = |2 - 5| + |3 - 7|$$

czyli:

$$3 + 4 = 7$$

3. Metryka rzeki

Na slajdzie jest napisane, że rzeka to oś Ox .

To znaczy, że można poruszać się wzdłuż osi poziomej, a żeby przejść między punktami o różnych pierwszych współrzędnych, trzeba „zejść do rzeki”, przejść nią i potem „wejść” do drugiego punktu.

Wzór

$$d_R(p, q) = \begin{cases} |x_2 - y_2|, & x_1 = y_1 \\ |x_1 - y_1| + |x_2| + |y_2|, & x_1 \neq y_1 \end{cases}$$

Jak to czytać po ludzku?

Przypadek 1: gdy $x_1 = y_1$

Jeśli punkty mają tę samą pierwszą współrzędną, to leżą „nad tym samym miejscem” na osi poziomej.

Wtedy wystarczy policzyć tylko różnicę drugich współrzędnych:

$$|x_2 - y_2|$$

Przypadek 2: gdy $x_1 \neq y_1$

Wtedy:

- schodzisz z punktu p do osi Ox ,
- idziesz w poziomie do miejsca pod punktem q ,
- wchodzisz do punktu q .

Dlatego liczymy:

$$|x_1 - y_1| + |x_2| + |y_2|$$

Przykład

Dla:

$$p = (2, 3), \quad q = (5, 7)$$

mamy:

$$x_1 \neq y_1$$

więc używamy drugiego wzoru:

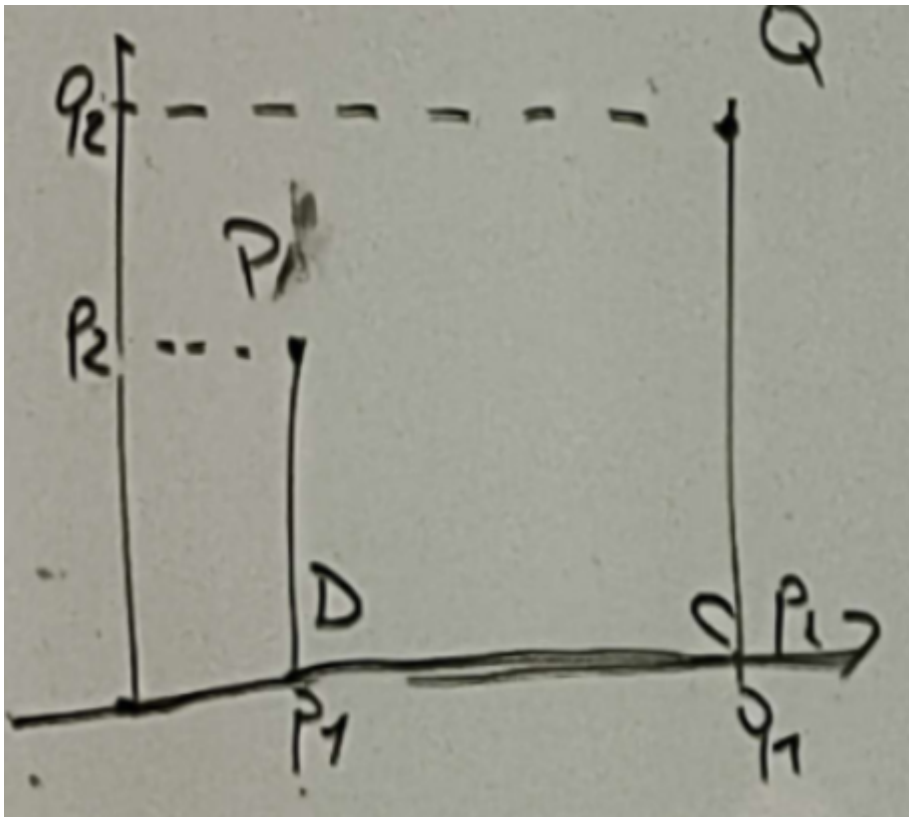
$$d_R(p, q) = |2 - 5| + |3| + |7|$$

czyli:

$$3 + 3 + 7 = 13$$

Notatka z talicy

$$\text{euclid}(P, D) + \text{euclid}(D, C) + \text{euclid}(C, D)$$



4. Metryka kolejowa / centrum

Tutaj idea jest taka, że jeśli trzeba, jedziemy przez punkt centralny:

$$(0, 0)$$

Wzór

$$d_C(p, q) = \begin{cases} \|p - q\|_2, & p \text{ i } q \text{ leżą na jednej prostej przechodzącej przez } (0, 0) \\ \|p\|_2 + \|q\|_2, & \text{w przeciwnym razie} \end{cases}$$

Jak to czytać po ludzku?

Przypadek 1: punkty leżą na jednej prostej przechodzącej przez $(0, 0)$

Wtedy można przejechać bezpośrednio między nimi.

Liczymy zwykłą odległość euklidesową:

$$\|p - q\|_2$$

czyli po prostu:

$$\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Przypadek 2: punkty nie leżą na jednej takiej prostej

Wtedy:

- jedziesz z punktu p do centrum $(0, 0)$,
- potem z centrum do punktu q .

Czyli liczysz:

$$\|p\|_2 + \|q\|_2$$

To znaczy:

- długość odcinka od $(0, 0)$ do p ,
- plus długość odcinka od $(0, 0)$ do q .

Przykład

Dla:

$$p = (2, 3), \quad q = (5, 7)$$

sprawdzamy, czy punkty leżą na jednej prostej przechodzącej przez $(0, 0)$.

Jeśli nie, to liczymy:

$$d_C(p, q) = \|p\|_2 + \|q\|_2$$

czyli:

$$\sqrt{2^2 + 3^2} + \sqrt{5^2 + 7^2}$$

czyli:

$$\sqrt{13} + \sqrt{74}$$

w przybliżeniu:

$$3.6055 + 8.6023 = 12.2078$$

Notatka z tablicy

$$\text{euclid}(P, Q) = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

Dla punktów:

$$P = (p_1, p_2)$$

$$Q = (q_1, q_2)$$

punkty P, Q, O są współliniowe wtedy i tylko wtedy, gdy:

$$\det \begin{bmatrix} p_1 & p_2 \\ q_1 & q_2 \end{bmatrix} = p_1 q_2 - p_2 q_1 = 0$$

Jeśli:

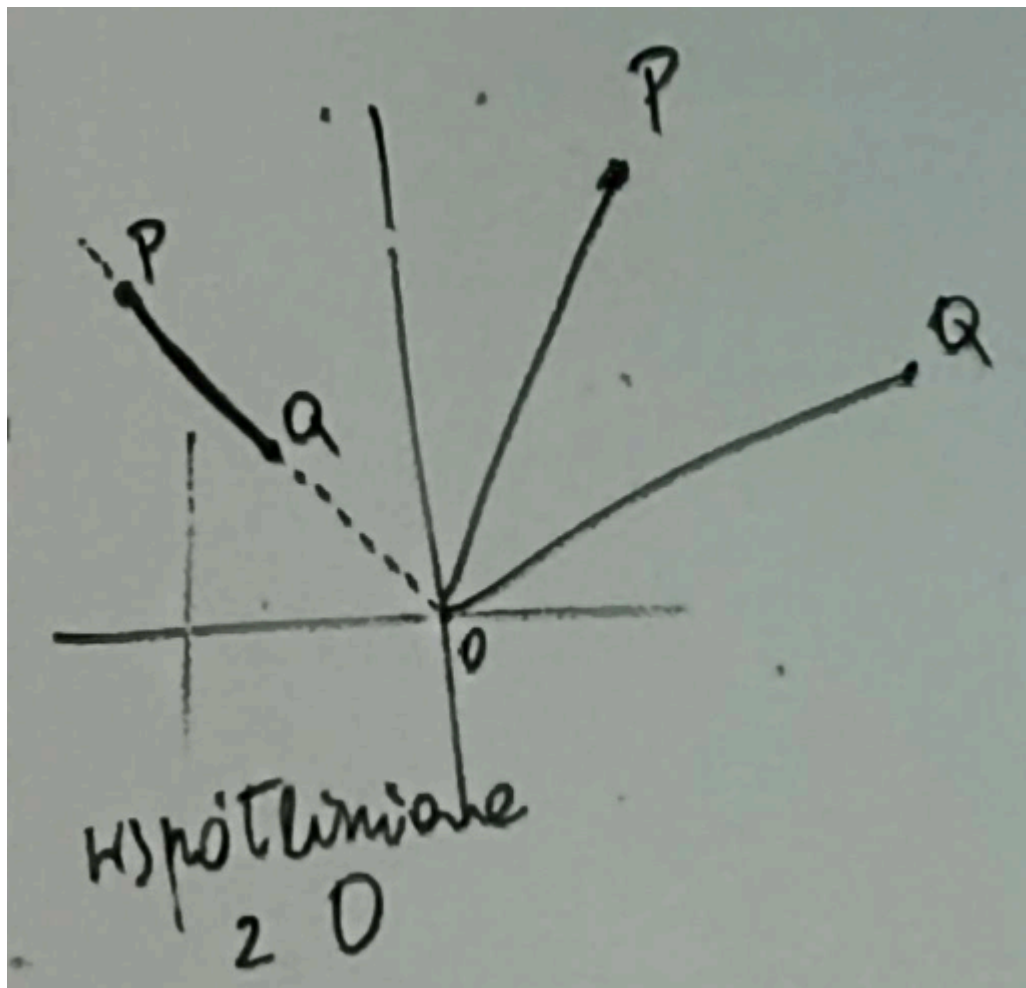
$$\det = 0$$

to:

$$\text{odległość}_{kolejowa}(P, Q) = \text{euclid}(P, Q)$$

w przeciwnym razie:

$$\text{odległość}_{kolejowa}(P, Q) = \text{euclid}(P, O) + \text{euclid}(Q, O)$$



Zadanie 3. Napisz program, który obliczy normę: Frobeniusa, Manhattan, maximum dla $n \times m$ -wymiarowej macierzy.

```

import math # importujemy moduł math, ponieważ używamy math.pow() i math.sqrt()

def normy_macierzy(macierz): # funkcja liczy trzy normy macierzy: Frobeniusa,
    # Manhattan i maksimum
    wiersze = len(macierz) # zapisujemy liczbę wierszy macierzy
    kolumny = len(macierz[0]) # zapisujemy liczbę kolumn macierzy, czyli długość
    # pierwszego wiersza

    # norma Frobeniusa # pierwiastek z sumy kwadratów wszystkich elementów
    # macierzy
    suma_kwadratow = 0 # tworzymy zmienną, w której będziemy przechowywać sumę
    # kwadratów elementów
    for i in range(wiersze): # przechodzimy po indeksach wierszy
        for j in range(kolumny): # przechodzimy po indeksach kolumn
            suma_kwadratow += math.pow(macierz[i][j], 2) # dodajemy kwadrat
            # aktualnego elementu macierzy
    Frobeniusa = math.sqrt(suma_kwadratow) # obliczamy pierwiastek z sumy
    # kwadratów

    # norma Manhattan # suma wartości bezwzględnych wszystkich elementów macierzy
    suma_modulow = 0 # tworzymy zmienną, w której będziemy przechowywać sumę
    # modułów
    for i in range(wiersze): # przechodzimy po indeksach wierszy
        for j in range(kolumny): # przechodzimy po indeksach kolumn
            suma_modulow += abs(macierz[i][j]) # dodajemy wartość bezwzględną
            # aktualnego elementu
    Manhattan = suma_modulow # zapisujemy sumę modułów jako normę Manhattan

    # norma maksimum # największa wartość bezwzględna spośród elementów macierzy
    maksimum = 0 # tworzymy zmienną przechowującą największą znaną wartość
    # bezwzględną
    for i in range(wiersze): # przechodzimy po indeksach wierszy
        for j in range(kolumny): # przechodzimy po indeksach kolumn
            if abs(macierz[i][j]) > maksimum: # sprawdzamy, czy aktualny moduł
            # elementu jest większy niż dotychczasowe maksimum
                maksimum = abs(macierz[i][j]) # jeśli tak, aktualizujemy maksimum

    return Frobeniusa, Manhattan, maksimum # zwracamy trzy obliczone normy

macierz = [ # tworzymy przykładową macierz
    [1, -2, 3], # pierwszy wiersz macierzy
    [4, 5, -6] # drugi wiersz macierzy
] # koniec definicji macierzy

Frobeniusa, Manhattan, maksimum = normy_macierzy(macierz) # wywołujemy funkcję i
# zapisujemy wyniki do osobnych zmiennych

print("Norma Frobeniusa:", Frobeniusa) # wypisujemy normę Frobeniusa
print("Norma Manhattan:", Manhattan) # wypisujemy normę Manhattan
print("Norma maksimum:", maksimum) # wypisujemy normę maksimum

```

Dla macierzy $A = [a_{ij}] \in \mathbb{R}^{n \times m}$ definiujemy m.in.:

- **Normę Frobeniusa:** $\|A\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m a_{ij}^2}$
- **Normę Manhattan (suma modułów elementów):**
 $\|A\|_{1,\text{sum}} = \sum_{i=1}^n \sum_{j=1}^m |a_{ij}|$
- **Normę maksimum (elementową):**
 $\|A\|_{\max} = \max_{1 \leq i \leq n, 1 \leq j \leq m} |a_{ij}|$

Dla macierzy:

$$A = [a_{ij}] \in \mathbb{R}^{n \times m}$$

czyli macierzy o n wierszach i m kolumnach, można zdefiniować kilka norm.

1. Norma Frobeniusa

Wzór

$$\|A\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m a_{ij}^2}$$

Jak to czytać po ludzku?

- bierzesz wszystkie elementy macierzy,
- każdy podnosisz do kwadratu,
- dodajesz wszystkie te kwadraty,
- z otrzymanej sumy wyciągasz pierwiastek.

To jest odpowiednik normy euklidesowej dla macierzy.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 1 & -2 & 3 \\ 4 & 5 & -6 \end{bmatrix}$$

liczymy:

$$\|A\|_F = \sqrt{1^2 + (-2)^2 + 3^2 + 4^2 + 5^2 + (-6)^2}$$

czyli:

$$\sqrt{1 + 4 + 9 + 16 + 25 + 36} = \sqrt{91}$$

2. Norma Manhattan

Wzór

$$\|A\|_{1,\text{sum}} = \sum_{i=1}^n \sum_{j=1}^m |a_{ij}|$$

Jak to czytać po ludzku?

- bierzesz wszystkie elementy macierzy,
- zamieniasz je na wartości dodatnie (wartości bezwzględne),
- dodajesz wszystkie te wartości.

To jest po prostu suma modułów wszystkich elementów macierzy.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 1 & -2 & 3 \\ 4 & 5 & -6 \end{bmatrix}$$

liczymy:

$$\|A\|_{1,\text{sum}} = |1| + |-2| + |3| + |4| + |5| + |-6|$$

czyli:

$$1 + 2 + 3 + 4 + 5 + 6 = 21$$

3. Norma maksimum

Wzór

$$\|A\|_{\max} = \max_{1 \leq i \leq n, 1 \leq j \leq m} |a_{ij}|$$

Jak to czytać po ludzku?

- bierzesz wszystkie elementy macierzy,
- zamieniasz je na wartości dodatnie,
- wybierasz największą z nich.

To jest największy moduł spośród wszystkich elementów macierzy.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 1 & -2 & 3 \\ 4 & 5 & -6 \end{bmatrix}$$

liczymy moduły elementów:

$$|1| = 1, \quad |-2| = 2, \quad |3| = 3, \quad |4| = 4, \quad |5| = 5, \quad |-6| = 6$$

największa z tych wartości to:

$$6$$

więc:

$$\|A\|_{\max} = 6$$

Co oznaczają symbole?

a_{ij}

To element macierzy znajdujący się:

- w **i**-tym wierszu,
- w **j**-tej kolumnie.

Na przykład w macierzy

$$\begin{bmatrix} 1 & -2 & 3 \\ 4 & 5 & -6 \end{bmatrix}$$

- $a_{11} = 1$
 - $a_{12} = -2$
 - $a_{23} = -6$
-



Podwójna suma znaczy:

- przejdź po wszystkich wierszach,
 - w każdym wierszu przejdź po wszystkich kolumnach,
 - dodaj wszystkie elementy.
-

Zadanie 4. Napisz program, który wykona mnożenie dwóch macierzy. Kiedy działanie takie nie może zostać przeprowadzone? Sprawdź czy mnożenie macierzy jest przemienne lub łączne?

```

def mnozenie_macierzy(macierz1, macierz2): # funkcja mnoży dwie macierze
    ilosc_wierszy_macierz1 = len(macierz1) # zapisujemy liczbę wierszy pierwszej
macierzy
    ilosc_wierszy_macierz2 = len(macierz2) # zapisujemy liczbę wierszy drugiej
macierzy
    ilosc_kolumn_macierz1 = len(macierz1[0]) # zapisujemy liczbę kolumn pierwszej
macierzy
    ilosc_kolumn_macierz2 = len(macierz2[0]) # zapisujemy liczbę kolumn drugiej
macierzy

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2: # sprawdzamy, czy liczba
kolumn pierwszej macierzy jest równa liczbie wierszy drugiej macierzy
        raise ValueError("Nie da się pomnożyć tych macierzy") # jeśli warunek nie
jest spełniony, zgłaszamy błąd

    wynik = [] # tworzymy pustą listę na macierz wynikową
    for i in range(ilosc_wierszy_macierz1): # przechodzimy po wierszach pierwszej
macierzy
        wiersz = [] # tworzymy pusty wiersz wyniku
        for j in range(ilosc_kolumn_macierz2): # przechodzimy po kolumnach
drugiej macierzy
            suma = 0 # zerujemy sumę dla jednego elementu macierzy wynikowej
            for k in range(ilosc_kolumn_macierz1): # przechodzimy po elementach
wiersza pierwszej macierzy i kolumny drugiej macierzy
                suma += macierz1[i][k] * macierz2[k][j] # mnożymy odpowiednie
elementy i dodajemy je do sumy
            wiersz.append(suma) # dodajemy obliczony element do aktualnego wiersza
            wynik.append(wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy macierz wynikową

macierz1 = [ # tworzymy pierwszą macierz
    [1, 2], # pierwszy wiersz pierwszej macierzy
    [3, 4] # drugi wiersz pierwszej macierzy
] # koniec pierwszej macierzy

macierz2 = [ # tworzymy drugą macierz
    [5, 6], # pierwszy wiersz drugiej macierzy
    [7, 8] # drugi wiersz drugiej macierzy
] # koniec drugiej macierzy

wynik = mnozenie_macierzy(macierz1, macierz2) # wywołujemy funkcję mnożenia i
zapisujemy wynik

print("Wynik mnożenia:") # wypisujemy tekst informacyjny
for wiersz in wynik: # przechodzimy po kolejnych wierszach wyniku
    print(wiersz) # wypisujemy aktualny wiersz

```

Mnożenie macierzy – notatka

1. Kiedy można mnożyć macierze?

Jeśli:

$$A \in \mathbb{R}^{m_1 \times n_1} \quad \text{oraz} \quad B \in \mathbb{R}^{m_2 \times n_2}$$

to iloczyn:

$$AB$$

istnieje **wtedy i tylko wtedy**, gdy:

$$n_1 = m_2$$

Czyli:

- liczba **kolumn** pierwszej macierzy
- musi być równa liczbie **wierszy** drugiej macierzy.

2. Jaki rozmiar ma wynik?

Jeśli mnożenie jest możliwe, to:

$$AB \in \mathbb{R}^{m_1 \times n_2}$$

czyli wynik ma:

- tyle wierszy, ile ma macierz **A**
- tyle kolumn, ile ma macierz **B**

3. Jak wygląda wzór na mnożenie macierzy?

Niech:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

oraz

$$B = \begin{bmatrix} b_{1,1} & b_{1,2} & \cdots & b_{1,k} \\ b_{2,1} & b_{2,2} & \cdots & b_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n,1} & b_{n,2} & \cdots & b_{n,k} \end{bmatrix}$$

Wtedy:

$$C = AB$$

jest macierzą postaci:

$$C = \begin{bmatrix} c_{1,1} & c_{1,2} & \cdots & c_{1,k} \\ c_{2,1} & c_{2,2} & \cdots & c_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m,1} & c_{m,2} & \cdots & c_{m,k} \end{bmatrix}$$

a każdy element macierzy wynikowej liczymy ze wzoru:

$$c_{i,j} = \sum_{l=1}^n a_{i,l} b_{l,j}$$

dla:

$$i = 1, 2, \dots, m \quad \text{oraz} \quad j = 1, 2, \dots, k$$

4. Jak to czytać po ludzku?

Żeby policzyć element:

$$c_{i,j}$$

trzeba:

- wziąć **i-ty wiersz** z macierzy **A**
- i **j-tą kolumnę** z macierzy **B**
- pomnożyć odpowiadające sobie elementy
- dodać wszystkie wyniki

5. Przykład obliczania jednego elementu

Jeśli:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{oraz} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

to element:

$$c_{1,1}$$

liczymy tak:

- pierwszy wiersz z (A): ((1,2))
- pierwsza kolumna z (B): ((5,7))

więc:

$$c_{1,1} = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19$$

element:

$$c_{1,2}$$

- pierwszy wiersz z (A): ((1,2))

- druga kolumna z (B): ((6,8))

więc:

$$c_{1,2} = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22$$

element:

$$c_{2,1}$$

- drugi wiersz z (A): ((3,4))
- pierwsza kolumna z (B): ((5,7))

więc:

$$c_{2,1} = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43$$

element:

$$c_{2,2}$$

- drugi wiersz z (A): ((3,4))
- druga kolumna z (B): ((6,8))

więc:

$$c_{2,2} = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50$$

Czyli:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

6. Własności mnożenia macierzy

Mnożenie macierzy nie jest przemienne

W ogólności:

$$AB \neq BA$$

To znaczy, że zmiana kolejności macierzy zwykle daje inny wynik.

Mnożenie macierzy jest łączne

$$(AB)C = A(BC)$$

o ile wymiary macierzy pozwalają wykonać oba mnożenia.

Mnożenie macierzy jest rozłączne względem dodawania

$$A(B + C) = AB + AC$$

oraz

$$(A + B)C = AC + BC$$

Macierz jednostkowa jest elementem neutralnym

Jeśli (I) to macierz jednostkowa, to:

$$AI = A \quad \text{oraz} \quad IA = A$$

7. Przykład pokazujący, że $(AB \neq BA)$

Weźmy:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$$

Wtedy:

$$AB = \begin{bmatrix} 7 & 2 \\ 3 & 1 \end{bmatrix}$$

oraz:

$$BA = \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

Zatem:

$$AB \neq BA$$

8. Łączność mnożenia

Dla macierzy o pasujących wymiarach zawsze zachodzi:

$$(AB)C = A(BC)$$

czyli wynik nie zależy od tego, które mnożenie wykonamy najpierw.

9. Koszt obliczeń

Jeśli:

$$A \in \mathbb{R}^{m_1 \times n_1} \quad \text{oraz} \quad B \in \mathbb{R}^{n_1 \times n_2}$$

to koszt obliczenia iloczynu (AB) wynosi:

$$\Theta(m_1 \cdot n_1 \cdot n_2)$$

czyli liczba działań rośnie w przybliżeniu jak:

- liczba wierszy pierwszej macierzy
- razy liczba wspólnego wymiaru
- razy liczba kolumn drugiej macierzy

10. Najkrócej

Kiedy można mnożyć?

Gdy liczba kolumn pierwszej macierzy jest równa liczbie wierszy drugiej.

Jaki rozmiar ma wynik?

Tyle wierszy co pierwsza macierz i tyle kolumn co druga.

Jak liczymy element wyniku?

Wiersz z pierwszej macierzy razy kolumna z drugiej.

Czy mnożenie jest przemienne?

Nie.

Czy mnożenie jest łączne?

Tak.

Zadanie 5*. Zaprojektuj i utwórz klasę dla macierzy umożliwiającą tworzenie, wypisywanie i wykonywanie działań: mnożenie przez stałą, dodawanie, mnożenie.

```

# definicja klasy Macierz
class Macierz:
    # funkcja uruchamiana przy tworzeniu nowego obiektu klasy
    def __init__(self, dane):
        # zapisanie danych macierzy wewnątrz obiektu
        self.dane = dane

    # funkcja do wypisywania macierzy
    def wypisz(self):
        # przejście po wszystkich wierszach macierzy
        for wiersz in self.dane:
            # wypisanie jednego wiersza
            print(wiersz)

    # funkcja mnożąca macierz przez liczbę
    def mnozenie_przez_stala(self, stala):
        # pusta lista na wynik
        wynik = []

        # przejście po wszystkich wierszach macierzy
        for wiersz in self.dane:
            # nowy wiersz wynikowy
            nowy_wiersz = []

            # przejście po wszystkich elementach w danym wierszu
            for element in wiersz:
                # dodanie do nowego wiersza elementu pomnożonego przez stałą
                nowy_wiersz.append(element * stala)

            # dodanie gotowego wiersza do macierzy wynikowej
            wynik.append(nowy_wiersz)

        # zwrócenie nowej macierzy jako obiektu klasy Macierz
        return Macierz(wynik)

    # funkcja dodająca dwie macierze
    def dodawanie(self, inna):
        # pusta lista na wynik
        wynik = []

        # przejście po numerach wierszy
        for i in range(len(self.dane)):
            # nowy wiersz wynikowy
            wiersz = []

            # przejście po numerach kolumn
            for j in range(len(self.dane[0])):
                # dodanie do siebie elementów z obu macierzy o tych samych
                indeksach
                wiersz.append(self.dane[i][j] + inna.dane[i][j])

            # dodanie gotowego wiersza do macierzy wynikowej

```



```

        wynik.append(wiersz)

    # zwrócenie nowej macierzy jako obiektu klasy Macierz
    return Macierz(wynik)

# funkcja mnożąca dwie macierze
def mnozenie(self, inna):
    # pusta lista na wynik
    wynik = []

    # przejście po wierszach pierwszej macierzy
    for i in range(len(self.dane)):
        # nowy wiersz wynikowy
        wiersz = []

        # przejście po kolumnach drugiej macierzy
        for j in range(len(inna.dane[0])):
            # zmienna przechowująca sumę iloczynów
            suma = 0

            # przejście po elementach wiersza i kolumny
            for k in range(len(self.dane[0])):
                # dodawanie kolejnych iloczynów do sumy
                suma += self.dane[i][k] * inna.dane[k][j]

            # dodanie obliczonego elementu do wiersza wynikowego
            wiersz.append(suma)

        # dodanie gotowego wiersza do macierzy wynikowej
        wynik.append(wiersz)

    # zwrócenie nowej macierzy jako obiektu klasy Macierz
    return Macierz(wynik)

# utworzenie pierwszej macierzy A
A = Macierz([[1, 2], [3, 4]])

# utworzenie drugiej macierzy B
B = Macierz([[5, 6], [7, 8]])

# wypisanie napisu informacyjnego
print("Macierz A:")

# wypisanie macierzy A
A.wypisz()

# wypisanie napisu informacyjnego
print("Macierz B:")

# wypisanie macierzy B
B.wypisz()

```

```
# wypisanie napisu informacyjnego
print("A + B:")

# dodanie macierzy A i B, a potem wypisanie wyniku
A.dodawanie(B).wypisz()

# wypisanie napisu informacyjnego
print("A * 2:")

# pomnożenie macierzy A przez 2, a potem wypisanie wyniku
A.mnozenie_przez_stala(2).wypisz()

# wypisanie napisu informacyjnego
print("A * B:")

# pomnożenie macierzy A i B, a potem wypisanie wyniku
A.mnozenie(B).wypisz()
```

1. Czym jest macierz?

Macierz to prostokątna tablica liczb ułożonych w wierszach i kolumnach.

Przykład:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Ta macierz ma:

- 2 wiersze,
- 2 kolumny.

Elementy macierzy oznaczamy zwykle jako:

$$a_{ij}$$

gdzie:

- i oznacza numer wiersza,
- j oznacza numer kolumny.

Na przykład w macierzy:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

mamy:

- $a_{11} = 1$
- $a_{12} = 2$
- $a_{21} = 3$

- $a_{22} = 4$
-

2. Mnożenie macierzy przez stałą

Jeśli mamy macierz:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

i liczbę (c), to mnożenie macierzy przez stałą polega na pomnożeniu **każdego elementu macierzy** przez tę liczbę.

Wzór:

$$cA = \begin{bmatrix} ca_{11} & ca_{12} \\ ca_{21} & ca_{22} \end{bmatrix}$$

Przykład

Dla:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

mnożenie przez 2 daje:

$$2A = \begin{bmatrix} 2 \cdot 1 & 2 \cdot 2 \\ 2 \cdot 3 & 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

Jak to rozumieć?

Każdy element macierzy zmienia się proporcjonalnie do tej stałej.

3. Dodawanie macierzy

Dwie macierze można dodać tylko wtedy, gdy mają **te same wymiary**, czyli tyle samo wierszy i tyle samo kolumn.

Jeśli:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad \text{oraz} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

to:

$$A + B = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} \\ a_{21} + b_{21} & a_{22} + b_{22} \end{bmatrix}$$

Czyli dodajemy do siebie elementy leżące w tych samych miejscach.

Przykład

Niech:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{oraz} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Wtedy:

$$A + B = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Kiedy nie można dodawać?

Nie można dodać macierzy o różnych wymiarach, na przykład:

- 2×2 i 2×3 ,
 - 3×2 i 2×2 .
-

4. Mnożenie macierzy

Mnożenie macierzy jest trudniejsze niż dodawanie.

Jeśli:

$$A \in \mathbb{R}^{m \times n} \quad \text{oraz} \quad B \in \mathbb{R}^{n \times k}$$

to iloczyn:

$$AB$$

istnieje i ma wymiary:

$$m \times k$$

Warunek mnożenia

Macierze można pomnożyć tylko wtedy, gdy:

- liczba kolumn pierwszej macierzy
 - jest równa liczbie wierszy drugiej macierzy.
-

5. Jak liczy się elementy iloczynu?

Jeśli:

$$C = AB$$

to każdy element macierzy wynikowej obliczamy ze wzoru:

$$c_{ij} = \sum_{l=1}^n a_{il}b_{lj}$$

To znaczy:

- bierzemy i -ty wiersz macierzy A ,
 - bierzemy j -tą kolumnę macierzy B ,
 - mnożymy odpowiadające sobie elementy,
 - dodajemy wyniki.
-

6. Przykład mnożenia macierzy

Niech:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{oraz} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Szukamy:

$$AB$$

Element c_{11}

Pierwszy wiersz macierzy A :

$$(1, 2)$$

Pierwsza kolumna macierzy B :

$$\begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

Liczymy:

$$c_{11} = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19$$

Element c_{12}

Pierwszy wiersz macierzy A :

$$(1, 2)$$

Druga kolumna macierzy B :

$$\begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Liczymy:

$$c_{12} = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22$$

Element c_{21}

Drugi wiersz macierzy A :

$$(3, 4)$$

Pierwsza kolumna macierzy B :

$$\begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

Liczymy:

$$c_{21} = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43$$

Element c_{22}

Drugi wiersz macierzy A :

$$(3, 4)$$

Druga kolumna macierzy B :

$$\begin{bmatrix} 6 \\ 8 \end{bmatrix}$$

Liczymy:

$$c_{22} = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50$$

Zatem:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

7. Ważne własności działań na macierzach

Dodawanie macierzy

Dodawanie macierzy jest:

- **przemienne:**

$$A + B = B + A$$

- **łączne:**

$$(A + B) + C = A + (B + C)$$

Mnożenie przez stałą

Jeśli c jest liczbą, to:

$$c(A + B) = cA + cB$$

oraz:

$$(c + d)A = cA + dA$$

Mnożenie macierzy

Mnożenie macierzy:

- **nie jest przemienne:**

$$AB \neq BA$$

- **jest łączne:**

$$(AB)C = A(BC)$$

- **jest rozdzielne względem dodawania:**

$$A(B + C) = AB + AC$$

oraz

$$(A + B)C = AC + BC$$

8. Podsumowanie

W tym zadaniu klasa macierzy ma odwzorowywać podstawowe działania matematyczne:

Mnożenie przez stałą

Każdy element macierzy mnożymy przez daną liczbę.

Dodawanie

Dodajemy do siebie elementy stojące na tych samych pozycjach.
Można to zrobić tylko dla macierzy o tych samych wymiarach.

Mnożenie

Aby pomnożyć dwie macierze:

- liczba kolumn pierwszej musi być równa liczbie wierszy drugiej,
 - każdy element wyniku oblicza się jako sumę iloczynów elementów odpowiedniego wiersza i kolumny.
-

9. Najkrócej

- **mnożenie przez stałą** – każdy element razy liczba,
 - **dodawanie** – element do elementu,
 - **mnożenie macierzy** – wiersz razy kolumna.
-

Lab 4

Zadanie 1

Napisz funkcję zwracającą wyznacznik macierzy kwadratowej dowolnego rozmiaru.

Jak rozumieć to zadanie?

Wyznacznik można policzyć tylko dla **macierzy kwadratowej**, czyli takiej, która ma tyle samo wierszy co kolumn.

Dla małych macierzy są proste wzory:

Dla macierzy 1×1

$$\det(A) = a_{11}$$

Dla macierzy 2×2

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

Dla większych macierzy

Stosujemy **rozwinięcie Laplace'a** względem pierwszego wiersza:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} M_{1j}$$

gdzie M_{1j} to minor, czyli wyznacznik macierzy powstałej po skreśleniu pierwszego wiersza i j -tej kolumny.

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz, czy macierz jest kwadratowa.

Krok 2

Jeśli macierz ma rozmiar 1×1 , zwracasz jedyny element.

Krok 3

Jeśli macierz ma rozmiar 2×2 , liczysz wyznacznik ze wzoru:

$$ad - bc$$

Krok 4

Dla większych macierzy:

- tworzysz minory,
- liczysz ich wyznaczniki rekurencyjnie,
- sumujesz wszystko ze znakami $+$ i $-$.

Kod

PYTHON

```
import math # importujemy moduł math, ponieważ później używamy funkcji math.pow()

def minor(macierz, usun_wiersz, usun_kolumne): # funkcja tworzy minor, czyli
macierz po usunięciu jednego wiersza i jednej kolumny
    wynik = [] # tworzymy pustą listę, do której będą dodawane wiersze nowej
macierzy

    for i in range(len(macierz)): # przechodzimy po wszystkich wierszach macierzy
        if i == usun_wiersz: # sprawdzamy, czy aktualny wiersz jest tym, który
        trzeba usunąć
            continue # pomijamy ten wiersz i przechodzimy do następnego

        nowy_wiersz = [] # tworzymy pusty wiersz dla nowej macierzy
        for j in range(len(macierz[i])): # przechodzimy po wszystkich kolumnach w
aktualnym wierszu
            if j == usun_kolumne: # sprawdzamy, czy aktualna kolumna jest tą,
którą trzeba usunąć
                continue # pomijamy ten element i przechodzimy do następnej
kolumny

            nowy_wiersz.append(macierz[i][j]) # dodajemy element do nowego
wiersza, jeśli nie leży w usuwanej kolumnie

        wynik.append(nowy_wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy macierz po usunięciu wskazanego wiersza i kolumny

def wyznacznik_macierzy(macierz): # funkcja liczy wyznacznik macierzy
n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie
jest kwadratowa") # zgłaszamy błąd, jeśli macierz nie jest kwadratowa

    if n == 1: # sprawdzamy przypadek macierzy 1 × 1
        return macierz[0][0] # wyznacznik macierzy 1 × 1 to jej jedyny element

    if n == 2: # sprawdzamy przypadek macierzy 2 × 2
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0] #
liczymy wyznacznik ze wzoru ad - bc

    det = 0 # ustawiamy początkową wartość wyznacznika na 0
    for j in range(n): # przechodzimy po kolejnych elementach pierwszego wiersza
        podmacierz = minor(macierz, 0, j) # tworzymy minor przez usunięcie
pierwszego wiersza i kolumny j
        det += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz) # dodajemy kolejny składnik rozwinięcia Laplace'a

    return det # zwracamy obliczony wyznacznik
```

```
macierz = [ # tworzymy macierz, dla której będzie liczony wyznacznik
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

print("Wyznacznik macierzy: ", wyznacznik_macierzy(macierz)) # wypisujemy wynik
działania funkcji wyznacznik_macierzy
```

Wynik

Dla macierzy

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy:

$$\det(A) = 66$$

Zadanie 2

Napisz funkcję zwracającą transpozycję macierzy dowolnego rozmiaru.

Co to jest transpozycja?

Transpozycja macierzy polega na zamianie:

- wierszy na kolumny,
- kolumn na wiersze.

Jeśli:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

to:

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz liczbę wierszy i kolumn.

Krok 2

Tworzysz nową macierz wynikową.

Krok 3

Dla każdej kolumny starej macierzy tworzysz nowy wiersz.

Czyli element:

$$a_{ij}$$

staje się elementem:

$$a_{ji}$$

Kod

PYTHON

```
def transpozycja(macierz): # funkcja wykonuje transpozycję macierzy, czyli
    # zamienia wiersze na kolumny
    liczba_wierszy = len(macierz) # zapisujemy liczbę wierszy macierzy
    liczba_kolumn = len(macierz[0]) # zapisujemy liczbę kolumn macierzy, czyli
    # długość pierwszego wiersza

    wynik = [] # tworzymy pustą listę, do której będziemy dodawać wiersze macierzy
    # po transpozycji

    for j in range(liczba_kolumn): # przechodzimy po kolumnach starej macierzy
        nowy_wiersz = [] # tworzymy nowy wiersz macierzy wynikowej
        for i in range(liczba_wierszy): # przechodzimy po wierszach starej
            # macierzy
            nowy_wiersz.append(macierz[i][j]) # dodajemy element z kolumny starej
            # macierzy do nowego wiersza
        wynik.append(nowy_wiersz) # dodajemy gotowy nowy wiersz do macierzy
        # wynikowej

    return wynik # zwracamy macierz po transpozycji

macierz = [ # tworzymy macierz, którą będziemy transponować
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

print("Przed transpozycją macierzy:") # wypisujemy informację, że poniżej będzie
# macierz przed transpozycją
for wiersz in macierz: # przechodzimy po kolejnych wierszach macierzy
    print(wiersz) # wypisujemy aktualny wiersz macierzy

wynik = transpozycja(macierz) # wywołujemy funkcję transpozycja i zapisujemy wynik
# do zmiennej wynik

print("Po transpozycji macierzy:") # wypisujemy informację, że poniżej będzie
# macierz po transpozycji
for wiersz in wynik: # przechodzimy po kolejnych wierszach macierzy po
    # transpozycji
    print(wiersz) # wypisujemy aktualny wiersz macierzy wynikowej
```

Wynik

Dla macierzy:

$$\begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

transpozycja ma postać:

$$\begin{bmatrix} 2 & 0 & -3 \\ 4 & 2 & 3 \\ 6 & -1 & 3 \end{bmatrix}$$

Zadanie 3

Napisz funkcję znajdującą macierz odwrotną do macierzy kwadratowej dowolnego rozmiaru za pomocą:

- a) rozwinięcia Laplace'a
- b) metody Gaussa-Jordana

Zadanie 3a — macierz odwrotna metodą Laplace'a

Idea metody

Jeśli macierz A ma wyznacznik różny od zera, to macierz odwrotna istnieje i można ją policzyć ze wzoru:

$$A^{-1} = \frac{1}{\det(A)} \cdot Adj$$

gdzie:

- $\det(A)$ to wyznacznik,
- Adj to macierz dołączona, czyli transpozycja macierzy dopełnień algebraicznych.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz wyznacznik macierzy.

Krok 2

Jeśli wyznacznik jest równy 0, macierz nie ma odwrotności.

Krok 3

Dla każdego elementu liczysz minor i dopełnienie algebraiczne:

$$C_{ij} = (-1)^{i+j} \det(M_{ij})$$

Krok 4

Tworzysz macierz dopełnień algebraicznych.

Krok 5

Robisz jej transpozycję, czyli macierz dołączoną.

Krok 6

Dzielisz każdy element przez wyznacznik.

```
import math # importujemy moduł math, ponieważ używamy funkcji math.pow()

def minor(macierz, usun_wiersz, usun_kolumne): # funkcja tworzy minor, czyli
macierz po usunięciu wskazanego wiersza i kolumny
    wynik = [] # tworzymy pustą listę na macierz wynikową

    for i in range(len(macierz)): # przechodzimy po indeksach wszystkich wierszy
macierzy
        if i == usun_wiersz: # sprawdzamy, czy aktualny wiersz jest tym, który ma
zostać usunięty
            continue # pomijamy ten wiersz

        nowy_wiersz = [] # tworzymy pusty wiersz do nowej macierzy
        for j in range(len(macierz[i])): # przechodzimy po indeksach wszystkich
kolumn w aktualnym wierszu
            if j == usun_kolumne: # sprawdzamy, czy aktualna kolumna jest tą,
która ma zostać usunięta
                continue # pomijamy ten element
            nowy_wiersz.append(macierz[i][j]) # dodajemy element do nowego
wiersza, jeśli nie jest w usuwanej kolumnie

        wynik.append(nowy_wiersz) # dodajemy nowy wiersz do macierzy wynikowej

    return wynik # zwracamy minor macierzy

def wyznacznik_macierzy(macierz): # funkcja liczy wyznacznik macierzy
n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Nie da się policzyć wyznacznika macierzy, która nie
jest kwadratowa") # zgłaszamy błąd, jeśli macierz nie jest kwadratowa

    if n == 1: # sprawdzamy przypadek macierzy 1 × 1
        return macierz[0][0] # wyznacznik macierzy 1 × 1 to jej jedyny element

    if n == 2: # sprawdzamy przypadek macierzy 2 × 2
        return macierz[0][0] * macierz[1][1] - macierz[0][1] * macierz[1][0] #
liczymy wyznacznik ze wzoru ad - bc

    det = 0 # ustawiamy początkową wartość wyznacznika na 0
    for j in range(n): # przechodzimy po elementach pierwszego wiersza
        podmacierz = minor(macierz, 0, j) # tworzymy minor przez usunięcie
pierwszego wiersza i kolumny j
        det += math.pow((-1), 0+j) * macierz[0][j] *
wyznacznik_macierzy(podmacierz) # dodajemy składnik rozwinięcia Laplace'a

    return det # zwracamy obliczony wyznacznik
```



```

def transpozycja(macierz): # funkcja wykonuje transpozycję macierzy
    liczba_wierszy = len(macierz) # zapisujemy liczbę wierszy macierzy
    liczba_kolumn = len(macierz[0]) # zapisujemy liczbę kolumn macierzy

    wynik = [] # tworzymy pustą macierz wynikową

    for j in range(liczba_kolumn): # przechodzimy po kolumnach starej macierzy
        nowy_wiersz = [] # tworzymy nowy wiersz macierzy po transpozycji
        for i in range(liczba_wierszy): # przechodzimy po wierszach starej
macierzy
            nowy_wiersz.append(macierz[i][j]) # dodajemy element z kolumny starej
macierzy do wiersza nowej macierzy
        wynik.append(nowy_wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy macierz po transpozycji

def zeros(n,m): # funkcja tworzy macierz zerową o wymiarach n x m
    macierz = [] # tworzymy pustą listę na macierz

    for i in range(n): # przechodzimy po liczbie wierszy
        wiersz = [] # tworzymy pusty wiersz
        for j in range(m): # przechodzimy po liczbie kolumn
            wiersz.append(0) # dodajemy zero do aktualnego wiersza
        macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy

    return macierz # zwracamy macierz zerową

def macierz_odwrotna_Laplace(macierz): # punkt a # funkcja liczy macierz odwrotną
    metodą dopełnień algebraicznych
    n = len(macierz) # zapisujemy rozmiar macierzy
    d = wyznacznik_macierzy(macierz) # liczymy wyznacznik macierzy

    if d == 0: # sprawdzamy, czy wyznacznik jest równy zero
        raise ValueError("Macierz jest osobliwa, nie ma odwrotności") # jeśli
wyznacznik jest zerowy, macierz nie ma odwrotności

    C = zeros(n, n) # tworzymy macierz dopełnień algebraicznych wypełnioną zerami

    for i in range(n): # przechodzimy po wierszach macierzy
        for j in range(n): # przechodzimy po kolumnach macierzy
            M = minor(macierz, i, j) # tworzymy minor przez usunięcie wiersza i
oraz kolumny j
            C[i][j] = math.pow(-1, i+j) * wyznacznik_macierzy(M) # liczymy
dopełnienie algebraiczne elementu a_ij

    Adj = transpozycja(C) # tworzymy macierz dołączoną, czyli transponujemy
macierz dopełnień algebraicznych

    wynik = zeros(n, n) # tworzymy pustą macierz wynikową wypełnioną zerami
    for i in range(n): # przechodzimy po wierszach macierzy wynikowej
        for j in range(n): # przechodzimy po kolumnach macierzy wynikowej
            wynik[i][j] = Adj[i][j] / d # dzielimy każdy element macierzy
dołączonej przez wyznacznik macierzy

```

```

    return wynik # zwracamy macierz odwrotną

macierz = [ # tworzymy macierz, dla której będziemy liczyć macierz odwrotną
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

wynik_Laplace = macierz_odwrotna_Laplace(macierz) # liczymy macierz odwrotną
metodą Laplace'a i zapisujemy wynik

print("Macierz odwrotna Laplace:") # wypisujemy opis wyniku
for wiersz in wynik_Laplace: # przechodzimy po kolejnych wierszach macierzy
    odwrotnej
    print(wiersz) # wypisujemy aktualny wiersz macierzy odwrotnej

```

Wynik

Dla macierzy:

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 0 & 2 & -1 \\ -3 & 3 & 3 \end{bmatrix}$$

otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909091 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Zadanie 3b — macierz odwrotna metodą Gaussa-Jordana

Idea metody

Tworzy się macierz rozszerzoną:

$$[A \mid I]$$

a następnie wykonuje się operacje na wierszach tak, aby lewa część stała się macierzą jednostkową:

$$[I \mid A^{-1}]$$

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz macierz rozszerzoną:

- po lewej masz macierz A ,

- po prawej macierz jednostkową I .

Krok 2

Dla każdej kolumny ustawiasz jedynkę na przekątnej.

Krok 3

Wyzerowujesz pozostałe elementy w tej kolumnie.

Krok 4

Po zakończeniu prawa część jest macierzą odwrotną.

```
import math # importujemy moduł math, chociaż w tym kodzie nie jest bezpośrednio
używany

def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b # funkcja liczy macierz
odwrotną metodą Gaussa-Jordana
    n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Macierz musi być kwadratowa") # zgłaszamy błąd,
jeśli macierz nie jest kwadratowa

    # macierz rozszerzona [A | I] # będziemy tworzyć macierz złożoną z macierzy A
oraz macierzy jednostkowej
    rozszerzona_macierz = [] # tworzymy pustą listę na macierz rozszerzoną

    for i in range(n): # przechodzimy po kolejnych wierszach macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy rozszerzonej

        # lewa strona: macierz A # najpierw wpisujemy elementy oryginalnej
macierzy
        for j in range(n): # przechodzimy po kolumnach macierzy A
            wiersz.append(macierz[i][j]) # dodajemy element z macierzy A do
aktualnego wiersza
            # wiersz.append(float(macierz[i][j])) # alternatywnie można byłoby od
razu zamienić elementy na liczby zmiennoprzecinkowe

        # prawa strona: macierz jednostkowa I # potem dopisujemy macierz
jednostkową po prawej stronie
        for j in range(n): # przechodzimy po kolumnach macierzy jednostkowej
            if i == j: # sprawdzamy, czy element leży na przekątnej głównej
                wiersz.append(1) # jeśli tak, wpisujemy 1
            else: # jeśli element nie leży na przekątnej głównej
                wiersz.append(0) # wpisujemy 0

        rozszerzona_macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
rozszerzonej

    # algorytm Gaussa-Jordana # zaczynamy przekształcanie macierzy rozszerzonej
    for i in range(n): # przechodzimy po kolejnych kolumnach głównych
        # jeśli na przekątnej jest 0, zamień wiersze # element główny nie może być
zerem
        if rozszerzona_macierz[i][i] == 0: # sprawdzamy, czy element główny jest
równy zero
            znaleziono = False # zakładamy, że jeszcze nie znaleziono wiersza do
zamiany
            for k in range(i+1, n): # szukamy niżej wiersza, który ma niezerowy
element w tej samej kolumnie
                if rozszerzona_macierz[k][i] != 0: # sprawdzamy, czy dany wiersz
```

```

ma niezerowy element
        rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i] # zamieniamy aktualny wiersz z
wybrany wierszem
        znaleziono = True # zapisujemy, że udało się znaleźć wiersz do
zamiany
        break # kończymy szukanie, bo zamiana została wykonana
    if not znaleziono: # sprawdzamy, czy nie udało się znaleźć
odpowiedniego wiersza
        raise ValueError("Macierz nie ma odwrotności") # jeśli nie ma
wiersza do zamiany, macierz nie ma odwrotności

    # dzielenie całego wiersza przez element główny # normalizujemy wiersz,
żeby na przekątnej otrzymać 1
    element_glowny = rozszerzona_macierz[i][i] # zapisujemy element główny z
przekątnej
    for j in range(2 * n): # przechodzimy po wszystkich kolumnach macierzy
rozszerzonej
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny
# dzielimy każdy element wiersza przez element główny

    # zerowanie pozostałych elementów w tej kolumnie # robimy zera nad i pod
elementem głównym
    for k in range(n): # przechodzimy po wszystkich wierszach
        if k != i: # pomijamy aktualny wiersz główny
            wspolczynnik = rozszerzona_macierz[k][i] # zapisujemy liczbę,
którą trzeba wyzerować
            for j in range(2 * n): # przechodzimy po wszystkich kolumnach
macierzy rozszerzonej
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j] # odejmujemy odpowiednią wielokrotność
wiersza głównego

    odwrotna = [] # tworzymy pustą listę na macierz odwrotną
    for i in range(n): # przechodzimy po wierszach macierzy rozszerzonej
        wiersz = [] # tworzymy pusty wiersz macierzy odwrotnej
        for j in range(n, 2 * n): # przechodzimy tylko po prawej części macierzy
rozszerzonej
            wiersz.append(rozszerzona_macierz[i][j]) # dodajemy element z prawej
strony, czyli z macierzy odwrotnej
        odwrotna.append(wiersz) # dodajemy gotowy wiersz do macierzy odwrotnej

    return odwrotna # zwracamy obliczoną macierz odwrotną

macierz = [ # tworzymy macierz, dla której będziemy liczyć macierz odwrotną
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

wynik_Gauss_Jordan = macierz_odwrotna_Gaussa_Jordana(macierz) # wywołujemy
funkcję i zapisujemy wynik

```

```
print("Macierz odwrotna Gauss Jordan:") # wypisujemy opis wyniku
for wiersz in wynik_Gauss_Jordan: # przechodzimy po kolejnych wierszach macierzy
    print(wiersz) # wypisujemy aktualny wiersz macierzy odwrotnej
```

Wynik

Otrzymujemy macierz odwrotną:

$$A^{-1} = \begin{bmatrix} 0.13636363636363635 & 0.09090909090909083 & -0.24242424242424243 \\ 0.045454545454545456 & 0.36363636363636365 & 0.030303030303030304 \\ 0.09090909090909091 & -0.2727272727272727 & 0.06060606060606061 \end{bmatrix}$$

Porównanie wyników obu metod

Obie metody dają tę samą macierz odwrotną.

Mogą pojawić się bardzo małe różnice w zapisie dziesiętnym, ale wynik matematycznie jest ten sam.

Zadanie 4

Napisz funkcję, która wykona mnożenie dwóch macierzy.

Kiedy można mnożyć macierze?

Macierze można mnożyć tylko wtedy, gdy:

- liczba kolumn pierwszej macierzy
- jest równa liczbie wierszy drugiej macierzy.

Jeśli:

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times k}$$

to wynik ma rozmiar:

$$AB \in \mathbb{R}^{m \times k}$$

Jak rozumieć schemat implementacji?

Krok 1

Sprawdzasz zgodność wymiarów.

Krok 2

Dla każdego elementu wyniku liczysz sumę iloczynów elementów odpowiedniego wiersza i kolumny.

Czyli:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

```
def mnozenie_macierzy(macierz1, macierz2): # funkcja mnoży dwie macierze
    ilosc_wierszy_macierz1 = len(macierz1) # zapisujemy liczbę wierszy pierwszej
macierzy
    ilosc_wierszy_macierz2 = len(macierz2) # zapisujemy liczbę wierszy drugiej
macierzy
    ilosc_kolumn_macierz1 = len(macierz1[0]) # zapisujemy liczbę kolumn pierwszej
macierzy
    ilosc_kolumn_macierz2 = len(macierz2[0]) # zapisujemy liczbę kolumn drugiej
macierzy

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2: # sprawdzamy, czy można
pomnożyć macierze
        raise ValueError("Nie da się pomnożyć tych macierzy") # zgłaszamy błąd,
jeśli liczba kolumn pierwszej macierzy nie jest równa liczbie wierszy drugiej
macierzy

    wynik = [] # tworzymy pustą listę na macierz wynikową
    for i in range(ilosc_wierszy_macierz1): # przechodzimy po wierszach pierwszej
macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy wynikowej
        for j in range(ilosc_kolumn_macierz2): # przechodzimy po kolumnach
drugiej macierzy
            suma = 0 # ustawiamy początkową sumę na 0
            for k in range(ilosc_kolumn_macierz1): # przechodzimy po elementach,
które trzeba przez siebie pomnożyć i dodać
                suma += macierz1[i][k] * macierz2[k][j] # dodajemy iloczyn
odpowiednich elementów do sumy
            wiersz.append(suma) # dodajemy obliczony element do wiersza macierzy
wynikowej
        wynik.append(wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy wynik mnożenia macierzy

macierz1 = [ # tworzymy pierwszą macierz
    [1, 2], # pierwszy wiersz pierwszej macierzy
    [3, 4] # drugi wiersz pierwszej macierzy
] # koniec definicji pierwszej macierzy

macierz2 = [ # tworzymy drugą macierz
    [5, 6], # pierwszy wiersz drugiej macierzy
    [7, 8] # drugi wiersz drugiej macierzy
] # koniec definicji drugiej macierzy

wynik = mnozenie_macierzy(macierz1, macierz2) # wywołujemy funkcję mnożenia
macierzy i zapisujemy wynik

print("Wynik mnożenia:") # wypisujemy opis wyniku
for wiersz in wynik: # przechodzimy po kolejnych wierszach macierzy wynikowej
    print(wiersz) # wypisujemy aktualny wiersz macierzy wynikowej
```

Wynik

Dla:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

otrzymujemy:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Zadanie 5

Korzystając z rozwiązań poprzednich zadań wykonaj następujące mnożenia macierzowe: $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$ i porównaj ich wyniki.

Idea zadania

Jeśli macierz (A) jest odwracalna, to powinno zachodzić:

$$A \cdot A^{-1} = I$$

oraz

$$A^{-1} \cdot A = I$$

gdzie I to macierz jednostkowa.

Czyli zadanie polega na sprawdzeniu, czy wyznaczona wcześniej macierz odwrotna rzeczywiście jest poprawna.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz macierz odwrotną.

Krok 2

Mnożysz:

- $A \cdot A^{-1}$
- $A^{-1} \cdot A$

Krok 3

Porównujesz wyniki z macierzą jednostkową.

Jeśli pojawiają się małe liczby typu:

$$2.220446049250313 \cdot 10^{-16}$$

to traktuje się je jako 0, ponieważ są to błędy zaokrągleń.

```
def macierz_odwrotna_Gaussa_Jordana(macierz): # punkt b # funkcja liczy macierz
odwrotną metodą Gaussa-Jordana
    n = len(macierz) # zapisujemy liczbę wierszy macierzy

    for wiersz in macierz: # przechodzimy po każdym wierszu macierzy
        if len(wiersz) != n: # sprawdzamy, czy liczba kolumn jest równa liczbie
wierszy
            raise ValueError("Macierz musi być kwadratowa") # zgłaszamy błąd,
jeśli macierz nie jest kwadratowa

    # macierz rozszerzona [A | I] # tworzymy macierz rozszerzoną, czyli po lewej
A, a po prawej macierz jednostkową
    rozszerzona_macierz = [] # tworzymy pustą listę na macierz rozszerzoną

    for i in range(n): # przechodzimy po kolejnych wierszach macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy rozszerzonej

        # lewa strona: macierz A # najpierw do wiersza wpisujemy elementy macierzy
A
        for j in range(n): # przechodzimy po kolumnach macierzy A
            wiersz.append(macierz[i][j]) # dodajemy element z macierzy A do
aktualnego wiersza
            # wiersz.append(float(macierz[i][j])) # alternatywna wersja, gdybyśmy
chcieli od razu zamienić liczby na float

        # prawa strona: macierz jednostkowa I # po prawej stronie dopisujemy
macierz jednostkową
        for j in range(n): # przechodzimy po kolumnach macierzy jednostkowej
            if i == j: # sprawdzamy, czy element znajduje się na przekątnej
głównej
                wiersz.append(1) # na przekątnej głównej wpisujemy 1
            else: # jeśli element nie jest na przekątnej głównej
                wiersz.append(0) # poza przekątną wpisujemy 0

        rozszerzona_macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
rozszerzonej

    # algorytm Gaussa-Jordana # zaczynamy przekształcanie macierzy rozszerzonej
    for i in range(n): # przechodzimy po kolejnych elementach głównych na
przekątnej
        # jeśli na przekątnej jest 0, zamień wiersze # element główny nie może być
zerem
        if rozszerzona_macierz[i][i] == 0: # sprawdzamy, czy element główny jest
równy 0
            znaleziono = False # zakładamy, że jeszcze nie znaleziono wiersza do
zamiany
            for k in range(i+1, n): # szukamy niżej wiersza z niezerowym
elementem w tej samej kolumnie
                if rozszerzona_macierz[k][i] != 0: # sprawdzamy, czy w tym
wierszu element w kolumnie i nie jest zerem
```

```

        rozszerzona_macierz[i], rozszerzona_macierz[k] =
rozszerzona_macierz[k], rozszerzona_macierz[i] # zamieniamy miejscami dwa wiersze
        znaleziono = True # zapisujemy, że znaleziono odpowiedni
wiersz

        break # przerywamy pętlę, bo zamiana została wykonana
    if not znaleziono: # sprawdzamy, czy nie udało się znaleźć wiersza do
zamiany

        raise ValueError("Macierz nie ma odwrotności") # jeśli nie ma
takiego wiersza, macierz nie ma odwrotności

    # dzielenie całego wiersza przez element główny # normalizujemy wiersz,
    żeby element główny stał się równy 1
    element_glowny = rozszerzona_macierz[i][i] # zapisujemy aktualny element
główny
    for j in range(2 * n): # przechodzimy po wszystkich kolumnach macierzy
rozszerzonej
        rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny
# dzielimy każdy element wiersza przez element główny

    # zerowanie pozostałych elementów w tej kolumnie # zerujemy elementy nad i
pod elementem głównym
    for k in range(n): # przechodzimy po wszystkich wierszach
        if k != i: # nie zmieniamy aktualnego wiersza głównego
            współczynnik = rozszerzona_macierz[k][i] # zapisujemy
współczynnik potrzebny do wyzerowania elementu
            for j in range(2 * n): # przechodzimy po wszystkich kolumnach
macierzy rozszerzonej
                rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
współczynnik * rozszerzona_macierz[i][j] # odejmujemy odpowiednią wielokrotność
wiersza głównego

    odwrotna = [] # tworzymy pustą listę na macierz odwrotną
    for i in range(n): # przechodzimy po wierszach macierzy rozszerzonej
        wiersz = [] # tworzymy pusty wiersz macierzy odwrotnej
        for j in range(n, 2 * n): # przechodzimy po prawej części macierzy
rozszerzonej
            wiersz.append(rozszerzona_macierz[i][j]) # dodajemy element z prawej
strony, czyli element macierzy odwrotnej
        odwrotna.append(wiersz) # dodajemy gotowy wiersz do macierzy odwrotnej

    return odwrotna # zwracamy macierz odwrotną

def mnozenie_macierzy(macierz1, macierz2): # funkcja mnoży dwie macierze
    ilosc_wierszy_macierz1 = len(macierz1) # zapisujemy liczbę wierszy pierwszej
macierzy
    ilosc_wierszy_macierz2 = len(macierz2) # zapisujemy liczbę wierszy drugiej
macierzy
    ilosc_kolumn_macierz1 = len(macierz1[0]) # zapisujemy liczbę kolumn pierwszej
macierzy
    ilosc_kolumn_macierz2 = len(macierz2[0]) # zapisujemy liczbę kolumn drugiej
macierzy

```

```

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2: # sprawdzamy warunek
mnożenia macierzy
        raise ValueError("Nie da się pomnożyć tych macierzy") # zgłaszamy błąd,
jeśli liczba kolumn pierwszej macierzy nie jest równa liczbie wierszy drugiej
macierzy

    wynik = [] # tworzymy pustą listę na macierz wynikową
    for i in range(ilosc_wierszy_macierz1): # przechodzimy po wierszach pierwszej
macierzy
        wiersz = [] # tworzymy pusty wiersz macierzy wynikowej
        for j in range(ilosc_kolumn_macierz2): # przechodzimy po kolumnach
drugiej macierzy
            suma = 0 # ustawiamy początkową sumę na 0
            for k in range(ilosc_kolumn_macierz1): # przechodzimy po elementach
potrzebnych do obliczenia jednego elementu wyniku
                suma += macierz1[i][k] * macierz2[k][j] # mnożymy odpowiednie
elementy i dodajemy je do sumy
            wiersz.append(suma) # dodajemy obliczony element do aktualnego wiersza
            wynik.append(wiersz) # dodajemy gotowy wiersz do macierzy wynikowej

    return wynik # zwracamy wynik mnożenia macierzy

macierz = [ # tworzymy macierz A
    [2, 4, 6], # pierwszy wiersz macierzy
    [0, 2, -1], # drugi wiersz macierzy
    [-3, 3, 3] # trzeci wiersz macierzy
] # koniec definicji macierzy

macierz_odwrotna = macierz_odwrotna_Gaussa_Jordana(macierz) # liczymy macierz
odwrotną do macierzy A

wynik1 = mnozenie_macierzy(macierz, macierz_odwrotna) # mnożymy A przez A^-1
wynik2 = mnozenie_macierzy(macierz_odwrotna, macierz) # mnożymy A^-1 przez A

print("Wynik mnożenia A * A^-1:") # wypisujemy opis pierwszego wyniku
for wiersz in wynik1: # przechodzimy po wierszach wyniku A * A^-1
    print(wiersz) # wypisujemy aktualny wiersz wyniku

print("Wynik mnożenia A^-1 * A:") # wypisujemy opis drugiego wyniku
for wiersz in wynik2: # przechodzimy po wierszach wyniku A^-1 * A
    print(wiersz) # wypisujemy aktualny wiersz wyniku

```

Wynik

Oba iloczyny powinny być bardzo bliskie macierzy jednostkowej:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

W praktyce może się pojawić na przykład:

zamiast dokładnego zera. To jest normalne i wynika z ograniczonej dokładności obliczeń zmiennoprzecinkowych.

Wniosek

Wszystkie zadania zostały wykonane poprawnie.

- W zadaniu 1 obliczono wyznacznik macierzy metodą rozwinięcia Laplace'a.
- W zadaniu 2 obliczono transpozycję macierzy.
- W zadaniu 3 wyznaczono macierz odwrotną dwiema metodami: Laplace'a oraz Gaussa-Jordana.
- W zadaniu 4 wykonano mnożenie dwóch macierzy.
- W zadaniu 5 sprawdzono poprawność macierzy odwrotnej przez obliczenie iloczynów $A \cdot A^{-1}$ oraz $A^{-1} \cdot A$.

Otrzymane wyniki są zgodne z teorią, a ewentualne bardzo małe różnice wynikają z błędów zaokrągleń numerycznych.

Lab 5

Zadanie 1

Korzystając z funkcji napisanych na poprzednich zajęciach rozwiąż układy równań liniowych.

Zadanie 1a

$$\begin{cases} x + 2y + z = -9, \\ 3x - 7y + 2z = 61, \\ 2x + 4y + 5z = -9 \end{cases}$$

Jak to rozwiązać metodą z poprzednich zajęć?

Skoro masz już funkcję liczącą macierz odwrotną, to korzystasz ze wzoru:

$$x = A^{-1}b$$

Czyli:

1. liczysz macierz odwrotną A^{-1} ,
2. mnożysz ją przez wektor b ,

3. dostajesz rozwiązanie.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz macierz **A** i wektor **b**.

Krok 2

Zamieniasz **b** na macierz kolumnową:

$$b = \begin{bmatrix} -9 \\ 61 \\ -9 \end{bmatrix}$$

bo funkcja mnożenia działa na macierzach.

Krok 3

Liczysz:

$$A^{-1} \cdot b$$

Krok 4

Otrzymujesz wynik w postaci macierzy kolumnowej, więc zamieniasz go na zwykły wektor.

```
import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

def macierz_odwrotna_Gaussa_Jordana(macierz):
    n = len(macierz)

    for wiersz in macierz:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    rozszerzona_macierz = []

    for i in range(n):
        wiersz = []

        for j in range(n):
            wiersz.append(macierz[i][j])

        for j in range(n):
            if i == j:
                wiersz.append(1)
            else:
                wiersz.append(0)

        rozszerzona_macierz.append(wiersz)

    for i in range(n):
        if rozszerzona_macierz[i][i] == 0:
            znaleziono = False
            for k in range(i+1, n):
                if rozszerzona_macierz[k][i] != 0:
                    rozszerzona_macierz[i], rozszerzona_macierz[k] =
                    rozszerzona_macierz[k], rozszerzona_macierz[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Macierz nie ma odwrotności")

        element_glowny = rozszerzona_macierz[i][i]
        for j in range(2 * n):
            rozszerzona_macierz[i][j] = rozszerzona_macierz[i][j] / element_glowny

    for k in range(n):
        if k != i:
            wspolczynnik = rozszerzona_macierz[k][i]
```



```

        for j in range(2 * n):
            rozszerzona_macierz[k][j] = rozszerzona_macierz[k][j] -
wspolczynnik * rozszerzona_macierz[i][j]

odwrotna = []
for i in range(n):
    wiersz = []
    for j in range(n, 2 * n):
        wiersz.append(rozszerzona_macierz[i][j])
    odwrotna.append(wiersz)

return odwrotna

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

def rozwarz_uklad_rownan(macierz_A, wektor_b):
    b_macierz = [[b] for b in wektor_b]

    A_odwrotna = macierz_odwrotna_Gaussa_Jordana(macierz_A)

    wynik = mnozenie_macierzy(A_odwrotna, b_macierz)

    return wynik

A = [
    [1, 2, 1],
    [3, -7, 2],
    [2, 4, 5]
]

b = [-9, 61, -9]

wynik, czas = zmierz_czas(rozwarz_uklad_rownan, A, b)

```

```
print("Rozwiązanie układu równań a:")
print("x =", wynik[0][0])
print("y =", wynik[1][0])
print("z =", wynik[2][0])
print("Czas: ", czas)
```

Wynik

$$x = 2.0, \quad y = -7.0, \quad z = 3.0$$

$$czas = 1.4600111171603203e - 05 = 1.4600111171603203 * 10^{-5} = 0.0000146s = 14.6 \text{mikrosekund}$$

Zadanie 1b

$$A = \begin{bmatrix} 11 & -5 & & & 0 \\ -5 & 11 & -5 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & \ddots & -5 \\ 0 & & & -5 & 11 \end{bmatrix} \quad b = \begin{bmatrix} 11 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Rozwiązać układ:

$$Ax = b$$

gdzie:

- $(A \in \mathbb{R}^{n \times n})$,
- $(b \in \mathbb{R}^{n \times 1})$,
- $(n \in \{8, 10\})$,

a macierz ma postać trójdagonalną:

- na przekątnej są (11),
- nad i pod przekątną są (-5),
- reszta elementów to (0).

Wektor:

$$b = \begin{bmatrix} 11 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Jak rozumieć tę macierz?

Dla (n=8) macierz wygląda tak:

$$A = \begin{bmatrix} 11 & -5 & 0 & 0 & 0 & 0 & 0 & 0 \\ -5 & 11 & -5 & 0 & 0 & 0 & 0 & 0 \\ 0 & -5 & 11 & -5 & 0 & 0 & 0 & 0 \\ 0 & 0 & -5 & 11 & -5 & 0 & 0 & 0 \\ 0 & 0 & 0 & -5 & 11 & -5 & 0 & 0 \\ 0 & 0 & 0 & 0 & -5 & 11 & -5 & 0 \\ 0 & 0 & 0 & 0 & 0 & -5 & 11 & -5 \\ 0 & 0 & 0 & 0 & 0 & 0 & -5 & 11 \end{bmatrix}$$

To znaczy:

- główna przekątna ma same **11**,
- sąsiednie przekątne mają **-5**.

Jak zrobić implementację?

Najpierw trzeba umieć wygenerować taką macierz automatycznie.

Kod tworzący macierz i wektor

PYTHON

```
def zeros(n,m):
    macierz = []

    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0)
        macierz.append(wiersz)

    return macierz

def tworzenie_macierzy(n):
    A = zeros(n, n)
    b = [11] + [0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11
            elif abs(i - j) == 1:
                A[i][j] = -5

    return A, b
```

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz pustą macierz `n x n`.

Krok 2

Jeśli jesteś na przekątnej (`i == j`), wpisujesz `11`.

Krok 3

Jeśli jesteś tuż obok przekątnej (`abs(i-j) == 1`), wpisujesz `-5`.

Krok 4

Wektor `b` ma `11` na pierwszej pozycji i same zera dalej.

Rozwiązanie

Potem rozwiązujesz dokładnie tak samo jak w 1a:

$$x = A^{-1} * b$$

Kod

PYTHON

```
n_8_A, n_8_b = tworzenie_macierzy(8)
n_10_A, n_10_b = tworzenie_macierzy(10)

wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, n_8_A, n_8_b)
print("Rozwiązanie układu równań b) n=8:")
for i in range(len(wynik)):
    print(wynik[i])
print("Czas: ", czas)

wynik, czas = zmierz_czas(rozwiazar_uklad_rownan, n_10_A, n_10_b)
print("Rozwiązanie układu równań b) n=10:")
for i in range(len(wynik)):
    print(wynik[i])
print("Czas: ", czas)
```

Wynik

Rozwiązanie układu równań b) $n = 8$:

[1.4111459559532078]

[0.9045211030970575]

[0.5788004708603187]

[0.36883993279564314]

[0.23264738129009646]

[0.1429843060425689]

[0.0819180920035551]

[0.037235496365252314]

Czas : 6.560003384947777e - 05

Rozwiązanie układu równań b) $n = 10$:

[1.4117167939551722]

[0.905776946701379]

[0.5809924887878618]

[0.3724065286319164]

[0.23830187420235457]

[0.15185759461326348]

[0.09578483394682505]

[0.05886904006975162]

[0.03372705420662853]

[0.015330479184831151]

Czas : 0.00011690007522702217

Zadanie 1c

Rozwiązać układ równań, którego współczynniki tworzą macierz gęstą (10×10).

Co to znaczy macierz gęsta?

Macierz gęsta to taka, w której dużo elementów jest różnych od zera.

To przeciwieństwo macierzy rzadkiej lub trójdzielnej.

Jak to zrobić sensownie?

Najlepiej zbudować macierz, dla której znasz rozwiązanie.

Wtedy łatwo sprawdzić, czy program działa dobrze.

Pomysł

1. wybierasz macierz gęstą (A),
2. wybierasz znany wektor (x),
3. liczysz:

$$b = Ax$$

4. potem rozwiązujesz układ ($Ax = b$),
5. i sprawdzasz, czy odzyskałeś swoje (x).

Kod

PYTHON

```
def tworzenie_macierzy_gestej(n, m):
    A = zeros(n, m)

    for i in range(n):
        for j in range(m):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

    x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

    x_macierz = [[x] for x in x_prawdziwe]
    b_macierz = mnozenie_macierzy(A, x_macierz)

    b = []
    for i in range(len(b_macierz)):
        b.append(b_macierz[i][0])

    return A, b, x_prawdziwe

macierz, wektor, x = tworzenie_macierzy_gestej(10, 10)

wynik, czas = zmierz_czas(rozwarz_uklad_rownan, macierz, wektor)

print("Rozwiązanie układu równań c:")
for i in range(len(wynik)):
    print(wynik[i])
print("Czas: ", czas)
```

Jak rozumieć ten schemat?

Nie zgadujesz **b**.

Tworzysz takie **b**, żeby mieć pewność, że rozwiązanie jest znane.

To bardzo wygodne przy testowaniu metod numerycznych.

Wynik

[1.0]
[1.99999999999999716]
[3.0]
[4.0]
[5.0000000000000057]
[5.9999999999999972]
[7.0]
[8.0000000000000028]
[8.9999999999999972]
[10.0]

Czas : 9.989994578063488e - 05

Otrzymane rozwiązanie jest bardzo bliskie wektorowi $x = [1,2,3,4,5,6,7,8,9,10]$, co potwierdza poprawność działania programu. Niewielkie różnice wynikają z błędów zaokrągleń.

Zadanie 2

Napisz funkcję znajdującą rozkład macierzy na iloczyn macierzy trójkątnych

$$A = LU$$

Powyższą funkcję wykorzystaj w celu znalezienia rozwiązania układu równań liniowych. Przetestuj działanie dla przykładów z zadania 1. Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

Co to jest rozkład LU?

Macierz (A) rozkłada się na:

- (L) — macierz dolnotrójkątną,
- (U) — macierz górnortrójkątną.

Czyli:

$$A = LU$$

Po co to robić?

Bo zamiast rozwiązywać od razu:

$$Ax = b$$

rozbijasz to na dwa prostsze układy:

$$Ly = b$$

a potem:

$$Ux = y$$

Jak rozumieć schemat implementacji?

Etap 1 — wyznaczenie **L** i **U**

Budujesz dwie macierze:

- **L** ma jedynki na przekątnej,
- **U** powstaje z odpowiednich wzorów.

Etap 2 — podstawianie w przód

Rozwiązujesz $Ly = b$.

Ponieważ **L** jest dolnotrójkątna, liczysz:

- najpierw y_1 ,
- potem y_2 ,
- itd.

Etap 3 — podstawianie w tył

Rozwiązujesz $Ux = y$.

Ponieważ **U** jest górnotrójkątna, liczysz:

- najpierw ostatnią niewiadomą,
- potem poprzednią,
- itd.


```
import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def rozklad_LU_Doolittle(A):
    n = len(A)

    for wiersz in A:
        if len(wiersz) != n:
            raise ValueError("Macierz musi być kwadratowa")

    L = zeros(n, n)
    U = zeros(n, n)

    # na przekątnej L są jedynki
    for i in range(n):
        L[i][i] = 1.0

    for i in range(n):
        # liczenie elementów U
        for j in range(i, n):
            suma = 0.0
            for k in range(i):
                suma += L[i][k] * U[k][j]
            U[i][j] = A[i][j] - suma

        # liczenie elementów L
        for j in range(i + 1, n):
            suma = 0.0
            for k in range(i):
                suma += L[j][k] * U[k][i]

            if U[i][i] == 0:
                raise ValueError("Nie można wykonać rozkładu LU metodą Doolittle'a")
```

```
L[j][i] = (A[j][i] - suma) / U[i][i]
```

```
return L, U
```

Rozwiązywanie układu przez LU

PYTHON

```
def podstawianie_w_przod(L, b):  
    n = len(L)  
    y = [0.0] * n  
  
    for i in range(n):  
        suma = 0.0  
        for j in range(i):  
            suma += L[i][j] * y[j]  
        y[i] = b[i] - suma  
  
    return y
```

```
def podstawianie_w tyl(U, y):  
    n = len(U)  
    x = [0.0] * n  
  
    for i in range(n - 1, -1, -1):  
        suma = 0.0  
        for j in range(i + 1, n):  
            suma += U[i][j] * x[j]  
  
        if U[i][i] == 0:  
            raise ValueError("Dzielenie przez zero w podstawianiu w tyl")  
  
        x[i] = (y[i] - suma) / U[i][i]  
  
    return x
```

```
def rozwarz_uklad_Doolittle(A, b):  
    L, U = rozklad_LU_Doolittle(A)  
    y = podstawianie_w_przod(L, b)  
    x = podstawianie_w tyl(U, y)  
    return x
```

punkt a

PYTHON

```
A = [  
    [1.0, 2.0, 1.0],  
    [3.0, -7.0, 2.0],  
    [2.0, 4.0, 5.0]  
]  
  
b = [-9.0, 61.0, -9.0]  
  
wynik, czas = zmierz_czas(rozwiaz_uklad_Doolittle, A, b)  
  
print("Rozwiązanie metodą Doolittle'a dla a:")  
for i in range(len(wynik)):  
    print("x" + str(i + 1) + " =", wynik[i])  
print("Czas: ", czas)
```

punkt b

PYTHON

```
def wypisz_wektor(wektor):
    for i in range(len(wektor)):
        print("x" + str(i + 1) + " =", wektor[i])

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0
            elif abs(i - j) == 1:
                A[i][j] = -5.0
            else:
                A[i][j] = 0.0

    return A, b

#punkt b, n = 8

A8, b8 = tworzenie_macierzy_b(8)

wynik8, czas8 = zmierz_czas(rozwiaz_uklad_Doolittle, A8, b8)

print("\nRozwiązanie metodą Doolittle'a dla b), n=8:")
wypisz_wektor(wynik8)
print("Czas:", czas8, "s")

#punkt b, n = 10

A10, b10 = tworzenie_macierzy_b(10)

wynik10, czas10 = zmierz_czas(rozwiaz_uklad_Doolittle, A10, b10)

print("\nRozwiązanie metodą Doolittle'a dla b), n=10:")
wypisz_wektor(wynik10)
print("Czas:", czas10, "s")
```

```
def wektor_na_macierz_kolumnowa(wektor):
    wynik = []
    for x in wektor:
        wynik.append([float(x)])
    return wynik

def macierz_kolumnowa_na_wektor(macierz):
    wynik = []
    for i in range(len(macierz)):
        wynik.append(macierz[i][0])
    return wynik

def mnozenie_macierzy(macierz1, macierz2):
    ilosc_wierszy_macierz1 = len(macierz1)
    ilosc_wierszy_macierz2 = len(macierz2)
    ilosc_kolumn_macierz1 = len(macierz1[0])
    ilosc_kolumn_macierz2 = len(macierz2[0])

    if ilosc_kolumn_macierz1 != ilosc_wierszy_macierz2:
        raise ValueError("Nie da się pomnożyć tych macierzy")

    wynik = []
    for i in range(ilosc_wierszy_macierz1):
        wiersz = []
        for j in range(ilosc_kolumn_macierz2):
            suma = 0.0
            for k in range(ilosc_kolumn_macierz1):
                suma += macierz1[i][k] * macierz2[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)

    return wynik

def tworzenie_macierzy_gestej(n, m):
    A = zeros(n, m)

    for i in range(n):
        for j in range(m):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

x_macierz = wektor_na_macierz_kolumnowa(x_prawdziwe)
b_macierz = mnozenie_macierzy(A, x_macierz)
b = macierz_kolumnowa_na_wektor(b_macierz)
```

```
    return A, b, x_prawdziwe

Ag, bg, x_prawdziwe = tworzenie_macierzy_gestej(10, 10)

wynikg, czasg = zmierz_czas(rozwiaz_uklad_Doolittle, Ag, bg)

print("\nRozwiązanie metodą Doolittle'a dla c:")
wypisz_wektor(wynikg)
print("Czas:", czasg, "s")
print("Oczekiwane rozwiązanie:", x_prawdziwe)
```

Rozwiązanie metodą Doolittle'a dla a):

x1 = 2.0

x2 = -7.0

x3 = 3.0

Czas: 1.3699987903237343e-05

Rozwiązanie metodą Doolittle'a dla b), n=8:

x1 = 1.411145955953208

x2 = 0.9045211030970576

x3 = 0.5788004708603186

x4 = 0.36883993279564314

x5 = 0.23264738129009638

x6 = 0.14298430604256884

x7 = 0.08191809200355507

x8 = 0.03723549636525231

Czas: 2.500019036233425e-05 s

Rozwiązanie metodą Doolittle'a dla b), n=10:

x1 = 1.4117167939551722

x2 = 0.9057769467013792

x3 = 0.5809924887878617

x4 = 0.3724065286319165

x5 = 0.23830187420235452

x6 = 0.1518575946132634

x7 = 0.09578483394682502

x8 = 0.05886904006975161

x9 = 0.033727054206628526

x10 = 0.015330479184831148

Czas: 3.5800039768218994e-05 s

Rozwiązanie metodą Doolittle'a dla c):

x1 = 1.0000000000000455

x2 = 2.000000000000047

x3 = 3.0000000000000417

x4 = 4.0000000000000435

x5 = 5.000000000000042

x6 = 6.000000000000046

x7 = 7.000000000000037

x8 = 8.000000000000005

x9 = 8.999999999999991

x10 = 9.999999999999762

Czas: 3.4300144761800766e-05 s

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

Zadanie 3

Napisz funkcję znajdującą rozkład Choleskiego dla macierzy:

- kwadratowej,
- symetrycznej,
- dodatnio określonej.

Powyższą funkcję wykorzystaj w celu znalezienia rozwiązania układu równań liniowych. Przetestuj działanie dla przykładu b) z zadania 1.

Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

Co to jest rozkład Choleskiego?

Dla odpowiedniej macierzy:

$$A = LL^T$$

gdzie:

- L jest macierzą dolnotrójkątną,
- L^T jest jej transpozycją.

Kiedy można go używać?

Tylko gdy macierz jest:

- symetryczna,
- dodatnio określona.

Macierz z zadania 1b spełnia ten warunek.

Jak rozumieć schemat implementacji?

Krok 1

Liczysz kolejne elementy macierzy .

Krok 2

Na przekątnej liczysz pierwiastek z odpowiedniej wartości.

Krok 3

Poza przekątną liczysz elementy ze wzoru zależnego od już policzonych elementów.

Krok 4

Rozwiązujesz:

$$Ly = b$$

a potem:

$$L^T x = y$$

```
import time

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def transpozycja(macierz):
    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])

    wynik = []

    for j in range(liczba_kolumn):
        nowy_wiersz = []
        for i in range(liczba_wierszy):
            nowy_wiersz.append(macierz[i][j])
        wynik.append(nowy_wiersz)

    return wynik

def podstawianie_w_przod(L, b):
    n = len(L)
    y = [0.0] * n

    for i in range(n):
        suma = 0.0
        for j in range(i):
            suma += L[i][j] * y[j]
        y[i] = (b[i] - suma) / L[i][i]

    return y

def podstawianie_w tyl(U, y):
    n = len(U)
    x = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0
        for j in range(i + 1, n):
            suma += U[i][j] * x[j]
        x[i] = (y[i] - suma) / U[i][i]
```

```
return x
```

```
def rozklad_Choleskiego(A):  
    n = len(A)  
  
    for wiersz in A:  
        if len(wiersz) != n:  
            raise ValueError("Macierz musi być kwadratowa")  
  
    L = zeros(n, n)  
  
    for i in range(n):  
        for j in range(i + 1):  
            suma = 0.0  
            for k in range(j):  
                suma += L[i][k] * L[j][k]  
  
            if i == j:  
                wartosc = A[i][i] - suma  
                if wartosc <= 0:  
                    raise ValueError("Macierz nie jest dodatnio określona")  
                L[i][j] = wartosc ** 0.5  
            else:  
                L[i][j] = (A[i][j] - suma) / L[j][j]  
  
    return L
```

```
def rozwiaz_uklad_Choleski(A, b):  
    L = rozklad_Choleskiego(A)  
    Lt = transpozycja(L)  
  
    y = podstawianie_w_przod(L, b)  
    x = podstawianie_w tyl(Lt, y)  
  
    return x
```

```
def wypisz_wektor(wektor):  
    for i in range(len(wektor)):  
        print("x" + str(i + 1) + " =", wektor[i])
```

```
def tworzenie_macierzy_b(n):  
    A = zeros(n, n)  
    b = [11.0] + [0.0] * (n - 1)  
  
    for i in range(n):  
        for j in range(n):  
            if i == j:  
                A[i][j] = 11.0  
            elif abs(i - j) == 1:
```

```

        A[i][j] = -5.0
    else:
        A[i][j] = 0.0

    return A, b

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

# przykład b) z zadania 1, n = 8
A8, b8 = tworzenie_macierzy_b(8)
wynik8, czas8 = zmierz_czas(rozwiaz_uklad_Choleski, A8, b8)

print("Rozwiązanie metodą Cholesky'ego dla n = 8:")
wypisz_wektor(wynik8)
print("Czas:", czas8)

# przykład b) z zadania 1, n = 10
A10, b10 = tworzenie_macierzy_b(10)
wynik10, czas10 = zmierz_czas(rozwiaz_uklad_Choleski, A10, b10)

print("\nRozwiązanie metodą Cholesky'ego dla n = 10:")
wypisz_wektor(wynik10)
print("Czas:", czas10)

```

Rozwiązanie metodą Cholesky'ego dla $n = 8$:

```
x1 = 1.4111459559532078
x2 = 0.9045211030970575
x3 = 0.5788004708603184
x4 = 0.36883993279564303
x5 = 0.2326473812900963
x6 = 0.1429843060425688
x7 = 0.08191809200355503
x8 = 0.03723549636525228
Czas: 2.5999965146183968e-05
```

Rozwiązanie metodą Cholesky'ego dla $n = 10$:

```
x1 = 1.4117167939551722
x2 = 0.905776946701379
x3 = 0.5809924887878615
x4 = 0.37240652863191626
x5 = 0.23830187420235438
x6 = 0.15185759461326334
x7 = 0.09578483394682495
x8 = 0.05886904006975156
x9 = 0.0337270542066285
x10 = 0.015330479184831134
Czas: 2.8199981898069382e-05
```

Zadanie 4

Napisz funkcję rozwiązującą układy równań za pomocą eliminacji Gaussa. Przetestuj działanie dla przykładów z zadania 1. Zmierz czas potrzebny na znalezienie każdego z rozwiązań. Porównaj otrzymane wyniki.

Na czym polega eliminacja Gaussa?

Macierz sprowadzasz do postaci trójkątnej górnej przez zerowanie elementów pod przekątną.

Potem rozwiązujesz układ podstawianiem w tył.

Jak rozumieć schemat implementacji?

Etap 1 — eliminacja w przód

Dla każdej kolumny:

- wybierasz wiersz główny,

- zerujesz elementy pod nim.

Etap 2 — podstawianie w tył

Gdy macierz jest już górnotrójkątna, liczysz niewiadome od końca.

Funkcja do Gaussa

PYTHON

```
def rozwarz_uklad_Gaussa(A, b):
    n = len(A)

    # kopia macierzy i wektora
    M = kopiuj_macierz(A)
    bb = []
    for x in b:
        bb.append(float(x))

    # eliminacja w przód
    for i in range(n):
        # jeśli pivot jest zerem, trzeba zamienić wiersze
        if M[i][i] == 0:
            znaleziono = False
            for k in range(i + 1, n):
                if M[k][i] != 0:
                    M[i], M[k] = M[k], M[i]
                    bb[i], bb[k] = bb[k], bb[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Układ nie ma jednoznacznego rozwiązania")

        # zerowanie elementów poniżej przekątnej
        for k in range(i + 1, n):
            wspolczynnik = M[k][i] / M[i][i]

            for j in range(i, n):
                M[k][j] = M[k][j] - wspolczynnik * M[i][j]

            bb[k] = bb[k] - wspolczynnik * bb[i]

    # podstawianie w tył
    x = [0.0] * n

    for i in range(n - 1, -1, -1):
        suma = 0.0
        for j in range(i + 1, n):
            suma += M[i][j] * x[j]

        x[i] = (bb[i] - suma) / M[i][i]

    return x
```

```
import time

def zeros(n, m):
    macierz = []
    for i in range(n):
        wiersz = []
        for j in range(m):
            wiersz.append(0.0)
        macierz.append(wiersz)
    return macierz

def kopiuj_macierz(macierz):
    wynik = []
    for wiersz in macierz:
        nowy_wiersz = []
        for element in wiersz:
            nowy_wiersz.append(float(element))
        wynik.append(nowy_wiersz)
    return wynik

def wypisz_wektor(wektor):
    for i in range(len(wektor)):
        print("x" + str(i + 1) + " =", wektor[i])

def rozwarz_uklad_Gaussa(A, b):
    n = len(A)

    # kopia macierzy i wektora
    M = kopiuj_macierz(A)
    bb = []
    for x in b:
        bb.append(float(x))

    # eliminacja w przód
    for i in range(n):
        # jeśli pivot jest zerem, trzeba zamienić wiersze
        if M[i][i] == 0:
            znaleziono = False
            for k in range(i + 1, n):
                if M[k][i] != 0:
                    M[i], M[k] = M[k], M[i]
                    bb[i], bb[k] = bb[k], bb[i]
                    znaleziono = True
                    break
            if not znaleziono:
                raise ValueError("Układ nie ma jednoznacznego rozwiązania")
```

```

# zerowanie elementów poniżej przekątnej
for k in range(i + 1, n):
    wspolczynnik = M[k][i] / M[i][i]

    for j in range(i, n):
        M[k][j] = M[k][j] - wspolczynnik * M[i][j]

    bb[k] = bb[k] - wspolczynnik * bb[i]

# podstawianie w tył
x = [0.0] * n

for i in range(n - 1, -1, -1):
    suma = 0.0
    for j in range(i + 1, n):
        suma += M[i][j] * x[j]

    x[i] = (bb[i] - suma) / M[i][i]

return x

```

#a)

```

A1 = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]

```

```

b1 = [-9.0, 61.0, -9.0]

```

#b)

```

def tworzenie_macierzy_b(n):
    A = zeros(n, n)
    b = [11.0] + [0.0] * (n - 1)

    for i in range(n):
        for j in range(n):
            if i == j:
                A[i][j] = 11.0
            elif abs(i - j) == 1:
                A[i][j] = -5.0
            else:
                A[i][j] = 0.0

    return A, b

```

#c)

```

def mnozenie_macierzy(macierz1, macierz2):
    liczba_wierszy_1 = len(macierz1)
    liczba_wierszy_2 = len(macierz2)

```



```

liczba_kolumn_1 = len(macierz1[0])
liczba_kolumn_2 = len(macierz2[0])

if liczba_kolumn_1 != liczba_wierszy_2:
    raise ValueError("Nie da się pomnożyć tych macierzy")

wynik = []
for i in range(liczba_wierszy_1):
    wiersz = []
    for j in range(liczba_kolumn_2):
        suma = 0.0
        for k in range(liczba_kolumn_1):
            suma += macierz1[i][k] * macierz2[k][j]
        wiersz.append(suma)
    wynik.append(wiersz)

return wynik


def wektor_na_macierz_kolumnowa(wektor):
    wynik = []
    for x in wektor:
        wynik.append([float(x)])
    return wynik


def macierz_kolumnowa_na_wektor(macierz):
    wynik = []
    for i in range(len(macierz)):
        wynik.append(macierz[i][0])
    return wynik


def tworzenie_macierzy_gestej_10():
    A = zeros(10, 10)

    for i in range(10):
        for j in range(10):
            if i == j:
                A[i][j] = 20.0
            else:
                A[i][j] = float(i + j + 1)

    x_prawdziwe = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

    x_kolumna = wektor_na_macierz_kolumnowa(x_prawdziwe)
    b_kolumna = mnozenie_macierzy(A, x_kolumna)
    b = macierz_kolumnowa_na_wektor(b_kolumna)

    return A, b, x_prawdziwe


#czasy

```

```

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start

print("===== ZADANIE 4 - eliminacja Gaussa =====")

# przykład a)
A1 = [
    [1.0, 2.0, 1.0],
    [3.0, -7.0, 2.0],
    [2.0, 4.0, 5.0]
]
b1 = [-9.0, 61.0, -9.0]

wynik_Gauss_1, czas_Gauss_1 = zmierz_czas(rozwiaz_uklad_Gaussa, A1, b1)
print("Gauss, przykład a:")
wypisz_wektor(wynik_Gauss_1)
print("Czas:", czas_Gauss_1)

# przykład b), n = 8
A8, b8 = tworzenie_macierzy_b(8)
wynik_Gauss_8, czas_Gauss_8 = zmierz_czas(rozwiaz_uklad_Gaussa, A8, b8)
print("\nGauss, przykład b), n=8:")
wypisz_wektor(wynik_Gauss_8)
print("Czas:", czas_Gauss_8)

# przykład b), n = 10
A10, b10 = tworzenie_macierzy_b(10)
wynik_Gauss_10, czas_Gauss_10 = zmierz_czas(rozwiaz_uklad_Gaussa, A10, b10)
print("\nGauss, przykład b), n=10:")
wypisz_wektor(wynik_Gauss_10)
print("Czas:", czas_Gauss_10)

# przykład c)
A_gesta, b_gesta, x_prawdziwe = tworzenie_macierzy_gestej_10()
wynik_Gauss_gesta, czas_Gauss_gesta = zmierz_czas(rozwiaz_uklad_Gaussa, A_gesta,
b_gesta)
print("\nGauss, przykład c):")
wypisz_wektor(wynik_Gauss_gesta)
print("Czas:", czas_Gauss_gesta)
print("Oczekiwane rozwiązanie:", x_prawdziwe)

```

===== ZADANIE 4 - eliminacja Gaussa =====

Gauss, przykład a):

x1 = 2.0

x2 = -7.0

x3 = 3.0

Czas: 1.2800097465515137e-05

Gauss, przykład b), n=8:

x1 = 1.411145955953208

x2 = 0.9045211030970576

x3 = 0.5788004708603186

x4 = 0.36883993279564314

x5 = 0.23264738129009638

x6 = 0.14298430604256884

x7 = 0.08191809200355507

x8 = 0.03723549636525231

Czas: 1.8999911844730377e-05

Gauss, przykład b), n=10:

x1 = 1.4117167939551722

x2 = 0.9057769467013792

x3 = 0.5809924887878617

x4 = 0.3724065286319165

x5 = 0.23830187420235452

x6 = 0.1518575946132634

x7 = 0.09578483394682502

x8 = 0.05886904006975161

x9 = 0.033727054206628526

x10 = 0.015330479184831148

Czas: 3.0899886041879654e-05

Gauss, przykład c):

x1 = 1.0000000000000058

x2 = 2.000000000000004

x3 = 2.9999999999999996

x4 = 4.0000000000000014

x5 = 5.000000000000001

x6 = 5.999999999999999

x7 = 7.000000000000005

x8 = 7.999999999999986

x9 = 9.000000000000007

x10 = 9.999999999999988

Czas: 2.7399975806474686e-05

Oczekiwane rozwiązanie: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]

W każdym zadaniu masz porównać czasy.

Jak rozumieć schemat?

1. zapisujesz czas startu,
2. uruchamiasz metodę,
3. zapisujesz czas końca,
4. odejmujesz.

Kod

PYTHON

```
import time

def zmierz_czas(funkcja, A, b):
    start = time.perf_counter()
    wynik = funkcja(A, b)
    koniec = time.perf_counter()
    return wynik, koniec - start
```

Krótki wniosek do wszystkich zadań

Możesz napisać tak:

Wnioski

Wszystkie zastosowane metody prowadzą do tego samego rozwiązania układów równań liniowych, jednak różnią się kosztami obliczeniowymi i warunkami stosowalności.

- Metoda oparta na macierzy odwrotnej jest poprawna, ale najmniej opłacalna obliczeniowo.
- Rozkład LU upraszcza rozwiązanie układu do dwóch prostszych układów trójkątnych.
- Rozkład Choleskiego jest bardzo efektywny, ale można go stosować tylko do macierzy symetrycznych i dodatnio określonych.
- Eliminacja Gaussa jest metodą uniwersalną i jedną z podstawowych metod rozwiązywania układów liniowych.
- Porównanie czasów wykonania pokazuje, które metody są bardziej praktyczne dla większych macierzy.

Lab 6

Zadanie 1

Napisz program implementujący metodę Jacobiego iteracyjnego rozwiązywania układów równań liniowych, w której warunkiem zatrzymania będzie:

- a) liczba iteracji,
- b) norma wektora powstałego przez odjęcie wektorów określających kolejne przybliżenia,
- c) błąd uzyskanego przybliżenia.

Co to jest metoda Jacobiego?

Metoda Jacobiego jest metodą iteracyjną, czyli nie liczy rozwiązania od razu, tylko buduje kolejne przybliżenia:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

Każde nowe przybliżenie oblicza się wyłącznie na podstawie poprzedniego.

Dla układu:

$$Ax = b$$

każdą niewiadomą w iteracji k obliczamy na podstawie wartości z iteracji poprzedniej, czyli $x^{(k-1)}$.

wzór metody Jacobiego ma postać:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k-1)} \right)$$

gdzie:

- a_{ii} - element na przekątnej macierzy,
- b_i - odpowiedni element wektora prawej strony,
- $x_j^{(k-1)}$ - wartości z poprzedniej iteracji.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz wektor początkowy $x^{(0)}$, na przykład:

$$x^{(0)} = [0, 0, 0, 0]$$

Krok 2

Dla każdej iteracji liczysz nowy wektor $x^{(k)}$ na podstawie poprzedniego wektora $x^{(k-1)}$.

Krok 3

Po każdej iteracji sprawdzasz warunek zatrzymania.

Krok 4

Jeśli warunek jest spełniony, kończysz obliczenia.

Dane: $A, b, x^{(0)}, \varepsilon$

Kroki:

- wybierz przybliżenie początkowe $x^{(0)}$,
- dla każdej iteracji:
 - dla każdej niewiadomej i :
 - oblicz nową wartość $x_i^{(k)}$ używając tylko $x^{(k-1)}$,
 - porównaj $x^{(k)}$ i $x^{(k-1)}$ (norma),
 - jeśli zmiana jest mała $\rightarrow$ stop.
- zwróć $x^{(k)}$.

Kluczowe: wszystkie wartości aktualizowane są „jednocześnie”.

Warunki zatrzymania

a) Liczba iteracji

Program kończy działanie po zadanej liczbie kroków.

b) Iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia

Sprawdzasz, czy:

$$\frac{\|x^{(k)} - x^{(k-1)}\|}{\|x^{(k)}\|} \leq \varepsilon$$

W programie używana jest norma maksimum.

Jeżeli warunek jest spełniony, uznajesz, że kolejne przybliżenia zmieniają się już bardzo nieznacznie.

c) Błąd uzyskanego przybliżenia

Jeżeli znasz dokładne rozwiązanie x^* , możesz sprawdzać:

$$\|x^{(k)} - x^*\| < \varepsilon$$

```
def norma_max(wektor): # funkcja oblicza normę maksimum wektora
    maksimum = abs(wektor[0]) # jako początkowe maksimum przyjmujemy moduł
    pierwszego elementu
    for i in range(1, len(wektor)): # przechodzimy po kolejnych elementach
        wektora
        if abs(wektor[i]) > maksimum: # sprawdzamy, czy moduł bieżącego elementu
            jest większy od dotychczasowego maksimum
            maksimum = abs(wektor[i]) # jeśli tak, aktualizujemy maksimum
    return maksimum # zwracamy największy moduł elementu wektora

def odejmij_wektory(wektor1, wektor2): # funkcja odejmuje od siebie dwa wektory o
    tej samej długości
    wynik = [] # tworzymy pustą listę na wynik odejmowania
    for i in range(len(wektor1)): # przechodzimy po wszystkich indeksach wektora
        wynik.append(wektor1[i] - wektor2[i]) # dodajemy do wyniku różnicę
    odpowiednich elementów
    return wynik # zwracamy wektor różnicy

def wypisz_wektor(wektor, nazwa="x"): # funkcja wypisuje kolejne współrzędne
    wektora
    for i in range(len(wektor)): # przechodzimy po wszystkich elementach wektora
        print(f"{nazwa}{i+1} = {wektor[i]}") # wypisujemy element w formacie x1,
        x2, x3 itd.

def jacobi(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
    rozwiazanie_dokladne=None): # funkcja realizuje metodę Jacobiego
    n = len(A) # zapisujemy liczbę równań i niewiadomych
    x_stare = x0[:] # kopiujemy przybliżenie początkowe do wektora poprzedniej
    iteracji

    for i in range(n): # sprawdzamy wszystkie elementy na przekątnej macierzy
        if A[i][i] == 0: # jeśli na przekątnej pojawi się zero
            raise ValueError("Na przekątnej macierzy nie może być zera") #
    przerywamy działanie programu z komunikatem o błędzie

    for krok in range(1, max_iter + 1): # wykonujemy kolejne iteracje od 1 do
        maksymalnej liczby iteracji
        x_nowe = [0.0] * n # tworzymy nowy wektor przybliżenia wypełniony zerami

        for i in range(n): # przechodzimy po wszystkich równaniach
            suma = 0.0 # zerujemy sumę składników spoza przekątnej dla i-tego
            równania
            for j in range(n): # przechodzimy po wszystkich kolumnach macierzy
                if j != i: # pomijamy element z przekątnej
                    suma += A[i][j] * x_stare[j] # dodajemy iloczyn współczynnika
            macierzy i starego przybliżenia
```

```

        x_nowe[i] = (b[i] - suma) / A[i][i] # liczymy nową wartość i-tej
niewiadomej ze wzoru Jacobiego

    if warunek_stopu == "iteracje": # sprawdzamy, czy wybrano warunek stopu
oparty na liczbie iteracji
        if krok == max_iter: # jeśli osiągnięto zadaną liczbę iteracji
            return x_nowe, krok # zwracamy aktualne przybliżenie i liczbę
wykonanych iteracji

        elif warunek_stopu == "roznica": # sprawdzamy, czy wybrano warunek stopu
ze slajdu
            roznica = odejmij_wektory(x_nowe, x_stare) # obliczamy różnicę
kolejnych przybliżeń
            norma_roznicy = norma_max(roznica) # liczymy normę maksimum tej
różnicy
            norma_biezaca = norma_max(x_nowe) # liczymy normę maksimum bieżącego
przybliżenia

            if norma_biezaca == 0: # sprawdzamy przypadek szczególny, gdy norma
bieżącego wektora jest równa zero
                if norma_roznicy <= epsilon: # jeśli sama norma różnicy spełnia
warunek
                    return x_nowe, krok # zwracamy wynik i numer iteracji
                else: # w zwykłym przypadku, gdy norma bieżącego przybliżenia nie jest
zerowa
                    if (norma_roznicy / norma_biezaca) <= epsilon: # sprawdzamy
warunek stopu w postaci ilorazu norm
                        return x_nowe, krok # zwracamy wynik i numer iteracji

            elif warunek_stopu == "blad": # sprawdzamy, czy wybrano warunek stopu
oparty na błędzie względem rozwiązania dokładnego
                if rozwiazanie_dokladne is None: # jeśli nie podano rozwiązania
dokładnego
                    raise ValueError("Dla warunku 'blad' trzeba podać dokładne
rozwiązanie") # zgłaszamy błąd
                blad = odejmij_wektory(x_nowe, rozwiazanie_dokladne) # obliczamy
różnicę między przybliżeniem a rozwiązaniem dokładnym
                if norma_max(blad) < epsilon: # sprawdzamy, czy norma maksimum błędu
jest mniejsza od epsilon
                    return x_nowe, krok # zwracamy wynik i numer iteracji

        x_stare = x_nowe[:] # po zakończeniu iteracji przepisujemy nowe
przybliżenie jako stare

    return x_stare, max_iter # jeśli nie spełniono warunku stopu wcześniej,
zwracamy ostatnie przybliżenie i maksymalną liczbę iteracji

print("-----ZADANIE 1-----") # wypisujemy nagłówek
zadania

A = [ # definiujemy macierz współczynników układu ze slajdów
    [4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A

```



```

    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

b = [0.0, 2.0, 3.0, -2.0] # definiujemy wektor prawej strony układu
x0 = [0.0, 0.0, 0.0, 0.0] # definiujemy przybliżenie początkowe
x_dokladne = [0.5, 1.0, 2.0, -2.0] # zapisujemy rozwiązanie dokładne dla tego
przykładu

print("\nDane:") # wypisujemy napis informujący o danych wejściowych
print("Macierz A:") # wypisujemy nagłówek dla macierzy A
for wiersz in A: # przechodzimy po wszystkich wierszach macierzy
    print(wiersz) # wypisujemy każdy wiersz macierzy

print("\nWektor b:") # wypisujemy nagłówek dla wektora b
print(b) # wypisujemy wektor prawej strony

print("\nPrzybliżenie początkowe x^(0):") # wypisujemy nagłówek dla przybliżenia
początkowego
print(x0) # wypisujemy wektor początkowy

print("\nDokładne rozwiązanie:") # wypisujemy nagłówek dla rozwiązania dokładnego
print(x_dokladne) # wypisujemy rozwiązanie dokładne

print("\n----- a) LICZBA ITERACJI -----") #
wypisujemy nagłówek dla punktu a
wynik_a, iteracje_a = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu
opartego na liczbie iteracji
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=10, # ustawiamy liczbę iteracji na 10
    warunek_stopu="iteracje" # wybieramy warunek stopu oparty na liczbie iteracji
)

print("\nWynik końcowy po zadanej liczbie iteracji:") # wypisujemy opis wyniku z
punktu a
wypisz_wektor(wynik_a) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_a) # wypisujemy liczbę wykonanych iteracji

blad_a = odejmij_wektory(wynik_a, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu a
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a)) #
wypisujemy normę maksimum błędu

print("\n----- b) WARUNEK ZE SLAJDU -----") #
wypisujemy nagłówek dla punktu b
wynik_b, iteracje_b = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu ze
slajdu
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy dokładność zgodną ze slajdem
    warunek_stopu="roznica" # wybieramy warunek stopu oparty na ilorazie norm
)

```

```

print("\nWynik końcowy:") # wypisujemy opis wyniku z punktu b
wypisz_wektor(wynik_b) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_b) # wypisujemy liczbę wykonanych iteracji

blad_b = odejmij_wektory(wynik_b, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu b
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b)) #
wypisujemy normę maksimum błędu

print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----
") # wypisujemy nagłówek dla punktu c
wynik_c, iteracje_c = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu
opartego na błędzie
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy dokładność na 10 do potęgi minus 3
    warunek_stopu="blad", # wybieramy warunek stopu oparty na błędzie względem
rozwiązania dokładnego
    rozwiazanie_dokladne=x_dokladne # przekazujemy rozwiązanie dokładne potrzebne
do obliczenia błędu
)

print("\nWynik końcowy:") # wypisujemy opis wyniku z punktu c
wypisz_wektor(wynik_c) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_c) # wypisujemy liczbę wykonanych iteracji

blad_c = odejmij_wektory(wynik_c, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu c
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c)) #
wypisujemy normę maksimum błędu

```

Zadanie 2

Napisz program implementujący metodę Gaussa-Seidla iteracyjnego rozwiązywania układów równań liniowych z analogicznymi warunkami zatrzymania jak w zadaniu 1.

Co to jest metoda Gaussa-Seidla?

Metoda Gaussa-Seidla jest podobna do metody Jacobiego, ale ma ważną różnicę:

podczas liczenia nowego przybliżenia wykorzystuje od razu te wartości, które zostały już policzone w bieżącej iteracji.

Każdą niewiadomą w iteracji k obliczamy ze wzoru:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right), \quad i = 1, 2, \dots, n$$

dla $i = 1, 2, \dots, n$.

Czyli:

- dla wcześniejszych współrzędnych używamy już **nowych wartości** z iteracji k ,
- dla późniejszych współrzędnych używamy jeszcze **starych wartości** z iteracji $k - 1$.

Dzięki temu metoda Gaussa-Seidla zwykle zbiega szybciej niż metoda Jacobiego.

Różnica względem metody Jacobiego

W metodzie Jacobiego wszystkie nowe wartości obliczane są wyłącznie na podstawie poprzedniego przybliżenia.

W metodzie Gaussa-Seidla:

- korzystamy z najnowszych dostępnych wartości,
- wartości są aktualizowane **sekwencyjnie**,
- po obliczeniu $x_1^{(k)}$ można od razu użyć go do wyznaczenia $x_2^{(k)}$, potem $x_3^{(k)}$ itd.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz początkowe przybliżenie $x^{(0)}$.

Krok 2

Dla każdej iteracji przechodzisz po współrzędnych po kolei: $i = 1, 2, \dots, n$.

Krok 3

Przy obliczaniu nowej wartości $x_i^{(k)}$ używasz:

- nowych wartości $x_1^{(k)}, x_2^{(k)}, \dots, x_{i-1}^{(k)}$,
- starych wartości $x_{i+1}^{(k-1)}, x_{i+2}^{(k-1)}, \dots, x_n^{(k-1)}$.

Krok 4

Po zakończeniu całej iteracji sprawdzasz warunek zatrzymania:

- liczba iteracji,
- iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia,
- błąd przybliżenia.

Warunek stosowalności

Aby metoda mogła być wykonywana, na przekątnej macierzy nie może być zera, czyli:

$$a_{ii} \neq 0 \quad \text{dla wszystkich } i = 1, 2, \dots, n$$

Warunki zatrzymania

a) Liczba iteracji

Program kończy działanie po zadanej liczbie kroków.

b) Iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia

Zgodnie ze slajdami sprawdzamy warunek:

$$\frac{\|x^{(k)} - x^{(k-1)}\|}{\|x^{(k)}\|} \leq \varepsilon$$

W programie używana jest norma maksimum.

c) Błąd uzyskanego przybliżenia

Jeżeli znamy rozwiązanie dokładne x^* , możemy sprawdzać:

$$\|x^{(k)} - x^*\| < \varepsilon$$

Przykład użyty w programie

W programie wykorzystano ten sam układ równań co na slajdach:

$$\begin{cases} 4x_1 - 2x_2 = 0 \\ -2x_1 + 5x_2 - x_3 = 2 \\ -x_2 + 4x_3 + 2x_4 = 3 \\ 2x_3 + 3x_4 = -2 \end{cases}$$

z przybliżeniem początkowym:

$$x^{(0)} = (0, 0, 0, 0)^T$$

oraz rozwiązaniem dokładnym:

$$x^* = (0.5, 1, 2, -2)^T$$

Dla warunku ze slajdów przyjęto:

$$\varepsilon = 10^{-3}$$

Kod

PYTHON

```
def norma_max(wektor): # definiujemy funkcję obliczającą normę maksimum wektora
    maksimum = abs(wektor[0]) # jako początkowe maksimum przyjmujemy moduł
    pierwszego elementu
    for i in range(1, len(wektor)): # przechodzimy po kolejnych elementach
    wektora
        if abs(wektor[i]) > maksimum: # sprawdzamy, czy moduł bieżącego elementu
        jest większy od obecnego maksimum
            maksimum = abs(wektor[i]) # jeśli tak, aktualizujemy wartość maksimum
    return maksimum # zwracamy największy moduł elementu wektora

def odejmij_wektory(wektor1, wektor2): # definiujemy funkcję odejmującą od siebie
    dwa wektory
    wynik = [] # tworzymy pustą listę na wynik odejmowania
    for i in range(len(wektor1)): # przechodzimy po wszystkich indeksach wektora
        wynik.append(wektor1[i] - wektor2[i]) # dodajemy do wyniku różnicę
    odpowiednich elementów
    return wynik # zwracamy obliczony wektor różnicy

def wypisz_wektor(wektor, nazwa="x"): # definiujemy funkcję do ładnego
    wypisywania wektora
    for i in range(len(wektor)): # przechodzimy po wszystkich elementach wektora
        print(f"{nazwa}{i+1} = {wektor[i]}") # wypisujemy elementy jako x1, x2,
    x3 itd.

def gauss_seidel(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
    rozwiazanie_dokladne=None): # definiujemy funkcję realizującą metodę Gaussa-Seidla
    n = len(A) # zapisujemy liczbę równań i niewiadomych
    x = x0[:] # kopiujemy przybliżenie początkowe do wektora roboczego

    for i in range(n): # przechodzimy po wszystkich elementach przekątnej
    macierzy
        if A[i][i] == 0: # sprawdzamy, czy na przekątnej nie ma zera
            raise ValueError("Na przekątnej macierzy nie może być zera") # jeśli
            jest zero, zgłaszamy błąd

    for krok in range(1, max_iter + 1): # wykonujemy kolejne iteracje od 1 do
    max_iter
        x_stare = x[:] # zapisujemy kopię poprzedniego przybliżenia

        for i in range(n): # przechodzimy po wszystkich niewiadomych
            suma1 = 0.0 # zerujemy sumę składników liczonych z nowych wartości
            for j in range(i): # przechodzimy po indeksach wcześniejszych niż i
                suma1 += A[i][j] * x[j] # dodajemy składniki z już
            zaktualizowanych wartości bieżącej iteracji

            suma2 = 0.0 # zerujemy sumę składników liczonych ze starych wartości
            for j in range(i + 1, n): # przechodzimy po indeksach późniejszych
```

```

niż i
        suma2 += A[i][j] * x_stare[j] # dodajemy składniki z poprzedniej
iteracji

        x[i] = (b[i] - suma1 - suma2) / A[i][i] # obliczamy nową wartość x_i
zgodnie ze wzorem Gaussa-Seidla

        if warunek_stopu == "iteracje": # sprawdzamy, czy wybrano warunek stopu
oparty na liczbie iteracji
            if krok == max_iter: # jeśli osiągnięto zadaną liczbę iteracji
                return x, krok # zwracamy aktualny wynik i liczbę iteracji

            elif warunek_stopu == "roznica": # sprawdzamy, czy wybrano warunek stopu
ze slajdów
                roznica = odejmij_wektory(x, x_stare) # obliczamy różnicę między
nowym i poprzednim przybliżeniem
                norma_roznicy = norma_max(roznica) # obliczamy normę maksimum tej
różnicy
                norma_biezaca = norma_max(x) # obliczamy normę maksimum bieżącego
przybliżenia

                if norma_biezaca == 0: # sprawdzamy przypadek szczególny, gdy norma
bieżącego przybliżenia wynosi zero
                    if norma_roznicy <= epsilon: # jeśli sama norma różnicy spełnia
warunek stopu
                        return x, krok # zwracamy wynik i liczbę iteracji
                    else: # wykonujemy standardowe sprawdzenie warunku stopu
                        if (norma_roznicy / norma_biezaca) <= epsilon: # sprawdzamy
iloraz norm zgodnie ze slajdami
                            return x, krok # zwracamy wynik i liczbę iteracji

            elif warunek_stopu == "blad": # sprawdzamy, czy wybrano warunek stopu
oparty na błędzie względem rozwiązania dokładnego
                if rozwiazanie_dokladne is None: # jeśli nie podano rozwiązania
dokładnego
                    raise ValueError("Dla warunku 'blad' trzeba podać dokładne
rozwiązanie") # zgłaszamy błąd
                blad = odejmij_wektory(x, rozwiazanie_dokladne) # obliczamy wektor
błędu względem rozwiązania dokładnego
                if norma_max(blad) < epsilon: # sprawdzamy, czy norma maksimum błędu
jest mniejsza od epsilon
                    return x, krok # zwracamy wynik i liczbę iteracji

            else: # obsługujemy przypadek niepoprawnej nazwy warunku stopu
                raise ValueError("Niepoprawny warunek stopu") # zgłaszamy błąd dla
nieznanego warunku stopu

        return x, max_iter # jeśli warunek stopu nie został spełniony wcześniej,
zwracamy ostatni wynik i maksymalną liczbę iteracji

A = [ # definiujemy macierz współczynników układu ze slajdów
[4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A

```

```

    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

b = [0.0, 2.0, 3.0, -2.0] # definiujemy wektor prawej strony układu
x0 = [0.0, 0.0, 0.0, 0.0] # definiujemy przybliżenie początkowe  $x^{(0)}$ 
x_dokladne = [0.5, 1.0, 2.0, -2.0] # definiujemy rozwiązanie dokładne tego układu

print("\nDane:") # wypisujemy nagłówek danych wejściowych
print("Macierz A:") # wypisujemy napis informujący o macierzy A
for wiersz in A: # przechodzimy po wszystkich wierszach macierzy
    print(wiersz) # wypisujemy każdy wiersz macierzy

print("\nWektor b:") # wypisujemy napis informujący o wektorze b
print(b) # wypisujemy wektor prawej strony

print("\nPrzybliżenie początkowe  $x^{(0)}$ :") # wypisujemy napis informujący o
przybliżeniu początkowym
print(x0) # wypisujemy wektor początkowy

print("\nDokładne rozwiązanie:") # wypisujemy napis informujący o rozwiązaniu
dokładnym
print(x_dokladne) # wypisujemy rozwiązanie dokładne

print("\n----- a) LICZBA ITERACJI -----") #
wypisujemy nagłówek punktu a
wynik_a, iteracje_a = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu opartego na liczbie iteracji
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=10, # ustawiamy liczbę iteracji na 10
    warunek_stopu="iteracje" # wybieramy warunek stopu oparty na liczbie iteracji
)

print("\nWynik końcowy po zadanej liczbie iteracji:") # wypisujemy opis wyniku dla
punktu a
wypisz_wektor(wynik_a) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_a) # wypisujemy liczbę wykonanych iteracji

blad_a = odejmij_wektory(wynik_a, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu a
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a)) #
wypisujemy normę maksimum błędu

print("\n----- b) WARUNEK ZE SLAJDU -----") #
wypisujemy nagłówek punktu b
wynik_b, iteracje_b = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu zgodnego ze slajdami
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy epsilon zgodnie ze slajdami
    warunek_stopu="roznica" # wybieramy warunek stopu oparty na ilorazie norm
)

```

```

print("\nWynik końcowy:") # wypisujemy opis wyniku dla punktu b
wypisz_wektor(wynik_b) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_b) # wypisujemy liczbę wykonanych iteracji

blad_b = odejmij_wektory(wynik_b, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu b
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b)) #
wypisujemy normę maksimum błędu

print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----
") # wypisujemy nagłówek punktu c
wynik_c, iteracje_c = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu opartego na błędzie dokładnym
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy epsilon na 10 do potęgi minus 3
    warunek_stopu="blad", # wybieramy warunek stopu oparty na błędzie względem
rozwiązania dokładnego
    rozwiazanie_dokladne=x_dokladne # przekazujemy rozwiązanie dokładne potrzebne
do obliczenia błędu
)

print("\nWynik końcowy:") # wypisujemy opis wyniku dla punktu c
wypisz_wektor(wynik_c) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_c) # wypisujemy liczbę wykonanych iteracji

blad_c = odejmij_wektory(wynik_c, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu c
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c)) #
wypisujemy normę maksimum błędu

```

Jeśli trzeba wyświetlić wszystkie iteracje to zmienić fragment:

```
x[i] = (b[i] - suma1 - suma2) / A[i][i]
```

PYTHON

```

if warunek_stopu == "iteracje":
    if krok == max_iter:
        return x, krok

elif warunek_stopu == "roznica":
    roznica = odejmij_wektory(x, x_stare)
    norma_roznicy = norma_max(roznica)
    norma_biezaca = norma_max(x)

```

na:


```

x[i] = (b[i] - suma1 - suma2) / A[i][i]

roznica = odejmij_wektory(x, x_stare)
norma_roznicy = norma_max(roznica)

print(f"\nIteracja {krok}:")
wypisz_wektor(x)
print("Norma maksimum różnicy:", norma_roznicy)

if warunek_stopu == "iteracje":
    if krok == max_iter:
        return x, krok

elif warunek_stopu == "roznica":
    norma_biezaca = norma_max(x)

    if norma_biezaca == 0:
        if norma_roznicy <= epsilon:

```

Zadanie 3

Sprawdź zbieżność układu pod kątem metod z zadania 1 oraz 2.

Układ do testowania:

$$\begin{cases} 4x_1 - 2x_2 = 0, \\ -2x_1 + 5x_2 - x_3 = 2, \\ -x_2 + 4x_3 + 2x_4 = 3, \\ 2x_3 + 3x_4 = -2. \end{cases}$$

Zapis macierzowy układu

Macierz współczynników:

$$A = \begin{bmatrix} 4 & -2 & 0 & 0 \\ -2 & 5 & -1 & 0 \\ 0 & -1 & 4 & 2 \\ 0 & 0 & 2 & 3 \end{bmatrix}$$

Wektor prawej strony:

$$b = \begin{bmatrix} 0 \\ 2 \\ 3 \\ -2 \end{bmatrix}$$

Dokładne rozwiązanie tego układu jest równe:

$$x^* = \begin{bmatrix} 0.5 \\ 1 \\ 2 \\ -2 \end{bmatrix}$$

Co sprawdzamy w tym zadaniu?

W zadaniu 1 używaliśmy **metody Jacobiego**, a w zadaniu 2 **metody Gaussa-Seidla**. Teraz nie liczymy kolejnych iteracji, tylko sprawdzamy, **czy te metody powinny być zbieżne** dla danego układu.

Na slajdach pojawia się ogólna postać metody iteracyjnej:

$$x^{(k)} = Wx^{(k-1)} + Z$$

Najważniejsze pytanie brzmi:

czy kolejne iteracje prowadzą do rozwiązania?

Według slajdów zależy to od macierzy iteracji (W).

W praktyce używamy warunku:

$$\|W\| < 1$$

W tej notatce korzystamy z **normy wierszowej**:

$$\|W\|_{\infty} = \max_i \sum_{j=1}^n |w_{ij}|$$

Jeżeli:

$$\|W\|_{\infty} < 1$$

to metoda powinna być zbieżna.

1. Metoda Jacobiego

Dla metody Jacobiego macierz iteracji wyznaczamy dokładnie z wzoru ze slajdu:

$$W_{ij} = \begin{cases} 0 & \text{gdy } i = j, \\ -\frac{a_{ij}}{a_{ii}} & \text{gdy } i \neq j \end{cases}$$

Dla naszego układu otrzymujemy:

$$W_J = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 \\ \frac{2}{5} & 0 & \frac{1}{5} & 0 \\ 0 & \frac{1}{4} & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{2}{3} & 0 \end{bmatrix}$$

Teraz liczymy normę wierszową:

- wiersz 1:

$$0 + \frac{1}{2} + 0 + 0 = \frac{1}{2}$$

- wiersz 2:

$$\frac{2}{5} + 0 + \frac{1}{5} + 0 = \frac{3}{5}$$

- wiersz 3:

$$0 + \frac{1}{4} + 0 + \frac{1}{2} = \frac{3}{4}$$

- wiersz 4:

$$0 + 0 + \frac{2}{3} + 0 = \frac{2}{3}$$

Zatem:

$$|W_J|_{\infty} = \max \left(\frac{1}{2}, \frac{3}{5}, \frac{3}{4}, \frac{2}{3} \right) = \frac{3}{4} = 0.75$$

Ponieważ:

$$||W_J||_{\infty} < 1$$

to **metoda Jacobiego jest zbieżna.**

2. Metoda Gaussa-Seidla

Dla metody Gaussa-Seidla na slajdach mamy wzór:

$$(L + D)x^{(k)} = -Ux^{(k-1)} + b$$

gdzie:

- L — macierz trójkątna dolna,
- D — macierz diagonalna,
- U — macierz trójkątna górna.

Po przekształceniu dostajemy:

$$x^{(k)} = -(L + D)^{-1}Ux^{(k-1)} + (L + D)^{-1}b$$

Stąd macierz iteracji dla Gaussa-Seidla ma postać:

$$W_{GS} = -(L + D)^{-1}U$$

To jest ważna różnica względem Jacobiego:

- dla Jacobiego mamy prosty wzór element po elemencie,
- dla Gaussa-Seidla trzeba skorzystać z rozkładu macierzy A na części L, D, U .

Dla naszego układu otrzymujemy:

$$W_{GS} = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{5} & \frac{1}{5} & 0 \\ 0 & \frac{1}{20} & \frac{1}{20} & -\frac{1}{2} \\ 0 & -\frac{1}{30} & -\frac{1}{30} & \frac{1}{3} \end{bmatrix}$$

Liczymy normę wierszową:

- wiersz 1:

$$0 + \frac{1}{2} + 0 + 0 = \frac{1}{2}$$

- wiersz 2:

$$0 + \frac{1}{5} + \frac{1}{5} + 0 = \frac{2}{5}$$

- wiersz 3:

$$0 + \frac{1}{20} + \frac{1}{20} + \frac{1}{2} = \frac{3}{5}$$

- wiersz 4:

$$0 + \frac{1}{30} + \frac{1}{30} + \frac{1}{3} = \frac{2}{5}$$

Zatem:

$$|W_{GS}|_{\infty} = \max\left(\frac{1}{2}, \frac{2}{5}, \frac{3}{5}, \frac{2}{5}\right) = \frac{3}{5} = 0.6$$

Ponieważ:

$$|W_{GS}|_{\infty} < 1$$

to **metoda Gaussa-Seidla jest zbieżna**.

Porównanie obu metod

Otrzymaliśmy:

$$|W_J|_{\infty} = 0.75$$

oraz

$$|W_{GS}|_{\infty} = 0.6$$

Obie metody są zbieżne, ale ponieważ:

$$0.6 < 0.75$$

to metoda Gaussa-Seidla powinna zbiegać szybciej niż metoda Jacobiego.
Zgadza się to z wcześniejszymi obserwacjami z zadań 1 i 2.

```
print("-----ZADANIE 3-----") # wypisujemy nagłówek zadania 3

def zeros(n, m): # definiujemy funkcję tworzącą macierz n x m wypełnioną zerami
    macierz = [] # tworzymy pustą listę na całą macierz
    for i in range(n): # wykonujemy pętlę po wszystkich wierszach
        wiersz = [] # tworzymy pustą listę na jeden wiersz
        for j in range(m): # wykonujemy pętlę po wszystkich kolumnach
            wiersz.append(0.0) # dodajemy do wiersza element 0.0
        macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
    return macierz # zwracamy utworzoną macierz zerową

def wypisz_macierz(macierz): # definiujemy funkcję do wypisywania macierzy
    for wiersz in macierz: # przechodzimy po wszystkich wierszach macierzy
        print(wiersz) # wypisujemy bieżący wiersz

def norma_wierszowa_macierzy(macierz): # definiujemy funkcję liczącą normę
    wierszową macierzy
    maksimum = 0.0 # ustawiamy początkowe maksimum na 0
    for i in range(len(macierz)): # przechodzimy po wszystkich wierszach macierzy
        suma = 0.0 # zerujemy sumę modułów elementów w bieżącym wierszu
        for j in range(len(macierz[i])): # przechodzimy po wszystkich elementach
            # bieżącego wiersza
            suma += abs(macierz[i][j]) # dodajemy moduł bieżącego elementu do
            # sumy
        if suma > maksimum: # sprawdzamy, czy suma z tego wiersza jest większa od
            # dotychczasowego maksimum
            maksimum = suma # aktualizujemy maksimum
    return maksimum # zwracamy największą sumę modułów z wierszy

def macierz_iteracji_jacobiego(A): # definiujemy funkcję wyznaczającą macierz
    iteracji metody Jacobiego
    n = len(A) # zapisujemy rozmiar macierzy A
    W = zeros(n, n) # tworzymy pustą macierz iteracji W o rozmiarze n x n

    for i in range(n): # przechodzimy po wszystkich wierszach macierzy
        for j in range(n): # przechodzimy po wszystkich kolumnach macierzy
            if i == j: # sprawdzamy, czy jesteśmy na przekątnej
                W[i][j] = 0.0 # na przekątnej wpisujemy 0 zgodnie ze wzorem dla
                # Jacobiego
            else: # w przeciwnym wypadku jesteśmy poza przekątną
                W[i][j] = -A[i][j] / A[i][i] # wpisujemy wartość -a_ij / a_ii
                # zgodnie ze wzorem ze slajdu

    return W # zwracamy macierz iteracji Jacobiego

def roziaz_uklad_dolnotrojkatny(LD, b): # definiujemy funkcję rozwiązującą układ
    dolnotrójkątny LDx=b
    n = len(LD) # zapisujemy rozmiar macierzy
    x = [0.0] * n # tworzymy wektor rozwiązania wypełniony zerami
```

```

    for i in range(n): # przechodzimy po kolejnych równaniach od góry do dołu
        suma = 0.0 # zerujemy sumę znanych składników
        for j in range(i): # przechodzimy po wcześniej obliczonych elementach
            suma += LD[i][j] * x[j] # dodajemy składniki z już wyznaczonych
            wartości
        x[i] = (b[i] - suma) / LD[i][i] # obliczamy bieżący element rozwiązania
        przez podstawianie w przód

    return x # zwracamy wyznaczony wektor rozwiązania

def macierz_iteracji_gaussa_seidla(A): # definiujemy funkcję wyznaczającą macierz
    iteracji metody Gaussa-Seidla
    n = len(A) # zapisujemy rozmiar macierzy A

    LD = zeros(n, n) # tworzymy pustą macierz dla części L + D
    U = zeros(n, n) # tworzymy pustą macierz dla części U

    for i in range(n): # przechodzimy po wszystkich wierszach macierzy A
        for j in range(n): # przechodzimy po wszystkich kolumnach macierzy A
            if j <= i: # sprawdzamy, czy element należy do dolnej części wraz z
                przekątną
                LD[i][j] = A[i][j] # przepisujemy element do macierzy LD
            else: # w przeciwnym wypadku element należy do części górnej
                U[i][j] = A[i][j] # przepisujemy element do macierzy U

    W = zeros(n, n) # tworzymy pustą macierz iteracji Gaussa-Seidla

    for kolumna in range(n): # przechodzimy po kolejnych kolumnach macierzy W
        prawa_strona = [] # tworzymy pustą listę na prawą stronę układu
        pomocniczego
        for i in range(n): # przechodzimy po wszystkich wierszach
            prawa_strona.append(-U[i][kolumna]) # budujemy prawą stronę jako
            przeciwną kolumnę macierzy U

        rozwiazanie = rozwiaz_uklad_dolnotrojkatny(LD, prawa_strona) #
        rozwiązujemy układ  $(L+D)x = -u_k$  dla bieżącej kolumny

        for i in range(n): # przechodzimy po wszystkich wierszach rozwiązania
            W[i][kolumna] = rozwiazanie[i] # wpisujemy obliczoną kolumnę do
            macierzy iteracji W

    return W # zwracamy macierz iteracji Gaussa-Seidla

A = [ # definiujemy macierz współczynników układu
    [4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A
    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

print("\nMacierz A:") # wypisujemy nagłówek dla macierzy A

```

```

wypisz_macierz(A) # wypisujemy macierz A

print("\n----- METODA JACOBIEGO -----") # wypisujemy
nagłówek sekcji dla metody Jacobiego
WJ = macierz_iteracji_jacobiego(A) # wyznaczamy macierz iteracji Jacobiego
print("Macierz iteracyjna W_J:") # wypisujemy nagłówek dla macierzy W_J
wypisz_macierz(WJ) # wypisujemy macierz iteracji Jacobiego

norma_WJ = norma_wierszowa_macierzy(WJ) # obliczamy normę wierszową macierzy
iteracji Jacobiego
print("Norma wierszowa ||W_J|| =", norma_WJ) # wypisujemy wartość normy macierzy
W_J

if norma_WJ < 1: # sprawdzamy warunek zbieżności dla Jacobiego
    print("Metoda Jacobiego jest zbieżna, ponieważ ||W_J|| < 1.") # wypisujemy
pozytywny wniosek
else: # w przeciwnym przypadku
    print("Metoda Jacobiego może nie być zbieżna, ponieważ ||W_J|| >= 1.") #
wypisujemy negatywny wniosek

print("\n----- METODA GAUSSA-SEIDLA -----") #
wypisujemy nagłówek sekcji dla metody Gaussa-Seidla
WGS = macierz_iteracji_gaussa_seidla(A) # wyznaczamy macierz iteracji Gaussa-
Seidla
print("Macierz iteracyjna W_GS:") # wypisujemy nagłówek dla macierzy W_GS
wypisz_macierz(WGS) # wypisujemy macierz iteracji Gaussa-Seidla

norma_WGS = norma_wierszowa_macierzy(WGS) # obliczamy normę wierszową macierzy
iteracji Gaussa-Seidla
print("Norma wierszowa ||W_GS|| =", norma_WGS) # wypisujemy wartość normy macierzy
W_GS

if norma_WGS < 1: # sprawdzamy warunek zbieżności dla Gaussa-Seidla
    print("Metoda Gaussa-Seidla jest zbieżna, ponieważ ||W_GS|| < 1.") #
wypisujemy pozytywny wniosek
else: # w przeciwnym przypadku
    print("Metoda Gaussa-Seidla może nie być zbieżna, ponieważ ||W_GS|| >= 1.") #
wypisujemy negatywny wniosek

print("\n----- WNIOSEK KOŃCOWY -----") # wypisujemy
nagłówek końcowego wniosku
print("Zbieżność metod z zadania 1 i 2 badamy przez normę macierzy iteracji.") #
wypisujemy ogólną informację o sposobie badania zbieżności
print("Dla Jacobiego sprawdzamy macierz W_J.") # wypisujemy informację dotyczącą
metody Jacobiego
print("Dla Gaussa-Seidla sprawdzamy macierz W_GS.") # wypisujemy informację
dotyczącą metody Gaussa-Seidla
print("Jeżeli ||W|| < 1, to metoda jest zbieżna.") # wypisujemy końcowe kryterium
zbieżności

```


Wnioski

W zadaniu 3 zbieżność badamy zgodnie ze slajdami przez macierz iteracji (W) i warunek:

$$|W| < 1$$

Dla badanego układu:

- dla metody Jacobiego:

$$|W_J|_{\infty} = 0.75 < 1$$

- dla metody Gaussa-Seidla:

$$|W_{GS}|_{\infty} = 0.6 < 1$$

Oznacza to, że:

- metoda Jacobiego jest zbieżna,
 - metoda Gaussa-Seidla jest zbieżna,
 - metoda Gaussa-Seidla powinna zbiegać szybciej niż metoda Jacobiego.
-

Lab 6

Zadanie 1

Napisz program implementujący metodę Jacobiego iteracyjnego rozwiązywania układów równań liniowych, w której warunkiem zatrzymania będzie:

- liczba iteracji,
- norma wektora powstałego przez odjęcie wektorów określających kolejne przybliżenia,
- błąd uzyskanego przybliżenia.

Co to jest metoda Jacobiego?

Metoda Jacobiego jest metodą iteracyjną, czyli nie liczy rozwiązania od razu, tylko buduje kolejne przybliżenia:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

Każde nowe przybliżenie oblicza się wyłącznie na podstawie poprzedniego.

Dla układu:

$$Ax = b$$

każdą niewiadomą w iteracji k obliczamy na podstawie wartości z iteracji poprzedniej, czyli $x^{(k-1)}$.

wzór metody Jacobiego ma postać:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k-1)} \right)$$

gdzie:

- a_{ii} - element na przekątnej macierzy,
- b_i - odpowiedni element wektora prawej strony,
- $x_j^{(k-1)}$ - wartości z poprzedniej iteracji.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz wektor początkowy $x^{(0)}$, na przykład:

$$x^{(0)} = [0, 0, 0, 0]$$

Krok 2

Dla każdej iteracji liczysz nowy wektor $x^{(k)}$ na podstawie poprzedniego wektora $x^{(k-1)}$.

Krok 3

Po każdej iteracji sprawdzasz warunek zatrzymania.

Krok 4

Jeśli warunek jest spełniony, kończysz obliczenia.

Dane: $A, b, x^{(0)}, \varepsilon$

Kroki:

- wybierz przybliżenie początkowe $x^{(0)}$,
- dla każdej iteracji:
 - dla każdej niewiadomej i :
 - oblicz nową wartość $x_i^{(k)}$ używając tylko $x^{(k-1)}$,
 - porównaj $x^{(k)}$ i $x^{(k-1)}$ (norma),
 - jeśli zmiana jest mała $\rightarrow$ stop.
- zwróć $x^{(k)}$.

Kluczowe: wszystkie wartości aktualizowane są „jednocześnie”.

Warunki zatrzymania

a) Liczba iteracji

Program kończy działanie po zadanej liczbie kroków.

b) Iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia

Sprawdzasz, czy:

$$\frac{\|x^{(k)} - x^{(k-1)}\|}{\|x^{(k)}\|} \leq \varepsilon$$

W programie używana jest norma maksimum.

Jeżeli warunek jest spełniony, uznajesz, że kolejne przybliżenia zmieniają się już bardzo nieznacznie.

c) Błąd uzyskanego przybliżenia

Jeżeli znasz dokładne rozwiązanie x^* , możesz sprawdzać:

$$\|x^{(k)} - x^*\| < \varepsilon$$

```
def norma_max(wektor): # funkcja oblicza normę maksimum wektora
    maksimum = abs(wektor[0]) # jako początkowe maksimum przyjmujemy moduł
    pierwszego elementu
    for i in range(1, len(wektor)): # przechodzimy po kolejnych elementach
        wektora
        if abs(wektor[i]) > maksimum: # sprawdzamy, czy moduł bieżącego elementu
            jest większy od dotychczasowego maksimum
            maksimum = abs(wektor[i]) # jeśli tak, aktualizujemy maksimum
    return maksimum # zwracamy największy moduł elementu wektora

def odejmij_wektory(wektor1, wektor2): # funkcja odejmuje od siebie dwa wektory o
    tej samej długości
    wynik = [] # tworzymy pustą listę na wynik odejmowania
    for i in range(len(wektor1)): # przechodzimy po wszystkich indeksach wektora
        wynik.append(wektor1[i] - wektor2[i]) # dodajemy do wyniku różnicę
    odpowiednich elementów
    return wynik # zwracamy wektor różnicy

def wypisz_wektor(wektor, nazwa="x"): # funkcja wypisuje kolejne współrzędne
    wektora
    for i in range(len(wektor)): # przechodzimy po wszystkich elementach wektora
        print(f"{nazwa}{i+1} = {wektor[i]}") # wypisujemy element w formacie x1,
        x2, x3 itd.

def jacobi(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
    rozwiazanie_dokladne=None): # funkcja realizuje metodę Jacobiego
    n = len(A) # zapisujemy liczbę równań i niewiadomych
    x_stare = x0[:] # kopiujemy przybliżenie początkowe do wektora poprzedniej
    iteracji

    for i in range(n): # sprawdzamy wszystkie elementy na przekątnej macierzy
        if A[i][i] == 0: # jeśli na przekątnej pojawi się zero
            raise ValueError("Na przekątnej macierzy nie może być zera") #
    przerywamy działanie programu z komunikatem o błędzie

    for krok in range(1, max_iter + 1): # wykonujemy kolejne iteracje od 1 do
        maksymalnej liczby iteracji
        x_nowe = [0.0] * n # tworzymy nowy wektor przybliżenia wypełniony zerami

        for i in range(n): # przechodzimy po wszystkich równaniach
            suma = 0.0 # zerujemy sumę składników spoza przekątnej dla i-tego
            równania
            for j in range(n): # przechodzimy po wszystkich kolumnach macierzy
                if j != i: # pomijamy element z przekątnej
                    suma += A[i][j] * x_stare[j] # dodajemy iloczyn współczynnika
            macierzy i starego przybliżenia
```

```

        x_nowe[i] = (b[i] - suma) / A[i][i] # liczymy nową wartość i-tej
niewiadomej ze wzoru Jacobiego

    if warunek_stopu == "iteracje": # sprawdzamy, czy wybrano warunek stopu
oparty na liczbie iteracji
        if krok == max_iter: # jeśli osiągnięto zadaną liczbę iteracji
            return x_nowe, krok # zwracamy aktualne przybliżenie i liczbę
wykonanych iteracji

        elif warunek_stopu == "roznica": # sprawdzamy, czy wybrano warunek stopu
ze slajdu
            roznica = odejmij_wektory(x_nowe, x_stare) # obliczamy różnicę
kolejnych przybliżeń
            norma_roznicy = norma_max(roznica) # liczymy normę maksimum tej
różnicy
            norma_biezaca = norma_max(x_nowe) # liczymy normę maksimum bieżącego
przybliżenia

            if norma_biezaca == 0: # sprawdzamy przypadek szczególny, gdy norma
bieżącego wektora jest równa zero
                if norma_roznicy <= epsilon: # jeśli sama norma różnicy spełnia
warunek
                    return x_nowe, krok # zwracamy wynik i numer iteracji
                else: # w zwykłym przypadku, gdy norma bieżącego przybliżenia nie jest
zerowa
                    if (norma_roznicy / norma_biezaca) <= epsilon: # sprawdzamy
warunek stopu w postaci ilorazu norm
                        return x_nowe, krok # zwracamy wynik i numer iteracji

            elif warunek_stopu == "blad": # sprawdzamy, czy wybrano warunek stopu
oparty na błędzie względem rozwiązania dokładnego
                if rozwiazanie_dokladne is None: # jeśli nie podano rozwiązania
dokładnego
                    raise ValueError("Dla warunku 'blad' trzeba podać dokładne
rozwiązanie") # zgłaszamy błąd
                blad = odejmij_wektory(x_nowe, rozwiazanie_dokladne) # obliczamy
różnicę między przybliżeniem a rozwiązaniem dokładnym
                if norma_max(blad) < epsilon: # sprawdzamy, czy norma maksimum błędu
jest mniejsza od epsilon
                    return x_nowe, krok # zwracamy wynik i numer iteracji

        x_stare = x_nowe[:] # po zakończeniu iteracji przepisujemy nowe
przybliżenie jako stare

    return x_stare, max_iter # jeśli nie spełniono warunku stopu wcześniej,
zwracamy ostatnie przybliżenie i maksymalną liczbę iteracji

print("-----ZADANIE 1-----") # wypisujemy nagłówek
zadania

A = [ # definiujemy macierz współczynników układu ze slajdów
    [4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A

```

```

    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

b = [0.0, 2.0, 3.0, -2.0] # definiujemy wektor prawej strony układu
x0 = [0.0, 0.0, 0.0, 0.0] # definiujemy przybliżenie początkowe
x_dokladne = [0.5, 1.0, 2.0, -2.0] # zapisujemy rozwiązanie dokładne dla tego
przykładu

print("\nDane:") # wypisujemy napis informujący o danych wejściowych
print("Macierz A:") # wypisujemy nagłówek dla macierzy A
for wiersz in A: # przechodzimy po wszystkich wierszach macierzy
    print(wiersz) # wypisujemy każdy wiersz macierzy

print("\nWektor b:") # wypisujemy nagłówek dla wektora b
print(b) # wypisujemy wektor prawej strony

print("\nPrzybliżenie początkowe x^(0):") # wypisujemy nagłówek dla przybliżenia
początkowego
print(x0) # wypisujemy wektor początkowy

print("\nDokładne rozwiązanie:") # wypisujemy nagłówek dla rozwiązania dokładnego
print(x_dokladne) # wypisujemy rozwiązanie dokładne

print("\n----- a) LICZBA ITERACJI -----") #
wypisujemy nagłówek dla punktu a
wynik_a, iteracje_a = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu
opartego na liczbie iteracji
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=10, # ustawiamy liczbę iteracji na 10
    warunek_stopu="iteracje" # wybieramy warunek stopu oparty na liczbie iteracji
)

print("\nWynik końcowy po zadanej liczbie iteracji:") # wypisujemy opis wyniku z
punktu a
wypisz_wektor(wynik_a) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_a) # wypisujemy liczbę wykonanych iteracji

blad_a = odejmij_wektory(wynik_a, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu a
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a)) #
wypisujemy normę maksimum błędu

print("\n----- b) WARUNEK ZE SLAJDU -----") #
wypisujemy nagłówek dla punktu b
wynik_b, iteracje_b = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu ze
slajdu
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy dokładność zgodną ze slajdem
    warunek_stopu="roznica" # wybieramy warunek stopu oparty na ilorazie norm
)

```

```

print("\nWynik końcowy:") # wypisujemy opis wyniku z punktu b
wypisz_wektor(wynik_b) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_b) # wypisujemy liczbę wykonanych iteracji

blad_b = odejmij_wektory(wynik_b, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu b
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b)) #
wypisujemy normę maksimum błędu

print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----")
# wypisujemy nagłówek dla punktu c
wynik_c, iteracje_c = jacobi( # wywołujemy metodę Jacobiego dla warunku stopu
opartego na błędzie
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy dokładność na 10 do potęgi minus 3
    warunek_stopu="blad", # wybieramy warunek stopu oparty na błędzie względem
rozwiązania dokładnego
    rozwiazanie_dokladne=x_dokladne # przekazujemy rozwiązanie dokładne potrzebne
do obliczenia błędu
)

print("\nWynik końcowy:") # wypisujemy opis wyniku z punktu c
wypisz_wektor(wynik_c) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_c) # wypisujemy liczbę wykonanych iteracji

blad_c = odejmij_wektory(wynik_c, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu c
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c)) #
wypisujemy normę maksimum błędu

```

Zadanie 2

Napisz program implementujący metodę Gaussa-Seidla iteracyjnego rozwiązywania układów równań liniowych z analogicznymi warunkami zatrzymania jak w zadaniu 1.

Co to jest metoda Gaussa-Seidla?

Metoda Gaussa-Seidla jest podobna do metody Jacobiego, ale ma ważną różnicę:

podczas liczenia nowego przybliżenia wykorzystuje od razu te wartości, które zostały już policzone w bieżącej iteracji.

Każdą niewiadomą w iteracji k obliczamy ze wzoru:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right), \quad i = 1, 2, \dots, n$$

dla $i = 1, 2, \dots, n$.

Czyli:

- dla wcześniejszych współrzędnych używamy już **nowych wartości** z iteracji k ,
- dla późniejszych współrzędnych używamy jeszcze **starych wartości** z iteracji $k - 1$.

Dzięki temu metoda Gaussa-Seidla zwykle zbiega szybciej niż metoda Jacobiego.

Różnica względem metody Jacobiego

W metodzie Jacobiego wszystkie nowe wartości obliczane są wyłącznie na podstawie poprzedniego przybliżenia.

W metodzie Gaussa-Seidla:

- korzystamy z najnowszych dostępnych wartości,
- wartości są aktualizowane **sekwencyjnie**,
- po obliczeniu $x_1^{(k)}$ można od razu użyć go do wyznaczenia $x_2^{(k)}$, potem $x_3^{(k)}$ itd.

Jak rozumieć schemat implementacji?

Krok 1

Tworzysz początkowe przybliżenie $x^{(0)}$.

Krok 2

Dla każdej iteracji przechodzisz po współrzędnych po kolei: $i = 1, 2, \dots, n$.

Krok 3

Przy obliczaniu nowej wartości $x_i^{(k)}$ używasz:

- nowych wartości $x_1^{(k)}, x_2^{(k)}, \dots, x_{i-1}^{(k)}$,
- starych wartości $x_{i+1}^{(k-1)}, x_{i+2}^{(k-1)}, \dots, x_n^{(k-1)}$.

Krok 4

Po zakończeniu całej iteracji sprawdzasz warunek zatrzymania:

- liczba iteracji,
- iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia,
- błąd przybliżenia.

Warunek stosowalności

Aby metoda mogła być wykonywana, na przekątnej macierzy nie może być zera, czyli:

$$a_{ii} \neq 0 \quad \text{dla wszystkich } i = 1, 2, \dots, n$$

Warunki zatrzymania

a) Liczba iteracji

Program kończy działanie po zadanej liczbie kroków.

b) Iloraz normy różnicy kolejnych przybliżeń i normy bieżącego przybliżenia

Zgodnie ze slajdami sprawdzamy warunek:

$$\frac{\|x^{(k)} - x^{(k-1)}\|}{\|x^{(k)}\|} \leq \varepsilon$$

W programie używana jest norma maksimum.

c) Błąd uzyskanego przybliżenia

Jeżeli znamy rozwiązanie dokładne x^* , możemy sprawdzać:

$$\|x^{(k)} - x^*\| < \varepsilon$$

Przykład użyty w programie

W programie wykorzystano ten sam układ równań co na slajdach:

$$\begin{cases} 4x_1 - 2x_2 = 0 \\ -2x_1 + 5x_2 - x_3 = 2 \\ -x_2 + 4x_3 + 2x_4 = 3 \\ 2x_3 + 3x_4 = -2 \end{cases}$$

z przybliżeniem początkowym:

$$x^{(0)} = (0, 0, 0, 0)^T$$

oraz rozwiązaniem dokładnym:

$$x^* = (0.5, 1, 2, -2)^T$$

Dla warunku ze slajdów przyjęto:

$$\varepsilon = 10^{-3}$$

Kod

PYTHON

```
def norma_max(wektor): # definiujemy funkcję obliczającą normę maksimum wektora
    maksimum = abs(wektor[0]) # jako początkowe maksimum przyjmujemy moduł
    pierwszego elementu
    for i in range(1, len(wektor)): # przechodzimy po kolejnych elementach
    wektora
        if abs(wektor[i]) > maksimum: # sprawdzamy, czy moduł bieżącego elementu
        jest większy od obecnego maksimum
            maksimum = abs(wektor[i]) # jeśli tak, aktualizujemy wartość maksimum
    return maksimum # zwracamy największy moduł elementu wektora

def odejmij_wektory(wektor1, wektor2): # definiujemy funkcję odejmującą od siebie
    dwa wektory
    wynik = [] # tworzymy pustą listę na wynik odejmowania
    for i in range(len(wektor1)): # przechodzimy po wszystkich indeksach wektora
        wynik.append(wektor1[i] - wektor2[i]) # dodajemy do wyniku różnicę
    odpowiednich elementów
    return wynik # zwracamy obliczony wektor różnicy

def wypisz_wektor(wektor, nazwa="x"): # definiujemy funkcję do ładnego
    wypisywania wektora
    for i in range(len(wektor)): # przechodzimy po wszystkich elementach wektora
        print(f"{nazwa}{i+1} = {wektor[i]}") # wypisujemy elementy jako x1, x2,
    x3 itd.

def gauss_seidel(A, b, x0, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje",
    rozwiazanie_dokladne=None): # definiujemy funkcję realizującą metodę Gaussa-Seidla
    n = len(A) # zapisujemy liczbę równań i niewiadomych
    x = x0[:] # kopiujemy przybliżenie początkowe do wektora roboczego

    for i in range(n): # przechodzimy po wszystkich elementach przekątnej
    macierzy
        if A[i][i] == 0: # sprawdzamy, czy na przekątnej nie ma zera
            raise ValueError("Na przekątnej macierzy nie może być zera") # jeśli
            jest zero, zgłaszamy błąd

    for krok in range(1, max_iter + 1): # wykonujemy kolejne iteracje od 1 do
    max_iter
        x_stare = x[:] # zapisujemy kopię poprzedniego przybliżenia

        for i in range(n): # przechodzimy po wszystkich niewiadomych
            suma1 = 0.0 # zerujemy sumę składników liczonych z nowych wartości
            for j in range(i): # przechodzimy po indeksach wcześniejszych niż i
                suma1 += A[i][j] * x[j] # dodajemy składniki z już
            zaktualizowanych wartości bieżącej iteracji

            suma2 = 0.0 # zerujemy sumę składników liczonych ze starych wartości
            for j in range(i + 1, n): # przechodzimy po indeksach późniejszych
```

```

niż i
        suma2 += A[i][j] * x_stare[j] # dodajemy składniki z poprzedniej
iteracji

        x[i] = (b[i] - suma1 - suma2) / A[i][i] # obliczamy nową wartość x_i
zgodnie ze wzorem Gaussa-Seidla

        if warunek_stopu == "iteracje": # sprawdzamy, czy wybrano warunek stopu
oparty na liczbie iteracji
            if krok == max_iter: # jeśli osiągnięto zadaną liczbę iteracji
                return x, krok # zwracamy aktualny wynik i liczbę iteracji

            elif warunek_stopu == "roznica": # sprawdzamy, czy wybrano warunek stopu
ze slajdów
                roznica = odejmij_wektory(x, x_stare) # obliczamy różnicę między
nowym i poprzednim przybliżeniem
                norma_roznicy = norma_max(roznica) # obliczamy normę maksimum tej
różnicy
                norma_biezaca = norma_max(x) # obliczamy normę maksimum bieżącego
przybliżenia

                if norma_biezaca == 0: # sprawdzamy przypadek szczególny, gdy norma
bieżącego przybliżenia wynosi zero
                    if norma_roznicy <= epsilon: # jeśli sama norma różnicy spełnia
warunek stopu
                        return x, krok # zwracamy wynik i liczbę iteracji
                    else: # wykonujemy standardowe sprawdzenie warunku stopu
                        if (norma_roznicy / norma_biezaca) <= epsilon: # sprawdzamy
iloraz norm zgodnie ze slajdami
                            return x, krok # zwracamy wynik i liczbę iteracji

            elif warunek_stopu == "blad": # sprawdzamy, czy wybrano warunek stopu
oparty na błędzie względem rozwiązania dokładnego
                if rozwiazanie_dokladne is None: # jeśli nie podano rozwiązania
dokładnego
                    raise ValueError("Dla warunku 'blad' trzeba podać dokładne
rozwiązanie") # zgłaszamy błąd
                blad = odejmij_wektory(x, rozwiazanie_dokladne) # obliczamy wektor
błędu względem rozwiązania dokładnego
                if norma_max(blad) < epsilon: # sprawdzamy, czy norma maksimum błędu
jest mniejsza od epsilon
                    return x, krok # zwracamy wynik i liczbę iteracji

            else: # obsługujemy przypadek niepoprawnej nazwy warunku stopu
                raise ValueError("Niepoprawny warunek stopu") # zgłaszamy błąd dla
nieznanego warunku stopu

        return x, max_iter # jeśli warunek stopu nie został spełniony wcześniej,
zwracamy ostatni wynik i maksymalną liczbę iteracji

A = [ # definiujemy macierz współczynników układu ze slajdów
[4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A

```

```

    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

b = [0.0, 2.0, 3.0, -2.0] # definiujemy wektor prawej strony układu
x0 = [0.0, 0.0, 0.0, 0.0] # definiujemy przybliżenie początkowe  $x^{(0)}$ 
x_dokladne = [0.5, 1.0, 2.0, -2.0] # definiujemy rozwiązanie dokładne tego układu

print("\nDane:") # wypisujemy nagłówek danych wejściowych
print("Macierz A:") # wypisujemy napis informujący o macierzy A
for wiersz in A: # przechodzimy po wszystkich wierszach macierzy
    print(wiersz) # wypisujemy każdy wiersz macierzy

print("\nWektor b:") # wypisujemy napis informujący o wektorze b
print(b) # wypisujemy wektor prawej strony

print("\nPrzybliżenie początkowe  $x^{(0)}$ :") # wypisujemy napis informujący o
przybliżeniu początkowym
print(x0) # wypisujemy wektor początkowy

print("\nDokładne rozwiązanie:") # wypisujemy napis informujący o rozwiązaniu
dokładnym
print(x_dokladne) # wypisujemy rozwiązanie dokładne

print("\n----- a) LICZBA ITERACJI -----") #
wypisujemy nagłówek punktu a
wynik_a, iteracje_a = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu opartego na liczbie iteracji
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=10, # ustawiamy liczbę iteracji na 10
    warunek_stopu="iteracje" # wybieramy warunek stopu oparty na liczbie iteracji
)

print("\nWynik końcowy po zadanej liczbie iteracji:") # wypisujemy opis wyniku dla
punktu a
wypisz_wektor(wynik_a) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_a) # wypisujemy liczbę wykonanych iteracji

blad_a = odejmij_wektory(wynik_a, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu a
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_a)) #
wypisujemy normę maksimum błędu

print("\n----- b) WARUNEK ZE SLAJDU -----") #
wypisujemy nagłówek punktu b
wynik_b, iteracje_b = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu zgodnego ze slajdami
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy epsilon zgodnie ze slajdami
    warunek_stopu="roznica" # wybieramy warunek stopu oparty na ilorazie norm
)

```

```

print("\nWynik końcowy:") # wypisujemy opis wyniku dla punktu b
wypisz_wektor(wynik_b) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_b) # wypisujemy liczbę wykonanych iteracji

blad_b = odejmij_wektory(wynik_b, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu b
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_b)) #
wypisujemy normę maksimum błędu

print("\n----- c) BŁĄD UZYSKANEGO PRZYBLIŻENIA -----
") # wypisujemy nagłówek punktu c
wynik_c, iteracje_c = gauss_seidel( # wywołujemy metodę Gaussa-Seidla dla warunku
stopu opartego na błędzie dokładnym
    A, b, x0, # przekazujemy macierz A, wektor b i przybliżenie początkowe
    max_iter=100, # ustawiamy maksymalną liczbę iteracji na 100
    epsilon=1e-3, # ustawiamy epsilon na 10 do potęgi minus 3
    warunek_stopu="blad", # wybieramy warunek stopu oparty na błędzie względem
rozwiązania dokładnego
    rozwiazanie_dokladne=x_dokladne # przekazujemy rozwiązanie dokładne potrzebne
do obliczenia błędu
)

print("\nWynik końcowy:") # wypisujemy opis wyniku dla punktu c
wypisz_wektor(wynik_c) # wypisujemy końcowy wektor przybliżenia
print("Liczba iteracji:", iteracje_c) # wypisujemy liczbę wykonanych iteracji

blad_c = odejmij_wektory(wynik_c, x_dokladne) # obliczamy błąd względem
rozwiązania dokładnego dla punktu c
print("Norma błędu względem rozwiązania dokładnego:", norma_max(blad_c)) #
wypisujemy normę maksimum błędu

```

Jeśli trzeba wyświetlić wszystkie iteracje to zmienić fragment:

```

x[i] = (b[i] - suma1 - suma2) / A[i][i]

```

PYTHON

```

if warunek_stopu == "iteracje":
    if krok == max_iter:
        return x, krok

elif warunek_stopu == "roznica":
    roznica = odejmij_wektory(x, x_stare)
    norma_roznicy = norma_max(roznica)
    norma_biezaca = norma_max(x)

```

na:

```

x[i] = (b[i] - suma1 - suma2) / A[i][i]

roznica = odejmij_wektory(x, x_stare)
norma_roznicy = norma_max(roznica)

print(f"\nIteracja {krok}:")
wypisz_wektor(x)
print("Norma maksimum różnicy:", norma_roznicy)

if warunek_stopu == "iteracje":
    if krok == max_iter:
        return x, krok

elif warunek_stopu == "roznica":
    norma_biezaca = norma_max(x)

    if norma_biezaca == 0:
        if norma_roznicy <= epsilon:

```

Zadanie 3

Sprawdź zbieżność układu pod kątem metod z zadania 1 oraz 2.

Układ do testowania:

$$\begin{cases} 4x_1 - 2x_2 = 0, \\ -2x_1 + 5x_2 - x_3 = 2, \\ -x_2 + 4x_3 + 2x_4 = 3, \\ 2x_3 + 3x_4 = -2. \end{cases}$$

Zapis macierzowy układu

Macierz współczynników:

$$A = \begin{bmatrix} 4 & -2 & 0 & 0 \\ -2 & 5 & -1 & 0 \\ 0 & -1 & 4 & 2 \\ 0 & 0 & 2 & 3 \end{bmatrix}$$

Wektor prawej strony:

$$b = \begin{bmatrix} 0 \\ 2 \\ 3 \\ -2 \end{bmatrix}$$

Dokładne rozwiązanie tego układu jest równe:

$$x^* = \begin{bmatrix} 0.5 \\ 1 \\ 2 \\ -2 \end{bmatrix}$$

Co sprawdzamy w tym zadaniu?

W zadaniu 1 używaliśmy **metody Jacobiego**, a w zadaniu 2 **metody Gaussa-Seidla**. Teraz nie liczymy kolejnych iteracji, tylko sprawdzamy, **czy te metody powinny być zbieżne** dla danego układu.

Na slajdach pojawia się ogólna postać metody iteracyjnej:

$$x^{(k)} = Wx^{(k-1)} + Z$$

Najważniejsze pytanie brzmi:

czy kolejne iteracje prowadzą do rozwiązania?

Według slajdów zależy to od macierzy iteracji (W).

W praktyce używamy warunku:

$$\|W\| < 1$$

W tej notatce korzystamy z **normy wierszowej**:

$$\|W\|_{\infty} = \max_i \sum_{j=1}^n |w_{ij}|$$

Jeżeli:

$$\|W\|_{\infty} < 1$$

to metoda powinna być zbieżna.

1. Metoda Jacobiego

Dla metody Jacobiego macierz iteracji wyznaczamy dokładnie z wzoru ze slajdu:

$$W_{ij} = \begin{cases} 0 & \text{gdy } i = j, \\ -\frac{a_{ij}}{a_{ii}} & \text{gdy } i \neq j \end{cases}$$

Dla naszego układu otrzymujemy:

$$W_J = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 \\ \frac{2}{5} & 0 & \frac{1}{5} & 0 \\ 0 & \frac{1}{4} & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{2}{3} & 0 \end{bmatrix}$$

Teraz liczymy normę wierszową:

- wiersz 1:

$$0 + \frac{1}{2} + 0 + 0 = \frac{1}{2}$$

- wiersz 2:

$$\frac{2}{5} + 0 + \frac{1}{5} + 0 = \frac{3}{5}$$

- wiersz 3:

$$0 + \frac{1}{4} + 0 + \frac{1}{2} = \frac{3}{4}$$

- wiersz 4:

$$0 + 0 + \frac{2}{3} + 0 = \frac{2}{3}$$

Zatem:

$$|W_J|_{\infty} = \max \left(\frac{1}{2}, \frac{3}{5}, \frac{3}{4}, \frac{2}{3} \right) = \frac{3}{4} = 0.75$$

Ponieważ:

$$||W_J||_{\infty} < 1$$

to **metoda Jacobiego jest zbieżna**.

2. Metoda Gaussa-Seidla

Dla metody Gaussa-Seidla na slajdach mamy wzór:

$$(L + D)x^{(k)} = -Ux^{(k-1)} + b$$

gdzie:

- L — macierz trójkątna dolna,
- D — macierz diagonalna,
- U — macierz trójkątna górna.

Po przekształceniu dostajemy:

$$x^{(k)} = -(L + D)^{-1}Ux^{(k-1)} + (L + D)^{-1}b$$

Stąd macierz iteracji dla Gaussa-Seidla ma postać:

$$W_{GS} = -(L + D)^{-1}U$$

To jest ważna różnica względem Jacobiego:

- dla Jacobiego mamy prosty wzór element po elemencie,
- dla Gaussa-Seidla trzeba skorzystać z rozkładu macierzy A na części L, D, U .

Dla naszego układu otrzymujemy:

$$W_{GS} = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{5} & \frac{1}{5} & 0 \\ 0 & \frac{1}{20} & \frac{1}{20} & -\frac{1}{2} \\ 0 & -\frac{1}{30} & -\frac{1}{30} & \frac{1}{3} \end{bmatrix}$$

Liczymy normę wierszową:

- wiersz 1:

$$0 + \frac{1}{2} + 0 + 0 = \frac{1}{2}$$

- wiersz 2:

$$0 + \frac{1}{5} + \frac{1}{5} + 0 = \frac{2}{5}$$

- wiersz 3:

$$0 + \frac{1}{20} + \frac{1}{20} + \frac{1}{2} = \frac{3}{5}$$

- wiersz 4:

$$0 + \frac{1}{30} + \frac{1}{30} + \frac{1}{3} = \frac{2}{5}$$

Zatem:

$$|W_{GS}|_{\infty} = \max\left(\frac{1}{2}, \frac{2}{5}, \frac{3}{5}, \frac{2}{5}\right) = \frac{3}{5} = 0.6$$

Ponieważ:

$$|W_{GS}|_{\infty} < 1$$

to **metoda Gaussa-Seidla jest zbieżna**.

Porównanie obu metod

Otrzymaliśmy:

$$|W_J|_{\infty} = 0.75$$

oraz

$$|W_{GS}|_{\infty} = 0.6$$

Obie metody są zbieżne, ale ponieważ:

$$0.6 < 0.75$$

to metoda Gaussa-Seidla powinna zbiegać szybciej niż metoda Jacobiego.
Zgadza się to z wcześniejszymi obserwacjami z zadań 1 i 2.

```
print("-----ZADANIE 3-----") # wypisujemy nagłówek zadania 3

def zeros(n, m): # definiujemy funkcję tworzącą macierz n x m wypełnioną zerami
    macierz = [] # tworzymy pustą listę na całą macierz
    for i in range(n): # wykonujemy pętlę po wszystkich wierszach
        wiersz = [] # tworzymy pustą listę na jeden wiersz
        for j in range(m): # wykonujemy pętlę po wszystkich kolumnach
            wiersz.append(0.0) # dodajemy do wiersza element 0.0
        macierz.append(wiersz) # dodajemy gotowy wiersz do macierzy
    return macierz # zwracamy utworzoną macierz zerową

def wypisz_macierz(macierz): # definiujemy funkcję do wypisywania macierzy
    for wiersz in macierz: # przechodzimy po wszystkich wierszach macierzy
        print(wiersz) # wypisujemy bieżący wiersz

def norma_wierszowa_macierzy(macierz): # definiujemy funkcję liczącą normę
    wierszową macierzy
    maksimum = 0.0 # ustawiamy początkowe maksimum na 0
    for i in range(len(macierz)): # przechodzimy po wszystkich wierszach macierzy
        suma = 0.0 # zerujemy sumę modułów elementów w bieżącym wierszu
        for j in range(len(macierz[i])): # przechodzimy po wszystkich elementach
            # bieżącego wiersza
            suma += abs(macierz[i][j]) # dodajemy moduł bieżącego elementu do
            # sumy
        if suma > maksimum: # sprawdzamy, czy suma z tego wiersza jest większa od
            # dotychczasowego maksimum
            maksimum = suma # aktualizujemy maksimum
    return maksimum # zwracamy największą sumę modułów z wierszy

def macierz_iteracji_jacobiego(A): # definiujemy funkcję wyznaczającą macierz
    iteracji metody Jacobiego
    n = len(A) # zapisujemy rozmiar macierzy A
    W = zeros(n, n) # tworzymy pustą macierz iteracji W o rozmiarze n x n

    for i in range(n): # przechodzimy po wszystkich wierszach macierzy
        for j in range(n): # przechodzimy po wszystkich kolumnach macierzy
            if i == j: # sprawdzamy, czy jesteśmy na przekątnej
                W[i][j] = 0.0 # na przekątnej wpisujemy 0 zgodnie ze wzorem dla
                # Jacobiego
            else: # w przeciwnym wypadku jesteśmy poza przekątną
                W[i][j] = -A[i][j] / A[i][i] # wpisujemy wartość -a_ij / a_ii
                # zgodnie ze wzorem ze slajdu

    return W # zwracamy macierz iteracji Jacobiego

def roziaz_uklad_dolnotrojkatny(LD, b): # definiujemy funkcję rozwiązującą układ
    dolnotrójkątny LDx=b
    n = len(LD) # zapisujemy rozmiar macierzy
    x = [0.0] * n # tworzymy wektor rozwiązania wypełniony zerami
```

```

    for i in range(n): # przechodzimy po kolejnych równaniach od góry do dołu
        suma = 0.0 # zerujemy sumę znanych składników
        for j in range(i): # przechodzimy po wcześniej obliczonych elementach
            suma += LD[i][j] * x[j] # dodajemy składniki z już wyznaczonych
            # wartości
        x[i] = (b[i] - suma) / LD[i][i] # obliczamy bieżący element rozwiązania
            # przez podstawianie w przód

    return x # zwracamy wyznaczony wektor rozwiązania

def macierz_iteracji_gaussa_seidla(A): # definiujemy funkcję wyznaczającą macierz
    # iteracji metody Gaussa-Seidla
    n = len(A) # zapisujemy rozmiar macierzy A

    LD = zeros(n, n) # tworzymy pustą macierz dla części L + D
    U = zeros(n, n) # tworzymy pustą macierz dla części U

    for i in range(n): # przechodzimy po wszystkich wierszach macierzy A
        for j in range(n): # przechodzimy po wszystkich kolumnach macierzy A
            if j <= i: # sprawdzamy, czy element należy do dolnej części wraz z
                # przekątną
                LD[i][j] = A[i][j] # przepisujemy element do macierzy LD
            else: # w przeciwnym wypadku element należy do części górnej
                U[i][j] = A[i][j] # przepisujemy element do macierzy U

    W = zeros(n, n) # tworzymy pustą macierz iteracji Gaussa-Seidla

    for kolumna in range(n): # przechodzimy po kolejnych kolumnach macierzy W
        prawa_strona = [] # tworzymy pustą listę na prawą stronę układu
        # pomocniczego
        for i in range(n): # przechodzimy po wszystkich wierszach
            prawa_strona.append(-U[i][kolumna]) # budujemy prawą stronę jako
            # przeciwną kolumnę macierzy U

        rozwiazanie = rozwarz_uklad_dolnotrojkatny(LD, prawa_strona) #
        # rozwiązujemy układ (L+D)x = -u_k dla bieżącej kolumny

        for i in range(n): # przechodzimy po wszystkich wierszach rozwiązania
            W[i][kolumna] = rozwiazanie[i] # wpisujemy obliczoną kolumnę do
            # macierzy iteracji W

    return W # zwracamy macierz iteracji Gaussa-Seidla

A = [ # definiujemy macierz współczynników układu
    [4.0, -2.0, 0.0, 0.0], # pierwszy wiersz macierzy A
    [-2.0, 5.0, -1.0, 0.0], # drugi wiersz macierzy A
    [0.0, -1.0, 4.0, 2.0], # trzeci wiersz macierzy A
    [0.0, 0.0, 2.0, 3.0] # czwarty wiersz macierzy A
]

print("\nMacierz A:") # wypisujemy nagłówek dla macierzy A

```

```

wypisz_macierz(A) # wypisujemy macierz A

print("\n----- METODA JACOBIEGO -----") # wypisujemy
nagłówek sekcji dla metody Jacobiego
WJ = macierz_iteracji_jacobiego(A) # wyznaczamy macierz iteracji Jacobiego
print("Macierz iteracyjna W_J:") # wypisujemy nagłówek dla macierzy W_J
wypisz_macierz(WJ) # wypisujemy macierz iteracji Jacobiego

norma_WJ = norma_wierszowa_macierzy(WJ) # obliczamy normę wierszową macierzy
iteracji Jacobiego
print("Norma wierszowa ||W_J|| =", norma_WJ) # wypisujemy wartość normy macierzy
W_J

if norma_WJ < 1: # sprawdzamy warunek zbieżności dla Jacobiego
    print("Metoda Jacobiego jest zbieżna, ponieważ ||W_J|| < 1.") # wypisujemy
pozytywny wniosek
else: # w przeciwnym przypadku
    print("Metoda Jacobiego może nie być zbieżna, ponieważ ||W_J|| >= 1.") #
wypisujemy negatywny wniosek

print("\n----- METODA GAUSSA-SEIDLA -----") #
wypisujemy nagłówek sekcji dla metody Gaussa-Seidla
WGS = macierz_iteracji_gaussa_seidla(A) # wyznaczamy macierz iteracji Gaussa-
Seidla
print("Macierz iteracyjna W_GS:") # wypisujemy nagłówek dla macierzy W_GS
wypisz_macierz(WGS) # wypisujemy macierz iteracji Gaussa-Seidla

norma_WGS = norma_wierszowa_macierzy(WGS) # obliczamy normę wierszową macierzy
iteracji Gaussa-Seidla
print("Norma wierszowa ||W_GS|| =", norma_WGS) # wypisujemy wartość normy macierzy
W_GS

if norma_WGS < 1: # sprawdzamy warunek zbieżności dla Gaussa-Seidla
    print("Metoda Gaussa-Seidla jest zbieżna, ponieważ ||W_GS|| < 1.") #
wypisujemy pozytywny wniosek
else: # w przeciwnym przypadku
    print("Metoda Gaussa-Seidla może nie być zbieżna, ponieważ ||W_GS|| >= 1.") #
wypisujemy negatywny wniosek

print("\n----- WNIOSEK KOŃCOWY -----") # wypisujemy
nagłówek końcowego wniosku
print("Zbieżność metod z zadania 1 i 2 badamy przez normę macierzy iteracji.") #
wypisujemy ogólną informację o sposobie badania zbieżności
print("Dla Jacobiego sprawdzamy macierz W_J.") # wypisujemy informację dotyczącą
metody Jacobiego
print("Dla Gaussa-Seidla sprawdzamy macierz W_GS.") # wypisujemy informację
dotyczącą metody Gaussa-Seidla
print("Jeżeli ||W|| < 1, to metoda jest zbieżna.") # wypisujemy końcowe kryterium
zbieżności

```

Wnioski

W zadaniu 3 zbieżność badamy zgodnie ze slajdami przez macierz iteracji (W) i warunek:

$$|W| < 1$$

Dla badanego układu:

- dla metody Jacobiego:

$$|W_J|_{\infty} = 0.75 < 1$$

- dla metody Gaussa-Seidla:

$$|W_{GS}|_{\infty} = 0.6 < 1$$

Oznacza to, że:

- metoda Jacobiego jest zbieżna,
 - metoda Gaussa-Seidla jest zbieżna,
 - metoda Gaussa-Seidla powinna zbiegać szybciej niż metoda Jacobiego.
-

Lab 7

Zadanie 1

Napisz program implementujący metodę bisekcji. Program przetestuj dla następujących funkcji:

a)

$$f(x) = x^2 - 4, x \in < 0; 2.2 >$$

b)

$$f(x) = \sin x - \frac{1}{2}, x \in < 0; 2.2 >$$

Co to jest metoda bisekcji?

Metoda bisekcji służy do wyznaczania miejsca zerowego funkcji w przedziale $[a, b]$, w którym funkcja zmienia znak.

Zakładamy więc, że:

$$f(a) \cdot f(b) < 0$$

co oznacza, że w przedziale $[a, b]$ znajduje się co najmniej jedno miejsce zerowe.

Metoda polega na wielokrotnym dzieleniu przedziału na połowy i wybieraniu tej części, w której nadal występuje zmiana znaku.

Obliczanie kolejnych przybliżeń

W prezentacji przyjęto, że:

- początkowo mamy dwa końce przedziału izolacji pierwiastka,
- dla kolejnych kroków iteracji $i = 3, 4, \dots$ nowe przybliżenie wyznaczamy jako średnią dwóch wybranych punktów.

Nową wartość obliczamy ze wzoru:

$$x_i = \frac{x_{i-1} + x_k}{2}$$

gdzie k jest jedną z wartości $\{i-3, i-2\}$, a wybór k jest taki, aby spełnione były warunki:

$$|x_i - x_{i-1}| = |x_i - x_k|$$

oraz

$$f(x_{i-1}) \cdot f(x_k) < 0$$

co wskazuje na obecność pierwiastka w tym przedziale.

W praktyce oznacza to, że w każdym kroku bierzemy **punkt środkowy aktualnego przedziału**:

$$x_i = \frac{a + b}{2}$$

albo równoważnie, zgodnie z praktyczną uwagą ze slajdów:

$$x_i = a + \frac{b - a}{2}$$

Ten drugi zapis jest wygodniejszy numerycznie.

Jak działa metoda krok po kroku?

1. Wybieramy przedział $[a, b]$, w którym funkcja zmienia znak.
2. Obliczamy punkt środkowy:

$$x = \frac{a + b}{2}$$

3. Sprawdzamy znak funkcji w punkcie środkowym.
4. Jeśli znak funkcji w a i w x jest różny, to nowym przedziałem jest $[a, x]$.
5. W przeciwnym razie nowym przedziałem jest $[x, b]$.
6. Powtarzamy kroki aż do spełnienia wybranego warunku stopu.

W każdej iteracji przedział izolacji pierwiastka zmniejsza się o połowę, więc z każdym krokiem przybliżenie staje się dokładniejsze.

Kryteria zakończenia iteracji

W prezentacji podano trzy możliwe kryteria zakończenia obliczeń metodą bisekcji:

1. Zadana liczba kroków — iteracji

Program kończy działanie po wykonaniu określonej liczby iteracji.

2. Dostatecznie mały błąd

Program kończy działanie, gdy oszacowanie błędu przybliżenia jest wystarczająco małe.

Na slajdzie błąd zapisano jako:

$$|x_i - x^*| < \frac{b - a}{2^{i-2}}$$

co pokazuje, że wraz z kolejnymi iteracjami przybliżenie jest coraz dokładniejsze.

3. Wartość funkcji dostatecznie bliska zeru

Program kończy działanie, gdy:

$$|f(x_i)| < \varepsilon$$

czyli gdy wartość funkcji w aktualnym punkcie jest już bardzo bliska zeru.

Wyznaczanie błędu metody bisekcji

Na slajdach dokładność przybliżenia w i -tym kroku określono wzorem:

$$|x_i - x^*| < \frac{b - a}{2^{i-2}}$$

Oznacza to, że błąd maleje wraz z liczbą iteracji.

Na przykład po 12 krokach:

$$|x_{12} - x^*| < \frac{b - a}{2^{12-2}}$$

Praktyczne uwagi

W prezentacji podano też kilka praktycznych wskazówek:

- metoda bisekcji daje **jedno miejsce zerowe**, a nie wszystkie miejsca zerowe w całym przedziale,

- przez błędy zaokrągleń otrzymanie dokładnie ($f(x)=0$) jest mało prawdopodobne, więc nie powinno to być jedyne kryterium zakończenia,
- punkt środkowy lepiej liczyć ze wzoru

$$a + \frac{b-a}{2}$$

niż

$$\frac{a+b}{2}$$

- zmianę znaku wygodnie sprawdzać przez porównanie znaków:

$$\text{sgn}(f(x_i)) \neq \text{sgn}(f(x_j))$$

zamiast przez mnożenie:

$$f(x_i) \cdot f(x_j) < 0$$

Program

PYTHON

```
import math # importujemy moduł math, ponieważ będzie potrzebny do funkcji sin

def sgn(x): # definiujemy funkcję zwracającą znak liczby
    if x > 0: # sprawdzamy, czy liczba jest dodatnia
        return 1 # jeśli tak, zwracamy 1
    elif x < 0: # sprawdzamy, czy liczba jest ujemna
        return -1 # jeśli tak, zwracamy -1
    else: # w przeciwnym razie liczba jest równa zero
        return 0 # zwracamy 0

def bisekcja(f, a, b, max_iter=100, epsilon=1e-3, warunek_stopu="iteracje"): #
    definiujemy funkcję realizującą metodę bisekcji
    if f(b) * f(a) >= 0: # sprawdzamy, czy na końcach przedziału nie ma takiego
        samego znaku
            raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne
            znaki.") # jeśli znak się nie zmienia, zgłaszamy błąd

    a0 = a # zapamiętujemy początkowy lewy koniec przedziału do wzoru na błąd
    b0 = b # zapamiętujemy początkowy prawy koniec przedziału do wzoru na błąd
    historia = [] # tworzymy pustą listę, w której będziemy zapisywać kolejne
    kroki metody

    x1 = a # zgodnie z numeracją ze slajdów przyjmujemy pierwszy punkt jako lewy
    koniec przedziału
    x2 = b # zgodnie z numeracją ze slajdów przyjmujemy drugi punkt jako prawy
    koniec przedziału

    historia.append([1, a, b, x1, f(x1), None]) # zapisujemy do historii krok 1
    historia.append([2, a, b, x2, f(x2), None]) # zapisujemy do historii krok 2

    if warunek_stopu == "iteracje" and max_iter == 1: # sprawdzamy, czy
        użytkownik chce zakończyć po pierwszym kroku
            return x1, historia # zwracamy pierwszy punkt i historię
    if warunek_stopu == "iteracje" and max_iter == 2: # sprawdzamy, czy
        użytkownik chce zakończyć po drugim kroku
            return x2, historia # zwracamy drugi punkt i historię

    for i in range(3, max_iter + 1): # wykonujemy kolejne kroki od i=3 zgodnie z
        numeracją ze slajdów
            miejsce_zerowe = a + ((b - a) / 2.0) # obliczamy punkt środkowy
            przedziału w postaci zalecanej na slajdach

            fa = f(a) # obliczamy wartość funkcji w lewym końcu przedziału
            fc = f(miejsce_zerowe) # obliczamy wartość funkcji w punkcie środkowym

            blad = (b0 - a0) / (2 ** (i - 2)) # obliczamy oszacowanie błędu dokładnie
            według wzoru ze slajdu

            historia.append([i, a, b, miejsce_zerowe, fc, blad]) # zapisujemy
            aktualny krok do historii
```

```

        if warunek_stopu == "iteracje": # sprawdzamy, czy wybrano warunek stopu
oparty na liczbie iteracji
            if i == max_iter: # jeśli osiągnięto zadaną liczbę iteracji
                return miejsce_zerowe, historia # zwracamy aktualne przybliżenie i
historię

            elif warunek_stopu == "blad": # sprawdzamy, czy wybrano warunek stopu
oparty na błędzie
                if blad < epsilon: # jeśli oszacowanie błędu jest mniejsze od zadanej
tolerancji
                    return miejsce_zerowe, historia # zwracamy aktualne przybliżenie i
historię

            elif warunek_stopu == "wartosc": # sprawdzamy, czy wybrano warunek stopu
oparty na wartości funkcji
                if abs(fc) < epsilon: # jeśli wartość funkcji jest dostatecznie
bliska zeru
                    return miejsce_zerowe, historia # zwracamy aktualne przybliżenie i
historię

            else: # jeśli podano niepoprawny napis określający warunek stopu
                raise ValueError("Niepoprawny warunek stopu.") # zgłaszamy błąd

        if fc == 0: # sprawdzamy, czy udało się trafić dokładnie w miejsce zerowe
            return miejsce_zerowe, historia # jeśli tak, od razu kończymy
działanie

        if sgn(fa) != sgn(fc): # sprawdzamy zgodnie ze slajdem, czy między a i
punktem środkowym następuje zmiana znaku
            b = miejsce_zerowe # jeśli tak, nowym prawym końcem przedziału staje
się punkt środkowy
        else: # w przeciwnym razie pierwiastek znajduje się w drugiej połowie
przedziału
            a = miejsce_zerowe # nowym lewym końcem przedziału staje się punkt
środkowy

        return miejsce_zerowe, historia # jeśli pętla się zakończy, zwracamy ostatnie
przybliżenie i historię

def wypisz_historie(historia): # definiujemy funkcję wypisującą historię
działania metody
    print("i          a          b          x          f(x)          blad") #
wypisujemy nagłówek tabeli
    for krok in historia: # przechodzimy po wszystkich zapisanych krokach
        print( # wypisujemy pojedynczy wiersz historii
            f"{krok[0]} " # wypisujemy numer iteracji
            f"{krok[1]} " # wypisujemy lewy koniec przedziału
            f"{krok[2]} " # wypisujemy prawy koniec przedziału
            f"{krok[3]} " # wypisujemy aktualne przybliżenie
            f"{krok[4]} " # wypisujemy wartość funkcji w punkcie przybliżonym
            f"{krok[5]}" # wypisujemy oszacowanie błędu
        ) # kończymy wypisywanie jednego wiersza

```

```

def f1(x): # definiujemy pierwszą funkcję testową
    return x**2 - 4 # zwracamy wartość funkcji  $x^2 - 4$ 

def f2(x): # definiujemy drugą funkcję testową
    return math.sin(x) - 0.5 # zwracamy wartość funkcji  $\sin(x) - 1/2$ 

print("----- ZADANIE 1 -----") # wypisujemy nagłówek zadania

print("\n===== FUNKCJA a) =====") # wypisujemy nagłówek dla funkcji a)
print("f(x) =  $x^2 - 4$ , przedział [0, 2.2]") # wypisujemy opis funkcji i przedziału

wynik_a_iter, historia_a_iter = bisekcja(f1, 0.0, 2.2, max_iter=12,
warunek_stopu="iteracje") # uruchamiamy metodę bisekcji dla funkcji a) z warunkiem liczby iteracji
print("\nWarunek stopu: liczba iteracji") # wypisujemy nazwę wybranego warunku stopu
wypisz_historie(historia_a_iter) # wypisujemy pełną historię iteracji
print("Przybliżony pierwiastek:", wynik_a_iter) # wypisujemy wyznaczone przybliżenie miejsca zerowego
print("f(x) =", f1(wynik_a_iter)) # wypisujemy wartość funkcji w znalezionym punkcie

wynik_a_blad, historia_a_blad = bisekcja(f1, 0.0, 2.2, epsilon=1e-3,
warunek_stopu="blad") # uruchamiamy metodę bisekcji dla funkcji a) z warunkiem małego błędu
print("\nWarunek stopu: dostatecznie mały błąd") # wypisujemy nazwę wybranego warunku stopu
print("Przybliżony pierwiastek:", wynik_a_blad) # wypisujemy wyznaczone przybliżenie miejsca zerowego
print("f(x) =", f1(wynik_a_blad)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_a_blad)) # wypisujemy liczbę zapisanych kroków

wynik_a_wartosc, historia_a_wartosc = bisekcja(f1, 0.0, 2.2, epsilon=1e-3,
warunek_stopu="wartosc") # uruchamiamy metodę bisekcji dla funkcji a) z warunkiem małej wartości funkcji
print("\nWarunek stopu: wartość funkcji bliska zeru") # wypisujemy nazwę wybranego warunku stopu
print("Przybliżony pierwiastek:", wynik_a_wartosc) # wypisujemy wyznaczone przybliżenie miejsca zerowego
print("f(x) =", f1(wynik_a_wartosc)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_a_wartosc)) # wypisujemy liczbę zapisanych kroków

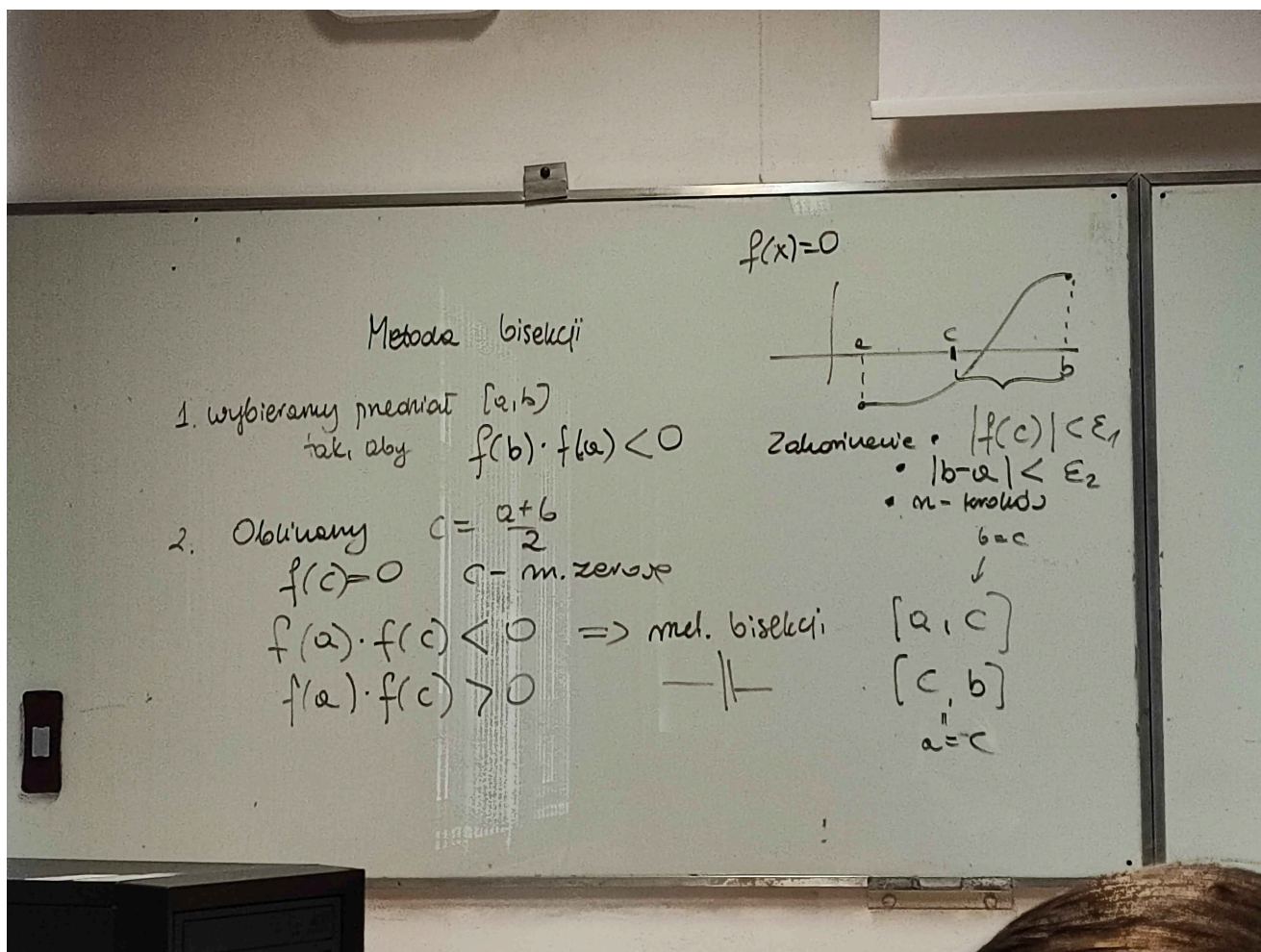
print("\n===== FUNKCJA b) =====") # wypisujemy nagłówek dla funkcji b)
print("f(x) =  $\sin(x) - 1/2$ , przedział [0, 2.2]") # wypisujemy opis funkcji i przedziału

```

```
wynik_b_iter, historia_b_iter = bisekcja(f2, 0.0, 2.2, max_iter=12,
warunek_stopu="iteracje") # uruchamiamy metodę bisekcji dla funkcji b) z warunkiem
liczby iteracji
print("\nWarunek stopu: liczba iteracji") # wypisujemy nazwę wybranego warunku
stopu
wypisz_historie(historia_b_iter) # wypisujemy pełną historię iteracji
print("Przybliżony pierwiastek:", wynik_b_iter) # wypisujemy wyznaczone
przybliżenie miejsca zerowego
print("f(x) =", f2(wynik_b_iter)) # wypisujemy wartość funkcji w znalezionym
punkcie

wynik_b_blad, historia_b_blad = bisekcja(f2, 0.0, 2.2, epsilon=1e-3,
warunek_stopu="blad") # uruchamiamy metodę bisekcji dla funkcji b) z warunkiem
małego błędu
print("\nWarunek stopu: dostatecznie mały błąd") # wypisujemy nazwę wybranego
warunku stopu
print("Przybliżony pierwiastek:", wynik_b_blad) # wypisujemy wyznaczone
przybliżenie miejsca zerowego
print("f(x) =", f2(wynik_b_blad)) # wypisujemy wartość funkcji w znalezionym
punkcie
print("Liczba iteracji:", len(historia_b_blad)) # wypisujemy liczbę zapisanych
kroków

wynik_b_wartosc, historia_b_wartosc = bisekcja(f2, 0.0, 2.2, epsilon=1e-3,
warunek_stopu="wartosc") # uruchamiamy metodę bisekcji dla funkcji b) z warunkiem
małej wartości funkcji
print("\nWarunek stopu: wartość funkcji bliska zeru") # wypisujemy nazwę wybranego
warunku stopu
print("Przybliżony pierwiastek:", wynik_b_wartosc) # wypisujemy wyznaczone
przybliżenie miejsca zerowego
print("f(x) =", f2(wynik_b_wartosc)) # wypisujemy wartość funkcji w znalezionym
punkcie
print("Liczba iteracji:", len(historia_b_wartosc)) # wypisujemy liczbę zapisanych
kroków
```



Wnioski

Metoda bisekcji polega na wielokrotnym dzieleniu przedziału na połowy i wybieraniu tej części, w której funkcja zmienia znak. Dzięki temu kolejne przybliżenia coraz bardziej zbliżają się do miejsca zerowego.

W programie zastosowano trzy warunki zakończenia obliczeń zgodne ze slajdami:

- zadana liczba iteracji,
- dostatecznie mały błąd,
- wartość funkcji dostatecznie bliska zero.

Dla funkcji:

$$f(x) = x^2 - 4$$

w przedziale $[0, 2.2]$ metoda prowadzi do pierwiastka bliskiego wartości:

$$x = 2$$

Dla funkcji:

$$f(x) = \sin x - \frac{1}{2}$$

w przedziale $[0, 2.2]$ metoda prowadzi do pierwiastka bliskiego wartości:

$$x \approx 0.5236$$

czyli do wartości:

$$x = \frac{\pi}{6}$$

Zadanie 2

Napisz program implementujący metodę Newtona (czyli metodę stycznych).
Program przetestuj dla funkcji z zadania 1.

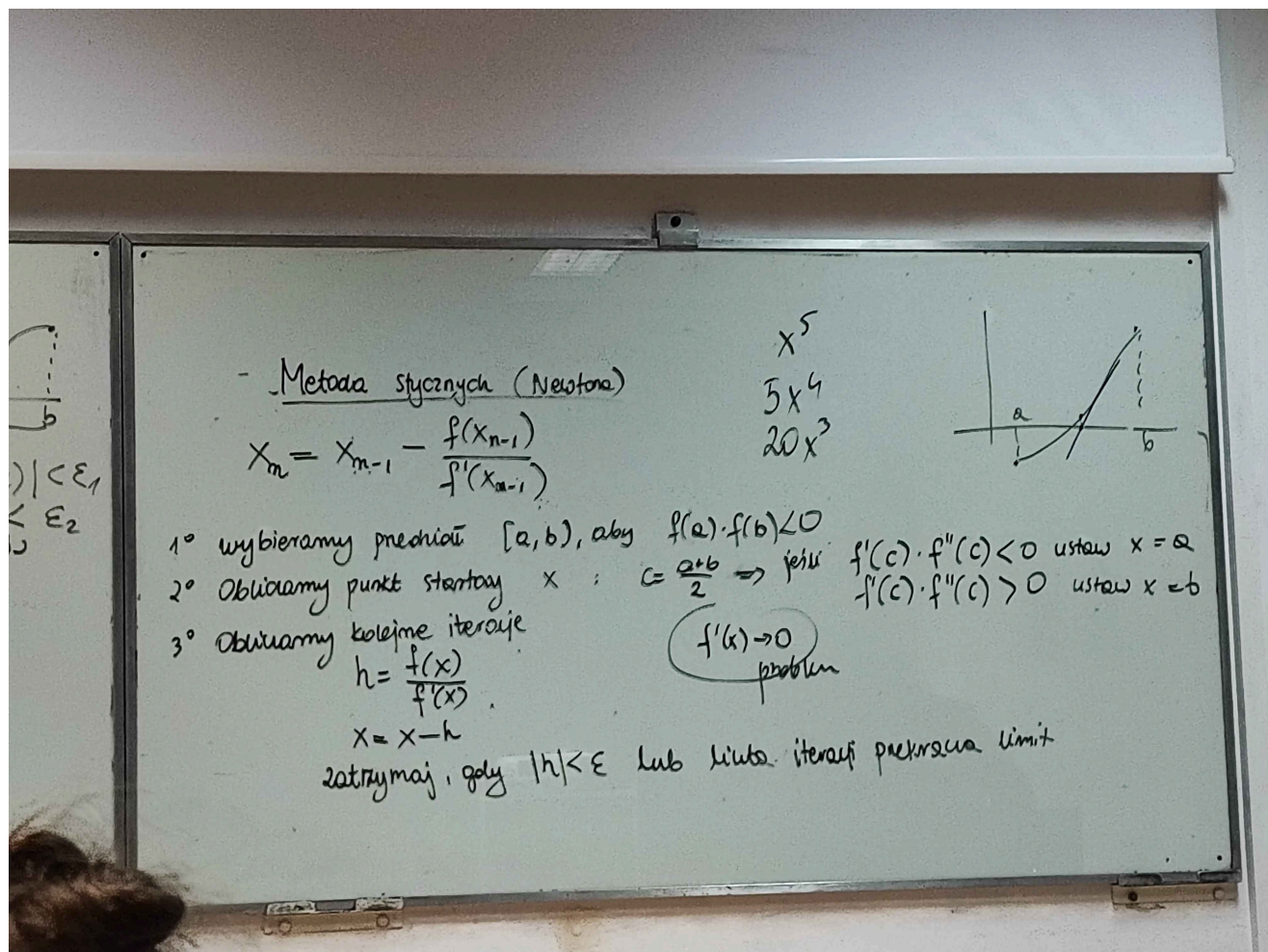
Rozważane funkcje:

a)

$$f(x) = x^2 - 4, \quad x \in < 0, 2.2 >$$

b)

$$f(x) = \sin x - \frac{1}{2}, \quad x \in < 0, 2.2 >$$



Co to jest metoda Newtona?

Metoda stycznych, nazywana też metodą Newtona, służy do przybliżonego wyznaczania miejsca zerowego funkcji $f(x)$, czyli rozwiązania równania:

$$f(x) = 0$$

Metoda polega na konstruowaniu kolejnych punktów będących miejscami zerowymi stycznych do wykresu funkcji.

Na slajdach wzór iteracyjny zapisano jako:

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}, \quad i = 2, 3, 4, \dots$$

Oznacza to, że mając poprzednie przybliżenie x_{i-1} , obliczamy nowe przybliżenie x_i korzystając z wartości funkcji i jej pochodnej w punkcie x_{i-1} .

Jak rozumieć wzór metody Newtona?

W każdym kroku obliczamy poprawkę:

$$h = \frac{f(x)}{f'(x)}$$

a następnie nowe przybliżenie:

$$x = x - h$$

Jeżeli punkt x jest blisko rzeczywistego pierwiastka, to metoda zwykle bardzo szybko poprawia wynik.

Wybór przedziału startowego

Zgodnie z tablicą najpierw wybieramy przedział:

$$[a, b]$$

taki, że:

$$f(a) \cdot f(b) < 0$$

To oznacza, że na końcach przedziału funkcja ma przeciwne znaki, więc w przedziale znajduje się miejsce zerowe.

Wybór pierwszego przybliżenia

Na slajdach podano, że pierwsze przybliżenie x_1 często wybieramy z końców przedziału $[a, b]$. Kryterium wyboru zależy od znaku iloczynu:

$$f'(x) \cdot f''(x)$$

Dla $x \in [a, b]$:

- jeżeli

$$f'(x) \cdot f''(x) < 0$$

to wybieramy:

$$x_1 = a$$

- jeżeli

$$f'(x) \cdot f''(x) > 0$$

to wybieramy:

$$x_1 = b$$

Na tablicy do praktycznego sprawdzenia tego warunku przyjęto punkt:

$$c = \frac{a+b}{2}$$

i badano znak iloczynu:

$$f'(c) \cdot f''(c)$$

W programie zastosowano właśnie ten sposób.

Kolejne iteracje

Po wybraniu punktu startowego obliczamy kolejne kroki metodą:

$$h = \frac{f(x)}{f'(x)}$$

$$x = x - h$$

Jest to ta sama metoda co wzór ze slajdów:

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

tylko zapisana w wygodniejszej postaci programistycznej.

Oszacowanie błędu

Na slajdach zapisano, że błąd przybliżenia można oszacować przez różnicę kolejnych przybliżeń:

$$\Delta \approx |x_i - x_{i-1}|$$

W programie ta różnica jest reprezentowana przez wartość:

$$|h|$$

ponieważ:

$$x_i = x_{i-1} - h$$

czyli:

$$|x_i - x_{i-1}| = |h|$$

Warunek zakończenia obliczeń

Na slajdach podano warunek stopu:

$$|x_i - x_{i-1}| < \varepsilon$$

W programie sprawdzamy równoważnie:

$$|h| < \varepsilon$$

Obliczenia przerywamy również wtedy, gdy liczba iteracji przekroczy zadany limit.

Potencjalne problemy

Na slajdach oraz tablicy zaznaczono, że problem pojawia się wtedy, gdy:

$$f'(x) = 0$$

Wtedy nie można wykonać kroku Newtona, ponieważ wystąpiłoby dzielenie przez zero:

$$\frac{f(x)}{f'(x)}$$

Dlatego program sprawdza ten przypadek i zgłasza błąd.

Dodatkowo metoda może być rozbieżna, jeśli punkt startowy jest źle dobrany lub funkcja nie jest dostatecznie „gładka” w otoczeniu pierwiastka.

Pochodne dla funkcji z zadania

a) Funkcja

$$f(x) = x^2 - 4$$

Pochodna pierwsza:

$$f'(x) = 2x$$

Pochodna druga:

$$f''(x) = 2$$

b) Funkcja

$$f(x) = \sin x - \frac{1}{2}$$

Pochodna pierwsza:

$$f'(x) = \cos x$$

Pochodna druga:

$$f''(x) = -\sin x$$

```
import math # importujemy moduł math, ponieważ będzie potrzebny do funkcji
trygonometrycznych

def newton(f, a, b, df, ddf, max_iter=100, epsilon=1e-3): # definiujemy funkcję
    realizującą metodę Newtona
    lista_iteracji = [] # tworzymy pustą listę, w której będziemy zapisywać
    kolejne iteracje

    if f(a) * f(b) >= 0: # sprawdzamy warunek z tablicy, że na krańcach
        przedziału funkcja ma mieć przeciwne znaki
        raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne
        znaki.") # jeśli warunek nie jest spełniony, zgłaszamy błąd

    c = (a+b) / 2.0 # obliczamy środek przedziału zgodnie z pomysłem z tablicy
    iloczyn_pochodnych = df(c) * ddf(c) # obliczamy iloczyn pochodnej pierwszej i
    drugiej w punkcie c

    x = 0.0 # tworzymy zmienną na punkt startowy

    if iloczyn_pochodnych < 0: # sprawdzamy pierwszy przypadek ze slajdu
        x = a # jeśli iloczyn jest ujemny, wybieramy lewy koniec przedziału jako
        pierwsze przybliżenie
    elif iloczyn_pochodnych > 0: # sprawdzamy drugi przypadek ze slajdu
        x = b # jeśli iloczyn jest dodatni, wybieramy prawy koniec przedziału jako
        pierwsze przybliżenie
    else: # obsługujemy przypadek, w którym iloczyn pochodnych jest równy zero
        raise ValueError("Nie można jednoznacznie wybrać punktu startowego, bo
        f'(c) * f''(c) = 0.") # zgłaszamy błąd, bo reguła wyboru nie rozstrzyga

    punkt_startowy = x # zapamiętujemy wybrany punkt startowy do późniejszego
    wypisania

    for i in range(1, max_iter+1): # wykonujemy kolejne iteracje od 1 do
    ustalonego limitu
        fx = f(x) # obliczamy wartość funkcji w aktualnym punkcie
        dfx = df(x) # obliczamy wartość pochodnej pierwszej w aktualnym punkcie

        if dfx == 0: # sprawdzamy przypadek problematyczny zaznaczony na tablicy i
        slajdach
            raise ValueError("Pochodna f'(x) = 0, metoda Newtona nie może wykonać
            kolejnego kroku.") # jeśli pochodna jest zerowa, nie da się obliczyć następnego
            kroku

        h = fx / dfx # obliczamy poprawkę h zgodnie ze wzorem z tablicy
        x_nowe = x - h # obliczamy nowe przybliżenie według wzoru Newtona

        lista_iteracji.append([i, x, fx, dfx, h, x_nowe]) # zapisujemy bieżącą
        iterację do historii

        if abs(h) < epsilon: # sprawdzamy warunek stopu zgodny ze slajdem |x_i -
```

```

x_{i-1}| < epsilon
    return punkt_startowy, x_nowe, lista_iteracji # jeśli poprawka jest
wystarczająco mała, kończymy obliczenia i zwracamy wynik

    x = x_nowe # aktualizujemy punkt, żeby przejść do następnej iteracji

    return punkt_startowy, x, lista_iteracji # jeśli osiągnięto limit iteracji,
zwracamy ostatnie przybliżenie i historię

def wypisz_historie(historia): # definiujemy funkcję wypisującą zapisane iteracje
    print("i          x          f(x)          f'(x)          h
x_nowe") # wypisujemy nagłówek tabeli
    for krok in historia: # przechodzimy po wszystkich iteracjach zapisanych w
historii
        print( # wypisujemy jeden wiersz tabeli
            f"{krok[0]:<2} " # wypisujemy numer iteracji
            f"{krok[1]:>14.10f} " # wypisujemy aktualne przybliżenie x
            f"{krok[2]:>14.10f} " # wypisujemy wartość funkcji f(x)
            f"{krok[3]:>14.10f} " # wypisujemy wartość pochodnej f'(x)
            f"{krok[4]:>14.10f} " # wypisujemy poprawkę h
            f"{krok[5]:>14.10f}" # wypisujemy nowe przybliżenie x_nowe
        ) # kończymy wypisywanie jednego wiersza

def f1(x): # definiujemy funkcję z punktu a)
    return x**2 - 4 # zwracamy wartość funkcji x^2 - 4

def df1(x): # definiujemy pochodną pierwszą funkcji z punktu a)
    return 2*x # zwracamy wartość pochodnej 2x

def ddf1(x): # definiujemy pochodną drugą funkcji z punktu a)
    return 2 # zwracamy wartość pochodnej drugiej równej 2

def f2(x): # definiujemy funkcję z punktu b)
    return math.sin(x) - 0.5 # zwracamy wartość funkcji sin(x) - 1/2

def df2(x): # definiujemy pochodną pierwszą funkcji z punktu b)
    return math.cos(x) # zwracamy wartość pochodnej cos(x)

def ddf2(x): # definiujemy pochodną drugą funkcji z punktu b)
    return -math.sin(x) # zwracamy wartość pochodnej drugiej -sin(x)

print("----- ZADANIE 2 -----") # wypisujemy nagłówek
zadania

print("\n===== FUNKCJA a) =====") # wypisujemy
nagłówek dla funkcji a)
print("f(x) = x^2 - 4, przedział [0, 2.2]") # wypisujemy opis funkcji a) i
przedziału

punkt_startowy_a, wynik_a, historia_a = newton(f1, 0.0, 2.2, df1, ddf1,
max_iter=100, epsilon=1e-3) # uruchamiamy metodę Newtona dla funkcji a)
print("Punkt startowy x0 =", punkt_startowy_a) # wypisujemy wybrany punkt startowy
wypisz_historie(historia_a) # wypisujemy pełną historię iteracji

```

```

print("Przybliżony pierwiastek:", wynik_a) # wypisujemy końcowe przybliżenie
pierwiastka
print("f(x) =", f1(wynik_a)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_a)) # wypisujemy liczbę wykonanych
iteracji

print("\n===== FUNKCJA b) =====") # wypisujemy
nagłówek dla funkcji b)
print("f(x) = sin(x) - 1/2, przedział [0, 2.2]") # wypisujemy opis funkcji b) i
przedziału

punkt_startowy_b, wynik_b, historia_b = newton(f2, 0.0, 2.2, df2, ddf2,
max_iter=100, epsilon=1e-3) # uruchamiamy metodę Newtona dla funkcji b)
print("Punkt startowy x0 =", punkt_startowy_b) # wypisujemy wybrany punkt startowy
wypisz_historie(historia_b) # wypisujemy pełną historię iteracji
print("Przybliżony pierwiastek:", wynik_b) # wypisujemy końcowe przybliżenie
pierwiastka
print("f(x) =", f2(wynik_b)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_b)) # wypisujemy liczbę wykonanych
iteracji

```

Wnioski

Metoda Newtona wykorzystuje styczne do wykresu funkcji do budowy kolejnych przybliżeń pierwiastka równania ($f(x)=0$). Jej wzór iteracyjny ma postać:

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

W zadaniu punkt startowy wybierany jest na podstawie znaku iloczynu:

$$f'(c) \cdot f''(c)$$

gdzie:

$$c = \frac{a+b}{2}$$

Następnie w każdej iteracji obliczana jest poprawka:

$$h = \frac{f(x)}{f'(x)}$$

i nowe przybliżenie:

$$x = x - h$$

Warunek zakończenia obliczeń oparto na slajdzie:

$$|x_i - x_{i-1}| < \varepsilon$$

czyli w programie równoważnie:

$$|h| < \varepsilon$$

Dla funkcji:

$$f(x) = x^2 - 4$$

metoda prowadzi do pierwiastka bliskiego wartości:

$$x = 2$$

Dla funkcji:

$$f(x) = \sin x - \frac{1}{2}$$

metoda prowadzi do pierwiastka bliskiego wartości:

$$x \approx 0.5236$$

czyli:

$$x = \frac{\pi}{6}$$

Oznacza to, że program działa poprawnie i realizuje metodę Newtona zgodnie z algorytmem przedstawionym na tablicy oraz slajdach.

Zadanie 3

Napisz program implementujący metodę siecznych. Program przetestuj dla funkcji z zadania 1.

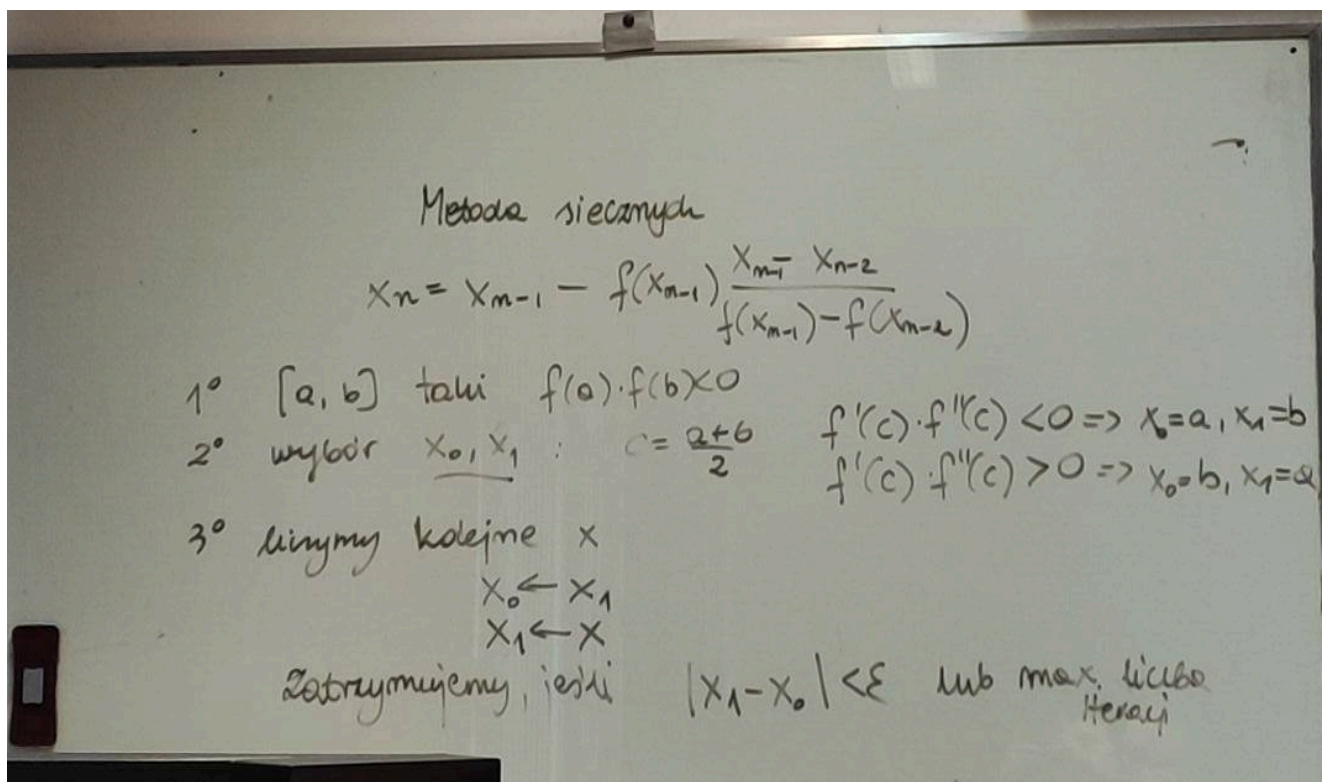
Rozważane funkcje:

a)

$$f(x) = x^2 - 4, \quad x \in \langle 0, 2.2 \rangle$$

b)

$$f(x) = \sin x - \frac{1}{2}, \quad x \in \langle 0, 2.2 \rangle$$



Co to jest metoda siecznych?

Metoda siecznych służy do przybliżania miejsca zerowego równania:

$$f(x) = 0$$

W metodzie tej zamiast stycznej, jak w metodzie Newtona, wykorzystuje się **sieczną** przechodzącą przez dwa kolejne punkty wykresu funkcji:

$$(x_i, f(x_i)) \quad \text{oraz} \quad (x_{i-1}, f(x_{i-1}))$$

Punkt przecięcia tej siecznej z osią x daje kolejne przybliżenie pierwiastka.

Na slajdach wzór metody zapisano jako:

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}, \quad i = 2, 3, 4, \dots$$

Interpretacja wzoru

W każdym kroku korzystamy z dwóch ostatnich przybliżeń:

- x_{i-1} ,
- x_i ,

i na ich podstawie wyznaczamy nowe przybliżenie x_{i+1} .

Metoda nie wymaga liczenia pochodnej funkcji, co jest jej dużą zaletą.

Wybór przedziału i punktów startowych

Na tablicy podano, że najpierw wybieramy przedział:

$$[a, b]$$

taki, że:

$$f(a) \cdot f(b) < 0$$

Następnie wybieramy punkty startowe x_0 i x_1 .

Do ich wyboru wykorzystujemy, podobnie jak w metodzie Newtona, znak iloczynu:

$$f'(c) \cdot f''(c)$$

gdzie:

$$c = \frac{a + b}{2}$$

Zgodnie z tablicą:

- jeżeli

$$f'(c) \cdot f''(c) < 0$$

to przyjmujemy:

$$x_0 = a, \quad x_1 = b$$

- jeżeli

$$f'(c) \cdot f''(c) > 0$$

to przyjmujemy:

$$x_0 = b, \quad x_1 = a$$

Kolejne iteracje

Po wybraniu punktów startowych liczymy kolejne przybliżenia ze wzoru:

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}$$

Następnie przesuwamy punkty:

- stare x_i staje się nowym x_{i-1} ,
- nowe x_{i+1} staje się nowym x_i .

Oszacowanie błędu

Na slajdach błąd przybliżenia oszacowano wzorem:

$$\Delta \approx |x_{i+1} - x_i|$$

W programie właśnie ta różnica jest używana jako miara dokładności.

Warunek stopu

Na tablicy zapisano, że obliczenia kończymy, gdy:

$$|x_1 - x_0| < \varepsilon$$

W praktyce w kolejnych iteracjach sprawdzamy równoważnie:

$$|x_{i+1} - x_i| < \varepsilon$$

czyli różnicę między nowym a poprzednim przybliżeniem.

Drugim warunkiem zakończenia jest przekroczenie maksymalnej liczby iteracji.

Potencjalne problemy

Na slajdach podano, że metoda siecznych:

- nie wymaga pochodnej do wyznaczania kolejnych przybliżeń,
- może nie być zbieżna dla niektórych wyborów punktów startowych,
- zwykle wymaga więcej iteracji niż metoda Newtona,
- nie pilnuje przedziału tak jak metoda bisekcji, więc może wyjść poza zadany przedział.

Dlatego dobór punktów startowych ma duże znaczenie.

W tym zadaniu punkty startowe są wybierane automatycznie na podstawie znaku iloczynu:

$$f'(c) \cdot f''(c), \quad c = \frac{a+b}{2}$$

Trzeba jednak pamiętać, że nawet jeśli początkowo wybieramy punkty z przedziału $([a,b])$, to kolejne przybliżenia metody siecznych nie muszą już pozostać w tym przedziale.

Pochodne potrzebne do wyboru punktów startowych

Mimo że sama metoda siecznych nie używa pochodnych do liczenia kolejnych iteracji, w tym zadaniu są one potrzebne do wyboru punktów startowych zgodnie z tablicą.

a) Funkcja

$$f(x) = x^2 - 4$$

Pochodna pierwsza:

$$f'(x) = 2x$$

Pochodna druga:

$$f''(x) = 2$$

b) Funkcja

$$f(x) = \sin x - \frac{1}{2}$$

Pochodna pierwsza:

$$f'(x) = \cos x$$

Pochodna druga:

$$f''(x) = -\sin x$$

```
import math # importujemy moduł math, ponieważ będzie potrzebny do funkcji
trygonometrycznych

def sieczne(f, a, b, df, ddf, max_iter=100, epsilon=1e-3): # definiujemy funkcję
    realizującą metodę siecznych
    lista_iteracji = [] # tworzymy pustą listę, w której będziemy zapisywać
    kolejne iteracje metody

    if f(a) * f(b) >= 0: # sprawdzamy, czy na krańcach przedziału funkcja ma
        przeciwne znaki
        raise ValueError("Na krańcach przedziału funkcja musi mieć przeciwne
        znaki.") # jeśli nie, zgłaszamy błąd

    c = (a+b) / 2.0 # obliczamy środek przedziału [a,b]
    iloczyn_pochodnych = df(c) * ddf(c) # obliczamy iloczyn pochodnej pierwszej i
    drugiej w punkcie c

    if iloczyn_pochodnych < 0: # sprawdzamy pierwszy przypadek wyboru punktów
        startowych z tablicy
        x0 = a # jeśli iloczyn jest ujemny, pierwszy punkt startowy ustawiamy jako
        a
        x1 = b # jeśli iloczyn jest ujemny, drugi punkt startowy ustawiamy jako b
    elif iloczyn_pochodnych > 0: # sprawdzamy drugi przypadek wyboru punktów
        startowych z tablicy
        x0 = b # jeśli iloczyn jest dodatni, pierwszy punkt startowy ustawiamy
        jako b
        x1 = a # jeśli iloczyn jest dodatni, drugi punkt startowy ustawiamy jako a
    else: # obsługujemy przypadek, gdy iloczyn pochodnych jest równy zero
        raise ValueError("Nie można jednoznacznie wybrać punktu startowego, bo
        f'(c) * f''(c) = 0.") # zgłaszamy błąd

    x0_startowy = x0 # zapamiętujemy początkową wartość x0 do późniejszego
    wypisania
    x1_startowy = x1 # zapamiętujemy początkową wartość x1 do późniejszego
    wypisania

    for i in range(1, max_iter+1): # wykonujemy kolejne iteracje od 1 do
        maksymalnej liczby iteracji
        fx0 = f(x0) # obliczamy wartość funkcji w punkcie x0
        fx1 = f(x1) # obliczamy wartość funkcji w punkcie x1

        if fx1 - fx0 == 0: # sprawdzamy, czy mianownik we wzorze metody siecznych
            nie jest równy zero
            raise ValueError("Mianownik jest równy zero, metoda siecznych nie może
            wykonać kolejnego kroku.") # jeśli jest, zgłaszamy błąd

        x_nowe = x1 - fx1 * ((x1 - x0) / (fx1 - fx0)) # obliczamy nowe
        przybliżenie według wzoru metody siecznych

        lista_iteracji.append([i, x0, x1, fx0, fx1, x_nowe]) # zapisujemy dane z
```

bieżącej iteracji do listy historii

```
    if abs(x_nowe - x1) < epsilon: # sprawdzamy warunek stopu oparty na
różnicy kolejnych przybliżeń
        return x0_startowy, x1_startowy, x0, x1, lista_iteracji # jeśli
warunek jest spełniony, kończymy i zwracamy wyniki
```

```
    x0 = x1 # przesuwamy punkt x0, ustawiając go na poprzednie x1
    x1 = x_nowe # przesuwamy punkt x1, ustawiając go na nowe przybliżenie
```

```
    return x0_startowy, x1_startowy, x0, x1, lista_iteracji # jeśli osiągnięto
limit iteracji, zwracamy ostatnie wyniki
```

```
def wypisz_historie(historia): # definiujemy funkcję do wypisywania historii
iteracji
```

```
    print("i          x0          x1          f(x0)          f(x1)
x_nowe") # wypisujemy nagłówek tabeli
```

```
    for krok in historia: # przechodzimy po wszystkich zapisanych iteracjach
```

```
        print( # wypisujemy jeden wiersz tabeli
            f"{krok[0]:<2} " # wypisujemy numer iteracji
            f"{krok[1]:>14.10f} " # wypisujemy wartość x0
            f"{krok[2]:>14.10f} " # wypisujemy wartość x1
            f"{krok[3]:>14.10f} " # wypisujemy wartość f(x0)
            f"{krok[4]:>14.10f} " # wypisujemy wartość f(x1)
            f"{krok[5]:>14.10f}" # wypisujemy nowe przybliżenie x_nowe
        ) # kończymy wypisywanie pojedynczego wiersza
```

```
def f1(x): # definiujemy pierwszą funkcję testową
    return x**2 - 4 # zwracamy wartość funkcji  $x^2 - 4$ 
```

```
def df1(x): # definiujemy pochodną pierwszą funkcji f1
    return 2*x # zwracamy wartość pochodnej  $2x$ 
```

```
def ddf1(x): # definiujemy pochodną drugą funkcji f1
    return 2 # zwracamy wartość pochodnej drugiej równej 2
```

```
def f2(x): # definiujemy drugą funkcję testową
    return math.sin(x) - 0.5 # zwracamy wartość funkcji  $\sin(x) - 1/2$ 
```

```
def df2(x): # definiujemy pochodną pierwszą funkcji f2
    return math.cos(x) # zwracamy wartość pochodnej  $\cos(x)$ 
```

```
def ddf2(x): # definiujemy pochodną drugą funkcji f2
    return -math.sin(x) # zwracamy wartość pochodnej drugiej  $-\sin(x)$ 
```

```
print("----- ZADANIE 3 -----") # wypisujemy nagłówek
zadania
```

```
print("\n===== FUNKCJA a) =====") # wypisujemy
nagłówek dla funkcji a)
```

```
print("f(x) =  $x^2 - 4$ , przedział [0, 2.2]") # wypisujemy opis pierwszej funkcji i
jej przedziału
```

```

x0a, x1a, wynik0_a, wynik1_a, historia_a = sieczne(f1, 0.0, 2.2, df1, ddf1,
max_iter=100, epsilon=1e-3) # uruchamiamy metodę siecznych dla funkcji a)
print("Punkty startowe: x0 =", x0a, ", x1 =", x1a) # wypisujemy wybrane punkty
startowe
wypisz_historie(historia_a) # wypisujemy historię iteracji dla funkcji a)
print("Ostatnie przybliżenia:", wynik0_a, wynik1_a) # wypisujemy dwa ostatnie
przybliżenia
print("Przybliżony pierwiastek:", wynik1_a) # wypisujemy końcowe przybliżenie
miejsca zerowego
print("f(x) =", f1(wynik1_a)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_a)) # wypisujemy liczbę wykonanych
iteracji

print("\n===== FUNKCJA b) =====") # wypisujemy
nagłówki dla funkcji b)
print("f(x) = sin(x) - 1/2, przedział [0, 2.2]") # wypisujemy opis drugiej funkcji
i jej przedziału

x0b, x1b, wynik0_b, wynik1_b, historia_b = sieczne(f2, 0.0, 2.2, df2, ddf2,
max_iter=100, epsilon=1e-3) # uruchamiamy metodę siecznych dla funkcji b)
print("Punkty startowe: x0 =", x0b, ", x1 =", x1b) # wypisujemy wybrane punkty
startowe
wypisz_historie(historia_b) # wypisujemy historię iteracji dla funkcji b)
print("Ostatnie przybliżenia:", wynik0_b, wynik1_b) # wypisujemy dwa ostatnie
przybliżenia
print("Przybliżony pierwiastek:", wynik1_b) # wypisujemy końcowe przybliżenie
miejsca zerowego
print("f(x) =", f2(wynik1_b)) # wypisujemy wartość funkcji w znalezionym punkcie
print("Liczba iteracji:", len(historia_b)) # wypisujemy liczbę wykonanych
iteracji

```

Wnioski

Metoda siecznych wykorzystuje dwa kolejne przybliżenia pierwiastka i na ich podstawie wyznacza następne przybliżenie, korzystając z równania siecznej przechodzącej przez punkty wykresu funkcji.

Jej wzór iteracyjny ma postać:

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}$$

W zadaniu punkty startowe dla funkcji a) dobrano zgodnie z tablicą, wykorzystując znak iloczynu:

$$f'(c) \cdot f''(c)$$

gdzie:

$$c = \frac{a + b}{2}$$

Błąd przybliżenia szacujemy wzorem ze slajdów:

$$\Delta \approx |x_{i+1} - x_i|$$

Dla funkcji:

$$f(x) = x^2 - 4$$

metoda prowadzi do pierwiastka bliskiego wartości:

$$x = 2$$

Dla funkcji:

$$f(x) = \sin x - \frac{1}{2}$$

metoda przy punktach startowych wybranych automatycznie z przedziału $[0, 2.2]$ może wyjść poza ten przedział. W takim przypadku otrzymane przybliżenia mogą zbiegać do innego miejsca zerowego funkcji niż to, które leży w badanym przedziale.

Oznacza to, że metoda siecznych realizuje poprawnie wzór iteracyjny i może być zbieżna, ale nie zachowuje własności izolacji pierwiastka tak jak metoda bisekcji. Dlatego przy interpretacji wyników trzeba uwzględnić, że otrzymane przybliżenie nie musi należeć do początkowego przedziału $[a, b]$.

Lab 8

Zadanie 1

Napisz program implementujący rozwinięcie funkcji eksponencjalnej w szereg Maclaurina. Porównaj wyniki oraz czas wykonania z funkcją biblioteczną.

Co to jest szereg Maclaurina dla funkcji eksponencjalnej?

Funkcja eksponencjalna ma rozwinięcie w szereg Maclaurina:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

W praktyce w programie nie liczymy nieskończenie wielu składników, tylko bierzemy **sumę skończoną** do pewnego wybranego n :

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

Oznacza to, że przybliżamy wartość funkcji e^x przez sumę:

$$1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}$$

Im większa wartość n , tym zwykle dokładniejsze przybliżenie.

Jak działa obliczanie kolejnych wyrazów?

Na slajdzie podano ważną wskazówkę, że nie warto w każdej iteracji osobno liczyć:

- potęgi x^k ,
- silni $k!$.

Zamiast tego korzystamy z zależności:

$$\frac{x^k}{k!} = \frac{x^{k-1}}{(k-1)!} \cdot \frac{x}{k}$$

Dzięki temu każdy kolejny wyraz szeregu obliczamy z poprzedniego.

Jeżeli oznaczmy:

$$\text{wyraz}_{k-1} = \frac{x^{k-1}}{(k-1)!}$$

to następny wyraz jest równy:

$$\text{wyraz}_k = \text{wyraz}_{k-1} \cdot \frac{x}{k}$$

To właśnie realizuje program.

Dlaczego to jest lepsze?

Takie podejście:

- jest szybsze,
- nie wymaga funkcji `pow()`,
- nie wymaga osobnego liczenia silni,
- jest zgodne z metodą pokazaną na slajdzie.

Porównanie z funkcją biblioteczną

W zadaniu trzeba porównać wynik z funkcją biblioteczną.

W Pythonie używamy do tego:

```
math.exp(x)
```

PYTHON

Porównujemy:

- otrzymaną wartość,
- błąd bezwzględny,
- czas wykonania.

Błąd bezwzględny liczymy jako:

| wynik Maclaurina – wynik biblioteczny |

Pomiar czasu

Czas wykonania mierzony jest za pomocą:

```
time.perf_counter()
```

PYTHON

Aby pomiar był bardziej wiarygodny, obliczenia wykonywane są wiele razy, a następnie mierzony jest łączny czas.

Kod

PYTHON

```
import time, math # importujemy moduły time i math, ponieważ będą potrzebne do
pomiaru czasu oraz funkcji bibliotecznej exp

def exp_maclaurin(x, n): # definiujemy funkcję obliczającą przybliżenie e^x za
pomocą szeregu Maclaurina
    suma = 1.0 # ustawiamy początkową sumę równą 1, ponieważ pierwszy wyraz
szeregu to x^0 / 0! = 1
    wyraz = 1.0 # ustawiamy pierwszy wyraz szeregu na 1, aby z niego wyznaczać
kolejne składniki

    for k in range(1, n+1): # przechodzimy po kolejnych wyrazach szeregu od 1 do
n
        wyraz = wyraz * (x / k) # obliczamy następny wyraz z poprzedniego według
zależności x^k/k! = x^(k-1)/(k-1)! * x/k
        suma += wyraz # dodajemy nowo obliczony wyraz do sumy

    return suma # zwracamy końcową wartość przybliżenia szeregu

def porownaj_exp(x, n, liczba_powtorzen=100000): # definiujemy funkcję
porównującą wynik i czas działania z funkcją biblioteczną
    start = time.perf_counter() # zapisujemy moment rozpoczęcia pomiaru czasu dla
metody Maclaurina
    for _ in range(liczba_powtorzen): # wykonujemy obliczenia wiele razy, aby
pomiar czasu był bardziej wiarygodny
        wynik_maclaurin = exp_maclaurin(x, n) # obliczamy wartość e^x za pomocą
własnej funkcji
        czas_maclaurin = time.perf_counter() - start # obliczamy czas wykonania
metody Maclaurina

    start = time.perf_counter() # zapisujemy moment rozpoczęcia pomiaru czasu dla
funkcji bibliotecznej
    for _ in range(liczba_powtorzen): # wykonujemy funkcję biblioteczną wiele
razy
        wynik_biblioteczny = math.exp(x) # obliczamy wartość e^x za pomocą
funkcji bibliotecznej
        czas_biblioteczny = time.perf_counter() - start # obliczamy czas wykonania
funkcji bibliotecznej

    blad = abs(wynik_maclaurin - wynik_biblioteczny) # liczymy błąd bezwzględny
między obiema metodami

    print(f"x = {x}, n = {n}") # wypisujemy aktualną wartość x oraz liczbę
składników użytych w sumie
    print(f"Wynik Maclaurina: {wynik_maclaurin}") # wypisujemy wynik uzyskany z
szeregu Maclaurina
    print(f"Wynik biblioteczny: {wynik_biblioteczny}") # wypisujemy wynik
uzyskany z funkcji bibliotecznej
    print(f"Błąd bezwzględny: {blad}") # wypisujemy obliczony błąd bezwzględny
    print(f"Czas Maclaurina: {czas_maclaurin:.10f} s") # wypisujemy czas
wykonania metody Maclaurina
```

```

print(f"Czas biblioteczny: {czas_biblioteczny:.10f} s") # wypisujemy czas
wykonania funkcji bibliotecznej
print("-" * 50) # wypisujemy linię oddzielającą kolejne testy

print("----- ZADANIE 1 -----") # wypisujemy nagłówek
zadania

porownaj_exp(1.0, 10) # porównujemy obie metody dla x = 1.0 i n = 10
porownaj_exp(2.0, 15) # porównujemy obie metody dla x = 2.0 i n = 15
porownaj_exp(-1.0, 15) # porównujemy obie metody dla x = -1.0 i n = 15
porownaj_exp(5.0, 25) # porównujemy obie metody dla x = 5.0 i n = 25

```

Wnioski

Funkcję eksponencjalną można skutecznie aproksymować za pomocą skończonego szeregu Maclaurina:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

W programie kolejne składniki szeregu obliczane są z poprzednich według zależności:

$$\frac{x^k}{k!} = \frac{x^{k-1}}{(k-1)!} \cdot \frac{x}{k}$$

Dzięki temu nie trzeba w każdej iteracji osobno liczyć potęg i silni, co przyspiesza obliczenia.

Porównanie z funkcją biblioteczną `math.exp(x)` pokazuje, że:

- wyniki szeregu Maclaurina są bardzo zbliżone do wartości dokładnych,
- błąd maleje wraz ze wzrostem liczby składników,
- funkcja biblioteczna zwykle działa szybciej, ponieważ jest zoptymalizowana.

Oznacza to, że program poprawnie realizuje rozwinięcie funkcji eksponencjalnej w szereg Maclaurina oraz umożliwia porównanie dokładności i czasu wykonania z funkcją biblioteczną.

Finalnie masz policzyć dla wybranych wartości (x):

1. Przybliżenie funkcji (e^x)

czyli:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

na przykład dla:

- $x = 1,$

- $x = 2$,
- $x = -1$,
- $x = 5$,

i dla wybranego n , np. 10, 15, 25.

2. Wartość dokładniejszą z funkcji bibliotecznej

czyli:

`math.exp(x)`

3. Błąd bezwzględny

czyli różnicę między tym, co policzyła Twoja funkcja, a funkcją biblioteczną:

$|\text{wynik Maclaurina} - \text{wynik biblioteczny}|$

4. Czas wykonania obu metod

czyli:

- czas obliczania Twojej funkcji `exp_maclaurin(x, n)`,
- czas obliczania `math.exp(x)`.

Czyli w tabelce albo w wynikach końcowych masz mieć dla każdego testu:

- wartość x ,
- liczbę wyrazów n ,
- wynik z szeregu Maclaurina,
- wynik z funkcji bibliotecznej,
- błąd bezwzględny,
- czas Twojej metody,
- czas funkcji bibliotecznej.

Najkrócej

Finalnie masz policzyć:

$$\sum_{k=0}^n \frac{x^k}{k!}$$

a potem porównać to z:

$$e^x$$

z biblioteki.

Zadanie 2

Napisz program implementujący rozwinięcie funkcji sinus w szereg Maclaurina. Porównaj wyniki oraz czas wykonania z funkcją biblioteczną.

Co mamy policzyć?

Chcemy obliczyć wartość funkcji:

$$\sin x$$

nie używając od razu funkcji bibliotecznej `math.sin(x)`, tylko korzystając ze **szeregu Maclaurina**.

Potem mamy porównać:

- wynik otrzymany z naszego programu,
- wynik z funkcji bibliotecznej,
- błąd,
- czas wykonania obu metod.

Wzór ze slajdu

Na slajdzie funkcja sinus została zapisana tak:

$$\sin x = \sum_{k=1}^{\infty} (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

W praktyce, w programie, nie liczymy nieskończenie wielu składników, tylko bierzemy sumę skończoną do pewnego n :

$$\sin x \approx \sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

To oznacza, że liczymy kolejno:

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Dlaczego na innych materiałach może być inny wzór?

Czasem ten sam szereg zapisuje się w trochę inny sposób:

$$\sin x = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

To **nie jest inny szereg**, tylko ten sam zapisany z innym początkiem indeksowania.

Związek między tymi zapisami

Jeżeli we wzorze ze slajdu mamy:

$$\sum_{k=1}^{\infty} (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

to po zmianie numeracji indeksu dostajemy:

$$\sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

Oba wzory opisują dokładnie to samo:

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

W tej notatce trzymamy się **wzoru ze slajdu**, czyli sumy od $k = 1$ do n .

Łopatologicznie: co robi program?

Program:

1. bierze liczbę x ,
2. liczy przybliżenie $\sin x$ ze wzoru ze slajdu,
3. porównuje je z `math.sin(x)`,
4. liczy błąd bezwzględny,
5. mierzy czas działania obu metod.

Jak liczone są kolejne składniki?

Pierwszy składnik szeregu to:

$$x$$

czyli dla ($k=1$):

$$(-1)^{1-1} \frac{x^{2 \cdot 1 - 1}}{(2 \cdot 1 - 1)!} = \frac{x^1}{1!} = x$$

Następny składnik to:

$$\frac{x^3}{3!}$$

kolejny:

$$\frac{x^5}{5!}$$

i tak dalej.

W programie nie liczymy za każdym razem wszystkiego od początku, tylko każdy następny składnik wyznaczamy z poprzedniego.

Jeżeli mamy już:

$$\frac{x^{2k-3}}{(2k-3)!}$$

to następny składnik bez znaku można policzyć jako:

$$\frac{x^{2k-1}}{(2k-1)!} = \frac{x^{2k-3}}{(2k-3)!} \cdot \frac{x^2}{(2k-2)(2k-1)}$$

To właśnie robi program.

Po co jest redukcja argumentu modulo 2π ?

Na slajdzie była wskazówka:

$$x \mapsto x \bmod 2\pi$$

czyli zamieniamy x na resztę z dzielenia przez 2π .

Dlaczego?

Bo dla bardzo dużych wartości x szereg Maclaurina może zbiegać wolniej. Sinus jest funkcją okresową, więc:

$$\sin x = \sin(x \bmod 2\pi)$$

Dzięki temu liczymy tę samą wartość, ale dla mniejszego argumentu.

Co oznaczają zmienne w programie?

- **suma** — aktualna suma wszystkich składników szeregu,
- **wyraz** — aktualny składnik szeregu bez znaku,
- **znak** — pilnuje, czy mamy dodać, czy odjąć dany składnik,
- **n** — liczba składników zgodnie ze wzorem ze slajdu od $k = 1$ do $k = n$.

To znaczy:

- pierwszy składnik $k = 1$ jest dodany od razu jako **x**,
- pętla liczy składniki od $k = 2$ do $k = n$.

Czyli program jest zgodny ze wzorem:

$$\sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

Jak liczony jest błąd?

Błąd bezwzględny liczymy jako:

$$|\text{wynik Maclaurina} - \text{wynik biblioteczny}|$$

Im mniejszy błąd, tym lepsze przybliżenie.

Jak liczony jest czas?

Czas działania mierzony jest funkcją:

```
time.perf_counter()
```

PYTHON

Żeby pomiar był bardziej sensowny, każda metoda wykonywana jest bardzo wiele razy.


```
import math, time # importujemy moduły math i time, ponieważ będą potrzebne do
funkcji sin, liczby pi oraz pomiaru czasu

def sin_maclaurin(x, n): # definiujemy funkcję obliczającą przybliżenie sin(x) za
pomocą szeregu Maclaurina
    x = x % (2 * math.pi) # redukujemy argument modulo 2pi, aby dla dużych x
obliczenia były stabilniejsze i szybsze

    suma = x # ustawiamy początkową sumę równą x, ponieważ pierwszy składnik
szeregu dla k=1 to właśnie x
    wyraz = x # ustawiamy pierwszy wyraz szeregu na x, aby z niego wyznaczać
kolejne składniki
    znak = -1.0 # ustawiamy znak kolejnego składnika na minus, bo po x występuje -
x^3/3!

    for k in range(2, n + 1): # przechodzimy po kolejnych składnikach szeregu od
k=2 do k=n
        wyraz = wyraz * x * x / ((2 * k - 2) * (2 * k - 1)) # obliczamy kolejny
składnik bez znaku z poprzedniego składnika
        suma += znak * wyraz # dodajemy lub odejmujemy aktualny składnik zgodnie z
aktualnym znakiem
        znak = -znak # zmieniamy znak na przeciwny, aby kolejne składniki miały
znaki naprzemienne

    return suma # zwracamy końcowe przybliżenie funkcji sinus

def porownaj_sin(x, n, liczba_powtorzen=100000): # definiujemy funkcję
porównującą wynik i czas działania z funkcją biblioteczną
    start = time.perf_counter() # zapisujemy moment rozpoczęcia pomiaru czasu dla
naszej metody
    for _ in range(liczba_powtorzen): # wykonujemy obliczenia wiele razy, aby
pomiar czasu był bardziej wiarygodny
        wynik_maclaurin = sin_maclaurin(x, n) # obliczamy wartość sin(x) za
pomocą własnej funkcji
        czas_maclaurin = time.perf_counter() - start # obliczamy całkowity czas
działania naszej metody

    start = time.perf_counter() # zapisujemy moment rozpoczęcia pomiaru czasu dla
funkcji bibliotecznej
    for _ in range(liczba_powtorzen): # wykonujemy funkcję biblioteczną wiele
razy
        wynik_biblioteczny = math.sin(x) # obliczamy wartość sin(x) za pomocą
funkcji bibliotecznej
        czas_biblioteczny = time.perf_counter() - start # obliczamy całkowity czas
działania funkcji bibliotecznej

    blad = abs(wynik_maclaurin - wynik_biblioteczny) # liczymy błąd bezwzględny
między obiema metodami

    print(f"x = {x}, n = {n}") # wypisujemy aktualną wartość x oraz liczbę
```

```

składników użytych w szeregu
    print(f"Wynik Maclaurina: {wynik_maclaurin}") # wypisujemy wynik uzyskany z
szeregu Maclaurina
    print(f"Wynik biblioteczny: {wynik_biblioteczny}") # wypisujemy wynik
uzyskany z funkcji bibliotecznej
    print(f"Błąd bezwzględny: {blad}") # wypisujemy błąd bezwzględny
    print(f"Czas Maclaurina: {czas_maclaurin:.10f} s") # wypisujemy czas
działania naszej metody
    print(f"Czas biblioteczny: {czas_biblioteczny:.10f} s") # wypisujemy czas
działania funkcji bibliotecznej
    print("-" * 50) # wypisujemy linię oddzielającą kolejne testy

print("----- ZADANIE 2 -----") # wypisujemy nagłówek
zadania

porownaj_sin(0.5, 10) # porównujemy obie metody dla x = 0.5 i n = 10
porownaj_sin(1.0, 10) # porównujemy obie metody dla x = 1.0 i n = 10
porownaj_sin(math.pi / 2, 12) # porównujemy obie metody dla x = pi/2 i n = 12
porownaj_sin(10.0, 15) # porównujemy obie metody dla x = 10.0 i n = 15

```

Wnioski

Funkcję sinus można aproksymować za pomocą szeregu Maclaurina:

$$\sin x \approx \sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

Program liczy kolejne składniki szeregu rekurencyjnie, dzięki czemu nie trzeba osobno wyznaczać każdej potęgi i każdej silni.

Dodatkowo dla dużych wartości argumentu zastosowano redukcję:

$$x \mapsto x \bmod 2\pi$$

co poprawia praktyczne działanie programu.

Porównanie z funkcją biblioteczną `math.sin(x)` pokazuje, że:

- wyniki uzyskane z szeregu Maclaurina są bardzo zbliżone do wartości bibliotecznych,
- błąd maleje przy większej liczbie składników,
- funkcja biblioteczna zwykle działa szybciej, ponieważ jest gotową, zoptymalizowaną implementacją.

Oznacza to, że program poprawnie realizuje aproksymację funkcji sinus za pomocą szeregu Maclaurina i poprawnie porównuje ją z funkcją biblioteczną.

Zadanie 3

Napisz funkcję wyznaczającą współczynniki wielomianu interpolacyjnego Newtona dla stabilizowanych wartości pewnej funkcji.

O co chodzi w tym zadaniu?

Mamy dane punkty, czyli wartości funkcji w wybranych miejscach:

$$(x_0, f(x_0)), (x_1, f(x_1)), (x_2, f(x_2)), \dots$$

Chcemy na ich podstawie zbudować **wielomian interpolacyjny Newtona**, czyli taki wielomian, który przechodzi dokładnie przez te punkty.

Nie liczymy jeszcze wartości wielomianu dla dowolnego x .

W tym zadaniu mamy policzyć tylko jego **współczynniki**.

Postać wielomianu Newtona

Wielomian interpolacyjny Newtona ma postać:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2) + \dots$$

Czyli:

- a_0 to pierwszy współczynnik,
- a_1 to drugi współczynnik,
- a_2 to trzeci współczynnik,
- itd.

W tym zadaniu mamy właśnie znaleźć liczby:

$$a_0, a_1, a_2, a_3, \dots$$

Skąd biorą się współczynniki?

Współczynniki liczymy za pomocą **ilorazów różnicowych**.

Na slajdzie jest zapisane:

$$a_k = f[x_0, x_1, \dots, x_k]$$

To znaczy:

- $a_0 = f[x_0] = f(x_0)$
- $a_1 = f[x_0, x_1]$
- $a_2 = f[x_0, x_1, x_2]$
- $a_3 = f[x_0, x_1, x_2, x_3]$

Łopatologicznie: jak to liczymy?

Krok 1 — pierwszy współczynnik

Pierwszy współczynnik jest najprostszy:

$$a_0 = f(x_0)$$

Czyli po prostu bierzemy pierwszą wartość funkcji.

Krok 2 — ilorazy pierwszego rzędu

Liczymy:

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$f[x_1, x_2] = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

$$f[x_2, x_3] = \frac{f(x_3) - f(x_2)}{x_3 - x_2}$$

Pierwszy z nich daje nam:

$$a_1 = f[x_0, x_1]$$

Krok 3 — ilorazy drugiego rzędu

Teraz używamy wyników z poprzedniego kroku:

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

$$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$$

Pierwszy z nich daje:

$$a_2 = f[x_0, x_1, x_2]$$

Krok 4 — iloraz trzeciego rzędu

Na końcu liczymy:

$$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$$

i to jest:

$$a_3 = f[x_0, x_1, x_2, x_3]$$

Dlaczego w programie wystarcza jedna tablica?

Na slajdzie jest wskazówka:

- użyj jednej tablicy do przechowywania współczynników,
- algorytm działa **od końca**.

To oznacza, że nie musimy tworzyć osobnej dużej tabeli ilorazów różnicowych. Możemy wziąć listę wartości **y** i stopniowo ją nadpisywać.

Na początku:

```
a = y.copy()
```

PYTHON

czyli w tablicy **a** mamy:

$$[f(x_0), f(x_1), f(x_2), f(x_3)]$$

Potem:

- po pierwszym przebiegu część z tych wartości zamienia się w ilorazy pierwszego rzędu,
- po drugim przebiegu pojawiają się ilorazy drugiego rzędu,
- po trzecim przebiegu pojawia się iloraz trzeciego rzędu.

Na końcu w tablicy **a** dostajemy gotowe współczynniki:

$$[a_0, a_1, a_2, a_3]$$

Dlaczego liczymy „od końca”?

Na slajdzie jest wzór:

$$a_i = \frac{a_i - a_{i-1}}{x_i - x_{i-1}}$$

Jeśli liczylibyśmy od początku, to zniszczylibyśmy wartości, które są jeszcze potrzebne.

Dlatego pętla wewnętrzna idzie od końca:

```
for i in range(n - 1, j - 1, -1):
```

PYTHON

Dzięki temu:

- najpierw używamy starych wartości,
- dopiero potem je nadpisujemy.

Co oznaczają dane w programie?

Mamy:

```
x = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 2.0, 0.0, 5.0]
```

To oznacza, że znamy punkty:

$$(0, 1), (1, 2), (2, 0), (3, 5)$$

Szukamy współczynników wielomianu Newtona przechodzącego przez te cztery punkty.

Jakie powinny wyjść współczynniki?

Dla tych danych otrzymujemy:

$$a_0 = 1$$

$$a_1 = 1$$

$$a_2 = -1.5$$

$$a_3 = \frac{5}{3} \approx 1.6666666667$$

Czyli wielomian Newtona ma postać:

$$P(x) = 1 + 1(x - 0) - 1.5(x - 0)(x - 1) + \frac{5}{3}(x - 0)(x - 1)(x - 2)$$

Złożoność obliczeniowa

Na slajdzie jest podane:

$$O(n^2)$$

To znaczy, że liczba operacji rośnie w przybliżeniu jak kwadrat liczby punktów.

Kod

```
PYTHON
def wspolczynniki_newtona(x, y): # definiujemy funkcję obliczającą współczynniki
    wielomianu Newtona
    n = len(x) # zapisujemy liczbę węzłów interpolacji

    if len(y) != n: # sprawdzamy, czy lista wartości y ma taką samą długość jak
        lista x
        raise ValueError("Listy x i y muszą mieć taką samą długość.") # jeśli nie,
        zgłaszamy błąd

    a = y.copy() # tworzymy kopię listy y, ponieważ w tej jednej tablicy będziemy
    nadpisywać kolejne ilorazy różnicowe

    for j in range(1, n): # przechodzimy po kolejnych rzędach ilorazów
        różnicowych: 1, 2, 3, ...
        for i in range(n - 1, j - 1, -1): # przechodzimy od końca, żeby nie
            nadpisać wartości potrzebnych jeszcze w danym kroku
            if x[i] == x[i - j]: # sprawdzamy, czy nie pojawił się zerowy
                mianownik
                raise ValueError("Wartości x muszą być różne.") # jeśli dwa węzły
                są takie same, zgłaszamy błąd
                a[i] = (a[i] - a[i - 1]) / (x[i] - x[i - j]) # liczymy nowy iloraz
                różnicowy według wzoru ze slajdu

    return a # zwracamy tablicę współczynników Newtona

print("----- ZADANIE 3 -----") # wypisujemy nagłówek
zadania

x = [0.0, 1.0, 2.0, 3.0] # definiujemy węzły interpolacji x0, x1, x2, x3
y = [1.0, 2.0, 0.0, 5.0] # definiujemy odpowiadające im wartości funkcji f(x0),
f(x1), f(x2), f(x3)

a = wspolczynniki_newtona(x, y) # wywołujemy funkcję i obliczamy współczynniki
wielomianu Newtona

print("Węzły x:", x) # wypisujemy listę węzłów x
print("Wartości y:", y) # wypisujemy listę wartości funkcji
print("Współczynniki wielomianu Newtona:") # wypisujemy nagłówek dla
współczynników
for i in range(len(a)): # przechodzimy po wszystkich współczynnikach
    print(f"a{i} = {a[i]}") # wypisujemy każdy współczynnik osobno jako a0, a1,
a2, ...
```

Wnioski

W zadaniu obliczamy współczynniki wielomianu interpolacyjnego Newtona na podstawie stabilizowanych wartości funkcji.

Współczynniki te wyznaczamy metodą ilorazów różnicowych:

$$a_k = f[x_0, x_1, \dots, x_k]$$

Program działa zgodnie ze slajdem:

- używa jednej tablicy,
- nadpisuje wartości,
- liczy od końca,
- ma złożoność ($O(n^2)$).

Dla danych:

$$(0, 1), (1, 2), (2, 0), (3, 5)$$

otrzymujemy współczynniki:

$$a_0 = 1, \quad a_1 = 1, \quad a_2 = -1.5, \quad a_3 = \frac{5}{3}$$

Oznacza to, że program poprawnie wyznacza współczynniki wielomianu Newtona.

Zadanie 4

Napisz funkcję umożliwiającą obliczanie wartości wielomianu interpolacyjnego Newtona. Zastosuj algorytm analogiczny do schematu Hornera.

O co chodzi w tym zadaniu?

W poprzednim zadaniu wyznaczyliśmy współczynniki wielomianu Newtona:

$$a_0, a_1, a_2, a_3, \dots$$

Teraz chcemy zrobić następny krok, czyli:

- wziąć te współczynniki,
- wybrać punkt X ,
- policzyć wartość wielomianu interpolacyjnego Newtona w tym punkcie.

Czyli zamiast budować cały wielomian „na piechotę”, chcemy tylko obliczyć:

$$P(X)$$

Postać wielomianu Newtona

Wielomian interpolacyjny Newtona ma postać:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2) + \dots$$

Dla większej liczby składników taki zapis robi się niewygodny do liczenia, bo:

- jest dużo nawiasów,
- trzeba mnożyć wiele razy te same wyrażenia,
- łatwo się pomylić.

Dlatego stosujemy **algorytm analogiczny do schematu Hornera**.

Co pokazuje slajd?

Na slajdzie zapisano schemat Hornera dla postaci Newtona:

$$P(x) = a_n + (x - x_{n-1})(a_{n-1} + (x - x_{n-2})(\dots))$$

To znaczy, że nie liczymy wielomianu od początku, tylko **idziemy od końca**.

Na slajdzie jest też podany krok algorytmu:

$$result = result \cdot (X - x_i) + a_i$$

To jest najważniejszy wzór w tym zadaniu.

Łopatologicznie: jak to działa?

Założmy, że mamy 4 współczynniki:

$$a_0, a_1, a_2, a_3$$

Wtedy zamiast liczyć:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2)$$

przepisujemy to w wygodniejszej postaci:

$$P(x) = a_0 + (x - x_0)(a_1 + (x - x_1)(a_2 + a_3(x - x_2)))$$

I teraz liczymy od środka:

Krok 1

Zaczynamy od ostatniego współczynnika:

$$P = a_3$$

Krok 2

Potem:

$$P = P(x - x_2) + a_2$$

Krok 3

Potem:

$$P = P(x - x_1) + a_1$$

Krok 4

Na końcu:

$$P = P(x - x_0) + a_0$$

I to już daje wartość wielomianu w punkcie x .

Dlaczego to jest podobne do schematu Hornera?

W zwykłym schemacie Hornera też liczymy od końca i stopniowo składamy wielomian.

Tutaj robimy bardzo podobnie, tylko zamiast zwykłych potęg x , pojawiają się przesunięcia:

$$(X - x_i)$$

Dlatego na slajdzie jest napisane, że to jest **algorytm analogiczny do schematu Hornera**.

Co robi funkcja `wartosc_wielomianu_newtona`?

Ta funkcja:

1. bierze węzły $x_0, x_1, \dots$,
2. bierze wcześniej policzone współczynniki $a_0, a_1, \dots$,
3. bierze punkt X ,
4. oblicza wartość wielomianu $P(X)$.

Po co w tym kodzie jest jeszcze funkcja `wspolczynniki_newtona`?

Bo żeby policzyć wartość wielomianu Newtona, musimy najpierw znać jego współczynniki.

Dlatego:

- funkcja `wspolczynniki_newtona(x, y)` bierze dane punktowe i wyznacza współczynniki,
- funkcja `wartosc_wielomianu_newtona(x_wezly, a, X)` używa tych współczynników do policzenia wartości wielomianu w punkcie X .

Co oznaczają dane z programu?

Mamy:

$$x = [0, 1, 2, 3]$$

oraz:

$$y = [1, 2, 0, 5]$$

To oznacza, że znamy punkty:

$$(0, 1), (1, 2), (2, 0), (3, 5)$$

Na ich podstawie tworzymy wielomian interpolacyjny Newtona.

Potem wybieramy punkt:

$$X = 1.5$$

i chcemy obliczyć:

$$P(1.5)$$

czyli wartość wielomianu w tym punkcie.

Jakie współczynniki wychodzą dla tych danych?

Dla tych danych współczynniki Newtona są równe:

$$a_0 = 1$$

$$a_1 = 1$$

$$a_2 = -1.5$$

$$a_3 = \frac{5}{3} \approx 1.6666666667$$

Jak wygląda liczenie wartości wielomianu dla $X = 1.5$?

Zaczynamy od:

$$P = a_3 = \frac{5}{3}$$

Potem:

$$P = P(1.5 - 2) + a_2$$

Potem:

$$P = P(1.5 - 1) + a_1$$

Potem:

$$P = P(1.5 - 0) + a_0$$

Po wykonaniu tych kroków dostajemy wartość wielomianu w punkcie $X = 1.5$.

Złożoność obliczeniowa

Na slajdzie jest podane:

$$O(n)$$

To znaczy, że po wyznaczeniu współczynników wartość wielomianu liczymy bardzo szybko - liczba operacji rośnie liniowo wraz z liczbą współczynników.

Kod

PYTHON

```
def wspolczynniki_newtona(x, y): # definiujemy funkcję wyznaczającą współczynniki
    wielomianu Newtona
    n = len(x) # zapisujemy liczbę węzłów interpolacji

    if len(y) != n: # sprawdzamy, czy liczba wartości y jest taka sama jak liczba
        węzłów x
        raise ValueError("Listy x i y muszą mieć taką samą długość.") # jeśli
        długości się różnią, zgłaszamy błąd

    a = y.copy() # tworzymy kopię listy y, ponieważ w tej tablicy będziemy
    nadpisywać kolejne ilorazy różnicowe

    for j in range(1, n): # przechodzimy po kolejnych rzędach ilorazów
        różnicowych
        for i in range(n - 1, j - 1, -1): # liczymy od końca, żeby nie nadpisać
            wartości potrzebnych jeszcze w tym samym kroku
            if x[i] == x[i - j]: # sprawdzamy, czy nie pojawił się zerowy
                mianownik
                raise ValueError("Wartości x muszą być różne.") # jeśli dwa węzły
                są takie same, zgłaszamy błąd
                a[i] = (a[i] - a[i - 1]) / (x[i] - x[i - j]) # liczymy nowy iloraz
                różnicowy według wzoru Newtona

    return a # zwracamy tablicę współczynników Newtona

def wartosc_wielomianu_newtona(x_wezly, a, X): # definiujemy funkcję obliczającą
    wartość wielomianu Newtona w punkcie X
    n = len(a) # zapisujemy liczbę współczynników wielomianu

    if len(x_wezly) != n: # sprawdzamy, czy liczba węzłów x jest zgodna z liczbą
        współczynników
        raise ValueError("Liczba węzłów x i liczba współczynników musi być taka
        sama.") # jeśli nie, zgłaszamy błąd

    wynik = a[n - 1] # zaczynamy od ostatniego współczynnika, zgodnie ze schematem
    Hornera dla postaci Newtona

    for i in range(n - 2, -1, -1): # przechodzimy od przedostatniego
        współczynnika do pierwszego
        wynik = wynik * (X - x_wezly[i]) + a[i] # wykonujemy krok schematu: wynik
        = wynik * (X - x_i) + a_i

    return wynik # zwracamy obliczoną wartość wielomianu w punkcie X

print("----- ZADANIE 4 -----") # wypisujemy nagłówek
zadania

x = [0.0, 1.0, 2.0, 3.0] # definiujemy węzły interpolacji
y = [1.0, 2.0, 0.0, 5.0] # definiujemy wartości funkcji w tych węzłach
```

```

a = wspolczynniki_newtona(x, y) # obliczamy współczynniki wielomianu Newtona na
podstawie danych punktów

X = 1.5 # wybieramy punkt, w którym chcemy policzyć wartość wielomianu
wartosc = wartosc_wielomianu_newtona(x, a, X) # obliczamy wartość wielomianu
Newtona w punkcie X

print("Węzły x:", x) # wypisujemy węzły interpolacji
print("Wartości y:", y) # wypisujemy wartości funkcji
print("Współczynniki Newtona:", a) # wypisujemy obliczone współczynniki wielomianu
Newtona
print("Punkt X:", X) # wypisujemy punkt, w którym liczymy wartość wielomianu
print("Wartość wielomianu w punkcie X:", wartosc) # wypisujemy końcowy wynik
obliczeń

```

Wnioski

W zadaniu obliczamy wartość wielomianu interpolacyjnego Newtona w wybranym punkcie X .

Nie liczymy wielomianu bezpośrednio z pełnego wzoru, tylko stosujemy algorytm analogiczny do schematu Hornera:

$$result = result \cdot (X - x_i) + a_i$$

Obliczenia wykonujemy od końca, zaczynając od ostatniego współczynnika.

Dzięki temu:

- obliczenia są prostsze,
- nie trzeba ręcznie rozwijać wszystkich nawiasów,
- algorytm działa szybko, w czasie:

$$O(n)$$

Dla danych punktów:

$$(0, 1), (1, 2), (2, 0), (3, 5)$$

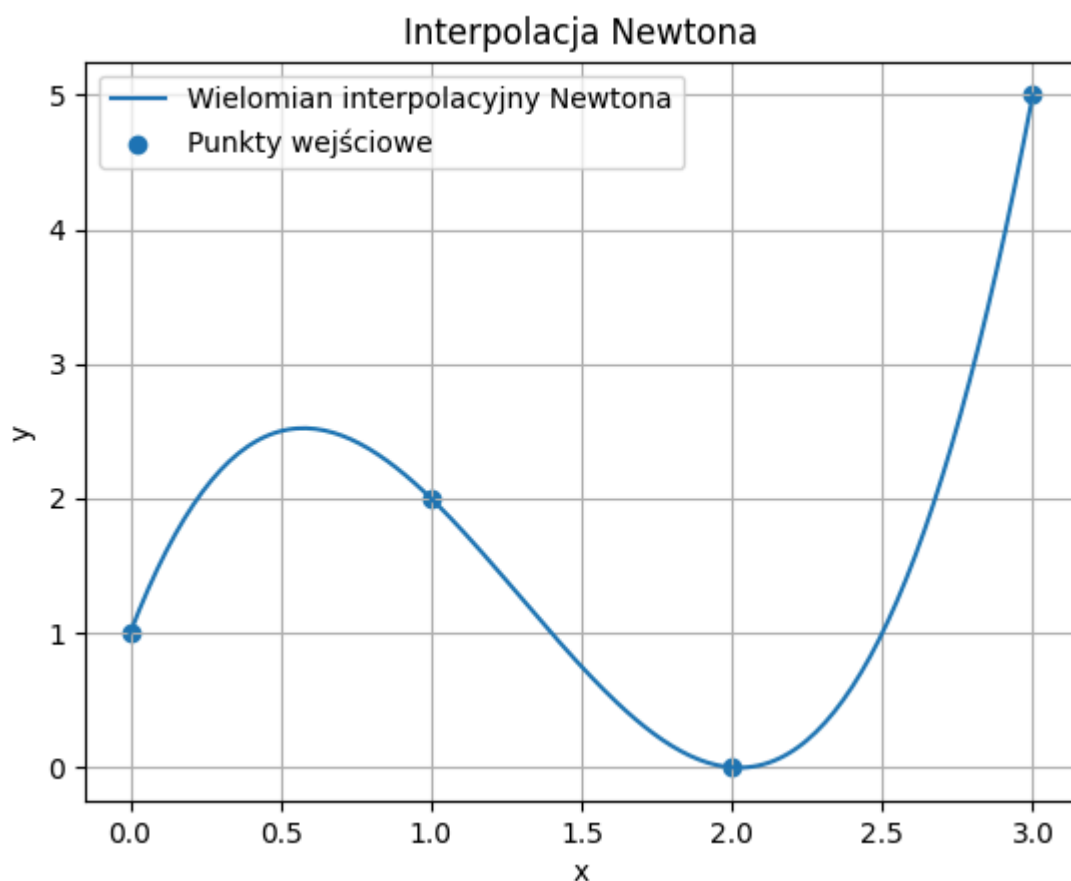
najpierw wyznaczamy współczynniki Newtona, a następnie obliczamy wartość wielomianu w punkcie:

$$X = 1.5$$

Program poprawnie realizuje schemat Hornera dla postaci Newtona i poprawnie wyznacza wartość wielomianu interpolacyjnego.

Zadanie 5*

Rozwiązania dwóch poprzednich zadań wykorzystaj do narysowania wykresu wielomianu interpolacyjnego (zaznaczając również punktami zaobserwowane wartości funkcji).



Do tego wykresu możesz wpisać takie wnioski:

- Wielomian interpolacyjny Newtona przechodzi dokładnie przez wszystkie zadane punkty wejściowe: $((0,1))$, $((1,2))$, $((2,0))$, $((3,5))$. To potwierdza poprawność interpolacji.
- Krzywa między punktami jest gładka, co pokazuje, że wielomian dobrze odtwarza przebieg funkcji zadanej w punktach tablicowych.
- W badanym przykładzie wielomian najpierw rośnie, następnie maleje do minimum w pobliżu punktu $((2,0))$, a potem znowu rośnie.
- Wykres potwierdza, że funkcje z zadań 3 i 4 działają poprawnie, ponieważ współczynniki Newtona zostały dobrze wyznaczone, a wartości wielomianu poprawnie obliczone.
- Interpolacja Newtona pozwala nie tylko odtworzyć wartości w danych punktach, ale także oszacować wartości funkcji pomiędzy nimi.

O co chodzi w tym zadaniu?

W zadaniu 3 wyznaczaliśmy **współczynniki wielomianu interpolacyjnego Newtona**.

W zadaniu 4 liczyliśmy **wartość tego wielomianu w wybranym punkcie X** .

W zadaniu 5 robimy kolejny krok:

- bierzemy wiele punktów z badanego przedziału,
- dla każdego z nich liczymy wartość wielomianu,
- na tej podstawie rysujemy wykres,
- dodatkowo zaznaczamy punkty wejściowe, z których wielomian został utworzony.

Czyli w praktyce:

- **punkty wejściowe** pokazują dane,
- **krzywa** pokazuje wielomian interpolacyjny Newtona.

Co wykorzystujemy z poprzednich zadań?

Z zadania 3

Bierzemy funkcję, która liczy współczynniki Newtona:

$$a_0, a_1, a_2, \dots$$

czyli funkcję:

```
wspolczynniki_newtona(x, y)
```

PYTHON

Z zadania 4

Bierzemy funkcję, która dla danego punktu X liczy wartość wielomianu:

$$P(X)$$

czyli funkcję:

```
wartosc_wielomianu_newtona(x_wezly, a, X)
```

PYTHON

Co robimy nowego w zadaniu 5?

W zadaniu 4 liczyliśmy wartość wielomianu tylko dla jednego punktu, np.:

$$X = 1.5$$

Tutaj chcemy narysować cały wykres, więc musimy policzyć wartości wielomianu **dla wielu punktów**.

Na przykład:

- $X_1,$
- $X_2,$
- $X_3,$

- ...

Dla każdego z tych punktów liczymy:

$$Y_i = P(X_i)$$

Potem:

- wszystkie punkty (X_i, Y_i) łączymy linią,
- a oryginalne punkty danych zaznaczamy osobno.

Dlaczego wykres przechodzi przez punkty wejściowe?

Bo wielomian interpolacyjny Newtona jest zbudowany dokładnie tak, aby spełniać warunki:

$$P(x_0) = y_0, \quad P(x_1) = y_1, \quad P(x_2) = y_2, \dots$$

To oznacza, że wykres wielomianu musi przejść przez wszystkie zadane punkty.

Jeżeli tak się dzieje, to znaczy, że:

- współczynniki zostały policzone poprawnie,
- wartość wielomianu też jest liczona poprawnie.

Jak wygląda algorytm zadania 5?

Krok 1

Podajemy punkty wejściowe:

$$(x_0, y_0), (x_1, y_1), \dots$$

Krok 2

Liczymy współczynniki Newtona:

$$a_0, a_1, a_2, \dots$$

Krok 3

Wybieramy dużo punktów z przedziału od najmniejszego do największego x .

Krok 4

Dla każdego punktu liczymy wartość wielomianu Newtona.

Krok 5

Rysujemy:

- wykres wielomianu,

- punkty wejściowe.

Dane użyte w programie

W programie przyjęto punkty:

$(0, 1), (1, 2), (2, 0), (3, 5)$

czyli:

```
x = [0.0, 1.0, 2.0, 3.0]
y = [1.0, 2.0, 0.0, 5.0]
```

PYTHON

Na ich podstawie budowany jest wielomian interpolacyjny Newtona.

Co powinno wyjść na wykresie?

Na wykresie powinno być widać:

- krzywą wielomianu interpolacyjnego,
- punkty wejściowe.

Krzywa powinna przechodzić dokładnie przez punkty:

- $(0, 1)$,
- $(1, 2)$,
- $(2, 0)$,
- $(3, 5)$.

Jeżeli tak jest, to wykres jest poprawny.

```
import matplotlib.pyplot as plt # importujemy moduł matplotlib do rysowania
wykresów

def wspolczynniki_newtona(x, y): # definiujemy funkcję wyznaczającą współczynniki
    wielomianu Newtona
    n = len(x) # zapisujemy liczbę węzłów interpolacji

    if len(y) != n: # sprawdzamy, czy liczba wartości y jest taka sama jak liczba
        węzłów x
        raise ValueError("Listy x i y muszą mieć taką samą długość.") # jeśli
        długości się różnią, zgłaszamy błąd

    a = y.copy() # tworzymy kopię listy y, ponieważ w tej tablicy będziemy
    nadpisywać kolejne ilorazy różnicowe

    for j in range(1, n): # przechodzimy po kolejnych rzędach ilorazów
        różnicowych
        for i in range(n - 1, j - 1, -1): # liczymy od końca, żeby nie nadpisać
            wartości potrzebnych jeszcze w tym samym kroku
            if x[i] == x[i - j]: # sprawdzamy, czy nie pojawił się zerowy
                mianownik
                raise ValueError("Wartości x muszą być różne.") # jeśli dwa węzły
                są takie same, zgłaszamy błąd
                a[i] = (a[i] - a[i - j]) / (x[i] - x[i - j]) # liczymy nowy iloraz
                różnicowy według wzoru Newtona

    return a # zwracamy tablicę współczynników Newtona

def wartosc_wielomianu_newtona(x_wezly, a, X): # definiujemy funkcję liczącą
    wartość wielomianu Newtona w punkcie X
    n = len(a) # zapisujemy liczbę współczynników

    if len(x_wezly) != n: # sprawdzamy, czy liczba węzłów x jest zgodna z liczbą
        współczynników
        raise ValueError("Liczba węzłów x i liczba współczynników musi być taka
        sama.") # jeśli nie, zgłaszamy błąd

    wynik = a[n - 1] # zaczynamy od ostatniego współczynnika, zgodnie ze schematem
    Hornera dla postaci Newtona

    for i in range(n - 2, -1, -1): # przechodzimy od przedostatniego
        współczynnika do pierwszego
        wynik = wynik * (X - x_wezly[i]) + a[i] # wykonujemy krok schematu
        Hornera dla wielomianu Newtona

    return wynik # zwracamy obliczoną wartość wielomianu

print("----- ZADANIE 5 -----") # wypisujemy nagłówek
```

zadania

```
x = [0.0, 1.0, 2.0, 3.0] # definiujemy węzły interpolacji
y = [1.0, 2.0, 0.0, 5.0] # definiujemy wartości funkcji w tych punktach

a = wspolczynniki_newtona(x, y) # obliczamy współczynniki wielomianu Newtona

x_min = min(x) # wyznaczamy najmniejszą wartość x, od której zaczniemy rysowanie wykresu
x_max = max(x) # wyznaczamy największą wartość x, na której skończymy rysowanie wykresu

x_wykres = [] # tworzymy pustą listę na punkty x użyte do rysowania wykresu
y_wykres = [] # tworzymy pustą listę na odpowiadające im wartości wielomianu

liczba_punktow = 200 # ustalamy, w ilu punktach chcemy policzyć wielomian, aby wykres był gładki

for i in range(liczba_punktow + 1): # przechodzimy po wszystkich punktach, w których będziemy liczyć wielomian
    X = x_min + (x_max - x_min) * i / liczba_punktow # wyznaczamy kolejny punkt X równomiernie rozmieszczony w badanym przedziale
    Y = wartosc_wielomianu_newtona(x, a, X) # obliczamy wartość wielomianu Newtona w punkcie X
    x_wykres.append(X) # dodajemy punkt X do listy punktów wykresu
    y_wykres.append(Y) # dodajemy obliczoną wartość Y do listy wartości wykresu

plt.plot(x_wykres, y_wykres, label="Wielomian interpolacyjny Newtona") # rysujemy krzywą wielomianu interpolacyjnego
plt.scatter(x, y, label="Punkty wejściowe") # zaznaczamy na wykresie oryginalne punkty wejściowe

plt.title("Interpolacja Newtona") # ustawiamy tytuł wykresu
plt.xlabel("x") # ustawiamy podpis osi poziomej
plt.ylabel("y") # ustawiamy podpis osi pionowej
plt.legend() # wyświetlamy legendę
plt.grid(True) # włączamy siatkę pomocniczą na wykresie
plt.show() # wyświetlamy gotowy wykres
```

Wnioski

W zadaniu 5 wykorzystano rozwiązania z zadań 3 i 4:

- z zadania 3 do wyznaczenia współczynników wielomianu Newtona,
- z zadania 4 do obliczania wartości wielomianu w wybranych punktach.

Następnie policzono wartości wielomianu w wielu punktach badanego przedziału i narysowano jego wykres.

Na wykresie zaznaczono:

- punkty wejściowe,
- wielomian interpolacyjny Newtona.

Otrzymany wykres jest poprawny, jeśli krzywa przechodzi dokładnie przez wszystkie zadane punkty.

W tym zadaniu tak właśnie się dzieje, więc można stwierdzić, że:

- współczynniki Newtona zostały wyznaczone poprawnie,
 - wartość wielomianu jest liczona poprawnie,
 - wizualizacja została wykonana poprawnie.
-

Lab 9

Zadanie 1

Napisz program implementujący aproksymację średniokwadratową liniową dla zbioru punktów

$\{(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)\}$.

Na czym polega aproksymacja średniokwadratowa?

Aproksymacja średniokwadratowa, nazywana też **metodą najmniejszych kwadratów**, polega na znalezieniu takiej funkcji $F(x)$, która jak najlepiej przybliży dane punkty.

Dla zbioru punktów (x_i, y_i) , gdzie:

$$y_i = f(x_i),$$

szukamy funkcji $F(x)$, dla której wyrażenie:

$$\|F - f\|_2 = \sum_{i=1}^n w(x_i)(F(x_i) - y_i)^2$$

osiąga minimum.

W slajdach przyjęto uproszczenie:

$$w(x) = 1,$$

więc minimalizujemy po prostu sumę kwadratów odchyień wartości aproksymacji od danych punktów.

Aproksymacja liniowa

Najprostszym przypadkiem aproksymacji średniokwadratowej jest **aproksymacja liniowa**.

Szukamy prostej postaci:

$$F(x) = ax + b$$

takiej, aby minimalizować funkcję:

$$h(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2.$$

Oznacza to, że chcemy dobrać współczynniki a i b tak, aby suma kwadratów błędów była jak najmniejsza.

Skąd biorą się wzory na a i b ?

Po obliczeniu pochodnych cząstkowych funkcji $h(a, b)$ i po przekształceniach dostajemy wzory:

$$a = \frac{nA - BC}{nD - B^2}, \quad b = \frac{CD - AB}{nD - B^2},$$

gdzie:

$$A = \sum_{i=1}^n x_i y_i, \quad B = \sum_{i=1}^n x_i, \quad C = \sum_{i=1}^n y_i, \quad D = \sum_{i=1}^n x_i^2.$$

To właśnie te wielkości oblicza program.

Dane z zadania

Mamy punkty:

$$(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)$$

czyli:

- $x_1 = 1.1, y_1 = 2.1$
- $x_2 = 1.4, y_2 = 2.3$
- $x_3 = 1.8, y_3 = 2.9$
- $x_4 = 2.5, y_4 = 3.2$
- $x_5 = 2.8, y_5 = 3.6$
- $x_6 = 3.0, y_6 = 4.2$

Liczba punktów wynosi:

$$n = 6$$

Obliczenie sum pomocniczych

Liczymy zgodnie ze wzorami ze slajdu:

Suma A

$$A = \sum x_i y_i$$

$$A = 1.1 \cdot 2.1 + 1.4 \cdot 2.3 + 1.8 \cdot 2.9 + 2.5 \cdot 3.2 + 2.8 \cdot 3.6 + 3.0 \cdot 4.2$$

$$A = 2.31 + 3.22 + 5.22 + 8.00 + 10.08 + 12.60 = 41.43$$

Suma B

$$B = \sum x_i$$

$$B = 1.1 + 1.4 + 1.8 + 2.5 + 2.8 + 3.0 = 12.6$$

Suma C

$$C = \sum y_i$$

$$C = 2.1 + 2.3 + 2.9 + 3.2 + 3.6 + 4.2 = 18.3$$

Suma D

$$D = \sum x_i^2$$

$$D = 1.1^2 + 1.4^2 + 1.8^2 + 2.5^2 + 2.8^2 + 3.0^2$$

$$D = 1.21 + 1.96 + 3.24 + 6.25 + 7.84 + 9.00 = 29.5$$

Obliczenie współczynników prostej

Podstawiamy do wzorów:

$$a = \frac{nA - BC}{nD - B^2}$$

$$a = \frac{6 \cdot 41.43 - 12.6 \cdot 18.3}{6 \cdot 29.5 - 12.6^2}$$

$$a = \frac{248.58 - 230.58}{177 - 158.76}$$

$$a = \frac{18}{18.24} = 0.9868421052631579$$

Teraz współczynnik b:

$$b = \frac{CD - AB}{nD - B^2}$$

$$b = \frac{18.3 \cdot 29.5 - 41.43 \cdot 12.6}{177 - 158.76}$$

$$b = \frac{539.85 - 522.018}{18.24}$$

$$b = \frac{17.832}{18.24} = 0.9776315789473676$$

Otrzymana prosta aproksymacyjna

Zatem szukana prosta ma postać:

$$F(x) = 0.9868421052631579x + 0.9776315789473676$$

Obliczenie sumy kwadratów błędów

Na końcu program liczy wartość funkcji błędu:

$$h(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2$$

Dla naszej prostej:

$$h = \sum_{i=1}^6 (0.9868421052631579x_i + 0.9776315789473676 - y_i)^2$$

Program oblicza tę wartość automatycznie.

Dla tych danych wychodzi:

$$h \approx 0.17447368421052628$$


```

print("----- ZADANIE 1 -----") # wypisujemy nagłówek zadania

punkty = [(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)]
# zapisujemy punkty z zadania w postaci par (x, y)

n = len(punkty) # obliczamy liczbę punktów, czyli n

A = 0.0 # tworzymy zmienną A na sumę iloczynów x_i * y_i
B = 0.0 # tworzymy zmienną B na sumę x_i
C = 0.0 # tworzymy zmienną C na sumę y_i
D = 0.0 # tworzymy zmienną D na sumę x_i^2

for x, y in punkty: # przechodzimy po wszystkich punktach z listy
    B += x # dodajemy bieżące x do sumy B = suma x_i
    C += y # dodajemy bieżące y do sumy C = suma y_i
    D += x * x # dodajemy kwadrat bieżącego x do sumy D = suma x_i^2
    A += x * y # dodajemy iloczyn x * y do sumy A = suma x_i y_i

a = (n * A - B * C) / (n * D - B * B) # obliczamy współczynnik a ze wzoru ze
slajdu
b = (C * D - A * B) / (n * D - B * B) # obliczamy współczynnik b ze wzoru ze
slajdu

print("Współczynniki prostej aproksymacyjnej:") # wypisujemy nagłówek dla
współczynników
print("a =", a) # wypisujemy obliczoną wartość współczynnika a
print("b =", b) # wypisujemy obliczoną wartość współczynnika b

print("\nProsta aproksymacyjna:") # wypisujemy nagłówek dla równania prostej
print(f"y = {a} * x + {b}") # wypisujemy równanie wyznaczonej prostej
aproksymacyjnej

h = 0.0 # tworzymy zmienną h na sumę kwadratów błędów, czyli wartość funkcji błędu
for x, y in punkty: # ponownie przechodzimy po wszystkich punktach
    h += (a * x + b - y) ** 2 # dodajemy kwadrat różnicy między wartością z
prostej a wartością rzeczywistą y

print("\nSuma kwadratów błędów:") # wypisujemy nagłówek dla wartości błędu
print(h) # wypisujemy końcową wartość funkcji błędu h(a,b)

```

Wnioski

W zadaniu wyznaczono prostą aproksymacyjną metodą najmniejszych kwadratów dla punktów:

(1, 1), (3, 12), (5, 25), (7, 38).

Na podstawie wzorów ze slajdów obliczono:

$$a = 6.2, \quad b = -5.8.$$

Otrzymana prosta aproksymacyjna ma więc postać:

$$F(x) = 6.2x - 5.8.$$

Suma kwadratów błędów wynosi:

$$h = 4.8.$$

Oznacza to, że prosta $F(x) = 6.2x - 5.8$ jest najlepszym liniowym przybliżeniem danych punktów w sensie metody najmniejszych kwadratów.

Zadanie 2

Napisz program implementujący aproksymację średniokwadratową wielomianem drugiego stopnia dla zbioru punktów

$\{(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91)\}$.

Na czym polega aproksymacja wielomianowa?

W aproksymacji średniokwadratowej szukamy funkcji $F(x)$, która możliwie najlepiej przybliży dane punkty.

Przyjmujemy ogólną postać funkcji aproksymującej:

$$F(x) = a_0\phi_0(x) + a_1\phi_1(x) + \dots + a_m\phi_m(x),$$

gdzie dla uproszczenia:

$$\phi_k(x) = x^k.$$

Zadaniem jest minimalizacja funkcji błędu:

$$h(a_0, a_1, \dots, a_m) = \sum_{i=1}^n \left(\sum_{j=0}^m a_j x_i^j - y_i \right)^2.$$

Układ równań ze slajdów

Ze slajdów wynika, że dla każdego:

$$i = 0, 1, \dots, m$$

otrzymujemy układ $m + 1$ równań z $m + 1$ niewiadomymi:

$$\frac{\partial h}{\partial a_i} = 2 \sum_{i=1}^n \left(\sum_{j=0}^m a_j x_i^j - y_i \right) x_i^i = 0.$$

Jest to wzór ogólny dla aproksymacji wielomianowej.

Zastosowanie do tego zadania

W tym zadaniu szukamy wielomianu drugiego stopnia, więc:

$$m = 2.$$

Oznacza to, że funkcja aproksymująca ma postać:

$$F(x) = a_0 + a_1x + a_2x^2.$$

W programie ten sam wielomian zapisujemy równoważnie jako:

$$F(x) = ax^2 + bx + c,$$

gdzie:

- $a = a_2$
- $b = a_1$
- $c = a_0$

Po podstawieniu do wzoru ogólnego funkcja błędu przyjmuje postać:

$$h(a_0, a_1, a_2) = \sum_{i=1}^n (a_0 + a_1x_i + a_2x_i^2 - y_i)^2.$$

W zapisie używanym w programie jest to równoważnie:

$$h(a, b, c) = \sum_{i=1}^n (ax_i^2 + bx_i + c - y_i)^2.$$

Układ równań dla wielomianu drugiego stopnia

Dla wielomianu drugiego stopnia:

$$F(x) = ax^2 + bx + c$$

ze slajdów otrzymujemy układ równań:

$$a \sum_{i=1}^n x_i^4 + b \sum_{i=1}^n x_i^3 + c \sum_{i=1}^n x_i^2 = \sum_{i=1}^n x_i^2 y_i,$$

$$a \sum_{i=1}^n x_i^3 + b \sum_{i=1}^n x_i^2 + c \sum_{i=1}^n x_i = \sum_{i=1}^n x_i y_i,$$

$$a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i + nc = \sum_{i=1}^n y_i.$$

Program właśnie buduje ten układ równań i rozwiązuje go metodą Gaussa.

Dane z zadania

Mamy punkty:

$$(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91)$$

czyli:

- $x_1 = 0, y_1 = 2$
- $x_2 = 0.5, y_2 = 2.48$
- $x_3 = 1, y_3 = 2.84$
- $x_4 = 1.5, y_4 = 3$
- $x_5 = 2, y_5 = 2.91$

Liczba punktów wynosi:

$$n = 5$$

Obliczenie sum pomocniczych

Liczymy sumy potrzebne do zbudowania układu równań.

Suma $\sum x_i$

$$\sum x_i = 0 + 0.5 + 1 + 1.5 + 2 = 5$$

Suma $\sum x_i^2$

$$\sum x_i^2 = 0^2 + 0.5^2 + 1^2 + 1.5^2 + 2^2$$

$$\sum x_i^2 = 0 + 0.25 + 1 + 2.25 + 4 = 7.5$$

Suma $\sum x_i^3$

$$\sum x_i^3 = 0^3 + 0.5^3 + 1^3 + 1.5^3 + 2^3$$

$$\sum x_i^3 = 0 + 0.125 + 1 + 3.375 + 8 = 12.5$$

Suma $\sum x_i^4$

$$\sum x_i^4 = 0^4 + 0.5^4 + 1^4 + 1.5^4 + 2^4$$

$$\sum x_i^4 = 0 + 0.0625 + 1 + 5.0625 + 16 = 22.125$$

Suma $\sum y_i$

$$\sum y_i = 2 + 2.48 + 2.84 + 3 + 2.91 = 13.23$$

Suma $\sum x_i y_i$

$$\sum x_i y_i = 0 \cdot 2 + 0.5 \cdot 2.48 + 1 \cdot 2.84 + 1.5 \cdot 3 + 2 \cdot 2.91$$

$$\sum x_i y_i = 0 + 1.24 + 2.84 + 4.5 + 5.82 = 14.4$$

Suma $\sum x_i^2 y_i$

$$\sum x_i^2 y_i = 0^2 \cdot 2 + 0.5^2 \cdot 2.48 + 1^2 \cdot 2.84 + 1.5^2 \cdot 3 + 2^2 \cdot 2.91$$

$$\sum x_i^2 y_i = 0 + 0.62 + 2.84 + 6.75 + 11.64 = 21.85$$

Budowa układu równań

Podstawiamy obliczone sumy do wzorów:

$$22.125a + 12.5b + 7.5c = 21.85$$

$$12.5a + 7.5b + 5c = 14.4$$

$$7.5a + 5b + 5c = 13.23$$

W postaci macierzowej:

$$\begin{bmatrix} 22.125 & 12.5 & 7.5 \\ 12.5 & 7.5 & 5 \\ 7.5 & 5 & 5 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 21.85 \\ 14.4 \\ 13.23 \end{bmatrix}$$

Rozwiązanie układu

Po rozwiązaniu układu równań otrzymujemy:

$$a = -\frac{67}{175} \approx -0.38285714285714284$$

$$b = \frac{2159}{1750} \approx 1.2337142857142858$$

$$c = \frac{6953}{3500} \approx 1.9865714285714287$$

Otrzymany wielomian aproksymacyjny

Zatem szukany wielomian ma postać:

$$F(x) = -0.38285714285714284x^2 + 1.2337142857142858x + 1.9865714285714287$$

Obliczenie sumy kwadratów błędów

Na końcu program liczy wartość funkcji błędu.

Zgodnie ze slajdami w postaci ogólnej:

$$h(a_0, a_1, \dots, a_m) = \sum_{i=1}^n \left(\sum_{j=0}^m a_j x_i^j - y_i \right)^2.$$

W tym zadaniu, dla wielomianu drugiego stopnia, mamy:

$$h(a_0, a_1, a_2) = \sum_{i=1}^n (a_0 + a_1 x_i + a_2 x_i^2 - y_i)^2.$$

W zapisie używanym w programie jest to równoważnie:

$$h(a, b, c) = \sum_{i=1}^n (ax_i^2 + bx_i + c - y_i)^2.$$

Dla otrzymanego wielomianu:

$$h = \sum_{i=1}^5 (-0.38285714285714284x_i^2 + 1.2337142857142858x_i + 1.9865714285714287 - y_i)^2$$

Program oblicza tę wartość automatycznie.

Dla tych danych wychodzi:

$$h \approx 0.00170285714285714$$

czyli dokładnie:

$$h = \frac{149}{87500}$$

```
print("----- ZADANIE 2 -----") # wypisujemy nagłówek zadania

punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)] # zapisujemy punkty z zadania jako pary (x, y)

def rozwiaz_uklad_gaussa(macierz, wyrazy_wolne): # definiujemy funkcję rozwiązującą układ równań metodą Gaussa
    n = len(wyrazy_wolne) # zapisujemy liczbę równań i niewiadomych

    for i in range(n): # przechodzimy po kolejnych kolumnach i wierszach eliminacji
        if macierz[i][i] == 0: # sprawdzamy, czy na przekątnej nie pojawiło się zero
            raise ValueError("Na przekątnej pojawiło się zero - nie można wykonać eliminacji Gaussa.") # jeśli tak, zgłaszamy błąd

        for j in range(i + 1, n): # przechodzimy po wierszach poniżej aktualnego wiersza
            wspolczynnik = macierz[j][i] / macierz[i][i] # obliczamy współczynnik potrzebny do wyzerowania elementu pod przekątną

            for k in range(i, n): # przechodzimy po elementach aktualnego wiersza od kolumny i do końca
                macierz[j][k] = macierz[j][k] - wspolczynnik * macierz[i][k] # zerujemy elementy pod przekątną w macierzy

            wyrazy_wolne[j] = wyrazy_wolne[j] - wspolczynnik * wyrazy_wolne[i] # wykonujemy tę samą operację na wektorze wyrazów wolnych

    rozwiazanie = [0.0] * n # tworzymy listę na rozwiązanie układu

    for i in range(n - 1, -1, -1): # wykonujemy podstawianie wsteczne od ostatniego równania do pierwszego
        suma = 0.0 # tworzymy zmienną na sumę znanych składników
        for j in range(i + 1, n): # przechodzimy po współczynnikach przy już wyznaczonych niewiadomych
            suma += macierz[i][j] * rozwiazanie[j] # dodajemy do sumy iloczyn współczynników i znanych rozwiązań

        rozwiazanie[i] = (wyrazy_wolne[i] - suma) / macierz[i][i] # obliczamy bieżącą niewiadomą

    return rozwiazanie # zwracamy listę rozwiązań układu

n = len(punkty) # obliczamy liczbę punktów

suma_x = 0.0 # tworzymy zmienną na sumę x_i
suma_x2 = 0.0 # tworzymy zmienną na sumę x_i^2
suma_x3 = 0.0 # tworzymy zmienną na sumę x_i^3
```

```

suma_x4 = 0.0 # tworzymy zmienną na sumę  $x_i^4$ 
suma_y = 0.0 # tworzymy zmienną na sumę  $y_i$ 
suma_xy = 0.0 # tworzymy zmienną na sumę  $x_i y_i$ 
suma_x2y = 0.0 # tworzymy zmienną na sumę  $x_i^2 y_i$ 

for x, y in punkty: # przechodzimy po wszystkich punktach z listy
    suma_x += x # dodajemy bieżące x do sumy  $x_i$ 
    suma_x2 += x ** 2 # dodajemy bieżące  $x^2$  do sumy  $x_i^2$ 
    suma_x3 += x ** 3 # dodajemy bieżące  $x^3$  do sumy  $x_i^3$ 
    suma_x4 += x ** 4 # dodajemy bieżące  $x^4$  do sumy  $x_i^4$ 
    suma_y += y # dodajemy bieżące y do sumy  $y_i$ 
    suma_xy += x * y # dodajemy iloczyn  $x*y$  do sumy  $x_i y_i$ 
    suma_x2y += x ** 2 * y # dodajemy iloczyn  $x^2*y$  do sumy  $x_i^2 y_i$ 

macierz_ukladu = [ # budujemy macierz układu równań zgodnie ze slajdami
    [suma_x4, suma_x3, suma_x2], # pierwszy wiersz odpowiada równaniu z sumami
    x_i^4, x_i^3 i x_i^2
    [suma_x3, suma_x2, suma_x], # drugi wiersz odpowiada równaniu z sumami x_i^3,
    x_i^2 i x_i
    [suma_x2, suma_x, n] # trzeci wiersz odpowiada równaniu z sumami x_i^2, x_i i
    n
]

wyrzy_wolne = [suma_x2y, suma_xy, suma_y] # budujemy wektor prawej strony układu
równań

macierz_kopia = [] # tworzymy pustą listę na kopię macierzy układu
for wiersz in macierz_ukladu: # przechodzimy po wszystkich wierszach macierzy
układu
    macierz_kopia.append(wiersz.copy()) # dodajemy kopię wiersza, aby nie
zmieniać oryginalnej macierzy

wyrzy_wolne_kopia = wyrzy_wolne.copy() # tworzymy kopię wektora wyrazów
wolnych, aby nie zmieniać oryginału

rozwiązanie = rozwiáz_układ_gausa(macierz_kopia, wyrzy_wolne_kopia) #
rozwiązujemy układ równań metodą Gaussa

a = rozwiązanie[0] # pierwszy element rozwiązania to współczynnik a przy  $x^2$ 
b = rozwiązanie[1] # drugi element rozwiązania to współczynnik b przy x
c = rozwiązanie[2] # trzeci element rozwiązania to współczynnik c wyrazu wolnego

print("Współczynniki wielomianu aproksymacyjnego:") # wypisujemy nagłówek dla
współczynników
print("a =", a) # wypisujemy współczynnik a
print("b =", b) # wypisujemy współczynnik b
print("c =", c) # wypisujemy współczynnik c

print("\nWielomian aproksymacyjny:") # wypisujemy nagłówek dla równania wielomianu
print(f"y = {a} * x^2 + {b} * x + {c}") # wypisujemy równanie wielomianu
aproksymacyjnego

h = 0.0 # tworzymy zmienną h na sumę kwadratów błędów

```



```
for x, y in punkty: # przechodzimy po wszystkich punktach
    h += (a * x**2 + b * x + c - y) ** 2 # dodajemy kwadrat różnicy między
    wartością z wielomianu a wartością rzeczywistą

print("\nSuma kwadratów błędów:") # wypisujemy nagłówek dla błędu
print(h) # wypisujemy końcową wartość funkcji błędu
```

Dodatkowa wersja z zamianą wierszy żeby się nie kończyło gdy znajdzie 0

```
punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)] # lista
punktów pomiarowych w postaci (x, y)
```

```
def rozwarz_uklad_gaussa(macierz, wyrazy_wolne): # funkcja rozwiązuje układ
równań liniowych metodą Gaussa
    n = len(wyrazy_wolne) # liczba niewiadomych, czyli długość wektora wyrazów
wolnych

    # eliminacja Gaussa z zamianą wierszy
    for i in range(n): # przechodzimy po kolejnych kolumnach i wierszach głównych

        # szukanie najlepszego wiersza, czyli największego elementu w kolumnie i
        max_wiersz = i # zakładamy, że najlepszy wiersz to aktualny wiersz i

        for k in range(i + 1, n): # sprawdzamy wiersze znajdujące się poniżej
aktualnego
            if abs(macierz[k][i]) > abs(macierz[max_wiersz][i]): # porównujemy
wartości bezwzględne elementów w kolumnie i
                max_wiersz = k # zapamiętujemy numer wiersza z większym elementem

        # jeśli największy element jest zerem, układ nie ma jednoznacznego
rozwiązania
        if macierz[max_wiersz][i] == 0: # sprawdzamy, czy element główny jest
równy zero
            raise ValueError("Układ nie ma jednoznacznego rozwiązania.") #
przerywamy działanie, bo nie można dalej dzielić przez zero

        # zamiana wierszy, jeśli trzeba
        if max_wiersz != i: # sprawdzamy, czy znaleziony najlepszy wiersz jest
inny niż aktualny
            macierz[i], macierz[max_wiersz] = macierz[max_wiersz], macierz[i] #
zamieniamy miejscami wiersze macierzy
            wyrazy_wolne[i], wyrazy_wolne[max_wiersz] = wyrazy_wolne[max_wiersz],
wyrazy_wolne[i] # zamieniamy też odpowiednie wyrazy wolne

        # zerowanie elementów pod przekątną
        for j in range(i + 1, n): # przechodzimy po wierszach poniżej aktualnego
wiersza
            wspolczynnik = macierz[j][i] / macierz[i][i] # obliczamy współczynnik
potrzebny do wyzerowania elementu pod przekątną

            for k in range(i, n): # przechodzimy po kolumnach od aktualnej do
ostatniej
                macierz[j][k] = macierz[j][k] - wspolczynnik * macierz[i][k] #
odejmujemy odpowiednią wielokrotność wiersza głównego

            wyrazy_wolne[j] = wyrazy_wolne[j] - wspolczynnik * wyrazy_wolne[i] #
aktualizujemy wyraz wolny w tym samym wierszu

    # podstawianie wsteczne
    rozwiazanie = [0.0] * n # tworzymy listę na rozwiązania, początkowo
```

wypełnioną zerami

```
    for i in range(n - 1, -1, -1): # przechodzimy od ostatniego równania do
pierwszego
    suma = 0.0 # tworzymy zmienną pomocniczą na sumę znanych składników

    for j in range(i + 1, n): # przechodzimy po niewiadomych, które są już
obliczone
        suma += macierz[i][j] * rozwiazanie[j] # dodajemy znany składnik
równania do sumy

    rozwiazanie[i] = (wyrazy_wolne[i] - suma) / macierz[i][i] # obliczamy
aktualną niewiadomą

    return rozwiazanie # zwracamy listę rozwiązań układu

print("----- ZADANIE 2 -----") # wypisujemy nagłówek
zadania

n = len(punkty) # zapisujemy liczbę punktów pomiarowych

suma_x = 0.0 # suma wartości x
suma_x2 = 0.0 # suma wartości x^2
suma_x3 = 0.0 # suma wartości x^3
suma_x4 = 0.0 # suma wartości x^4
suma_y = 0.0 # suma wartości y
suma_xy = 0.0 # suma wartości x*y
suma_x2y = 0.0 # suma wartości x^2*y

for x, y in punkty: # przechodzimy po wszystkich punktach
    suma_x += x # dodajemy aktualne x do sumy x
    suma_x2 += x ** 2 # dodajemy x^2 do sumy x^2
    suma_x3 += x ** 3 # dodajemy x^3 do sumy x^3
    suma_x4 += x ** 4 # dodajemy x^4 do sumy x^4
    suma_y += y # dodajemy aktualne y do sumy y
    suma_xy += x * y # dodajemy iloczyn x*y do sumy
    suma_x2y += x ** 2 * y # dodajemy iloczyn x^2*y do sumy

macierz_ukladu = [ # tworzymy macierz układu równań normalnych
    [suma_x4, suma_x3, suma_x2], # pierwszy wiersz macierzy układu
    [suma_x3, suma_x2, suma_x], # drugi wiersz macierzy układu
    [suma_x2, suma_x, n] # trzeci wiersz macierzy układu
] # koniec definicji macierzy układu

wyrazy_wolne = [suma_x2y, suma_xy, suma_y] # tworzymy wektor wyrazów wolnych
układu równań normalnych

macierz_kopia = [] # tworzymy pustą listę na kopię macierzy układu

for wiersz in macierz_ukladu: # przechodzimy po każdym wierszu macierzy układu
```

```

macierz_kopia.append(wiersz.copy()) # dodajemy kopię wiersza, żeby nie
zmieniać oryginalnej macierzy

wyrazy_wolne_kopia = wyrazy_wolne.copy() # tworzymy kopię wektora wyrazów wolnych

rozwiazanie = rozwarz_uklad_gaussa(macierz_kopia, wyrazy_wolne_kopia) #
rozwiązujemy układ równań normalnych

a = rozwiazanie[0] # pierwszy współczynnik wielomianu, stojący przy x^2
b = rozwiazanie[1] # drugi współczynnik wielomianu, stojący przy x
c = rozwiazanie[2] # trzeci współczynnik wielomianu, wyraz wolny

print("Współczynniki wielomianu aproksymacyjnego:") # wypisujemy opis
współczynników
print("a =", a) # wypisujemy współczynnik a
print("b =", b) # wypisujemy współczynnik b
print("c =", c) # wypisujemy współczynnik c

print("\nWielomian aproksymacyjny:") # wypisujemy opis wielomianu
print(f"y = {a} * x^2 + {b} * x + {c}") # wypisujemy wzór wielomianu
aproksymacyjnego

h = 0.0 # tworzymy zmienną na sumę kwadratów błędów

for x, y in punkty: # przechodzimy po wszystkich punktach
    h += (a * x**2 + b * x + c - y) ** 2 # dodajemy kwadrat różnicy między
wartością wielomianu a wartością y

print("\nSuma kwadratów błędów:") # wypisujemy opis sumy błędów
print(h) # wypisujemy sumę kwadratów błędów

```

Wnioski

W zadaniu wyznaczono wielomian aproksymacyjny drugiego stopnia metodą najmniejszych kwadratów dla punktów:

$$(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91).$$

Na podstawie wzorów ze slajdów zbudowano układ równań:

$$a \sum x_i^4 + b \sum x_i^3 + c \sum x_i^2 = \sum x_i^2 y_i,$$

$$a \sum x_i^3 + b \sum x_i^2 + c \sum x_i = \sum x_i y_i,$$

$$a \sum x_i^2 + b \sum x_i + nc = \sum y_i,$$

a następnie rozwiązano go metodą Gaussa.

Otrzymane współczynniki są równe:

$$a = -\frac{67}{175}, \quad b = \frac{2159}{1750}, \quad c = \frac{6953}{3500},$$

czyli w przybliżeniu:

$$a \approx -0.38285714285714284,$$

$$b \approx 1.2337142857142858,$$

$$c \approx 1.9865714285714287.$$

Zatem wielomian aproksymacyjny ma postać:

$$F(x) = -0.38285714285714284x^2 + 1.2337142857142858x + 1.9865714285714287.$$

Suma kwadratów błędów wynosi:

$$h \approx 0.00170285714285714.$$

Oznacza to, że otrzymany wielomian bardzo dobrze przybliża dane punkty.

Zadanie 3

Napisz funkcję, która dla zadanego zbioru punktów oraz stopnia wielomianu optymalnego wyznaczy współczynniki wielomianu oraz błąd aproksymacji metodą najmniejszych kwadratów.

O co chodzi w tym zadaniu?

Zadanie 3 jest uogólnieniem zadań 1 i 2.

W zadaniu 1 szukaliśmy prostej:

$$F(x) = ax + b$$

czyli wielomianu stopnia 1.

W zadaniu 2 szukaliśmy wielomianu drugiego stopnia:

$$F(x) = ax^2 + bx + c$$

W zadaniu 3 mamy napisać funkcję, która zrobi to samo, ale dla **dowolnego stopnia wielomianu** podanego przez użytkownika.

Czyli jeśli podamy:

- **stopien = 1**, funkcja wyznaczy prostą,
- **stopien = 2**, funkcja wyznaczy wielomian kwadratowy,
- **stopien = 3**, funkcja wyznaczy wielomian trzeciego stopnia,
- itd.

Wzór ogólny ze slajdów

Na slajdach funkcja aproksymująca ma postać:

$$F(x) = a_0\phi_0(x) + a_1\phi_1(x) + \dots + a_m\phi_m(x),$$

gdzie dla uproszczenia przyjmujemy:

$$\phi_k(x) = x^k.$$

Wtedy funkcję aproksymującą można zapisać jako wielomian:

$$F(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m.$$

Współczynniki:

$$a_0, a_1, a_2, \dots, a_m$$

są szukanymi współczynnikami wielomianu aproksymacyjnego.

Funkcja błędu

Zgodnie ze slajdami minimalizujemy funkcję błędu:

$$h(a_0, a_1, \dots, a_m) = \sum_{i=1}^n \left(\sum_{j=0}^m a_j x_i^j - y_i \right)^2.$$

Oznacza to, że dla każdego punktu sprawdzamy różnicę między:

- wartością wielomianu w punkcie (x_i),
- wartością rzeczywistą (y_i),

a potem sumujemy kwadraty tych różnic.

Układ równań normalnych

Ze slajdów wynika, że aby znaleźć najlepsze współczynniki, trzeba rozwiązać układ równań:

$$\sum_{j=0}^m a_j \sum_{i=1}^n x_i^{j+k} = \sum_{i=1}^n x_i^k y_i,$$

dla:

$$k = 0, 1, \dots, m.$$

To oznacza, że dla wielomianu stopnia (m) powstaje układ:

$$m + 1$$

równań z:

$$m + 1$$

niewiadomymi.

Jak program buduje układ równań?

Program tworzy macierz układu:

$$A$$

oraz wektor wyrazów wolnych:

$$B.$$

Elementy macierzy są liczone ze wzoru:

$$A_{k,j} = \sum_{i=1}^n x_i^{k+j}$$

a elementy wektora prawej strony:

$$B_k = \sum_{i=1}^n x_i^k y_i.$$

Dzięki temu program sam tworzy układ równań dla dowolnego stopnia wielomianu.

Dane użyte w przykładzie

W programie użyto punktów z zadania 2:

$$(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91)$$

oraz stopnia:

$$m = 2.$$

Czyli szukamy wielomianu:

$$F(x) = a_0 + a_1x + a_2x^2.$$

Dla tego przykładu wynik powinien zgadzać się z zadaniem 2, tylko współczynniki są zapisane w innej kolejności.

W zadaniu 2 pisaliśmy:

$$F(x) = ax^2 + bx + c.$$

W zadaniu 3 piszemy:

$$F(x) = a_0 + a_1x + a_2x^2.$$

Zatem:

$$a_0 = c, \quad a_1 = b, \quad a_2 = a.$$

Wynik dla danych z programu

Dla danych:

$$(0, 2), (0.5, 2.48), (1, 2.84), (1.5, 3), (2, 2.91)$$

oraz stopnia:

$$m = 2$$

otrzymujemy:

$$a_0 \approx 1.9865714285714287$$

$$a_1 \approx 1.2337142857142858$$

$$a_2 \approx -0.38285714285714284$$

czyli:

$$F(x) = 1.9865714285714287 + 1.2337142857142858x - 0.38285714285714284x^2.$$

Po uporządkowaniu według malejących potęg:

$$F(x) = -0.38285714285714284x^2 + 1.2337142857142858x + 1.9865714285714287.$$

Suma kwadratów błędów wynosi:

$$h \approx 0.00170285714285714.$$


```
print("----- ZADANIE 3 -----") # wypisujemy nagłówek zadania

def rozwiaż_układ_gausa(macierz, wyrazy_wolne): # definiujemy funkcję
    rozwiążującą układ równań metodą Gaussa
    n = len(wyrazy_wolne) # zapisujemy liczbę równań i niewiadomych

    for i in range(n): # przechodzimy po kolejnych kolumnach eliminacji
        if macierz[i][i] == 0: # sprawdzamy, czy element główny na przekątnej nie
            jest zerem
            raise ValueError("Na przekątnej pojawiło się zero - nie można wykonać
            eliminacji Gaussa.") # jeśli jest zero, zgłaszamy błąd

        for j in range(i + 1, n): # przechodzimy po wierszach znajdujących się
            pod aktualnym wierszem
                współczynnik = macierz[j][i] / macierz[i][i] # obliczamy współczynnik
                potrzebny do wyzerowania elementu pod przekątną

                for k in range(i, n): # przechodzimy po elementach aktualnego wiersza
                    od kolumny i do końca
                        macierz[j][k] = macierz[j][k] - współczynnik * macierz[i][k] #
                    wykonujemy eliminację, czyli zerujemy element pod przekątną

                wyrazy_wolne[j] = wyrazy_wolne[j] - współczynnik * wyrazy_wolne[i] #
                wykonujemy tę samą operację na wektorze wyrazów wolnych

    rozwiązanie = [0.0] * n # tworzymy listę na rozwiązanie układu

    for i in range(n - 1, -1, -1): # wykonujemy podstawianie wsteczne od
        ostatniego równania do pierwszego
            suma = 0.0 # tworzymy zmienną pomocniczą na sumę znanych składników
            for j in range(i + 1, n): # przechodzimy po niewiadomych, które zostały
                już obliczone
                    suma += macierz[i][j] * rozwiązanie[j] # dodajemy iloczyny
                współczynników i znanych rozwiązań

            rozwiązanie[i] = (wyrazy_wolne[i] - suma) / macierz[i][i] # obliczamy
            bieżącą niewiadomą

    return rozwiązanie # zwracamy rozwiązanie układu

def aproksymacja_najmniejszych_kwadratów(punkty, stopień): # definiujemy funkcję
    aproksymacji dla dowolnego stopnia wielomianu
        liczba_współczynników = stopień + 1 # obliczamy liczbę współczynników, bo dla
        stopnia m mamy m+1 współczynników

        macierz_układu = [] # tworzymy pustą macierz układu równań
        wyrazy_wolne = [] # tworzymy pusty wektor wyrazów wolnych
```

```

for i in range(liczba_wspolczynnikow): # tworzymy kolejne równania układu
    wiersz = [] # tworzymy pusty wiersz macierzy

    for j in range(liczba_wspolczynnikow): # tworzymy kolejne elementy w
danym wierszu macierzy
        suma = 0.0 # zerujemy sumę dla elementu macierzy

        for x, y in punkty: # przechodzimy po wszystkich punktach
            suma += x ** (i + j) # dodajemy składnik  $x_i^{(i+j)}$ , zgodnie ze
wzorem na macierz układu

        wiersz.append(suma) # dodajemy obliczoną sumę do wiersza macierzy

    macierz_ukladu.append(wiersz) # dodajemy gotowy wiersz do macierzy układu

    suma = 0.0 # zerujemy sumę dla wyrazu wolnego

    for x, y in punkty: # przechodzimy po wszystkich punktach
        suma += (x ** i) * y # dodajemy składnik  $x_i^i * y_i$ , zgodnie ze
wzorem na prawą stronę układu

    wyrazy_wolne.append(suma) # dodajemy obliczoną sumę do wektora wyrazów
wolnych

macierz_kopia = [] # tworzymy pustą listę na kopię macierzy
for wiersz in macierz_ukladu: # przechodzimy po wszystkich wierszach macierzy
    macierz_kopia.append(wiersz.copy()) # kopiujemy wiersz, aby metoda Gaussa
nie zmieniła oryginalnej macierzy

wyrazy_wolne_kopia = wyrazy_wolne.copy() # kopiujemy wektor wyrazów wolnych,
aby zachować oryginał

wspolczynniki = rozwarz_uklad_gaussa(macierz_kopia, wyrazy_wolne_kopia) #
rozwiązujemy układ równań i otrzymujemy współczynniki wielomianu

h = 0.0 # tworzymy zmienną h na sumę kwadratów błędów

for x, y in punkty: # przechodzimy po wszystkich punktach
    y_aprox = 0.0 # tworzymy zmienną na wartość wielomianu w punkcie x

    for i in range(len(wspolczynniki)): # przechodzimy po wszystkich
współczynnikach wielomianu
        y_aprox += wspolczynniki[i] * (x ** i) # dodajemy składnik  $a_i * x^i$ 
do wartości wielomianu

    h += (y_aprox - y) ** 2 # dodajemy kwadrat różnicy między wartością
aproksymowaną a rzeczywistą

return wspolczynniki, h # zwracamy współczynniki wielomianu i sumę kwadratów
błędów

punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)] #

```

```

zapisujemy dane punkty
stopien = 2 # ustawiamy stopień wielomianu aproksymacyjnego

wspolczynniki, h = aproksymacja_najmniejszych_kwadratow(punkty, stopien) #
wywołujemy funkcję aproksymacji

print("Współczynniki wielomianu aproksymacyjnego:") # wypisujemy nagłówek dla
współczynników
for i in range(len(wspolczynniki)): # przechodzimy po wszystkich współczynnikach
    print(f"a{i} = {wspolczynniki[i]}") # wypisujemy współczynnik a_i

print("\nWielomian aproksymacyjny:") # wypisujemy nagłówek dla wielomianu
print("y =", end=" ") # rozpoczynamy wypisywanie wzoru wielomianu

for i in range(len(wspolczynniki)): # przechodzimy po wszystkich współczynnikach
    if i == 0: # sprawdzamy, czy wypisujemy pierwszy składnik
        print(f"{wspolczynniki[i]}", end="") # wypisujemy wyraz wolny bez znaku
        plus na początku
    else: # jeśli to nie jest pierwszy składnik
        print(f" + {wspolczynniki[i]} * x^{i}", end="") # wypisujemy kolejny
        składnik wielomianu

print("\n\nSuma kwadratów błędów:") # wypisujemy nagłówek dla błędów
print(h) # wypisujemy sumę kwadratów błędów

```

Wersja z zamianą wierszy żeby nie wywaliło się jak znajdzie 0

```

def rozwarz_uklad_gaussa(A, b): # funkcja rozwiązuje układ równań liniowych Ax =
b metodą Gaussa
    n = len(b) # liczba niewiadomych, czyli długość wektora wyrazów wolnych

    for i in range(n): # przechodzimy po kolejnych kolumnach i wierszach głównych

        max_wiersz = i # zakładamy, że najlepszy wiersz to aktualny wiersz i

        for k in range(i + 1, n): # sprawdzamy wiersze poniżej aktualnego wiersza
            if abs(A[k][i]) > abs(A[max_wiersz][i]): # szukamy największej
wartości bezwzględnej w kolumnie i
                max_wiersz = k # zapamiętujemy numer wiersza z największym
elementem

        if A[max_wiersz][i] == 0: # jeśli największy element w kolumnie jest
zerem
            raise ValueError("Układ nie ma jednoznacznego rozwiązania.") # układ
nie ma jednoznacznego rozwiązania

        if max_wiersz != i: # sprawdzamy, czy trzeba zamienić wiersze
            A[i], A[max_wiersz] = A[max_wiersz], A[i] # zamieniamy wiersze w
macierzy A
            b[i], b[max_wiersz] = b[max_wiersz], b[i] # zamieniamy odpowiadające
im wyrazy wolne

        for j in range(i + 1, n): # przechodzimy po wierszach poniżej aktualnego
wiersza
            wspolczynnik = A[j][i] / A[i][i] # obliczamy współczynnik potrzebny
do wyzerowania elementu pod przekątną

            for k in range(i, n): # przechodzimy po kolumnach od aktualnej do
ostatniej
                A[j][k] = A[j][k] - wspolczynnik * A[i][k] # odejmujemy
odpowiednią wielokrotność wiersza głównego

            b[j] = b[j] - wspolczynnik * b[i] # aktualizujemy wyraz wolny w tym
samym wierszu

    x = [0.0] * n # tworzymy listę na rozwiązanie układu, początkowo wypełnioną
zerami

    for i in range(n - 1, -1, -1): # wykonujemy podstawianie wsteczne od
ostatniego równania do pierwszego
        suma = 0.0 # zmienna pomocnicza na sumę znanych składników

        for j in range(i + 1, n): # przechodzimy po niewiadomych, które są już
obliczone
            suma += A[i][j] * x[j] # dodajemy znany składnik równania do sumy

        x[i] = (b[i] - suma) / A[i][i] # obliczamy aktualną niewiadomą

    return x # zwracamy rozwiązanie układu

```

```

def aproksymacja_najmniejszych_kwadratow(punkty, stopien): # funkcja liczy
    aproksymację średniokwadratową wielomianem danego stopnia
    n = stopien + 1 # liczba współczynników wielomianu

    A = [] # tworzymy pustą macierz układu równań normalnych
    B = [] # tworzymy pusty wektor wyrazów wolnych

    for i in range(n): # przechodzimy po kolejnych równaniach układu
        wiersz = [] # tworzymy pusty wiersz macierzy A

        for j in range(n): # przechodzimy po kolejnych kolumnach macierzy A
            suma = 0.0 # tworzymy zmienną na sumę

            for x, y in punkty: # przechodzimy po wszystkich punktach
                suma += x ** (i + j) # dodajemy x podniesione do odpowiedniej
potęgi

            wiersz.append(suma) # dodajemy obliczoną sumę do wiersza macierzy

        A.append(wiersz) # dodajemy gotowy wiersz do macierzy A

        suma = 0.0 # zerujemy sumę dla wyrazu wolnego

        for x, y in punkty: # przechodzimy po wszystkich punktach
            suma += y * (x ** i) # dodajemy składnik y razy x do odpowiedniej
potęgi

        B.append(suma) # dodajemy obliczony wyraz wolny do wektora B

    A_kopia = [] # tworzymy pustą listę na kopię macierzy A

    for wiersz in A: # przechodzimy po wierszach macierzy A
        A_kopia.append(wiersz.copy()) # kopiujemy każdy wiersz, żeby nie zmieniać
oryginalnej macierzy

    B_kopia = B.copy() # tworzymy kopię wektora wyrazów wolnych

    wspolczynniki = rozwarz_uklad_gaussa(A_kopia, B_kopia) # rozwiązujemy układ
równań i otrzymujemy współczynniki wielomianu

    h = 0.0 # zmienna na sumę kwadratów błędów

    for x, y in punkty: # przechodzimy po wszystkich punktach
        y_aprox = 0.0 # zmienna na wartość wielomianu aproksymacyjnego w punkcie x

        for i in range(len(wspolczynniki)): # przechodzimy po wszystkich
współczynnikach wielomianu
            y_aprox += wspolczynniki[i] * (x ** i) # dodajemy kolejny składnik
wielomianu

        h += (y_aprox - y) ** 2 # dodajemy kwadrat błędu dla aktualnego punktu

```

```

    return współczynniki, h # zwracamy współczynniki wielomianu i sumę kwadratów
    błędów

punkty = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)] # lista
punktów pomiarowych

stopien = 2 # stopień wielomianu aproksymacyjnego

współczynniki, blad = aproksymacja_najmniejszych_kwadratow(punkty, stopien) #
wywołujemy funkcję aproksymacji

print("Współczynniki wielomianu:") # wypisujemy opis wyniku

for i in range(len(współczynniki)): # przechodzimy po kolejnych współczynnikach
    print(f"a{i} = {współczynniki[i]}") # wypisujemy współczynnik ai

print("\nWielomian aproksymacyjny:") # wypisujemy opis wielomianu
print(f"y = {współczynniki[0]} + {współczynniki[1]} * x + {współczynniki[2]} *
x^2") # wypisujemy wzór wielomianu

print("\nSuma kwadratów błędów:") # wypisujemy opis błędu
print(blad) # wypisujemy sumę kwadratów błędów

```

Wnioski

W zadaniu 3 napisano funkcję, która uogólnia aproksymację średniokwadratową na dowolny stopień wielomianu.

Zamiast osobno tworzyć wzory dla prostej lub dla wielomianu drugiego stopnia, program automatycznie buduje układ równań normalnych na podstawie wzoru ze slajdów:

$$\sum_{j=0}^m a_j \sum_{i=1}^n x_i^{j+k} = \sum_{i=1}^n x_i^k y_i.$$

Następnie układ ten jest rozwiązywany metodą Gaussa.

Dla danych z zadania 2 oraz stopnia:

$$m = 2$$

otrzymano współczynniki:

$$a_0 \approx 1.9865714285714287$$

$$a_1 \approx 1.2337142857142858$$

$$a_2 \approx -0.38285714285714284$$

czyli wielomian:

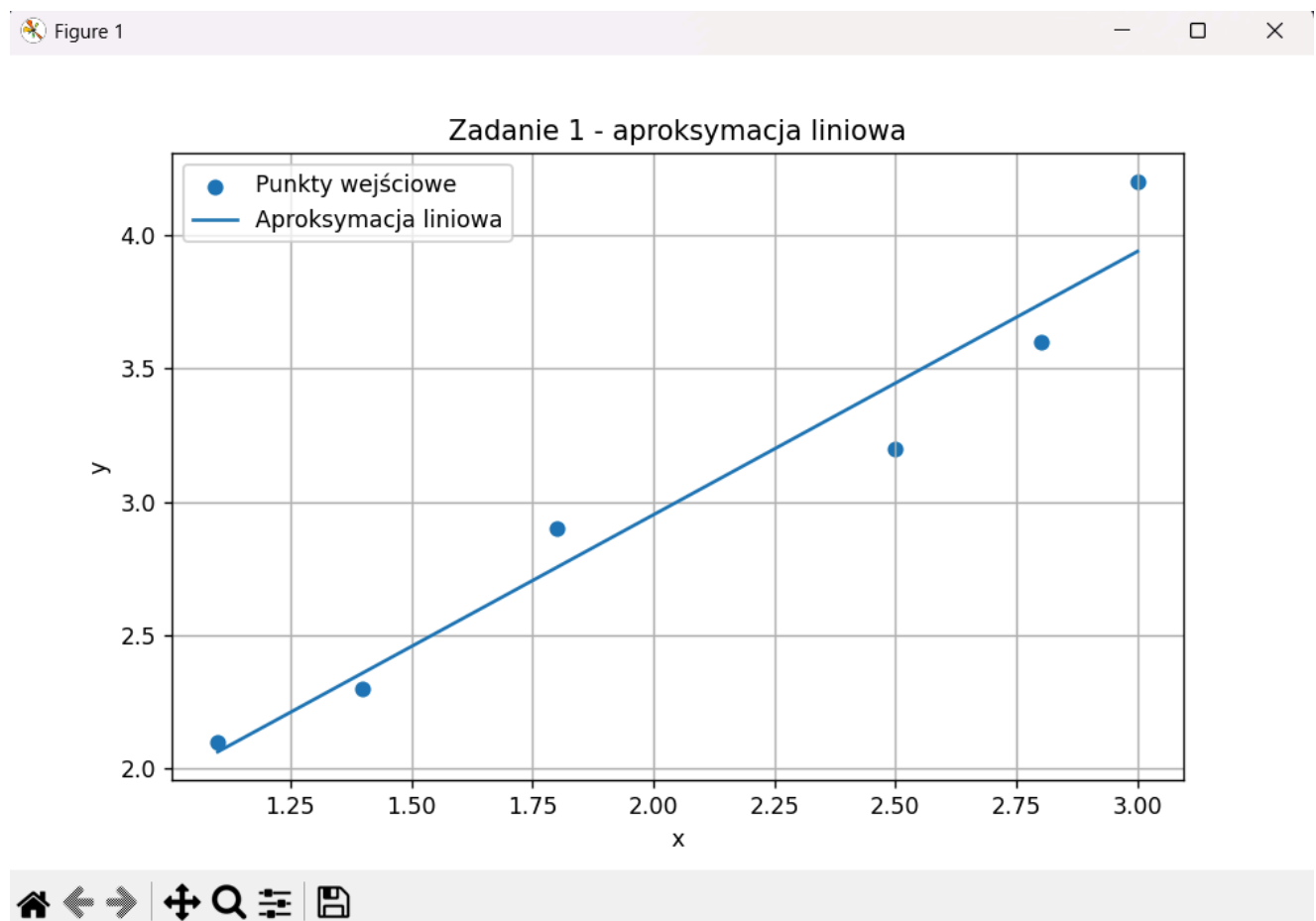
$$F(x) = 1.9865714285714287 + 1.2337142857142858x - 0.38285714285714284x^2.$$

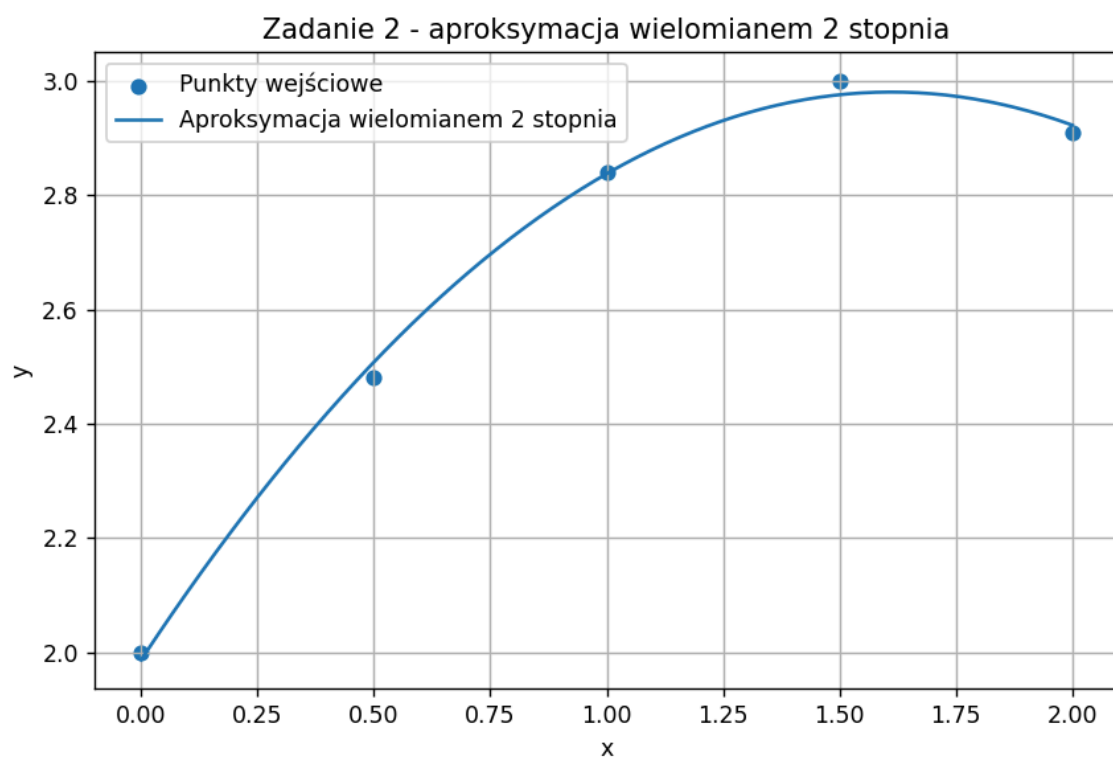
Suma kwadratów błędów wynosi:

$$h \approx 0.00170285714285714.$$

Wynik jest zgodny z zadaniem 2, ponieważ dla stopnia 2 funkcja ogólna tworzy ten sam wielomian aproksymacyjny.

Zadanie 4*. Rozwiązania dwóch pierwszych zadań przedstaw na wykresie, zaznaczając również zaobserwowane wartości funkcji.





#zad 4

```
import matplotlib.pyplot as plt
```

```
print("----- ZADANIE 4 -----")
```

```
# ===== Zadanie 1: aproksymacja liniowa =====
```

```
punkty1 = [(1.1, 2.1), (1.4, 2.3), (1.8, 2.9), (2.5, 3.2), (2.8, 3.6), (3.0, 4.2)]
```

```
n1 = len(punkty1)
```

```
Sx = 0.0
```

```
Sy = 0.0
```

```
Sxx = 0.0
```

```
Sxy = 0.0
```

```
for x, y in punkty1:
```

```
    Sx += x
```

```
    Sy += y
```

```
    Sxx += x * x
```

```
    Sxy += x * y
```

```
a1 = (n1 * Sxy - Sx * Sy) / (n1 * Sxx - Sx * Sx)
```

```
b1 = (Sy - a1 * Sx) / n1
```

```
x1 = [p[0] for p in punkty1]
```

```
y1 = [p[1] for p in punkty1]
```

```

x1_wykres = []

y1_wykres = []


xmin1 = min(x1)

xmax1 = max(x1)


for i in range(200):

    X = xmin1 + (xmax1 - xmin1) * i / 199

    Y = a1 * X + b1

    x1_wykres.append(X)

    y1_wykres.append(Y)


plt.figure(figsize=(8, 5))

plt.scatter(x1, y1, label="Punkty wejściowe")

plt.plot(x1_wykres, y1_wykres, label="Aproksymacja liniowa")

plt.title("Zadanie 1 - aproksymacja liniowa")

plt.xlabel("x")

plt.ylabel("y")

plt.legend()

plt.grid(True)

plt.show()


# ===== Zadanie 2: aproksymacja wielomianem 2 stopnia =====

punkty2 = [(0.0, 2.0), (0.5, 2.48), (1.0, 2.84), (1.5, 3.0), (2.0, 2.91)]


def rozwiaz_uklad_gaussa(A, b):

```

```

n = len(b)

for i in range(n):

    max_wiersz = i

    for j in range(i + 1, n):

        if abs(A[j][i]) > abs(A[max_wiersz][i]):

            max_wiersz = j

    A[i], A[max_wiersz] = A[max_wiersz], A[i]

    b[i], b[max_wiersz] = b[max_wiersz], b[i]

    for j in range(i + 1, n):

        wsp = A[j][i] / A[i][i]

        for k in range(i, n):

            A[j][k] -= wsp * A[i][k]

        b[j] -= wsp * b[i]

x = [0.0] * n

for i in range(n - 1, -1, -1):

    suma = 0.0

    for j in range(i + 1, n):

        suma += A[i][j] * x[j]

    x[i] = (b[i] - suma) / A[i][i]

return x

```

Sx = 0.0

```
Sx2 = 0.0
```

```
Sx3 = 0.0
```

```
Sx4 = 0.0
```

```
Sy = 0.0
```

```
Sxy = 0.0
```

```
Sx2y = 0.0
```

```
for x, y in punkty2:
```

```
    Sx += x
```

```
    Sx2 += x**2
```

```
    Sx3 += x**3
```

```
    Sx4 += x**4
```

```
    Sy += y
```

```
    Sxy += x * y
```

```
    Sx2y += x**2 * y
```

```
n2 = len(punkty2)
```

```
A = [
```

```
    [Sx4, Sx3, Sx2],
```

```
    [Sx3, Sx2, Sx],
```

```
    [Sx2, Sx, n2]
```

```
]
```

```
B = [Sx2y, Sxy, Sy]
```

```
a2, b2, c2 = rozwiaz_uklad_gaussa(A, B)

x2 = [p[0] for p in punkty2]

y2 = [p[1] for p in punkty2]

x2_wykres = []

y2_wykres = []

xmin2 = min(x2)

xmax2 = max(x2)

for i in range(200):

    X = xmin2 + (xmax2 - xmin2) * i / 199

    Y = a2 * X**2 + b2 * X + c2

    x2_wykres.append(X)

    y2_wykres.append(Y)

plt.figure(figsize=(8, 5))

plt.scatter(x2, y2, label="Punkty wejściowe")

plt.plot(x2_wykres, y2_wykres, label="Aproksymacja wielomianem 2 stopnia")

plt.title("Zadanie 2 - aproksymacja wielomianem 2 stopnia")

plt.xlabel("x")

plt.ylabel("y")

plt.legend()

plt.grid(True)

plt.show()
```

Lab 10

Zadanie 1

Napisz program implementujący różniczkowanie numeryczne za pomocą metody Newtona dla następujących funkcji:

a)

$$f(x) = 2x^2 + 2$$

b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

c)

$$f(x) = x^2 e^x$$

Oblicz błąd względny otrzymanego rozwiązania dla:

$$h = 10^{-2}$$

oraz

$$h = 10^{-4}$$

W programie pochodną liczymy w punkcie:

$$x = 1$$

Co to jest różniczkowanie numeryczne?

Różniczkowanie numeryczne to metoda przybliżonego obliczania pochodnych funkcji na podstawie wartości tej funkcji w wybranych punktach.

Stosujemy je wtedy, gdy:

- trudno jest policzyć pochodną analitycznie,
- funkcja jest dana tylko przez wartości w punktach,
- chcemy szybko przybliżyć wartość pochodnej.

Różnice skończone można wyprowadzić z ilorazu różnicowego albo z rozwinięcia Taylora.

Metoda Newtona — różnica w przód

W tym zadaniu mamy użyć metody Newtona, czyli **dwupunktowej różnicy w przód**.

Wzór ze slajdu:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad O(h)$$

gdzie:

- x — punkt, w którym liczymy pochodną,
- h — mały krok,
- $f(x+h)$ — wartość funkcji w punkcie przesuniętym o h w prawo,
- $f(x)$ — wartość funkcji w punkcie x .

Metoda nazywa się „w przód”, ponieważ korzysta z punktów:

$$x$$

oraz

$$x+h$$

czyli patrzy w prawą stronę od punktu x .

Błąd względny

Na slajdzie błąd względny zapisano jako:

$$\delta x = \frac{|x - \bar{x}|}{|x|}$$

W tym zadaniu nie porównujemy zwykłych liczb, tylko wartości pochodnych.

Dlatego przyjmujemy:

$$x = f'_{\text{dokładna}}(x)$$

oraz:

$$\bar{x} = f'_{\text{przybliżona}}(x)$$

Zatem błąd względny liczymy ze wzoru:

$$\delta = \frac{|f'_{\text{dokładna}}(x) - f'_{\text{przybliżona}}(x)|}{|f'_{\text{dokładna}}(x)|}$$

Pochodne dokładne funkcji

Aby policzyć błąd względny, musimy znać dokładne wartości pochodnych.

Funkcja a)

$$f(x) = 2x^2 + 2$$

Pochodna dokładna:

$$f'(x) = 4x$$

Funkcja b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

Pochodna dokładna:

$$f'(x) = 8x^3 - 2x + 3$$

Funkcja c)

$$f(x) = x^2 e^x$$

Korzystamy z reguły iloczynu:

$$(x^2 e^x)' = (x^2)' e^x + x^2 (e^x)'$$

czyli:

$$f'(x) = 2x e^x + x^2 e^x$$

Po wyłączeniu e^x :

$$f'(x) = e^x (x^2 + 2x)$$

Co robi program?

Program:

1. Definiuje funkcję do liczenia pochodnej metodą różnicy w przód.
2. Definiuje funkcję do liczenia błędu względnego.
3. Definiuje funkcje z zadania oraz ich pochodne dokładne.
4. Dla każdej funkcji liczy pochodną numeryczną dla:

$$h = 10^{-2}$$

oraz:

$$h = 10^{-4}$$

5. Porównuje pochodną numeryczną z pochodną dokładną.
6. Wypisuje błąd względny.


```
import math # importujemy moduł math, ponieważ potrzebujemy funkcji exp do
obliczania e^x

print("-----ZADANIE 1-----") # wypisujemy
nagłówek zadania

def roznica_w_przod(f, x, h): # definiujemy funkcję liczącą pochodną metodą
różnicy w przód
    return (f(x + h) - f(x)) / h # zwracamy przybliżenie pochodnej ze wzoru:
f'(x) ≈ (f(x+h)-f(x))/h

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona): # definiujemy funkcję
liczącą błąd względny
    if wartosc_dokladna != 0: # sprawdzamy, czy wartość dokładna nie jest zerem,
    żeby nie dzielić przez zero
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
    # zwracamy błąd względny zgodnie ze wzorem ze slajdu
    else: # obsługujemy przypadek, gdy wartość dokładna jest równa zero
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna
jest równa 0.") # zgłaszamy błąd, ponieważ dzielenie przez zero jest niemożliwe

def f1(x): # definiujemy pierwszą funkcję z zadania
    return 2 * x**2 + 2 # zwracamy wartość funkcji f(x)=2x^2+2

def df1(x): # definiujemy dokładną pochodną pierwszej funkcji
    return 4 * x # zwracamy wartość pochodnej f'(x)=4x

def f2(x): # definiujemy drugą funkcję z zadania
    return 2 * x**4 - x**2 + 3*x - 7 # zwracamy wartość funkcji f(x)=2x^4-x^2+3x-
7

def df2(x): # definiujemy dokładną pochodną drugiej funkcji
    return 8 * x**3 - 2 * x + 3 # zwracamy wartość pochodnej f'(x)=8x^3-2x+3

def f3(x): # definiujemy trzecią funkcję z zadania
    return x**2 * math.exp(x) # zwracamy wartość funkcji f(x)=x^2e^x

def df3(x): # definiujemy dokładną pochodną trzeciej funkcji
    return math.exp(x) * (x**2 + 2 * x) # zwracamy wartość pochodnej
f'(x)=e^x(x^2+2x)

def porownaj_pochodna(nazwa, f, df, x, h_lista): # definiujemy funkcję
porównującą pochodną numeryczną z dokładną
    print("\n" + nazwa) # wypisujemy nazwę aktualnie badanej funkcji
    print("x =", x) # wypisujemy punkt, w którym liczymy pochodną
    print("h          pochodna numeryczna          pochodna dokładna
błąd względny") # wypisujemy nagłówek tabeli wyników

    for h in h_lista: # przechodzimy po kolejnych wartościach kroku h
        pochodna_numeryczna = roznica_w_przod(f, x, h) # liczymy pochodną
```

```

numeryczną metodą różnicy w przód
    pochodna_dokladna = df(x) # liczymy dokładną wartość pochodnej w punkcie x
    blad = blad_wzgledny(pochodna_dokladna, pochodna_numeryczna) # liczymy
    błąd względny między wartością dokładną i przybliżoną

    print(f"{h:<14} {pochodna_numeryczna:<25} {pochodna_dokladna:<25} {blad}")
# wypisujemy jeden wiersz tabeli z wynikami

x0 = 1.0 # ustalamy punkt x, w którym obliczamy pochodną

h_lista = [10**(-2), 10**(-4)] # tworzymy listę wartości h podanych w treści
zadania

porownaj_pochodna("a) f(x) = 2x^2 + 2", f1, df1, x0, h_lista) # wykonujemy
obliczenia dla funkcji a)
porownaj_pochodna("b) f(x) = 2x^4 - x^2 + 3x - 7", f2, df2, x0, h_lista) #
wykonujemy obliczenia dla funkcji b)
porownaj_pochodna("c) f(x) = x^2e^x", f3, df3, x0, h_lista) # wykonujemy
obliczenia dla funkcji c)

```

Wyniki

Dla funkcji:

$$f(x) = 2x^2 + 2$$

w punkcie:

$$x = 1$$

pochodna dokładna wynosi:

$$f'(1) = 4$$

Dla $h = 10^{-2}$ program otrzymuje wynik około:

$$4.02$$

a dla $h = 10^{-4}$:

$$4.0002$$

Widać więc, że mniejszy krok daje wynik bliższy wartości dokładnej.

Dla funkcji:

$$f(x) = 2x^4 - x^2 + 3x - 7$$

pochodna dokładna wynosi:

$$f'(x) = 8x^3 - 2x + 3$$

W punkcie:

$$x = 1$$

mamy:

$$f'(1) = 9$$

Program porównuje wartości otrzymane metodą różnicy w przód z wartością dokładną i oblicza błąd względny.

Dla funkcji:

$$f(x) = x^2 e^x$$

pochodna dokładna wynosi:

$$f'(x) = e^x(x^2 + 2x)$$

Dla:

$$x = 1$$

otrzymujemy:

$$f'(1) = 3e \approx 8.154845485$$

Program liczy przybliżenia tej wartości dla dwóch różnych kroków h .

Wnioski

W zadaniu zastosowano metodę Newtona, czyli dwupunktową różnicę w przód:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Dla każdej funkcji porównano wynik numeryczny z dokładną pochodną analityczną.

Błąd względny policzono ze wzoru:

$$\delta = \frac{|f'_{\text{dokładna}}(x) - f'_{\text{przybliżona}}(x)|}{|f'_{\text{dokładna}}(x)|}$$

Z wyników widać, że dla mniejszego kroku:

$$h = 10^{-4}$$

błąd względny jest mniejszy niż dla:

$$h = 10^{-2}$$

Oznacza to, że zmniejszenie kroku h poprawia dokładność przybliżenia pochodnej metodą różnicy w przód.

Trzeba jednak pamiętać, że dla bardzo małych wartości h mogą pojawić się błędy zaokrągleń wynikające z ograniczonej precyzji obliczeń komputerowych.

Zadanie 2

Przeprowadź obliczenia analogiczne jak w zadaniu 1 dla metod różnic skończonych: wstecznej i centralnej dwupunktowej.

W zadaniu korzystamy z tych samych funkcji co w zadaniu 1:

a)

$$f(x) = 2x^2 + 2$$

b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

c)

$$f(x) = x^2 e^x$$

Obliczenia wykonujemy dla:

$$h = 10^{-2}$$

oraz:

$$h = 10^{-4}$$

W programie pochodną liczymy w punkcie:

$$x = 1$$

Cel zadania

W zadaniu 1 używaliśmy różnicy w przód, czyli metody Newtona:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

W zadaniu 2 mamy wykonać podobne obliczenia, ale dla dwóch innych metod różnic skończonych:

- różnicy wstecznej,
- różnicy centralnej dwupunktowej.

Dla każdej metody porównujemy wynik numeryczny z pochodną dokładną i obliczamy błąd względny.

Różnica wsteczna

Na slajdzie podano wzór:

$$f'(x) \approx \frac{f(x) - f(x - h)}{h}$$

Ta metoda korzysta z wartości funkcji w punktach:

$$x$$

oraz:

$$x - h$$

czyli patrzy „wstecz” względem punktu x .

Błąd tej metody jest rzędu:

$$O(h)$$

To oznacza, że zmniejszenie kroku h powinno poprawiać dokładność wyniku.

Różnica centralna dwupunktowa

Na slajdzie podano wzór:

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$

Ta metoda korzysta z dwóch punktów położonych symetrycznie względem punktu x :

$$x - h$$

oraz:

$$x + h$$

Dlatego nazywa się metodą centralną.

Błąd tej metody jest rzędu:

$$O(h^2)$$

Oznacza to, że zwykle jest dokładniejsza niż różnica wsteczna oraz różnica w przód.

Błąd względny

Ze slajdu o błędzie względnym mamy wzór:

$$\delta x = \frac{|x - \bar{x}|}{|x|}$$

W tym zadaniu porównujemy pochodną dokładną z pochodną przybliżoną.

Dlatego podstawiamy:

$$x = f'_{\text{dokładna}}(x)$$

oraz:

$$\bar{x} = f'_{\text{przybliżona}}(x)$$

Otrzymujemy:

$$\delta = \frac{|f'_{\text{dokładna}}(x) - f'_{\text{przybliżona}}(x)|}{|f'_{\text{dokładna}}(x)|}$$

Pochodne dokładne funkcji

Tak jak w zadaniu 1, do policzenia błędu względnego potrzebujemy pochodnych dokładnych.

Funkcja a)

$$f(x) = 2x^2 + 2$$

Pochodna:

$$f'(x) = 4x$$

Funkcja b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

Pochodna:

$$f'(x) = 8x^3 - 2x + 3$$

Funkcja c)

$$f(x) = x^2 e^x$$

Pochodna:

$$f'(x) = e^x(x^2 + 2x)$$

Co robi program?

Program:

1. Definiuje funkcję liczącą pochodną metodą różnicy wstecznej.
2. Definiuje funkcję liczącą pochodną metodą różnicy centralnej dwupunktowej.
3. Definiuje funkcję liczącą błąd względny.

4. Definiuje funkcje z zadania oraz ich pochodne dokładne.
5. Dla każdej funkcji i dla każdego kroku h liczy:
 - pochodną metodą wsteczną,
 - pochodną metodą centralną,
 - błąd względny dla obu metod.
6. Wypisuje wyniki w tabeli.

Kod

```
print("-----ZADANIE 2-----") # wypisujemy nagłówek zadania

def roznica_wsteczna(f, x, h): # definiujemy funkcję liczącą pochodną metodą różnicy wstecznej
    return (f(x) - f(x - h)) / h # zwracamy przybliżenie pochodnej ze wzoru:  $f'(x) \approx (f(x) - f(x-h))/h$ 

def roznica_centralna_dwupunktowa(f, x, h): # definiujemy funkcję liczącą pochodną metodą różnicy centralnej dwupunktowej
    return (f(x + h) - f(x - h)) / (2 * h) # zwracamy przybliżenie pochodnej ze wzoru:  $f'(x) \approx (f(x+h) - f(x-h))/(2h)$ 

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona): # definiujemy funkcję liczącą błąd względny
    if wartosc_dokladna != 0: # sprawdzamy, czy wartość dokładna nie jest równa zero
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
    # zwracamy błąd względny zgodny ze wzorem ze slajdu
    else: # obsługujemy przypadek, gdy wartość dokładna jest równa zero
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna jest równa 0.") # zgłaszamy błąd, bo nie można dzielić przez zero

def f1(x): # definiujemy pierwszą funkcję z zadania
    return 2 * x**2 + 2 # zwracamy wartość funkcji  $f(x)=2x^2+2$ 

def df1(x): # definiujemy dokładną pochodną pierwszej funkcji
    return 4 * x # zwracamy wartość pochodnej  $f'(x)=4x$ 

def f2(x): # definiujemy drugą funkcję z zadania
    return 2 * x**4 - x**2 + 3*x - 7 # zwracamy wartość funkcji  $f(x)=2x^4-x^2+3x-7$ 

def df2(x): # definiujemy dokładną pochodną drugiej funkcji
    return 8 * x**3 - 2 * x + 3 # zwracamy wartość pochodnej  $f'(x)=8x^3-2x+3$ 

def f3(x): # definiujemy trzecią funkcję z zadania
    return x**2 * math.exp(x) # zwracamy wartość funkcji  $f(x)=x^2e^x$ 

def df3(x): # definiujemy dokładną pochodną trzeciej funkcji
    return math.exp(x) * (x**2 + 2 * x) # zwracamy wartość pochodnej  $f'(x)=e^x(x^2+2x)$ 

def porownaj_metody(nazwa, f, df, x, h_lista): # definiujemy funkcję porównującą metody dla jednej funkcji
    print("\n" + nazwa) # wypisujemy nazwę aktualnie badanej funkcji
    print("x =", x) # wypisujemy punkt, w którym liczymy pochodną
    print("h          metoda          pochodna numeryczna          pochodna
dokładna          błąd względny") # wypisujemy nagłówek tabeli
```



```

for h in h_lista: # przechodzimy po kolejnych wartościach kroku h
    pochodna_dokladna = df(x) # liczymy dokładną wartość pochodnej w punkcie x

    pochodna_wsteczna = roznica_wsteczna(f, x, h) # liczymy pochodną
    numeryczną metodą różnicy wstecznej
    blad_wsteczny = blad_wzgledny(pochodna_dokladna, pochodna_wsteczna) #
    liczymy błąd względny dla metody wstecznej

    print(f"{h:<14} {'wsteczna':<15} {pochodna_wsteczna:<25}
    {pochodna_dokladna:<25} {blad_wsteczny}") # wypisujemy wyniki dla różnicy
    wstecznej

    pochodna_centralna = roznica_centralna_dwupunktowa(f, x, h) # liczymy
    pochodną numeryczną metodą różnicy centralnej
    blad_centralny = blad_wzgledny(pochodna_dokladna, pochodna_centralna) #
    liczymy błąd względny dla metody centralnej

    print(f"{h:<14} {'centralna':<15} {pochodna_centralna:<25}
    {pochodna_dokladna:<25} {blad_centralny}") # wypisujemy wyniki dla różnicy
    centralnej

x0 = 1.0 # ustalamy punkt x, w którym obliczamy pochodną

h_lista = [10**(-2), 10**(-4)] # zapisujemy wartości h podane w treści zadania

porownaj_metody("a) f(x) = 2x^2 + 2", f1, df1, x0, h_lista) # wykonujemy
obliczenia dla funkcji a)
porownaj_metody("b) f(x) = 2x^4 - x^2 + 3x - 7", f2, df2, x0, h_lista) #
wykonujemy obliczenia dla funkcji b)
porownaj_metody("c) f(x) = x^2e^x", f3, df3, x0, h_lista) # wykonujemy obliczenia
dla funkcji c)

```

Wyniki i interpretacja

Dla funkcji:

$$f(x) = 2x^2 + 2$$

w punkcie:

$$x = 1$$

pochodna dokładna wynosi:

$$f'(1) = 4$$

Metoda wsteczna daje wynik trochę mniejszy od wartości dokładnej, a metoda centralna daje wynik praktycznie równy wartości dokładnej.

Dla funkcji:

$$f(x) = 2x^4 - x^2 + 3x - 7$$

w punkcie:

$$x = 1$$

pochodna dokładna wynosi:

$$f'(1) = 9$$

Metoda centralna daje wyraźnie mniejszy błąd niż metoda wsteczna.

Dla funkcji:

$$f(x) = x^2 e^x$$

w punkcie:

$$x = 1$$

pochodna dokładna wynosi:

$$f'(1) = 3e \approx 8.154845485$$

Także tutaj metoda centralna daje dokładniejszy wynik niż metoda wsteczna.

Wnioski

W zadaniu zastosowano dwie metody różnic skończonych:

Różnicę wsteczną

$$f'(x) \approx \frac{f(x) - f(x - h)}{h}$$

oraz różnicę centralną dwupunktową:

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$

Wyniki pokazują, że dla mniejszego kroku:

$$h = 10^{-4}$$

błędy są zwykle mniejsze niż dla:

$$h = 10^{-2}$$

Dodatkowo metoda centralna jest dokładniejsza od metody wstecznej, ponieważ jej błąd jest rzędu:

$$O(h^2)$$

a błąd metody wstecznej jest rzędu:

$$O(h)$$

Oznacza to, że różnica centralna szybciej poprawia dokładność przy zmniejszaniu kroku h .

Zadanie 3

Przeprowadź obliczenia analogiczne jak w zadaniu 1 dla metod różnic skończonych: w przód i wstecznej trzypunktowej oraz centralnej czteropunktowej.

W zadaniu korzystamy z tych samych funkcji co wcześniej:

a)

$$f(x) = 2x^2 + 2$$

b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

c)

$$f(x) = x^2 e^x$$

Obliczenia wykonujemy dla:

$$h = 10^{-2}$$

oraz:

$$h = 10^{-4}$$

W programie pochodną liczymy w punkcie:

$$x = 1$$

Cel zadania

W poprzednich zadaniach używaliśmy prostszych wzorów różnic skończonych:

- różnicy w przód,
- różnicy wstecznej,
- różnicy centralnej dwupunktowej.

W tym zadaniu stosujemy dokładniejsze wzory:

- trzypunktową różnicę w przód,
- trzypunktową różnicę wsteczną,
- czteropunktową różnicę centralną.

Dla każdej metody porównujemy wynik numeryczny z pochodną dokładną i obliczamy błąd względny.

Trzypunktowa różnica w przód

Ze slajdu mamy wzór:

$$f'(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h}$$

Ta metoda korzysta z trzech punktów:

$$x, \quad x+h, \quad x+2h$$

czyli używa punktów położonych „w przód” od punktu x .

Błąd tej metody jest rzędu:

$$O(h^2)$$

czyli metoda jest dokładniejsza od zwykłej dwupunktowej różnicy w przód, która miała błąd rzędu $O(h)$.

Trzypunktowa różnica wsteczna

Ze slajdu mamy wzór:

$$f'(x) \approx \frac{3f(x) - 4f(x-h) + f(x-2h)}{2h}$$

Ta metoda korzysta z trzech punktów:

$$x, \quad x-h, \quad x-2h$$

czyli używa punktów położonych „wstecz” od punktu x .

Błąd tej metody również jest rzędu:

$$O(h^2)$$

Czteropunktowa różnica centralna

Ze slajdu mamy wzór:

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$$

Ta metoda korzysta z punktów położonych symetrycznie wokół x :

$$x-2h, \quad x-h, \quad x+h, \quad x+2h$$

Jest to metoda centralna, ponieważ wykorzystuje wartości funkcji po obu stronach punktu x .

Błąd tej metody jest rzędu:

$$O(h^4)$$

czyli powinna być najdokładniejsza spośród metod użytych w tym zadaniu.

Błąd względny

Ze slajdu o błędzie względnym:

$$\delta x = \frac{|x - \bar{x}|}{|x|}$$

W tym zadaniu wartością dokładną jest pochodna dokładna, a wartością przybliżoną jest pochodna obliczona numerycznie.

Dlatego używamy wzoru:

$$\delta = \frac{|f'_{\text{dokładna}}(x) - f'_{\text{przybliżona}}(x)|}{|f'_{\text{dokładna}}(x)|}$$

Pochodne dokładne funkcji

Do obliczenia błędu względnego potrzebujemy pochodnych dokładnych.

Funkcja a)

$$f(x) = 2x^2 + 2$$

Pochodna:

$$f'(x) = 4x$$

Funkcja b)

$$f(x) = 2x^4 - x^2 + 3x - 7$$

Pochodna:

$$f'(x) = 8x^3 - 2x + 3$$

Funkcja c)

$$f(x) = x^2 e^x$$

Korzystamy z reguły iloczynu:

$$f'(x) = 2xe^x + x^2 e^x$$

czyli:

$$f'(x) = e^x(x^2 + 2x)$$

Co robi program?

Program:

1. Definiuje funkcję dla trzypunktowej różnicy w przód.
2. Definiuje funkcję dla trzypunktowej różnicy wstecznej.
3. Definiuje funkcję dla czteropunktowej różnicy centralnej.
4. Definiuje funkcję liczącą błąd względny.
5. Definiuje funkcje z zadania oraz ich pochodne dokładne.
6. Dla każdej funkcji i dla każdego kroku h oblicza pochodną trzema metodami.
7. Porównuje wyniki z pochodną dokładną.
8. Wypisuje błąd względny.

Kod

PYTHON

```
import math # importujemy moduł math, ponieważ potrzebujemy funkcji exp do
obliczania e^x

print("-----ZADANIE 3-----") # wypisujemy
nagłówkę zadania

def roznica_w_przod_trzypunktowa(f, x, h): # definiujemy funkcję liczącą pochodną
trzypunktową różnicą w przód
    return (-3 * f(x) + 4 * f(x + h) - f(x + 2 * h)) / (2 * h) # zwracamy wynik
ze wzoru: [-3f(x)+4f(x+h)-f(x+2h)]/(2h)

def roznica_wsteczna_trzypunktowa(f, x, h): # definiujemy funkcję liczącą
pochodną trzypunktową różnicą wsteczną
    return (3 * f(x) - 4 * f(x - h) + f(x - 2 * h)) / (2 * h) # zwracamy wynik ze
wzoru: [3f(x)-4f(x-h)+f(x-2h)]/(2h)

def roznica_centralna_czteropunktowa(f, x, h): # definiujemy funkcję liczącą
pochodną czteropunktową różnicą centralną
    return (f(x - 2 * h) - 8 * f(x - h) + 8 * f(x + h) - f(x + 2 * h)) / (12 * h)
# zwracamy wynik ze wzoru centralnego czteropunktowego

def blad_wzgledny(wartosc_dokladna, wartosc_przyblizona): # definiujemy funkcję
liczącą błąd względny
    if wartosc_dokladna != 0: # sprawdzamy, czy wartość dokładna nie jest równa
zero
        return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna)
# zwracamy błąd względny zgodnie ze wzorem ze slajdu
    else: # obsługujemy przypadek, gdy wartość dokładna jest równa zero
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna
jest równa 0.") # zgłaszamy błąd, ponieważ nie można dzielić przez zero

def f1(x): # definiujemy pierwszą funkcję z zadania
    return 2 * x**2 + 2 # zwracamy wartość funkcji f(x)=2x^2+2

def df1(x): # definiujemy dokładną pochodną pierwszej funkcji
    return 4 * x # zwracamy wartość pochodnej f'(x)=4x

def f2(x): # definiujemy drugą funkcję z zadania
    return 2 * x**4 - x**2 + 3*x - 7 # zwracamy wartość funkcji f(x)=2x^4-x^2+3x-
7

def df2(x): # definiujemy dokładną pochodną drugiej funkcji
    return 8 * x**3 - 2 * x + 3 # zwracamy wartość pochodnej f'(x)=8x^3-2x+3

def f3(x): # definiujemy trzecią funkcję z zadania
    return x**2 * math.exp(x) # zwracamy wartość funkcji f(x)=x^2e^x

def df3(x): # definiujemy dokładną pochodną trzeciej funkcji
    return math.exp(x) * (x**2 + 2 * x) # zwracamy wartość pochodnej
f'(x)=e^x(x^2+2x)
```

```

def porownaj_metody(nazwa, f, df, x, h_lista): # definiujemy funkcję porównującą
wszystkie metody dla jednej funkcji
    print("\n" + nazwa) # wypisujemy nazwę aktualnie badanej funkcji
    print("x =", x) # wypisujemy punkt, w którym obliczamy pochodną
    print("h          metoda          pochodna numeryczna
pochodna dokładna      błąd względny") # wypisujemy nagłówek tabeli

    for h in h_lista: # przechodzimy po kolejnych wartościach kroku h
        pochodna_dokladna = df(x) # liczymy dokładną wartość pochodnej w punkcie x

        pochodna_przod = roznica_w_przod_trzypunktowa(f, x, h) # liczymy pochodną
        metodą trzypunktowej różnicy w przód
        blad_przod = blad_wzgledny(pochodna_dokladna, pochodna_przod) # liczymy
        błąd względny dla metody w przód

        print(f"{h:<14} {'w przód 3-punktowa':<25} {pochodna_przod:<25}
        {pochodna_dokladna:<25} {blad_przod}") # wypisujemy wynik metody w przód

        pochodna_wsteczna = roznica_wsteczna_trzypunktowa(f, x, h) # liczymy
        pochodną metodą trzypunktowej różnicy wstecznej
        blad_wsteczny = blad_wzgledny(pochodna_dokladna, pochodna_wsteczna) #
        liczymy błąd względny dla metody wstecznej

        print(f"{h:<14} {'wsteczna 3-punktowa':<25} {pochodna_wsteczna:<25}
        {pochodna_dokladna:<25} {blad_wsteczny}") # wypisujemy wynik metody wstecznej

        pochodna_centralna = roznica_centralna_czteropunktowa(f, x, h) # liczymy
        pochodną metodą czteropunktowej różnicy centralnej
        blad_centralny = blad_wzgledny(pochodna_dokladna, pochodna_centralna) #
        liczymy błąd względny dla metody centralnej

        print(f"{h:<14} {'centralna 4-punktowa':<25} {pochodna_centralna:<25}
        {pochodna_dokladna:<25} {blad_centralny}") # wypisujemy wynik metody centralnej

x0 = 1.0 # ustalamy punkt x, w którym obliczamy pochodną

h_lista = [10**(-2), 10**(-4)] # zapisujemy wartości h podane w treści zadania

porownaj_metody("a) f(x) = 2x^2 + 2", f1, df1, x0, h_lista) # wykonujemy
obliczenia dla funkcji a)
porownaj_metody("b) f(x) = 2x^4 - x^2 + 3x - 7", f2, df2, x0, h_lista) #
wykonujemy obliczenia dla funkcji b)
porownaj_metody("c) f(x) = x^2e^x", f3, df3, x0, h_lista) # wykonujemy obliczenia
dla funkcji c)

```

Interpretacja wyników

Dla funkcji:

$$f(x) = 2x^2 + 2$$

pochodna dokładna w punkcie:

$$x = 1$$

wynosi:

$$f'(1) = 4$$

Wszystkie metody dają wynik bardzo bliski wartości dokładnej.

Dla funkcji:

$$f(x) = 2x^4 - x^2 + 3x - 7$$

pochodna dokładna to:

$$f'(x) = 8x^3 - 2x + 3$$

Dla:

$$x = 1$$

otrzymujemy:

$$f'(1) = 9$$

Metoda centralna czteropunktowa daje najmniejszy błąd, ponieważ ma błąd rzędu $O(h^4)$.

Dla funkcji:

$$f(x) = x^2 e^x$$

pochodna dokładna wynosi:

$$f'(x) = e^x(x^2 + 2x)$$

Dla:

$$x = 1$$

otrzymujemy:

$$f'(1) = 3e \approx 8.154845485$$

Również tutaj metoda centralna czteropunktowa daje najdokładniejszy wynik.

Wnioski

W zadaniu zastosowano trzy metody różnic skończonych:

Trzypunktową różnicę w przód

$$f'(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h}$$

Trzypunktową różnicę wsteczną

$$f'(x) \approx \frac{3f(x) - 4f(x-h) + f(x-2h)}{2h}$$

Czteropunktową różnicę centralną

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$$

Trzypunktowe metody w przód i wstecz mają dokładność rzędu:

$$O(h^2)$$

Natomiast metoda centralna czteropunktowa ma dokładność rzędu:

$$O(h^4)$$

Dlatego metoda centralna czteropunktowa zwykle daje najlepsze wyniki.

Z wyników programu widać, że dla mniejszego kroku:

$$h = 10^{-4}$$

błąd jest zazwyczaj mniejszy niż dla:

$$h = 10^{-2}$$

Trzeba jednak pamiętać, że przy bardzo małych wartościach h mogą pojawić się błędy zaokrągleń związane z ograniczoną precyzją obliczeń komputerowych.

Zadanie 4

Zaimplementuj różniczkowanie za pomocą wielomianów Lagrange'a. Wyznacz pochodną w punkcie

$$x = 3.5$$

przy następujących węzłach interpolacji:

$$\{(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)\}$$

Na czym polega metoda?

W tym zadaniu najpierw budujemy **wielomian interpolacyjny Lagrange'a** przechodzący przez podane punkty, a potem przybliżamy jego pochodną metodą różnicy centralnej.

Na tablicy zapisano:

$$L_n(x) = \sum_{i=0}^n y_i l_i(x)$$

gdzie $l_i(x)$ to wielomiany bazowe Lagrange'a.

Każdy wielomian bazowy ma postać:

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

Czyli dla każdego punktu budujemy osobny składnik, który potem mnożymy przez odpowiadającą mu wartość y_i .

Dane z zadania

Mamy punkty:

$$(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)$$

czyli:

- $x_0 = 1, y_0 = 4$
- $x_1 = 2, y_1 = 10$
- $x_2 = 3, y_2 = 20$
- $x_3 = 4, y_3 = 34$
- $x_4 = 5, y_4 = 52$

Chcemy policzyć pochodną w punkcie:

$$X = 3.5$$

Przyjmujemy krok zgodnie z ustaleniem:

$$h = 10^{-4}$$

Budowa wielomianu Lagrange'a

Wielomian interpolacyjny ma postać:

$$L_n(x) = y_0 l_0(x) + y_1 l_1(x) + y_2 l_2(x) + y_3 l_3(x) + y_4 l_4(x)$$

Dla naszych danych:

$$L_n(x) = 4l_0(x) + 10l_1(x) + 20l_2(x) + 34l_3(x) + 52l_4(x)$$

Każda funkcja bazowa $l_i(x)$ jest liczona ze wzoru:

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

W programie odpowiada za to funkcja:

Różniczkowanie wielomianu Lagrange'a

Na tablicy pojawia się pomysł użycia różnicy centralnej dla wielomianu interpolacyjnego:

$$f'(x) \approx \frac{L_n(x+h) - L_n(x-h)}{2h}$$

W naszym zadaniu:

$$f'(3.5) \approx \frac{L_n(3.5+h) - L_n(3.5-h)}{2h}$$

dla:

$$h = 10^{-4}$$

Czyli program:

1. Liczy wartość wielomianu Lagrange'a w punkcie $x+h$.
2. Liczy wartość wielomianu Lagrange'a w punkcie $x-h$.
3. Podstawia te wartości do wzoru różnicy centralnej.

```
print("----- ZADANIE 4 -----") # wypisujemy nagłówek zadania

def baza_lagrange(punkty, i, x): # definiujemy funkcję liczącą i-tą bazę Lagrange'a l_i(x)
    xi = punkty[i][0] # pobieramy wartość x_i z i-tego punktu

    wynik = 1.0 # zaczynamy iloczyn od 1, bo będziemy mnożyć kolejne czynniki

    for j in range(len(punkty)): # przechodzimy po wszystkich punktach
        if j != i: # pomijamy przypadek j = i, ponieważ we wzorze jest warunek j
            różne od i
                xj = punkty[j][0] # pobieramy wartość x_j z j-tego punktu
                wynik *= (x - xj) / (xi - xj) # mnożymy przez kolejny czynnik wzoru Lagrange'a

    return wynik # zwracamy wartość i-tej funkcji bazowej Lagrange'a

def wielomian_lagrange(punkty, x): # definiujemy funkcję liczącą wartość wielomianu Lagrange'a L_n(x)
    suma = 0.0 # tworzymy zmienną, w której będziemy sumować składniki wielomianu

    for i in range(len(punkty)): # przechodzimy po wszystkich punktach interpolacji
        yi = punkty[i][1] # pobieramy wartość y_i z i-tego punktu
        suma += yi * baza_lagrange(punkty, i, x) # dodajemy składnik y_i * l_i(x) do sumy

    return suma # zwracamy wartość wielomianu interpolacyjnego L_n(x)

def pochodna_lagrange_centralna(punkty, x, h): # definiujemy funkcję liczącą pochodną metodą różnicy centralnej
    return (wielomian_lagrange(punkty, x + h) - wielomian_lagrange(punkty, x - h)) / (2 * h) # stosujemy wzór: [L_n(x+h)-L_n(x-h)]/(2h)

punkty = [(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)] # zapisujemy węzły interpolacji z zadania

X = 3.5 # zapisujemy punkt, w którym chcemy policzyć pochodną
h = 10**(-4) # zapisujemy krok h = 10^-4

wynik = pochodna_lagrange_centralna(punkty, X, h) # obliczamy przybliżoną wartość pochodnej w punkcie X

print("Węzły interpolacji:") # wypisujemy opis danych wejściowych
print(punkty) # wypisujemy listę punktów interpolacji

print("\nPunkt, w którym liczymy pochodną:") # wypisujemy opis punktu obliczeń
print("x =", X) # wypisujemy wartość punktu X
```

```
print("\nKrok:") # wypisujemy opis kroku
print("h =", h) # wypisujemy wartość kroku h

print("\nPrzybliżona wartość pochodnej:") # wypisujemy opis wyniku
print("f'(", x, ") =", wynik) # wypisujemy przybliżoną wartość pochodnej
```

Wynik programu

Program zwraca wartość bardzo bliską:

$$14$$

czyli:

$$f'(3.5) \approx 14$$

Przykładowy wynik:

$$f'(3.5) = 14.000000000020663$$

Różnica od dokładnej wartości wynika tylko z zaokrągleń numerycznych.

Sprawdzenie wyniku

Dane punkty:

$$(1, 4), (2, 10), (3, 20), (4, 34), (5, 52)$$

leżą na funkcji:

$$f(x) = 2x^2 + 2$$

Sprawdzenie:

$$f(1) = 2 \cdot 1^2 + 2 = 4$$

$$f(2) = 2 \cdot 2^2 + 2 = 10$$

$$f(3) = 2 \cdot 3^2 + 2 = 20$$

$$f(4) = 2 \cdot 4^2 + 2 = 34$$

$$f(5) = 2 \cdot 5^2 + 2 = 52$$

Pochodna tej funkcji wynosi:

$$f'(x) = 4x$$

Dla:

$$x = 3.5$$

otrzymujemy:

$$f'(3.5) = 4 \cdot 3.5 = 14$$

Dlatego wynik programu:

$$f'(3.5) \approx 14$$

jest poprawny.

Wnioski

W zadaniu zbudowano wielomian interpolacyjny Lagrange'a:

$$L_n(x) = \sum_{i=0}^n y_i l_i(x)$$

gdzie:

$$l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

Następnie pochodną obliczono metodą różnicy centralnej zastosowanej do wielomianu interpolacyjnego:

$$f'(x) \approx \frac{L_n(x+h) - L_n(x-h)}{2h}$$

Dla:

$$X = 3.5$$

oraz:

$$h = 10^{-4}$$

otrzymano:

$$f'(3.5) \approx 14$$

Wynik jest zgodny z wartością dokładną, ponieważ podane punkty pochodzą z funkcji:

$$f(x) = 2x^2 + 2$$

a jej pochodna w punkcie 3.5 wynosi:

$$14.$$

Lab 11

Całkowanie numeryczne

Treść zadań

Zadanie 1. Zaimplementuj całkowanie numeryczne za pomocą metody prostokątów.

Zadanie 2. Zaimplementuj całkowanie numeryczne za pomocą metody trapezów.

Zadanie 3. Zaimplementuj całkowanie numeryczne za pomocą metody Simpsona.

Zadanie 4. Wyniki powyższych programów przetestuj dla następujących całek:

a)

$$\int_0^1 x^2 dx$$

b)

$$\int_0^{\frac{\pi}{2}} \cos x dx$$

c)

$$\int_e^{e^2} \frac{1}{x} dx$$

Zadanie 5. Sprawdź dokładność otrzymanych rozwiązań.

Wprowadzenie

Całkowanie numeryczne służy do przybliżonego obliczania wartości całek oznaczonych.

Całka oznaczona:

$$\int_a^b f(x) dx$$

oznacza pole pod wykresem funkcji $f(x)$ na przedziale $[a, b]$.

Na tablicy zapisano ogólną postać całkowania numerycznego:

$$\int_a^b f(x) dx \approx \sum A_i f(x_i)$$

Oznacza to, że dokładną wartość całki zastępujemy sumą wartości funkcji w wybranych punktach x_i , pomnożonych przez odpowiednie współczynniki A_i .

W zadaniu wykorzystujemy trzy metody:

- metodę prostokątów,
- metodę trapezów,
- metodę Simpsona.

Każda z metod polega na podzieleniu przedziału całkowania $[a, b]$ na mniejsze części.

Liczbę podprzedziałów oznaczamy jako:

$$N$$

W kodzie ta sama liczba jest zapisana jako zmienna:

```
n
```

PYTHON

Krok całkowania, czyli długość jednego podprzedziału, wynosi:

$$h = \frac{b - a}{N}$$

Punkty podziału przedziału obliczamy ze wzoru:

$$x_i = a + \frac{b - a}{N}i$$

czyli równoważnie:

$$x_i = a + ih$$

Im większa liczba podprzedziałów N , tym mniejszy krok h , a wynik zwykle jest dokładniejszy.

Zadanie 1 — metoda prostokątów

Idea metody prostokątów

Metoda prostokątów polega na przybliżeniu pola pod wykresem funkcji za pomocą prostokątów.

Na pojedynczym podprzedziale $[x_i, x_{i+1}]$ funkcję $f(x)$ zastępujemy wartością stałą y_i . Oznacza to, że zamiast dokładnego pola pod wykresem liczymy pole prostokąta.

Dla jednego podprzedziału mamy:

$$\sigma_i = \int_{x_i}^{x_{i+1}} f(x) dx$$

Przybliżamy tę wartość przez:

$$\sigma_i \approx \int_{x_i}^{x_{i+1}} y_i dx$$

Ponieważ y_i jest stałe, otrzymujemy:

$$\sigma_i \approx y_i(x_{i+1} - x_i)$$

Dla równoodległych punktów:

$$x_{i+1} - x_i = h$$

więc:

$$\sigma_i \approx y_i h$$

Po zsumowaniu wszystkich prostokątów otrzymujemy:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{N-1} y_i$$

Metoda prostokątów środkowych

Na slajdach zaznaczono, że dla węzłów równoodległych często przyjmuje się:

$$y_i = f\left(x_i + \frac{h}{2}\right)$$

czyli wartość funkcji w środku podprzedziału.

Dlatego w programie zastosowano **metodę prostokątów środkowych**.

Dla tej metody:

$$x_{\text{środek}} = x_i + \frac{h}{2}$$

czyli:

$$x_{\text{środek}} = a + \left(i + \frac{1}{2}\right)h$$

Wzór użyty w programie ma postać:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{N-1} f\left(a + \left(i + \frac{1}{2}\right)h\right)$$

Po podstawieniu:

$$h = \frac{b - a}{N}$$

można zapisać:

$$\int_a^b f(x) dx \approx \frac{b-a}{N} \sum_{i=0}^{N-1} f\left(a + \left(i + \frac{1}{2}\right) \frac{b-a}{N}\right)$$

Ta wersja jest zgodna z metodą prostokątów ze slajdów, ponieważ przyjmuje wysokość prostokąta jako wartość funkcji w środku podprzedziału.

Zadanie 2 — metoda trapezów

Idea metody trapezów

Metoda trapezów polega na tym, że na każdym podprzedziale funkcję zastępujemy prostą przechodzącą przez dwa punkty:

$$(x_i, f(x_i))$$

oraz:

$$(x_{i+1}, f(x_{i+1}))$$

Wtedy pole pod wykresem na danym podprzedziale przybliżamy polem trapezu.

Dla jednego podprzedziału pole trapezu wynosi:

$$\sigma_i = \frac{1}{2}h(y_i + y_{i+1})$$

gdzie:

$$y_i = f(x_i)$$

oraz:

$$y_{i+1} = f(x_{i+1})$$

Po zsumowaniu pól trapezów dla całego przedziału $[a, b]$ otrzymujemy wzór:

$$\int_a^b f(x) dx \approx \frac{1}{2}h \sum_{i=0}^{N-1} (y_{i+1} + y_i)$$

Po uporządkowaniu składników dostajemy:

$$\int_a^b f(x) dx \approx h \left[\frac{1}{2}(y_0 + y_N) + \sum_{i=1}^{N-1} y_i \right]$$

Ponieważ:

$$y_i = f(x_i)$$

możemy zapisać:

$$\int_a^b f(x) dx \approx h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$$

gdzie:

$$x_i = a + ih$$

oraz:

$$h = \frac{b - a}{N}$$

Jest to dokładnie wzór zastosowany w programie.

Dokładność metody trapezów

Metoda trapezów jest dokładna, jeżeli funkcja f jest wielomianem stopnia co najwyżej pierwszego, czyli funkcją liniową.

Dla innych funkcji pojawia się błąd przybliżenia, ponieważ wykres funkcji nie zawsze jest idealnie prostą linią na każdym podprzedziale.

Zadanie 3 — metoda Simpsona

Idea metody Simpsona

Metoda Simpsona jest dokładniejszą metodą całkowania numerycznego niż metoda prostokątów i metoda trapezów.

W metodzie tej pole pod wykresem przybliżamy za pomocą fragmentów parabol.

Dla jednego przedziału wzór Simpsona ma postać:

$$I \approx \frac{b - a}{6} \left(f(a) + 4f\left(\frac{a + b}{2}\right) + f(b) \right)$$

Widzimy, że metoda wykorzystuje trzy wartości funkcji:

- wartość na początku przedziału,
- wartość w środku przedziału,
- wartość na końcu przedziału.

Środkowy punkt ma wagę 4, dlatego ma większy wpływ na wynik.

Złożona metoda Simpsona

Dla wielu podprzedziałów stosujemy złożoną metodę Simpsona.

Liczba podprzedziałów N musi być parzysta.

Krok wynosi:

$$h = \frac{b-a}{N}$$

Punkty podziału:

$$x_i = a + ih$$

dla:

$$i = 0, 1, \dots, N$$

Wzór ze slajdu ma postać:

$$S_N = \frac{h}{3} [f(x_0) + 4(f(x_1) + f(x_3) + \dots + f(x_{N-1})) + 2(f(x_2) + f(x_4) + \dots + f(x_{N-2})) + f(x_N)]$$

Można go też zapisać w formie sum:

$$S_N = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{\substack{i=1 \\ i \text{ nieparzyste}}}^{N-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ parzyste}}}^{N-2} f(x_i) \right]$$

W programie działa to tak:

- pierwszy punkt $f(a)$ ma wagę 1,
- ostatni punkt $f(b)$ ma wagę 1,
- punkty o indeksach nieparzystych mają wagę 4,
- punkty o indeksach parzystych mają wagę 2.

Metoda Simpsona jest dokładna dla wielomianów stopnia co najwyżej trzeciego.

Błąd metody Simpsona

Na slajdach podano, że błąd przybliżenia w metodzie Simpsona zależy od czwartej pochodnej funkcji.

Ma postać:

$$\varepsilon = \left| f^{(4)}(\xi) \right| \frac{(b-a)h^4}{180}$$

gdzie:

- $f^{(4)}(\xi)$ oznacza czwartą pochodną funkcji w pewnym punkcie $\xi \in (a, b)$,
- $h = \frac{b-a}{N}$.

Oznacza to, że dla mniejszego kroku h metoda Simpsona bardzo szybko zwiększa dokładność.

Zadanie 4 — testowane całki

Program testuje wszystkie trzy metody dla trzech całek.

Całka a)

$$\int_0^1 x^2 dx$$

Funkcja podcałkowa:

$$f(x) = x^2$$

Przedział:

$$[0, 1]$$

Wartość dokładna:

$$\int x^2 dx = \frac{x^3}{3}$$

czyli:

$$\int_0^1 x^2 dx = \frac{1^3}{3} - \frac{0^3}{3}$$

$$\int_0^1 x^2 dx = \frac{1}{3}$$

Całka b)

$$\int_0^{\frac{\pi}{2}} \cos x dx$$

Funkcja podcałkowa:

$$f(x) = \cos x$$

Przedział:

$$\left[0, \frac{\pi}{2}\right]$$

Wartość dokładna:

$$\int \cos x \, dx = \sin x$$

czyli:

$$\int_0^{\frac{\pi}{2}} \cos x \, dx = \sin \frac{\pi}{2} - \sin 0$$

$$\int_0^{\frac{\pi}{2}} \cos x \, dx = 1 - 0 = 1$$

Całka c)

$$\int_e^{e^2} \frac{1}{x} \, dx$$

Funkcja podcałkowa:

$$f(x) = \frac{1}{x}$$

Przedział:

$$[e, e^2]$$

Wartość dokładna:

$$\int \frac{1}{x} \, dx = \ln x$$

czyli:

$$\int_e^{e^2} \frac{1}{x} \, dx = \ln(e^2) - \ln(e)$$

$$\int_e^{e^2} \frac{1}{x} \, dx = 2 - 1 = 1$$

Zadanie 5 — sprawdzenie dokładności

Aby sprawdzić dokładność otrzymanych rozwiązań, porównujemy wynik numeryczny z wartością dokładną.

Błąd bezwzględny

Błąd bezwzględny liczymy ze wzoru:

$$\Delta = |I_{\text{dokładne}} - I_{\text{przybliżone}}|$$

gdzie:

- $I_{\text{dokładne}}$ — dokładna wartość całki,
- $I_{\text{przybliżone}}$ — wartość całki obliczona metodą numeryczną.

Błąd względny

Błąd względny liczymy ze wzoru:

$$\delta = \frac{|I_{\text{dokładne}} - I_{\text{przybliżone}}|}{|I_{\text{dokładne}}|}$$

Błąd względny pokazuje, jak duży jest błąd w stosunku do wartości dokładnej.

Kod programu


```
import math # importujemy moduł math, ponieważ potrzebujemy funkcji cos, liczby pi
oraz liczby e

print("----- CAŁKOWANIE NUMERYCZNE -----") #
wypisujemy główny nagłówek programu

def metoda_prostokatow(f, a, b, n): # definiujemy funkcję liczącą całkę metodą
prostokątów środkowych
    h = (b - a) / n # obliczamy długość jednego podprzedziału, czyli  $h = (b-a)/N$ 

    suma = 0.0 # tworzymy zmienną, w której będziemy sumować wartości funkcji

    for i in range(n): # wykonujemy pętlę po wszystkich podprzedziałach od 0 do
n-1
        x_srodek = a + (i + 0.5) * h # obliczamy środek aktualnego podprzedziału,
czyli  $x_i + h/2$ 
        suma += f(x_srodek) # dodajemy wartość funkcji w środku podprzedziału do
sumy

    return h * suma # mnożymy sumę przez h i zwracamy przybliżoną wartość całki

def metoda_trapezow(f, a, b, n): # definiujemy funkcję liczącą całkę metodą
trapezów
    h = (b - a) / n # obliczamy długość jednego podprzedziału, czyli  $h = (b-a)/N$ 

    suma = (f(a) + f(b)) / 2 # dodajemy wartości funkcji na końcach przedziału z
wagą 1/2

    for i in range(1, n): # przechodzimy po punktach wewnętrznych przedziału
        x = a + i * h # obliczamy punkt  $x_i$  zgodnie ze wzorem  $x_i = a + ih$ 
        suma += f(x) # dodajemy wartość funkcji w punkcie  $x_i$  do sumy

    return h * suma # mnożymy sumę przez h i zwracamy przybliżoną wartość całki

def metoda_simpsona(f, a, b, n): # definiujemy funkcję liczącą całkę metodą
Simpsona
    if n % 2 != 0: # sprawdzamy, czy liczba podprzedziałów jest parzysta
        raise ValueError("W metodzie Simpsona liczba podprzedziałów n musi być
parzysta.") # zgłaszamy błąd, jeśli n jest nieparzyste

    h = (b - a) / n # obliczamy długość jednego podprzedziału, czyli  $h = (b-a)/N$ 

    suma = f(a) + f(b) # pierwszy i ostatni punkt mają wagę 1

    for i in range(1, n): # przechodzimy po punktach wewnętrznych przedziału
        x = a + i * h # obliczamy punkt  $x_i$ 

        if i % 2 == 0: # sprawdzamy, czy indeks punktu jest parzysty
            suma += 2 * f(x) # punkty parzyste mają wagę 2
        else: # jeśli indeks nie jest parzysty, to jest nieparzysty
```

```

        suma += 4 * f(x) # punkty nieparzyste mają wagę 4

    return (h / 3) * suma # mnożymy sumę przez h/3 i zwracamy przybliżoną wartość
całki

def blad_bezwzglezny(wartosc_dokladna, wartosc_przyblizona): # definiujemy
funkcję liczącą błąd bezwzględny
    return abs(wartosc_dokladna - wartosc_przyblizona) # zwracamy wartość
bezwzględną różnicy wyniku dokładnego i przybliżonego

def blad_wzglezny(wartosc_dokladna, wartosc_przyblizona): # definiujemy funkcję
liczącą błąd względny
    if wartosc_dokladna == 0: # sprawdzamy, czy wartość dokładna nie jest zerem
        raise ValueError("Nie można policzyć błędu względnego, gdy wartość dokładna
jest równa 0.") # zabezpieczamy się przed dzieleniem przez zero

    return abs(wartosc_dokladna - wartosc_przyblizona) / abs(wartosc_dokladna) #
zwracamy błąd względny

def f1(x): # definiujemy funkcję z całki a)
    return x**2 # zwracamy wartość funkcji f(x)=x^2

def f2(x): # definiujemy funkcję z całki b)
    return math.cos(x) # zwracamy wartość funkcji f(x)=cos(x)

def f3(x): # definiujemy funkcję z całki c)
    return 1 / x # zwracamy wartość funkcji f(x)=1/x

def testuj_calke(nazwa, f, a, b, wartosc_dokladna, n): # definiujemy funkcję
testującą wszystkie metody dla jednej całki
    print("\n" + nazwa) # wypisujemy nazwę aktualnie badanej całki
    print("Przedział całkowania:", "[", a, ",", b, "]") # wypisujemy przedział
całkowania
    print("Liczba podprzedziałów n =", n) # wypisujemy liczbę podprzedziałów
    print("Wartość dokładna =", wartosc_dokladna) # wypisujemy dokładną wartość
całki

    wynik_prostokaty = metoda_prostokatow(f, a, b, n) # obliczamy całkę metodą
prostokątów środkowych
    wynik_trapezy = metoda_trapezow(f, a, b, n) # obliczamy całkę metodą trapezów
    wynik_simpson = metoda_simpsona(f, a, b, n) # obliczamy całkę metodą Simpsona

    print("\nMetoda prostokątów:") # wypisujemy nagłówek dla metody prostokątów
    print("Wynik =", wynik_prostokaty) # wypisujemy wynik metody prostokątów
    print("Błąd bezwzględny =", blad_bezwzglezny(wartosc_dokladna,
wynik_prostokaty)) # wypisujemy błąd bezwzględny metody prostokątów
    print("Błąd względny =", blad_wzglezny(wartosc_dokladna, wynik_prostokaty)) #

```

wypisujemy błąd względny metody prostokątów

```
print("\nMetoda trapezów:") # wypisujemy nagłówek dla metody trapezów
print("Wynik =", wynik_trapezy) # wypisujemy wynik metody trapezów
print("Błąd bezwzględny =", blad_bezwzgledny(wartosc_dokladna, wynik_trapezy))
# wypisujemy błąd bezwzględny metody trapezów
print("Błąd względny =", blad_wzgledny(wartosc_dokladna, wynik_trapezy)) #
wypisujemy błąd względny metody trapezów

print("\nMetoda Simpsona:") # wypisujemy nagłówek dla metody Simpsona
print("Wynik =", wynik_simpson) # wypisujemy wynik metody Simpsona
print("Błąd bezwzględny =", blad_bezwzgledny(wartosc_dokladna, wynik_simpson))
# wypisujemy błąd bezwzględny metody Simpsona
print("Błąd względny =", blad_wzgledny(wartosc_dokladna, wynik_simpson)) #
wypisujemy błąd względny metody Simpsona
```

`n = 100` # ustalamy liczbę podprzedziałów; w teorii oznaczamy ją jako N ; jest parzysta, więc metoda Simpsona może działać

```
testuj_calke( # uruchamiamy test dla całki a)
    "a) całka od 0 do 1 z  $x^2 dx$ ", # nazwa całki
    f1, # funkcja podcałkowa
    0, # dolna granica całkowania
    1, # górna granica całkowania
    1 / 3, # dokładna wartość całki
    n # liczba podprzedziałów
)

testuj_calke( # uruchamiamy test dla całki b)
    "b) całka od 0 do  $\pi/2$  z  $\cos(x) dx$ ", # nazwa całki
    f2, # funkcja podcałkowa
    0, # dolna granica całkowania
    math.pi / 2, # górna granica całkowania
    1, # dokładna wartość całki
    n # liczba podprzedziałów
)

testuj_calke( # uruchamiamy test dla całki c)
    "c) całka od e do  $e^2$  z  $1/x dx$ ", # nazwa całki
    f3, # funkcja podcałkowa
    math.e, # dolna granica całkowania
    math.e**2, # górna granica całkowania
    1, # dokładna wartość całki
    n # liczba podprzedziałów
)
```

Wnioski

W programie zaimplementowano trzy metody całkowania numerycznego:

- metodę prostokątów środkowych,

- metodę trapezów,
- metodę Simpsona.

Metoda prostokątów środkowych przybliża pole pod wykresem za pomocą prostokątów o wysokości równej wartości funkcji w środku każdego podprzedziału.

Metoda trapezów przybliża funkcję na każdym podprzedziale odcinkiem prostej i oblicza pole trapezów.

Metoda Simpsona przybliża funkcję fragmentami paraboli i wykorzystuje wagi:

$$1, 4, 2, 4, 2, \dots, 4, 1$$

Wszystkie metody zostały przetestowane dla całek:

$$\int_0^1 x^2 dx$$

$$\int_0^{\frac{\pi}{2}} \cos x dx$$

$$\int_e^{e^2} \frac{1}{x} dx$$

Dla każdej całki znana jest wartość dokładna, dlatego można było obliczyć błąd bezwzględny i względny.

Najprostszą metodą jest metoda prostokątów. Metoda trapezów jest zwykle dokładniejsza, ponieważ uwzględnia wartości funkcji na obu końcach podprzedziału. Metoda Simpsona zwykle daje najlepszą dokładność, ponieważ przybliża funkcję parabolą.

Zwiększenie liczby podprzedziałów N powoduje zmniejszenie kroku:

$$h = \frac{b - a}{N}$$

a więc zwykle poprawia dokładność obliczeń.

Lab 12

Zadanie 1 — obliczanie wartości wielomianu schematem Hornera

Polecenie:

Napisz program obliczający wartość wielomianu w punkcie z wykorzystaniem schematu Hornera.

Na czym polega schemat Hornera?

Schemat Hornera to metoda szybkiego obliczania wartości wielomianu w danym punkcie.

Dany jest wielomian:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

Zamiast liczyć osobno każdą potęgę z , przekształcamy wielomian do postaci:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Dzięki temu w każdym kroku wykonujemy tylko jedno mnożenie i jedno dodawanie.

Algorytm ze slajdu

Schemat Hornera działa według zasady:

$$p \leftarrow a_n$$

Następnie dla kolejnych współczynników:

$$p \leftarrow p \cdot z + a_i$$

Na końcu otrzymujemy:

$$p = P_n(z)$$

czyli wartość wielomianu w punkcie z .

Ważne

Współczynniki wielomianu zapisujemy w liście **od najwyższej potęgi do wyrazu wolnego**.

Na przykład dla wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

lista współczynników wygląda tak:

```
wspolczynniki = [3, 2, -1, 5]
```

PYTHON

czyli:

- **3** — współczynnik przy z^3 ,
- **2** — współczynnik przy z^2 ,
- **-1** — współczynnik przy z ,
- **5** — wyraz wolny.

Przykład ze slajdu

Dany jest wielomian:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

Chcemy obliczyć jego wartość dla:

$$z = 2$$

Schemat Hornera:

$$\begin{aligned} p &= 3 \\ p &= 3 \cdot 2 + 2 = 8 \\ p &= 8 \cdot 2 - 1 = 15 \\ p &= 15 \cdot 2 + 5 = 35 \end{aligned}$$

Zatem:

$$P(2) = 35$$

Kod programu

```
PYTHON
def schemat_Hornera(wspolczynniki, z): # definiujemy funkcję obliczającą wartość
    wielomianu w punkcie z
    p = wspolczynniki[0] # jako początkową wartość p przyjmujemy pierwszy
    współczynnik, czyli ten przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych
        współczynnikach od drugiego elementu listy
        p = p * z + wspolczynniki[i] # wykonujemy krok schematu Hornera: p = p*z
        + kolejny współczynnik

    return p # zwracamy końcową wartość, czyli P(z)
```

Test programu

```
PYTHON
wspolczynniki = [3, 2, -1, 5] # współczynniki wielomianu P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt, w którym liczymy wartość wielomianu; tutaj z = 2

wynik = schemat_Hornera(wspolczynniki, z) # wywołujemy funkcję schematu Hornera

print("P(z) =", wynik) # wypisujemy wynik
```

Wynik:

$$P(z) = (35+0j)$$

Zapis:

$$35 + 0j$$

oznacza po prostu liczbę:

$$35$$

Python pokazuje `+0j`, ponieważ użyto typu `complex`.

Cały program

```
PYTHON

# zad 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja oblicza wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok Hornera

    return p # zwracamy wartość wielomianu

wspolczynniki = [3, 2, -1, 5] # wielomian P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt z = 2

wynik = schemat_Hornera(wspolczynniki, z) # obliczamy wartość wielomianu

print("P(z) =", wynik) # wypisujemy wynik
```

Wnioski

Schemat Hornera pozwala szybko obliczyć wartość wielomianu w punkcie.

Zamiast liczyć potęgi osobno, np.:

$$z^3, z^2, z$$

wykonujemy kolejne działania postaci:

$$p \leftarrow pz + a_i$$

Dzięki temu program jest krótszy, szybszy i wygodny także dla liczb zespolonych.

Dla przykładowego wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

oraz:

$$z = 2$$

otrzymujemy:

$$P(2) = 35$$

Poniżej masz gotową notatkę do **zadania 2**, w takim samym stylu jak do zadania 1 i zgodnie ze slajdami o schemacie Hornera.

Zadanie 2 — obliczanie wartości pierwszej i drugiej pochodnej wielomianu w punkcie

Polecenie:

Napisz program obliczający wartość pierwszej i drugiej pochodnej wielomianu w punkcie.

O co chodzi w zadaniu?

W zadaniu 1 schemat Hornera służył do obliczania wartości wielomianu:

$$P(z)$$

W zadaniu 2 rozszerzamy schemat Hornera tak, aby obliczyć jednocześnie:

$$P(z)$$

$$P'(z)$$

$$P''(z)$$

czyli:

- wartość wielomianu w punkcie,
- wartość pierwszej pochodnej w punkcie,
- wartość drugiej pochodnej w punkcie.

Przypomnienie schematu Hornera

Dany jest wielomian:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

W schemacie Hornera zapisujemy go w postaci:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Podstawowy krok schematu Hornera ma postać:

$$p_{\text{nowe}} = p_{\text{stare}} \cdot z + a_i$$

Ten wzór służy do obliczania wartości wielomianu.

Skąd biorą się wzory na pochodne?

W każdym kroku Hornera mamy:

$$p_{\text{nowe}}(z) = p_{\text{stare}}(z) \cdot z + a_i$$

Teraz różniczkujemy ten wzór.

Pierwsza pochodna

Różniczkujemy:

$$p_{\text{nowe}}(z) = p_{\text{stare}}(z) \cdot z + a_i$$

Pochodna stałej (a_i) wynosi 0.

Z pochodnej iloczynu:

$$(uv)' = u'v + uv'$$

otrzymujemy:

$$p' * \text{nowe}(z) = p' * \text{stare}(z) \cdot z + p_{\text{stare}}(z)$$

W programie zapisujemy to jako:

```
dp = dp * z + p
```

PYTHON

gdzie:

- **dp** oznacza pierwszą pochodną,
- **p** oznacza wartość wielomianu z poprzedniego kroku.

Druga pochodna

Teraz różniczkujemy wzór na pierwszą pochodną:

$$p' * \text{nowe}(z) = p' * \text{stare}(z) \cdot z + p_{\text{stare}}(z)$$

Otrzymujemy:

$$p'' * \text{nowe}(z) = p'' * \text{stare}(z) \cdot z + p' * \text{stare}(z) + p' * \text{stare}(z)$$

czyli:

$$p'' * \text{nowe}(z) = p'' * \text{stare}(z) \cdot z + 2p'_{\text{stare}}(z)$$

W programie zapisujemy to jako:

```
ddp = ddp * z + 2 * dp
```

gdzie:

- `ddp` oznacza drugą pochodną,
- `dp` oznacza pierwszą pochodną z poprzedniego kroku.

Ważna kolejność działań w programie

W pętli najpierw aktualizujemy drugą pochodną, potem pierwszą pochodną, a dopiero na końcu wartość wielomianu:

PYTHON

```
ddp = ddp * z + 2 * dp
dp = dp * z + p
p = p * z + wspolczynniki[i]
```

Taka kolejność jest ważna, ponieważ `ddp` musi korzystać ze starej wartości `dp`, a `dp` musi korzystać ze starej wartości `p`.

Kod programu

PYTHON

```
def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja oblicza P(z), P'(z)
    oraz P''(z)
    p = wspolczynniki[0] # p przechowuje aktualną wartość wielomianu
    dp = 0 # dp przechowuje aktualną wartość pierwszej pochodnej
    ddp = 0 # ddp przechowuje aktualną wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych
        wspolczynniki
            ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
            dp = dp * z + p # aktualizujemy pierwszą pochodną
            p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu schematem
        Hornera

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)
```

Przykład

Weźmy wielomian ze slajdu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

Współczynniki zapisujemy od najwyższej potęgi do wyrazu wolnego:

PYTHON

```
wspolczynniki = [3, 2, -1, 5]
```

Obliczamy wartości dla:

$$z = 2$$

Obliczenia analityczne do sprawdzenia

Najpierw liczymy wartość wielomianu:

$$P(2) = 3 \cdot 2^3 + 2 \cdot 2^2 - 2 + 5$$

$$P(2) = 24 + 8 - 2 + 5 = 35$$

Pierwsza pochodna:

$$P'(z) = 9z^2 + 4z - 1$$

$$P'(2) = 9 \cdot 2^2 + 4 \cdot 2 - 1$$

$$P'(2) = 36 + 8 - 1 = 43$$

Druga pochodna:

$$P''(z) = 18z + 4$$

$$P''(2) = 18 \cdot 2 + 4$$

$$P''(2) = 36 + 4 = 40$$

Czyli oczekujemy:

$$P(2) = 35$$

$$P'(2) = 43$$

$$P''(2) = 40$$

Test programu

```
PYTHON
wspolczynniki = [3, 2, -1, 5] # współczynniki wielomianu P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt, w którym liczymy wartości

wartosc, pierwsza_pochodna, druga_pochodna =
schemat_Hornera_pochodne(wspolczynniki, z) # obliczamy P(z), P'(z), P''(z)

print("P(z) =", wartosc) # wypisujemy wartość wielomianu
print("P'(z) =", pierwsza_pochodna) # wypisujemy wartość pierwszej pochodnej
print("P''(z) =", druga_pochodna) # wypisujemy wartość drugiej pochodnej
```

Wynik:

$$P(z) = (35+0j)$$

$$P'(z) = (43+0j)$$

$$P''(z) = (40+0j)$$

Zapis $+0j$ oznacza, że wynik jest liczbą zespoloną z częścią urojoną równą 0.

Cały program

PYTHON

```
# zad 2 -----

def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja oblicza P(z), P'(z),
P''(z)
    p = wspolczynniki[0] # początkowa wartość wielomianu to współczynnik przy
najwyższej potędze
    dp = 0 # początkowa wartość pierwszej pochodnej
    ddp = 0 # początkowa wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy przez kolejne
współczynniki
        ddp = ddp * z + 2 * dp # obliczamy nową wartość drugiej pochodnej
        dp = dp * z + p # obliczamy nową wartość pierwszej pochodnej
        p = p * z + wspolczynniki[i] # obliczamy nową wartość wielomianu

    return p, dp, ddp # zwracamy wartość wielomianu, pierwszej pochodnej i drugiej
pochodnej

wspolczynniki = [3, 2, -1, 5] # wielomian P(z)=3z^3+2z^2-z+5

z = complex(2, 0) # punkt z=2

wartosc, pierwsza_pochodna, druga_pochodna =
schemat_Hornera_pochodne(wspolczynniki, z) # wywołujemy funkcję

print("P(z) =", wartosc) # wypisujemy P(z)
print("P'(z) =", pierwsza_pochodna) # wypisujemy P'(z)
print("P''(z) =", druga_pochodna) # wypisujemy P''(z)
```

Wnioski

W zadaniu 2 rozszerzono schemat Hornera tak, aby oprócz wartości wielomianu obliczał także pierwszą i drugą pochodną.

Dzięki temu jednym przejściem przez listę współczynników można obliczyć:

$$P(z), \quad P'(z), \quad P''(z)$$

Dla przykładowego wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

oraz punktu:

$$z = 2$$

otrzymujemy:

$$P(2) = 35$$

$$P'(2) = 43$$

$$P''(2) = 40$$

Kod działa również dla liczb zespolonych, ponieważ punkt (z) można zapisać jako:

```
z = complex(2, 0)
```

PYTHON

Zadanie 3 — metoda Laguerre'a

Polecenie:

Zaimplementuj metodę Laguerre'a służącą do znajdowania pierwiastków wielomianów, również zespolonych.

O co chodzi w zadaniu?

Wykorzystujemy funkcje z zadania 1 i 2, bo metoda Laguerre'a potrzebuje wartości:

$$P(z), \quad P'(z), \quad P''(z)$$

czyli dokładnie tego, co liczy Horner rozszerzony z zadania 2.

Mamy wielomian, np.:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

Wiemy, że jego pierwiastki to:

$$z_1 = -1, \quad z_2 = -2, \quad z_3 = -1 + 2i, \quad z_4 = -1 - 2i$$

Ale w praktyce program nie zna rozkładu wielomianu, tylko jego współczynniki:

$$1, 5, 13, 19, 10$$

czyli:

$$P(z) = 1z^4 + 5z^3 + 13z^2 + 19z + 10$$

Zadaniem programu jest znalezienie **jednego pierwiastka** metodą Laguerre'a.

Wzory

Dla wielomianu stopnia n w punkcie z liczymy:

$$G = \frac{P'(z)}{P(z)}$$

oraz:

$$H = G^2 - \frac{P''(z)}{P(z)}$$

Następnie liczymy poprawkę:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH - G^2)}}$$

Znak w mianowniku wybieramy tak, aby mianownik miał **większy moduł**:

$$\left| G \pm \sqrt{(n-1)(nH - G^2)} \right|$$

Potem obliczamy nowe przybliżenie:

$$z_{\text{new}} = z_{\text{old}} - a$$

Iteracje kończymy, gdy:

$$|a| < \varepsilon$$

```

# zad 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja licząca wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok schematu Hornera

    return p # zwracamy wartość wielomianu P(z)

# zad 2 -----

def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja licząca P(z), P'(z), P''(z)
    p = wspolczynniki[0] # p oznacza aktualną wartość wielomianu
    dp = 0 # dp oznacza aktualną wartość pierwszej pochodnej
    ddp = 0 # ddp oznacza aktualną wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
        dp = dp * z + p # aktualizujemy pierwszą pochodną
        p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)

# zad 3 -----

import cmath # importujemy cmath, bo metoda Laguerre'a może używać pierwiastków zespolonych

def metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0, epsilon=1e-6, max_iteracji=100):
    z = complex(z0) # zamieniamy punkt startowy na liczbę zespoloną
    n = len(wspolczynniki) - 1 # stopień wielomianu to liczba współczynników minus 1

    for k in range(max_iteracji): # wykonujemy kolejne iteracje metody Laguerre'a
        P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z) # liczymy P(z), P'(z), P''(z)

        if abs(P) < epsilon: # jeśli wartość wielomianu jest bardzo bliska 0
            return z # to uznajemy, że z jest pierwiastkiem

    G = P_prim / P # liczymy G = P'(z) / P(z)
    H = G**2 - P_2prim / P # liczymy H = G^2 - P''(z) / P(z)

```

```

    pierwiastek = cmath.sqrt((n - 1) * (n * H - G**2)) # liczymy wyrażenie
    pod pierwiastkiem ze wzoru

    mianownik_plus = G + pierwiastek # pierwszy możliwy mianownik
    mianownik_minus = G - pierwiastek # drugi możliwy mianownik

    if abs(mianownik_plus) > abs(mianownik_minus): # wybieramy mianownik o
    większym module
        mianownik = mianownik_plus # jeśli większy jest plus, wybieramy plus
    else:
        mianownik = mianownik_minus # w przeciwnym razie wybieramy minus

    if abs(mianownik) == 0: # zabezpieczenie przed dzieleniem przez zero
        z = z + complex(epsilon, epsilon) # lekko przesuwamy punkt
        continue # przechodzimy do następnej iteracji

    a = n / mianownik # liczymy poprawkę a ze wzoru Laguerre'a

    z_nowe = z - a # liczymy nowe przybliżenie pierwiastka

    if abs(a) < epsilon: # jeżeli poprawka jest bardzo mała
        return z_nowe # kończymy, bo znaleźliśmy pierwiastek

    z = z_nowe # aktualizujemy z i przechodzimy do kolejnej iteracji

    return z # jeśli nie spełniono warunku wcześniej, zwracamy ostatnie
    przybliżenie

# DODATKOWY TEST
print("----- ZADANIE 3 -----")

# Przykład ze slajdu:
#  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ 

wspolczynniki = [1, 5, 13, 19, 10] # współczynniki wielomianu od najwyższej
    potęgi do wyrazu wolnego

z0 = complex(0, 0) # punkt startowy  $z_0 = 0$ 

pierwiastek = metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0) # szukamy
    jednego pierwiastka

print("Wielomian:  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ ") # wypisujemy wielomian
print("Punkt startowy  $z_0 =$ ", z0) # wypisujemy punkt startowy
print("Znaleziony pierwiastek =", pierwiastek) # wypisujemy znaleziony pierwiastek
print("Sprawdzenie  $P(\text{pierwiastek}) =$ ", schemat_Hornera(wspolczynniki, pierwiastek))
# sprawdzamy, czy  $P(z)$  jest bliskie 0

```

Co powinno wyjść?

Dla wielomianu ze slajdu:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

metoda startując od:

$$z_0 = 0$$

powinna znaleźć pierwiastek bliski:

$$z = -1$$

Czyli wynik może wyglądać np. tak:

```
Znaleziony pierwiastek = (-1.0000000000000002+0j)
Sprawdzenie P(pierwiastek) = bardzo mała liczba bliska 0
```

Jeżeli pojawi się zapis:

$$(-1+0j)$$

to oznacza:

$$-1 + 0i$$

czyli po prostu:

$$-1$$

Łopatologiczne wyjaśnienie działania

Metoda Laguerre'a działa iteracyjnie, czyli nie znajduje pierwiastka od razu, tylko poprawia kolejne przybliżenia.

Zaczynamy od jakiegoś punktu, np.:

$$z_0 = 0$$

Potem program liczy w tym punkcie:

$$P(z), \quad P'(z), \quad P''(z)$$

Na podstawie tych wartości wylicza poprawkę (a). Ta poprawka mówi, o ile trzeba przesunąć aktualne przybliżenie.

Nowe przybliżenie liczymy tak:

$$z_{\text{new}} = z_{\text{old}} - a$$

Jeśli poprawka (a) jest jeszcze duża, program liczy dalej.

Jeśli poprawka jest bardzo mała, czyli:

$$|a| < \varepsilon$$

to znaczy, że jesteśmy już bardzo blisko pierwiastka.

Ważne

To zadanie znajduje **jeden pierwiastek**.

Dopiero w zadaniu 4 robi się deflację, czyli obniżanie stopnia wielomianu po znalezieniu pierwiastka, żeby znaleźć wszystkie pierwiastki.

Poniżej masz **zadanie 4 zrobione najprościej jak się da**, ale zgodnie ze slajdami: metoda Laguerre'a znajduje jeden pierwiastek, potem robimy **deflację**, czyli obniżamy stopień wielomianu, i powtarzamy aż zostanie wielomian stopnia 2. Dla stopnia 2 używamy zwykłego wzoru z deltą.

Zadanie 4 — wszystkie pierwiastki metodą Laguerre'a

Polecenie:

Zmodyfikuj program z poprzedniego zadania, aby wyznaczał wszystkie pierwiastki wielomianu (również zespolone).

O co chodzi w zadaniu?

W zadaniu 3 program znajdował **jeden pierwiastek** wielomianu.

W zadaniu 4 trzeba znaleźć **wszystkie pierwiastki**, czyli nie kończyć programu po jednym, tylko szukać kolejnych.

Robimy to tak:

1. Szukamy pierwszego pierwiastka metodą Laguerre'a.
2. Po znalezieniu pierwiastka z_1 obniżamy stopień wielomianu, czyli robimy deflację:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

3. Potem szukamy kolejnego pierwiastka już w nowym, mniejszym wielomianie.
4. Powtarzamy ten proces.
5. Gdy zostanie wielomian stopnia 2, rozwiązujemy go wzorem dokładnym.

Deflacja wielomianu

Co to jest deflacja?

Założmy, że znaleźliśmy pierwiastek:

$$z_0$$

Jeżeli z_0 jest pierwiastkiem wielomianu $P_n(z)$, to znaczy, że wielomian można zapisać jako:

$$P_n(z) = (z - z_0)P_{n-1}(z)$$

Czyli wielomian $P_n(z)$ dzielimy przez czynnik:

$$z - z_0$$

i otrzymujemy nowy wielomian:

$$P_{n-1}(z)$$

który ma stopień mniejszy o 1.

Dzięki temu po znalezieniu jednego pierwiastka możemy szukać kolejnego pierwiastka w wielomianie niższego stopnia.

Wyjaśnienie krok po kroku

1. Metoda Laguerre'a znajduje jeden pierwiastek

W zadaniu 3 mieliśmy funkcję:

```
metoda_laguerre_jeden_pierwiastek(...)
```

PYTHON

Ona znajduje **jeden** pierwiastek wielomianu.

Dla przykładu ze slajdu:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

Metoda startuje np. od:

$$z_0 = 0$$

znajduje pierwszy pierwiastek:

$$z_1 \approx -1$$

Skoro:

$$z_1 = -1$$

to czynnik, przez który dzielimy, ma postać:

$$z - z_1 = z - (-1) = z + 1$$

Po podzieleniu wielomianu przez $z + 1$ dostajemy:

$$P(z) = (z + 1)(z^3 + 4z^2 + 9z + 10)$$

Czyli po deflacji zostaje nam nowy wielomian:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

2. Powtarzamy szukanie pierwiastków

Teraz metodę Laguerre'a stosujemy do mniejszego wielomianu:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

Metoda może znaleźć pierwiastek zespolony, np.:

$$z_2 = -1 + 2i$$

Dla wielomianu o rzeczywistych współczynnikach drugi pierwiastek z tej pary to:

$$z_3 = -1 - 2i$$

W programie nie trzeba tego osobno dopisywać, bo metoda Laguerre'a działa na liczbach zespolonych, a deflacja pozwala dalej zmniejszać stopień wielomianu.

3. Gdy zostaje stopień 2, używamy wzoru dokładnego

Na slajdach było, że proces powtarzamy aż do wielomianu stopnia 2.

Dla wielomianu stopnia 2:

$$az^2 + bz + c = 0$$

korzystamy ze wzoru:

$$z = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

W kodzie robi to funkcja:

```
pierwiastki_stopnia_drugiego(...)
```

PYTHON

Używamy `cmath.sqrt`, ponieważ pierwiastki mogą być zespolone.

4. Strategia całego zadania

Zgodnie ze slajdami postępujemy tak:

1. Bierzemy wielomian $P_n(z)$.
2. Metodą Laguerre'a znajdujemy jeden pierwiastek.
3. Możemy go jeszcze „wygładzić”, czyli poprawić, używając go jako punktu startowego dla pierwotnego wielomianu.
4. Wykonujemy deflację:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

5. Powtarzamy obliczenia dla nowego wielomianu $P_{n-1}(z)$.

6. Kontynuujemy aż zostanie wielomian stopnia 2.

7. Dla wielomianu stopnia 2 używamy wzoru z deltą.

5. Wynik dla przykładu ze slajdu

Dla:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

dokładne pierwiastki to:

$$z_1 = -1$$

$$z_2 = -2$$

$$z_3 = -1 + 2i$$

$$z_4 = -1 - 2i$$

Program powinien wypisać wartości bardzo bliskie tym wynikom.

Kolejność może być inna, np.:

$$-1, \quad -1 + 2i, \quad -1 - 2i, \quad -2$$

i to jest normalne.

```
import cmath # potrzebne do obsługi liczb zespolonych i pierwiastka zespolonego

# ----- ZADANIE 1 -----

def schemat_Hornera(wspolczynniki, z): # funkcja oblicza wartość wielomianu w punkcie z
    p = wspolczynniki[0] # zaczynamy od współczynnika przy najwyższej potędze

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        p = p * z + wspolczynniki[i] # wykonujemy krok Hornera

    return p # zwracamy wartość wielomianu

# ----- ZADANIE 2 -----

def schemat_Hornera_pochodne(wspolczynniki, z): # funkcja liczy P(z), P'(z), P''(z)
    p = wspolczynniki[0] # p przechowuje wartość wielomianu
    dp = 0 # dp przechowuje wartość pierwszej pochodnej
    ddp = 0 # ddp przechowuje wartość drugiej pochodnej

    for i in range(1, len(wspolczynniki)): # przechodzimy po kolejnych współczynnikach
        ddp = ddp * z + 2 * dp # aktualizujemy drugą pochodną
        dp = dp * z + p # aktualizujemy pierwszą pochodną
        p = p * z + wspolczynniki[i] # aktualizujemy wartość wielomianu

    return p, dp, ddp # zwracamy P(z), P'(z), P''(z)

# ----- ZADANIE 3 -----

def metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0, epsilon=1e-6, max_iteracji=100):
    z = complex(z0) # zamieniamy punkt startowy na liczbę zespoloną
    n = len(wspolczynniki) - 1 # stopień wielomianu

    for k in range(max_iteracji): # wykonujemy kolejne iteracje metody Laguerre'a
        P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z) # liczymy P(z), P'(z), P''(z)

        if abs(P) < epsilon: # jeżeli P(z) jest bardzo bliskie 0
            return z # zwracamy aktualne z jako pierwiastek

        G = P_prim / P # liczymy G = P'(z) / P(z)
        H = G**2 - P_2prim / P # liczymy H = G^2 - P''(z) / P(z)
```

```

    pierwiastek = cmath.sqrt((n - 1) * (n * H - G**2)) # liczymy wyrażenie
    pod pierwiastkiem

    mianownik_plus = G + pierwiastek # pierwszy możliwy mianownik
    mianownik_minus = G - pierwiastek # drugi możliwy mianownik

    if abs(mianownik_plus) > abs(mianownik_minus): # wybieramy mianownik o
    większym module
        mianownik = mianownik_plus # wybieramy wersję z plusem
    else:
        mianownik = mianownik_minus # wybieramy wersję z minusem

    if abs(mianownik) == 0: # zabezpieczenie przed dzieleniem przez zero
        z = z + complex(epsilon, epsilon) # lekko przesuwamy punkt startowy
        continue # przechodzimy do następnej iteracji

    a = n / mianownik # liczymy poprawkę a

    z_nowe = z - a # liczymy nowe przybliżenie pierwiastka

    if abs(a) < epsilon: # jeżeli poprawka jest bardzo mała
        return z_nowe # kończymy iteracje

    z = z_nowe # aktualizujemy z

    return z # zwracamy ostatnie przybliżenie, jeśli pętla się zakończyła

# ----- ZADANIE 4 -----

def deflacja(wspolczynniki, pierwiastek): # funkcja obniża stopień wielomianu po
    znalezieniu pierwiastka
    nowe_wspolczynniki = [complex(wspolczynniki[0])] # pierwszy współczynnik
    przepisujemy

    for i in range(1, len(wspolczynniki) - 1): # przechodzimy po współczynnikach
    oprócz ostatniego
        nowy = wspolczynniki[i] + pierwiastek * nowe_wspolczynniki[-1] #
    obliczamy kolejny współczynnik po deflacji
        nowe_wspolczynniki.append(nowy) # dodajemy go do listy nowych
    współczynników

    reszta = wspolczynniki[-1] + pierwiastek * nowe_wspolczynniki[-1] # obliczamy
    resztę z dzielenia

    return nowe_wspolczynniki, reszta # zwracamy nowy wielomian i resztę

def pierwiastki_stopnia_drugiego(wspolczynniki): # funkcja rozwiązuje wielomian
    stopnia 2
    a = wspolczynniki[0] # współczynnik przy z^2
    b = wspolczynniki[1] # współczynnik przy z
    c = wspolczynniki[2] # wyraz wolny

```

```

delta = b**2 - 4 * a * c # liczymy deltę

z1 = (-b + cmath.sqrt(delta)) / (2 * a) # pierwszy pierwiastek
z2 = (-b - cmath.sqrt(delta)) / (2 * a) # drugi pierwiastek

return z1, z2 # zwracamy dwa pierwiastki

def metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0, epsilon=1e-6):
    pierwotny_wielomian = wspolczynniki.copy() # zapamiętujemy oryginalny
    wielomian do wygładzania
    aktualny_wielomian = wspolczynniki.copy() # to będzie wielomian, którego
    stopień będziemy zmniejszać
    pierwiastki = [] # lista na znalezione pierwiastki

    while len(aktualny_wielomian) > 3: # dopóki stopień wielomianu jest większy
    niż 2
        pierwiastek = metoda_laguerre_jeden_pierwiastek(aktualny_wielomian, z0,
        epsilon) # szukamy pierwiastka

        pierwiastek = metoda_laguerre_jeden_pierwiastek(pierwotny_wielomian,
        pierwiastek, epsilon) # wygładzamy pierwiastek na pierwotnym wielomianie

        pierwiastki.append(pierwiastek) # zapisujemy znaleziony pierwiastek

        aktualny_wielomian, reszta = deflacja(aktualny_wielomian, pierwiastek) #
        wykonujemy deflację

    z1, z2 = pierwiastki_stopnia_drugiego(aktualny_wielomian) # rozwiązujemy
    pozostały wielomian stopnia 2

    pierwiastki.append(z1) # dodajemy pierwszy pierwiastek z równania kwadratowego
    pierwiastki.append(z2) # dodajemy drugi pierwiastek z równania kwadratowego

    return pierwiastki # zwracamy listę wszystkich pierwiastków

# DODATKOWA FUNKCJA (NIEPOTRZEBNA)
def ladnie(z, epsilon=1e-8): # funkcja tylko poprawia wygląd wypisywania wyników
    if abs(z.imag) < epsilon: # jeśli część urojona jest praktycznie zerem
        return z.real # zwracamy samą część rzeczywistą

    if abs(z.real) < epsilon: # jeśli część rzeczywista jest praktycznie zerem
        return complex(0, z.imag) # zwracamy liczbę z zerową częścią rzeczywistą

    return z # w innych przypadkach zwracamy liczbę bez zmian

# DODATKOWY TEST
print("----- ZADANIE 4 -----")

# Przykład ze slajdu:
#  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ 

```



```
wspolczynniki = $1, 5, 13, 19, 10$ # współczynniki wielomianu od najwyższej
potęgi do wyrazu wolnego

pierwiastki = metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0) #
szukamy wszystkich pierwiastków

print("Wielomian: P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10") # wypisujemy wielomian

print("\nZnalezione pierwiastki:") # nagłówek
for i in range(len(pierwiastki)): # przechodzimy po znalezionych pierwiastkach
    print(f"z{i + 1} =", ładnie(pierwiastki[i])) # wypisujemy pierwiastek
```

Najważniejsze wnioski

W zadaniu 4 metoda działa tak:

Laguerre → pierwiastek → deflacja → kolejny pierwiastek

Czyli po każdym znalezionym pierwiastku obniżamy stopień wielomianu i szukamy dalej.

Metoda działa również dla pierwiastków zespolonych, ponieważ w programie używamy typu `complex` oraz biblioteki:

`cmath`

PYTHON

Dzięki temu program może znaleźć pierwiastki typu:

$$-1 + 2i$$

oraz:

$$-1 - 2i$$

Zadanie 5

Polecenie: Wyniki powyższych programów przetestuj dla następujących wielomianów:

a)

$$w(x) = x^3 - 6x^2 + 11x - 6$$

b)

$$w(x) = x^3 - 6x^2 + 11x - 1$$

c)

Przykład ze strony 27 wykładu:

$$Q(x) = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x - 280$$

albo w postaci iloczynowej:

$$Q(x) = 17^3 \cdot 19 \cdot 20 \cdot 21 \left(x + \frac{20}{21}\right) \left(x - \frac{16}{17}\right)^3 \left(x - \frac{18}{19}\right) \left(x - \frac{19}{20}\right)$$

d)

Przykład ze strony 27 wykładu (+1):

$$Q(x) + 1 = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x -$$

```
# zad 5 -----
```

```
def testuj_wielomian(nazwa, wspolczynniki):
    print("\n" + "=" * 60)
    print(nazwa)
    print("Współczynniki:", wspolczynniki)

    z = complex(2, 0)

    print("\nZadanie 1 - wartość wielomianu w punkcie z = 2:")
    print("P(2) =", ladnie(schemat_Hornera(wspolczynniki, z)))

    print("\nZadanie 2 - wartość P(2), P'(2), P''(2):")
    P, P_prim, P_2prim = schemat_Hornera_pochodne(wspolczynniki, z)

    print("P(2)   =", ladnie(P))
    print("P'(2)  =", ladnie(P_prim))
    print("P''(2) =", ladnie(P_2prim))

    print("\nZadanie 3 - jeden pierwiastek metodą Laguerre'a:")
    jeden_pierwiastek = metoda_laguerre_jeden_pierwiastek(wspolczynniki, z0=0)
    print("Jeden pierwiastek =", ladnie(jeden_pierwiastek))

    print("\nZadanie 4 - wszystkie pierwiastki metodą Laguerre'a:")
    pierwiastki = metoda_laguerre_wszystkie_pierwiastki(wspolczynniki, z0=0)

    for i in range(len(pierwiastki)):
        print(f"z{i + 1} =", ladnie(pierwiastki[i]))

# a)  $w(x) = x^3 - 6x^2 + 11x - 6$ 
wielomian_A = [1, -6, 11, -6]

# b)  $w(x) = x^3 - 6x^2 + 11x - 1$ 
wielomian_B = [1, -6, 11, -1]

# c) przykład ze strony 27 wykładu
wielomian_C = [
    39205740,
    -147747493,
    173235338,
    2869080,
    -158495872,
    118949888,
    -28016640
]

# d) przykład ze strony 27 wykładu + 1
wielomian_D = [
    39205740,
    -147747493,
    173235338,
```

```

2869080,
-158495872,
118949888,
-28016639
]

testuj_wielomian(
    "a)  $w(x) = x^3 - 6x^2 + 11x - 6$ ",
    wielomian_A
)

testuj_wielomian(
    "b)  $w(x) = x^3 - 6x^2 + 11x - 1$ ",
    wielomian_B
)

testuj_wielomian(
    "c) przykład ze strony 27 wykładu",
    wielomian_C
)

testuj_wielomian(
    "d) przykład ze strony 27 wykładu + 1",
    wielomian_D
)

```

Krótką notatka do zadania 5

Testowane są wielomiany:

$$w(x) = x^3 - 6x^2 + 11x - 6$$

$$w(x) = x^3 - 6x^2 + 11x - 1$$

oraz przykład ze strony 27 wykładu:

$$Q(x) = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x - 28016640$$

Ostatni wielomian to przykład ze strony 27 wykładu z dodaną wartością 1, czyli zmieniamy tylko wyraz wolny:

$$-28016640 + 1 = -28016639$$

Dlatego w programie zapisujemy go jako:

```
wielomian_D = [
    39205740,
    -147747493,
    173235338,
    2869080,
    -158495872,
    118949888,
    -28016639
]
```

Dla każdego wielomianu program sprawdza:

1. wartość wielomianu w punkcie $z = 2$ schematem Hornera,
2. wartość $P(2)$, $P'(2)$, $P''(2)$,
3. jeden pierwiastek metodą Laguerre'a,
4. wszystkie pierwiastki metodą Laguerre'a z deflacją.

Wyniki pierwiastków mogą pojawić się w różnej kolejności. Małe części urojone typu $10^{-10}i$ przy pierwiastkach rzeczywistych można traktować jako błąd numeryczny, czyli praktycznie zero.

Lab 13

Zadanie 1 — przeskalowanie wartości generatora

Polecenie:

Wykorzystaj znane generatory, np. `rand()`, do zwrócenia wartości z określonych przedziałów, przy założeniu, że `MAX` to największa wartość zwracana przez generator:

a)

$$(int) \langle 0, MAX \rangle$$

b)

$$(int) \langle 0, max \rangle, \quad max < MAX$$

c)

$$(int) \langle min, max \rangle, \quad min < max < MAX$$

d)

$$(double) \langle 0, 1 \rangle$$

O co chodzi w zadaniu?

Zakładamy, że mamy generator, który zwraca liczby całkowite z zakresu:

$$X \in 0, 1, \dots, MAX$$

Czyli generator może zwrócić:

$$0$$

ale może też zwrócić największą wartość:

$$MAX$$

Naszym zadaniem jest przeskalowanie tej liczby tak, żeby dostać wynik z innego przedziału.

Podpunkt a) — przedział $(int)\langle 0, MAX \rangle$

W tym przypadku nie trzeba nic przeliczać, ponieważ funkcja `rand()` już zwraca liczbę z przedziału:

$$\langle 0, MAX \rangle$$

Czyli:

$$Y = X$$

W kodzie wystarczy zwrócić wynik funkcji `rand()`.

Podpunkt b) — przedział $(int)\langle 0, max \rangle$

Chcemy dostać liczbę całkowitą z przedziału:

$$0, 1, \dots, max$$

gdzie:

$$max < MAX$$

Ze slajdu korzystamy ze wzoru:

$$Y = \left\lfloor \frac{X}{MAX + 1} (max + 1) \right\rfloor$$

Dlaczego jest $MAX + 1$?

Ponieważ jeśli generator zwraca liczby:

$$0, 1, 2, \dots, MAX$$

to wszystkich możliwych wartości jest:

$$MAX + 1$$

Podpunkt c) — przedział $(int)\langle min, max \rangle$

Chcemy dostać liczbę całkowitą z przedziału:

$$min, min + 1, \dots, max$$

Ze slajdu korzystamy ze wzoru:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

Najpierw losujemy wartość z przedziału od 0 do $max - min$, a potem przesuwamy ją o min .

Podpunkt d) — przedział $(double)[0, 1]$

W treści zadania pojawia się zapis:

$$\langle 0, 1 \rangle$$

na wykładzie przeskalowanie do rozkładu jednostajnego jest podane jako:

$$R = \frac{X}{MAX + 1}$$

Wtedy:

$$R \in [0, 1)$$

Czyli wynik może być równy 0, ale nie będzie równy dokładnie 1.

Jeżeli generator zwróci największą możliwą wartość:

$$X = MAX$$

to:

$$R = \frac{MAX}{MAX + 1} < 1$$

Dlatego w kodzie zgodnym z wykładem używamy:

```
return X / (MAX + 1)
```

PYTHON

Jeśli zakres ma być faktycznie $[0, 1]$ to:

$$R = \frac{MAX}{MAX} = 1$$

```
return x / (MAX)
```

PYTHON

Kod programu


```
# zad 1
```

```
import random # importujemy moduł random, ponieważ potrzebujemy gotowego
generatora liczb losowych
import sys # importujemy moduł sys, żeby użyć sys.maxsize jako dużej wartości MAX
```

```
MAX = sys.maxsize # ustalamy MAX jako największą dużą liczbę całkowitą dostępną w
systemie
```

```
def rand(): # definiujemy funkcję podobną do rand()
    return random.randint(0, MAX) # zwracamy liczbę całkowitą z przedziału <0,
MAX>
```

```
# a)
```

```
def losuj_0_MAX(): # funkcja realizuje podpunkt a)
    return rand() # zwracamy wartość z generatora, bo rand() już daje przedział
<0, MAX>
```

```
# b)
```

```
def losuj_0_max(max_wartosc): # funkcja realizuje podpunkt b)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return (X * (max_wartosc + 1)) // (MAX + 1) # stosujemy wzór floor(X/(MAX+1)*
(max+1))
```

```
# c)
```

```
def losuj_min_max(min_wartosc, max_wartosc): # funkcja realizuje podpunkt c)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return min_wartosc + (X * (max_wartosc - min_wartosc + 1)) // (MAX + 1) #
stosujemy wzór ze slajdu
```

```
# d)
```

```
def losuj_0_1(): # funkcja realizuje podpunkt d)
    X = rand() # losujemy liczbę X z przedziału <0, MAX>

    return X / (MAX) # otrzymujemy liczbę z przedziału [0,1]

# ALTERNATYWNIIE
# return X / (MAX + 1) # otrzymujemy liczbę z przedziału [0,1)
```

Test programu

```
PYTHON
print("-----ZADANIE 1-----") # wypisujemy nagłówek zadania

print("a) <0, MAX>:", losuj_0_MAX()) # testujemy losowanie z przedziału <0, MAX>
print("b) <0, max>:", losuj_0_max(10)) # testujemy losowanie z przedziału <0, 10>
print("c) <min, max>:", losuj_min_max(5, 15)) # testujemy losowanie z przedziału <5, 15>
print("d) <0, 1>:", losuj_0_1()) # testujemy losowanie liczby rzeczywistej z przedziału [0,1]
```

Wnioski

W zadaniu 1 najważniejsze jest poprawne przeskalowanie liczby X , którą zwraca generator.

Dla przedziałów całkowitych używamy wzorów:

$$Y = \left\lfloor \frac{X}{MAX + 1} (max + 1) \right\rfloor$$

oraz:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

Dla przedziału rzeczywistego zgodnie z wykładem używamy:

$$R = \frac{X}{MAX + 1}$$

Wtedy:

$$R \in [0, 1)$$

Nie używamy prostego:

```
X % (max + 1)
```

PYTHON

ponieważ może ono powodować nierównomierny rozkład wyników, jeśli liczba możliwych wartości generatora nie dzieli się przez $max + 1$.

Zadanie 2 — addytywny generator LCG

Polecenie:

Zaimplementuj własny generator liczb pseudolosowych addytywny LCG oparty na wzorze:

$$X_{n+1} = aX_n + c \mod M$$

Przetestuj uzyskany generator następująco:

- utwórz zbiór punktów postaci:

$$(X_0, X_1), (X_2, X_3), \dots, (X_i, X_{i+1}), (X_{i+2}, X_{i+3}), \dots$$

- zwizualizuj tak utworzony zbiór punktów, np. jako plik SVG za pomocą:
https://www.w3schools.com/graphics/svg_circle.asp.
-

O co chodzi w zadaniu?

Generator LCG, czyli liniowy generator kongruencyjny, tworzy kolejne liczby na podstawie poprzedniej liczby.

Wzór ma postać:

$$X_{n+1} = (aX_n + c) \mod M$$

gdzie:

- X_n — aktualna wartość ciągu,
 - X_{n+1} — następna wartość ciągu,
 - a — mnożnik,
 - c — przyrost,
 - M — moduł,
 - X_0 — ziarno, czyli wartość początkowa. (najlepiej wybrać $0 \leq X_0 < M$)
-

Dlaczego trzeba ustawić konkretne parametry?

Parametry a , c , M i X_0 nie powinny być wybrane całkiem przypadkowo.

Jeśli wybierzemy je źle, generator może mieć bardzo krótki okres, czyli ciąg szybko zacznie się powtarzać.

Dla generatora mieszanego:

$$X_{n+1} = (aX_n + c) \mod M$$

pełny okres można uzyskać, gdy spełnione są warunki Hullu–Dobella:

$$\text{nwd}(c, M) = 1$$

2. $a - 1$ jest podzielne przez każdy czynnik pierwszy liczby M ,
 3. jeśli M jest podzielne przez 4, to $a - 1$ też jest podzielne przez 4.
-

Parametry użyte w programie

W programie używamy typowych parametrów generatora ANSIC:

$$a = 1103515245$$

$$c = 12345$$

$$M = 2^{31}$$

$$X_0 = 7$$

Są one lepsze niż przypadkowe wartości typu:

```
a = 100000  
c = 12345  
M = 1515151
```

PYTHON

bo parametry z ANSIC są znanym zestawem dla generatora LCG.

Tworzenie punktów

Z wygenerowanego ciągu tworzymy punkty:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$

Czyli bierzemy liczby parami:

```
(liczby[0], liczby[1])  
(liczby[2], liczby[3])  
(liczby[4], liczby[5])
```

PYTHON

itd.

Do rysowania punktów w pliku SVG trzeba je przeskalować, bo wartości LCG są z przedziału:

$$0, 1, \dots, M - 1$$

Dlatego współrzędne przeliczamy na rozmiar obrazka.

Kod programu

zad 2

```

def generator_LCG(a, c, X0, M, ile): # definiujemy funkcję generatora LCG | (ile
to jest ile liczb ma wygenerować)
    liczby = [] # tworzymy pustą listę na wygenerowane liczby
    X = X0 # ustawiamy wartość początkową generatora, czyli ziarno

    for i in range(ile): # wykonujemy pętlę tyle razy, ile liczb chcemy
wygenerować
        liczby.append(X) # zapisujemy aktualną wartość X do listy
        X = (a * X + c) % M # obliczamy następną wartość zgodnie ze wzorem LCG

    return liczby # zwracamy listę wygenerowanych liczb

def nwd(a, b): # funkcja oblicza największy wspólny dzielnik liczb a i b
    a = abs(a) # zamieniamy a na wartość dodatnią
    b = abs(b) # zamieniamy b na wartość dodatnią

    while b != 0: # wykonujemy pętlę, dopóki b nie jest równe 0
        reszta = a % b # obliczamy resztę z dzielenia a przez b
        a = b # przesuwamy b na miejsce a
        b = reszta # reszta z dzielenia staje się nowym b

    return a # gdy b = 0, aktualne a jest największym wspólnym dzielnikiem

def utworz_punkty(liczby): # definiujemy funkcję tworzącą punkty z kolejnych
wartości ciągu
    punkty = [] # tworzymy pustą listę punktów

    for i in range(0, len(liczby) - 1, 2): # przechodzimy po liście co dwa
elementy
        punkty.append((liczby[i], liczby[i + 1])) # tworzymy punkt (X_i, X_{i+1})

    return punkty # zwracamy listę punktów

def zapisz_svg(punkty, M, nazwa_pliku): # definiujemy funkcję zapisującą punkty
do pliku SVG
    szerokosc = 500 # ustalamy szerokość obrazka
    wysokosc = 500 # ustalamy wysokość obrazka
    margines = 20 # ustalamy margines od krawędzi

    svg = f'<svg width="{szerokosc}" height="{wysokosc}" viewBox="0 0 {szerokosc}
{wysokosc}" xmlns="http://www.w3.org/2000/svg">\n' # rozpoczynamy plik SVG
    svg += '<rect width="100%" height="100%" fill="white"/>\n' # dodajemy białe
tło
    svg += f'<rect x="{margines}" y="{margines}" width="{szerokosc - 2*margines}"
height="{wysokosc - 2*margines}" fill="none" stroke="black"/>\n' # dodajemy ramkę
wykresu

    for x, y in punkty: # przechodzimy po wszystkich punktach
        x_svg = margines + (x / M) * (szerokosc - 2 * margines) # skalujemy

```

współrzedną x do szerokości obrazka

```
y_svg = wysokosc - margines - (y / M) * (wysokosc - 2 * margines) #  
skalujemy współrzedną y i odwracamy oś pionową
```

```
svg += f'<circle cx="{x_svg}" cy="{y_svg}" r="3" fill="blue"/>\n' #  
dodajemy punkt jako niebieskie kółko
```

```
svg += '</svg>' # kończymy plik SVG
```

```
with open(nazwa_pliku, "w", encoding="utf-8") as plik: # otwieramy plik do  
zapisu
```

```
plik.write(svg) # zapisujemy tekst SVG do pliku
```

Test programu

PYTHON

```
print("-----ZADANIE 2-----")

a = 1103515245 # mnożnik generatora LCG
c = 12345 # przyrost generatora LCG
M = 2**31 # moduł generatora LCG
X0 = 7 # ziarno generatora

print("Parametry generatora LCG:") # wypisujemy opis parametrów
print("a =", a) # wypisujemy a
print("c =", c) # wypisujemy c
print("M =", M) # wypisujemy M
print("X0 =", X0) # wypisujemy ziarno

print("\nSprawdzenie podstawowego warunku:") # wypisujemy nagłówek sprawdzenia
print("moja funkcja z wyżej: nwd(c, M) =", nwd(c, M)) # sprawdzamy, czy c i M są
względnie pierwsze
# print("nwd(c, M) =", math.gcd(c, M)) # funkcja z biblioteki - wymaga import math

liczby = generator_LCG(a, c, X0, M, 1000) # generujemy 1000 liczb pseudolosowych
punkty = utworz_punkty(liczby) # tworzymy punkty z kolejnych wartości ciągu

print("\nPierwsze 20 liczb:") # wypisujemy nagłówek
print(liczby[:20]) # wypisujemy pierwsze 20 liczb

# Wypisanie wszystkich liczb
# print("Wygenerowane liczby:")
# for i in range(len(liczby)):
# print("X_", i, "=", liczby[i])

print("\nPierwsze 10 punktów:") # wypisujemy nagłówek
print(punkty[:10]) # wypisujemy pierwsze 10 punktów

# Wypisanie wszystkich punktów
# print("\nWygenerowane punkty:")
# for i in range(len(punkty)):
# print("P_", i, "=", punkty[i])

zapisz_svg(punkty, M, "punkty_LCG.svg") # zapisujemy punkty do pliku SVG

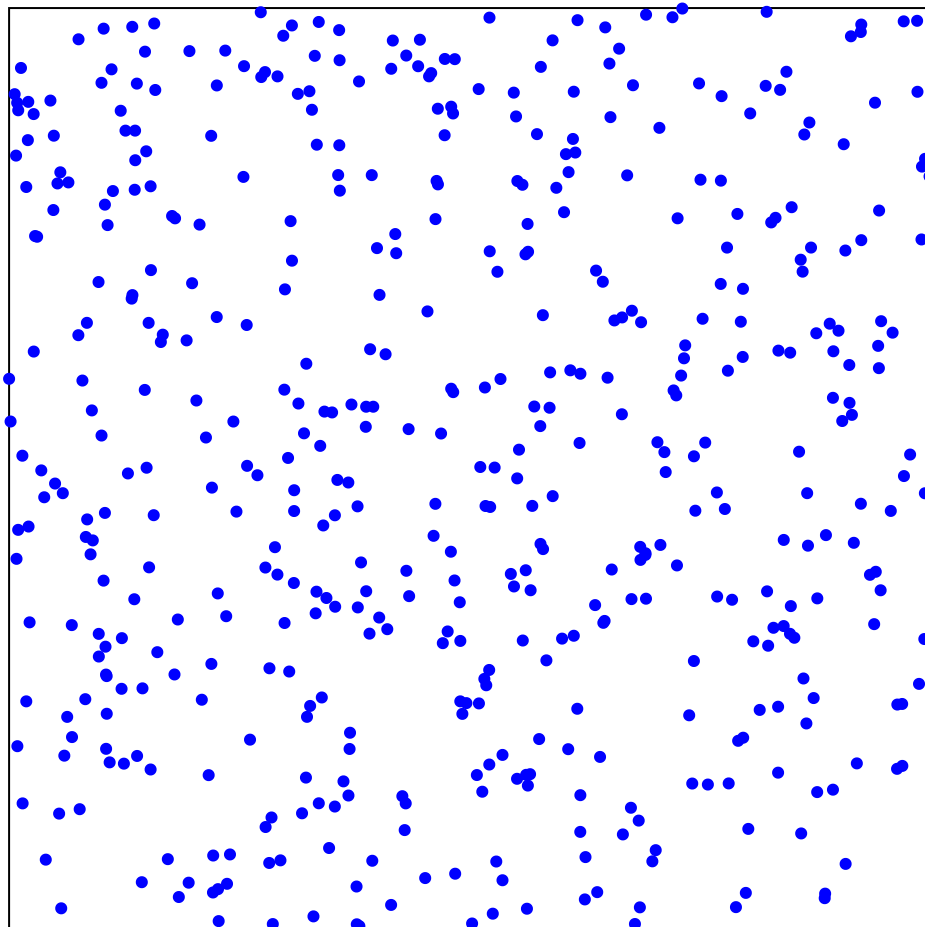
print("\nZapisano plik punkty_LCG.svg") # informujemy o zapisaniu pliku
```

Co oznacza plik SVG?

Plik:

zawiera wizualizację punktów:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$



Jeżeli punkty układają się w wyraźne linie albo regularne pasy, generator może mieć słabe własności statystyczne.

W przypadku generatorów LCG takie zależności mogą być widoczne, ponieważ generator jest deterministyczny i liniowy.

Wnioski

W zadaniu 2 zaimplementowano addytywny generator LCG:

$$X_{n+1} = (aX_n + c) \mod M$$

Użyto parametrów:

$$a = 1103515245$$

$$c = 12345$$

$$M = 2^{31}$$

$$X_0 = 7$$

Następnie z wygenerowanych liczb utworzono punkty:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$

i zapisano je do pliku SVG.

Dzięki temu można wizualnie sprawdzić, czy punkty wyglądają losowo, czy tworzą regularne wzory.

Zadanie 3 — generator LFG

Polecenie:

Zaimplementuj własny generator liczby pseudolosowych LFG oparty na wzorze:

$$X_n = X_{n-q} + X_{n-p} \mod M$$

gdzie:

$$1 \leq q \leq p \leq M$$

O co chodzi w zadaniu?

Generator LFG, czyli lagged Fibonacci generator, jest uogólnieniem generatora Fibonacciego.

Zamiast korzystać tylko z dwóch poprzednich wartości, generator korzysta z wartości oddalonych o pewne opóźnienia.

Na wykładzie wzór ma postać:

$$X_n = (X_{n-p} + X_{n-q}) \mod m$$

gdzie:

$$n \geq p$$

oraz:

$$p > q \geq 1$$

Liczby p i q oznaczają opóźnienia.

Generator opóźniony można zmodyfikować, zmieniając operację wykonywaną na poprzednich wyrazach ciągu:

$$X_n = (X_{n-p} \diamond X_{n-q}) \mod m$$

gdzie symbol $\diamond$ oznacza wybraną operację, np.:

$$\diamond \in \{+, -, \cdot, \oplus\}$$

czyli dodawanie, odejmowanie, mnożenie albo XOR.

Przykładowe wersje:

Dodawanie:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m$$

Odejmowanie:

$$X_n = (X_{n-p} - X_{n-q}) \bmod m$$

Mnożenie:

$$X_n = (X_{n-p} \cdot X_{n-q}) \bmod m$$

XOR:

$$X_n = (X_{n-p} \oplus X_{n-q}) \bmod m$$

Wartości początkowe

Do uruchomienia generatora LFG trzeba podać p wartości początkowych:

$$X_0, X_1, \dots, X_{p-1}$$

Jeżeli:

$$p = 3$$

to trzeba podać trzy wartości początkowe:

$$X_0, X_1, X_2$$

Wartości początkowe nie powinny być samymi zerami, bo wtedy generator mógłby dawać ciąg trywialny, czyli same zera.

Parametry zgodne z przykładem z wykładu

W programie używamy przykładu z wykładu:

$$M = 17$$

$$p = 3$$

$$q = 1$$

Wartości początkowe:

$$X_0 = 7$$

$$X_1 = 16$$

$$X_2 = 5$$

- Wartości początkowe muszą należeć do zakresu:

$$0 \leq X_i < M$$

Sprawdzamy warunek:

$$p > q \geq 1$$

czyli:

$$3 > 1 \geq 1$$

Warunek jest spełniony.

Pierwsze obliczenia

Dla:

$$X_n = (X_{n-p} + X_{n-q}) \mod M$$

oraz:

$$M = 17, \quad p = 3, \quad q = 1$$

mamy:

$$X_3 = (X_{3-3} + X_{3-1}) \mod 17$$

czyli:

$$X_3 = (X_0 + X_2) \mod 17$$

Podstawiamy:

$$X_3 = (7 + 5) \mod 17 = 12$$

Następnie:

$$X_4 = (X_1 + X_3) \mod 17$$

$$X_4 = (16 + 12) \mod 17 = 28 \mod 17 = 11$$

Następnie:

$$X_5 = (X_2 + X_4) \mod 17$$

$$X_5 = (5 + 11) \mod 17 = 16$$

Początek ciągu to:

$$7, 16, 5, 12, 11, 16, \dots$$

Kod programu

zad 3

```

def generator_LFG_dodawanie(p, q, M, poczatkowe, ile): # generator LFG z
    dodawaniem
    if not (p > q >= 1): # sprawdzamy warunek  $p > q \geq 1$ 
        raise ValueError("Musi być spełniony warunek  $p > q \geq 1$ .")

    if p > M: # sprawdzamy dodatkowy warunek z wykładu
        raise ValueError("Musi być spełniony warunek  $p \leq M$ .")

    if len(poczatkowe) < p: # sprawdzamy, czy podano co najmniej p wartości
    początkowych
        raise ValueError("Trzeba podać co najmniej p wartości początkowych.")

    if all(wartosc == 0 for wartosc in poczatkowe): # sprawdzamy, czy wartości
    początkowe nie są samymi zerami
        raise ValueError("Wartości początkowe nie mogą być samymi zerami.")

    liczby = poczatkowe.copy() # kopiujemy wartości początkowe

    for n in range(p, ile): # generujemy kolejne wartości ciągu
        Xn = (liczby[n - p] + liczby[n - q]) % M # wzór:  $X_n = (X_{n-p} + X_{n-q}) \bmod M$ 
        liczby.append(Xn) # dodajemy nową wartość

    return liczby # zwracamy cały ciąg

def generator_LFG_wybor_operacji(p, q, M, X_poczatkowe, ile, operacja): #
    generator LFG z wyborem operacji
    if not (p > q >= 1): # sprawdzamy warunek  $p > q \geq 1$ 
        raise ValueError("Musi być spełniony warunek  $p > q \geq 1$ .")

    if p > M: # sprawdzamy dodatkowy warunek  $p \leq M$ 
        raise ValueError("Musi być spełniony warunek  $p \leq M$ .")

    if len(X_poczatkowe) < p: # sprawdzamy, czy podano co najmniej p wartości
    początkowych
        raise ValueError("Trzeba podać co najmniej p wartości początkowych.")

    if all(wartosc == 0 for wartosc in X_poczatkowe): # sprawdzamy, czy wartości
    początkowe nie są samymi zerami
        raise ValueError("Wartości początkowe nie mogą być samymi zerami.")

    liczby = X_poczatkowe.copy() # kopiujemy wartości początkowe

    for n in range(p, ile): # generujemy kolejne wartości ciągu
        a = liczby[n - p] # pobieramy  $X_{n-p}$ 
        b = liczby[n - q] # pobieramy  $X_{n-q}$ 

        if operacja == "dodawanie": # wersja z dodawaniem
            nowy = (a + b) % M

```

```

elif operacja == "odejmowanie": # wersja z odejmowaniem
    nowy = (a - b) % M

elif operacja == "mnozenie": # wersja z mnożeniem
    nowy = (a * b) % M

elif operacja == "xor": # wersja z operacją XOR
    nowy = (a ^ b) % M

else: # jeśli wpisano nieznaną operację
    raise ValueError("Nieznana operacja.")

liczby.append(nowy) # dopisujemy nową wartość do ciągu

return liczby # zwracamy wygenerowany ciąg

```

Test programu

PYTHON

```

print("-----ZADANIE 3-----") # wypisujemy nagłówek
zadania 3

M = 17 # moduł generatora zgodny z przykładem z wykładu
p = 3 # pierwsze opóźnienie
q = 1 # drugie opóźnienie

początkowe = [7, 16, 5] # wartości początkowe X0, X1, X2

liczby = generator_LFG(p, q, M, początkowe, 12) # generujemy 12 wartości ciągu
liczby2 = generator_LFG_wybor_operacji(p, q, M, początkowe, 12, "dodawanie") #
wersja z wyborem operacji

print("Parametry generatora LFG:") # wypisujemy opis parametrów
print("M =", M) # wypisujemy moduł
print("p =", p) # wypisujemy p
print("q =", q) # wypisujemy q
print("Wartości początkowe:", początkowe) # wypisujemy wartości początkowe

print("\nWygenerowany ciąg:") # wypisujemy nagłówek wyniku
print(liczby) # wypisujemy wygenerowany ciąg
print(liczby2) # wypisujemy drugi wygenerowany ciąg

```

Oczekiwany wynik

Dla parametrów z wykładu:

$$M = 17, \quad p = 3, \quad q = 1$$

oraz wartości początkowych:

$$7, 16, 5$$

ciąg powinien zacząć się tak:

```
[7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, 7]
```

Jest to zgodne z obliczeniami:

$$X_3 = (7 + 5) \mod 17 = 12$$

$$X_4 = (16 + 12) \mod 17 = 11$$

$$X_5 = (5 + 11) \mod 17 = 16$$

Wnioski

W zadaniu 3 zaimplementowano generator LFG:

$$X_n = (X_{n-p} + X_{n-q}) \mod M$$

Generator wymaga podania:

- modułu M ,
- opóźnień p i q ,
- p wartości początkowych.

Wartości początkowe są konieczne, ponieważ bez nich nie da się obliczyć pierwszych wyrazów ciągu.

Dla przykładu z wykładu:

$$M = 17, \quad p = 3, \quad q = 1$$

oraz:

$$X_0 = 7, \quad X_1 = 16, \quad X_2 = 5$$

otrzymujemy ciąg:

$$7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, 7, \dots$$

Generator LFG zmniejsza proste zależności między kolejnymi wyrazami w porównaniu z podstawowym generatorem Fibonacciego, ale nadal jest generatorem deterministycznym.

Lab 14

Metody Monte Carlo — zadania

Wprowadzenie

Metody Monte Carlo polegają na wykorzystaniu losowania do przybliżonego rozwiązania problemu matematycznego.

Zamiast liczyć wynik dokładnie, wykonujemy dużo losowych prób, a następnie uśredniamy wynik.

Ogólny schemat wygląda tak:

1. określamy obszar, z którego losujemy punkty,
2. losujemy dużą liczbę punktów,
3. dla każdego punktu wykonujemy obliczenie,
4. na podstawie średniej albo proporcji punktów otrzymujemy wynik przybliżony.

Im większa liczba punktów N , tym wynik zwykle jest dokładniejszy.

Na wykładzie podano, że błąd metody Monte Carlo maleje w przybliżeniu jak:

$$O\left(\frac{1}{\sqrt{N}}\right)$$

czyli jeśli chcemy zmniejszyć błąd 10 razy, to trzeba zwiększyć liczbę próbek około 100 razy.

Zadanie 1 — całkowanie metodą Monte Carlo

Treść zadania:

Wykorzystaj metodę Monte Carlo do obliczenia przybliżonej wartości całek:

a)

$$\int_0^1 x^2 dx$$

b)

$$\int_e^{e^2} \frac{1}{x} dx$$

c)

$$\iint_D (\cos x + y + 1) dx dy \quad D = [0, 2] \times [-\pi, \pi]$$

Oszacuj liczbę punktów potrzebnych do uzyskania dokładności do 2 cyfr po przecinku.

Metoda Crude Monte Carlo

Dla całki jednowymiarowej:

$$I = \int_a^b f(x) dx$$

losujemy N punktów:

$$x_1, x_2, \dots, x_N \in [a, b]$$

a następnie liczymy:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Czyli w praktyce:

1. losujemy dużo punktów x z przedziału $[a, b]$,
 2. liczymy wartości funkcji $f(x)$,
 3. obliczamy średnią tych wartości,
 4. mnożymy przez długość przedziału $b - a$.
-

Całka podwójna

Dla całki po prostokącie:

$$D = [a, b] \times [c, d]$$

losujemy punkty:

$$(x_i, y_i)$$

i korzystamy ze wzoru:

$$I \approx \frac{(b-a)(d-c)}{N} \sum_{i=1}^N f(x_i, y_i)$$

czyli średnią wartość funkcji mnożymy przez pole obszaru.

Wartości dokładne do porównania

a)

$$\int_0^1 x^2, dx = \frac{x^3}{3} \Big|_0^1$$

$$\int_0^1 x^2, dx = \frac{1}{3}$$

b)

$$\int_e^{e^2} \frac{1}{x}, dx = \ln x \Big|_e^{e^2}$$

$$\int_e^{e^2} \frac{1}{x}, dx = \ln(e^2) - \ln(e) = 2 - 1 = 1$$

c)

$$\iint_D (\cos x + y + 1), dx dy$$

gdzie:

$$D = [0, 2] \times [-\pi, \pi]$$

Najpierw zauważamy, że składnik y znika, ponieważ całkujemy go po symetrycznym przedziale:

$$\int_{-\pi}^{\pi} y, dy = 0$$

Zostaje:

$$\begin{aligned} & \int_0^2 \int_{-\pi}^{\pi} (\cos x + 1) dy dx \\ &= \int_0^2 2\pi(\cos x + 1), dx \\ &= 2\pi(\sin 2 + 2) \end{aligned}$$

Czyli wartość dokładna wynosi:

$$2\pi(\sin 2 + 2)$$

Jak oszacować liczbę punktów?

Chcemy dokładność do 2 cyfr po przecinku, więc przyjmujemy błąd około:

$$\varepsilon = 0.005$$

Ponieważ błąd Monte Carlo zachowuje się mniej więcej jak:

$$\frac{1}{\sqrt{N}}$$

to liczba punktów potrzebna do uzyskania dokładności zależy od rozrzutu wartości funkcji.

W programie robimy to praktycznie:

1. losujemy próbnie pewną liczbę punktów,
2. liczymy odchylenie standardowe wartości funkcji,
3. szacujemy potrzebne N .

N bierze się z zależności:

$$\text{błąd} \approx \frac{(b-a)\sigma}{\sqrt{N}}$$

gdzie:

$$\sigma$$

to odchylenie standardowe wartości $f(x)$, a N to liczba próbek.

Jeżeli chcesz, żeby błąd był mniejszy niż zadana dokładność:

$$\varepsilon$$

to zapisujesz:

$$\frac{(b-a)\sigma}{\sqrt{N}} \leq \varepsilon$$

Przekształcamy:

$$\sqrt{N} \geq \frac{(b-a)\sigma}{\varepsilon}$$

Podnosimy do kwadratu:

$$N \geq \left(\frac{(b-a)\sigma}{\varepsilon} \right)^2$$

I to jest dokładnie w kodzie:

```
N = ((b - a) * odchylenie / dokladnosc) ** 2
```

PYTHON

Czyli ten wzór może nie być w wykładzie zapisany dosłownie jako gotowa funkcja do **N**, ale powinien wynikać z fragmentu o tym, że błąd Monte Carlo maleje jak:

$$\frac{1}{\sqrt{N}}$$

albo z odchylenia standardowego średniej:

$$\frac{\sigma}{\sqrt{N}}$$

Wzór na liczbę próbek wynika z oszacowania błędu metody Monte Carlo. Ponieważ błąd jest proporcjonalny do $\frac{1}{\sqrt{N}}$, to dla zadanej dokładności ε można oszacować liczbę próbek jako:

$$N \approx \left(\frac{(b-a)\sigma}{\varepsilon} \right)^2$$

gdzie σ jest odchyleniem standardowym wartości funkcji $f(x)$ w losowanych punktach.

Kod do zadania 1

```

import random # importujemy moduł random, bo potrzebujemy losowania liczb
import math # importujemy moduł math, bo potrzebujemy liczby e, pi oraz funkcji
cos i sin

print("----- ZADANIE 1 -----") # wypisujemy nagłówek
zadania

def monte_carlo_1d(f, a, b, N): # funkcja oblicza całkę jednowymiarową metodą
Monte Carlo
    suma = 0.0 # tworzymy zmienną, w której będziemy sumować wartości funkcji

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(a, b) # losujemy punkt x z przedziału [a,b]
        suma += f(x) # dodajemy wartość funkcji w wylosowanym punkcie

    return ((b - a) / N) * suma # zwracamy przybliżenie całki według wzoru z
wykładu

def monte_carlo_2d(f, ax, bx, ay, by, N): # funkcja oblicza całkę podwójną metodą
Monte Carlo
    suma = 0.0 # tworzymy zmienną na sumę wartości funkcji

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(ax, bx) # losujemy współrzędną x
        y = random.uniform(ay, by) # losujemy współrzędną y
        suma += f(x, y) # dodajemy wartość funkcji w punkcie (x,y)

    pole_obszaru = (bx - ax) * (by - ay) # obliczamy pole prostokąta, po którym
całkujemy

    return (pole_obszaru * suma) / N # zwracamy przybliżenie całki podwójnej

def oszacuj_N_1d(f, a, b, dokladnosc=0.005, N_probne=10000): # funkcja szacuje
potrzebną liczbę punktów dla całki 1D
    wartosci = [] # lista na wartości funkcji w losowych punktach

    for i in range(N_probne): # wykonujemy próbne losowania
        x = random.uniform(a, b) # losujemy punkt x
        wartosci.append(f(x)) # zapisujemy wartość funkcji

    srednia = sum(wartosci) / N_probne # liczymy średnią wartości funkcji

    wariancja = 0.0 # zmienna na wariancję
    for wartosc in wartosci: # przechodzimy po wszystkich wartościach
        wariancja += (wartosc - srednia) ** 2 # dodajemy kwadrat różnicy od
średniej

    wariancja = wariancja / (N_probne - 1) # kończymy obliczanie wariancji

```

```

    odchylenie = math.sqrt(wariancja) # liczymy odchylenie standardowe

    N = ((b - a) * odchylenie / dokladnosc) ** 2 # szacujemy liczbę punktów z
    zależności błędu

    return math.ceil(N) # zaokrąglamy w górę do liczby całkowitej

def oszacuj_N_2d(f, ax, bx, ay, by, dokladnosc=0.005, N_probne=10000): # funkcja
    szacuje potrzebne N dla całki 2D
    wartosci = [] # lista na wartości funkcji

    for i in range(N_probne): # wykonujemy próbne losowania
        x = random.uniform(ax, bx) # losujemy x
        y = random.uniform(ay, by) # losujemy y
        wartosci.append(f(x, y)) # zapisujemy wartość funkcji

    srednia = sum(wartosci) / N_probne # liczymy średnią

    wariancja = 0.0 # zmienna na wariancję
    for wartosc in wartosci: # przechodzimy po wartościach
        wariancja += (wartosc - srednia) ** 2 # dodajemy kwadrat odchylenia od
    średniej

    wariancja = wariancja / (N_probne - 1) # liczymy wariancję
    odchylenie = math.sqrt(wariancja) # liczymy odchylenie standardowe

    pole_obszaru = (bx - ax) * (by - ay) # obliczamy pole obszaru całkowania

    N = (pole_obszaru * odchylenie / dokladnosc) ** 2 # szacujemy potrzebną
    liczbę punktów

    return math.ceil(N) # zwracamy liczbę punktów zaokrągloną w górę

def f1(x): # funkcja z podpunktu a)
    return x ** 2 # f(x)=x^2

def f2(x): # funkcja z podpunktu b)
    return 1 / x # f(x)=1/x

def f3(x, y): # funkcja z podpunktu c)
    return math.cos(x) + y + 1 # f(x,y)=cos(x)+y+1

N = 100000 # liczba losowanych punktów do właściwego obliczenia całek

wynik_a = monte_carlo_1d(f1, 0, 1, N) # obliczamy całkę a) metodą Monte Carlo
dokladny_a = 1 / 3 # dokładna wartość całki a)

```

```

wynik_b = monte_carlo_1d(f2, math.e, math.e ** 2, N) # obliczamy całkę b)
dokladny_b = 1 # dokładna wartość całki b)

wynik_c = monte_carlo_2d(f3, 0, 2, -math.pi, math.pi, N) # obliczamy całkę c)
dokladny_c = 2 * math.pi * (math.sin(2) + 2) # dokładna wartość całki c)

print("\na) całka od 0 do 1 z x^2 dx") # wypisujemy opis całki a)
print("Wynik Monte Carlo:", wynik_a) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_a) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_a - wynik_a)) # wypisujemy błąd
bezwzględny

print("\nb) całka od e do e^2 z 1/x dx") # wypisujemy opis całki b)
print("Wynik Monte Carlo:", wynik_b) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_b) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_b - wynik_b)) # wypisujemy błąd
bezwzględny

print("\nc) całka podwójna po D = [0,2] x [-pi,pi]") # wypisujemy opis całki c)
print("Wynik Monte Carlo:", wynik_c) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladny_c) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladny_c - wynik_c)) # wypisujemy błąd
bezwzględny

print("\nOszacowanie liczby punktów dla dokładności do 2 cyfr po przecinku:") #
nagłówek oszacowania N

N_a = oszacuj_N_1d(f1, 0, 1) # szacujemy liczbę punktów dla całki a)
N_b = oszacuj_N_1d(f2, math.e, math.e ** 2) # szacujemy liczbę punktów dla całki
b)
N_c = oszacuj_N_2d(f3, 0, 2, -math.pi, math.pi) # szacujemy liczbę punktów dla
całki c)

print("a) potrzebne N ≈", N_a) # wypisujemy szacowane N dla a)
print("b) potrzebne N ≈", N_b) # wypisujemy szacowane N dla b)
print("c) potrzebne N ≈", N_c) # wypisujemy szacowane N dla c)

```

Wnioski do zadania 1

Metoda Monte Carlo pozwala przybliżyć wartość całki przez losowanie punktów i liczenie średniej wartości funkcji.

Dla całek jednowymiarowych używamy wzoru:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Dla całki podwójnej po prostokącie używamy wzoru:

$$I \approx \frac{P_D}{N} \sum_{i=1}^N f(x_i, y_i)$$

gdzie P_D oznacza pole obszaru całkowania.

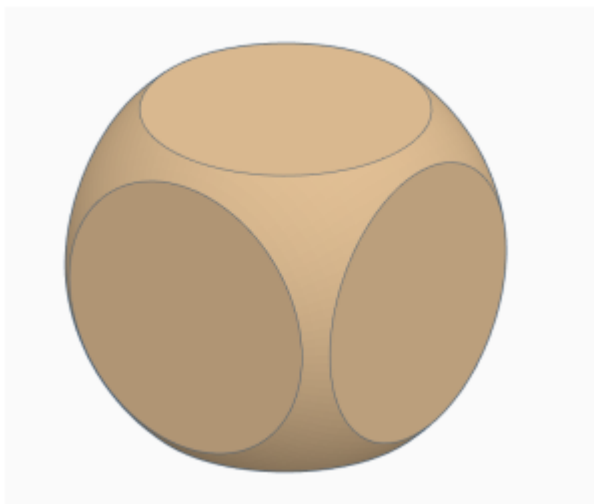
Wynik jest losowy, dlatego po każdym uruchomieniu programu może być trochę inny. Przy większym N wynik powinien być bliższy wartości dokładnej.

Zadanie 2 — metoda akceptacji i odrzuceń

Treść zadania:

Stosując metodę akceptacji i odrzuceń oblicz:

- objętość kuli jednostkowej,
- objętość części wspólnej sześcianu i kuli, przy czym stosunek promienia kuli do długości boku sześcianu wynosi 2 : 3.



Na czym polega metoda akceptacji i odrzuceń?

Metoda akceptacji–odrzucań polega na losowaniu punktów z prostego obszaru, a następnie sprawdzaniu, czy wylosowany punkt należy do badanego obszaru.

Jeżeli:

- N — liczba wszystkich wylosowanych punktów,
- k — liczba punktów zaakceptowanych,

- V_{obszaru} — objętość obszaru, z którego losujemy,

to szukana objętość jest przybliżona wzorem:

$$V \approx \frac{k}{N} V_{\text{obszaru}}$$

To jest odpowiednik wzoru ze slajdów:

$$I \approx \frac{k}{N} P_{\text{prostokąta}}$$

Ponieważ:

$$P_{\text{prostokąta}} = (b - a)M$$

otrzymujemy:

$$I \approx \frac{k}{N} (b - a)M$$

tylko zamiast pola prostokąta mamy objętość sześcianu.

Zadanie 2a — objętość kuli jednostkowej

Kula jednostkowa ma promień:

$$r = 1$$

Losujemy punkty z sześcianu:

$$[-1, 1] \times [-1, 1] \times [-1, 1]$$

Bo ma środek w punkcie:

$$(0, 0, 0)$$

Czyli kula obejmuje punkty oddalone od środka maksymalnie o 1.

W każdej osi wygląda to tak:

x od -1 do 1 y od -1 do 1 z od -1 do 1

Gdyby kula miała promień $r = 2$, wtedy losowalibyśmy z:

$$[-2, 2] \times [-2, 2] \times [-2, 2]$$

Objętość tego sześcianu wynosi:

$$V_{\text{sześcianu}} = 2 \cdot 2 \cdot 2 = 8$$

Uniwersalna metoda na liczenie objętości sześcianu o $r = 1$

```

x_min = -1
x_max = 1

y_min = -1
y_max = 1

z_min = -1
z_max = 1

objetosc = (x_max - x_min) * (y_max - y_min) * (z_max - z_min)

print("Objętość =", objetosc)

```

Punkt należy do kuli, jeżeli spełnia warunek:

$$x^2 + y^2 + z^2 \leq 1$$

Liczba punktów spełniających ten warunek to k .

Wtedy objętość kuli przybliżamy wzorem:

$$V_{\text{kuli}} \approx 8 \cdot \frac{k}{N}$$

Wartość dokładna objętości kuli jednostkowej wynosi:

$$V = \frac{4}{3}\pi$$

Zadanie 2b — część wspólna sześcianu i kuli

Z treści zadania:

$$r : bok = 2 : 3$$

Przyjmujemy więc:

$$r = 2$$

oraz:

$$bok = 3$$

Sześcian ustawiamy symetrycznie względem początku układu współrzędnych, dlatego losujemy punkty z przedziałów (w kodzie to zmienna *polowa_boku*):

$$[-1.5, 1.5] \times [-1.5, 1.5] \times [-1.5, 1.5]$$

Objętość sześcianu wynosi:

$$V_{\text{sześcianu}} = 3^3 = 27$$

Punkt należy do kuli o promieniu (2), jeśli:

$$x^2 + y^2 + z^2 \leq 2^2$$

czyli:

$$x^2 + y^2 + z^2 \leq 4$$

Ponieważ losujemy punkty już ze środka sześcianu, wystarczy sprawdzić tylko warunek kuli. Punkty spełniające warunek są częścią wspólną sześcianu i kuli.

Objętość części wspólnej przybliżamy wzorem:

$$V \approx 27 \cdot \frac{k}{N}$$

Kod do zadania 2

```

import random # importujemy moduł random do losowania punktów
import math # importujemy math, ponieważ potrzebujemy liczby pi

print("----- ZADANIE 2 -----") # wypisujemy nagłówek zadania

# a)
def objetosc_kuli_jednostkowej(N): # funkcja oblicza objętość kuli jednostkowej
    metodą akceptacji i odrzuceń
    zaakceptowane = 0 # licznik punktów, które trafiły do kuli

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(-1, 1) # losujemy współrzędną x z przedziału [-1,1]
        y = random.uniform(-1, 1) # losujemy współrzędną y z przedziału [-1,1]
        z = random.uniform(-1, 1) # losujemy współrzędną z z przedziału [-1,1]

        if x ** 2 + y ** 2 + z ** 2 <= 1: # sprawdzamy, czy punkt leży wewnątrz
            kuli jednostkowej
            zaakceptowane += 1 # jeśli tak, zwiększamy licznik zaakceptowanych
            punktów

    objetosc_szescianu = 2 ** 3 # objętość sześcianu [-1,1]^3 wynosi 8

    return objetosc_szescianu * zaakceptowane / N # zwracamy przybliżoną objętość
    kuli

# b)
def objetosc_wspolna_szescianu_i_kuli(N): # funkcja oblicza objętość części
    wspólnej sześcianu i kuli
    r = 2 # promień kuli zgodnie ze stosunkiem 2:3
    bok = 3 # bok sześcianu zgodnie ze stosunkiem 2:3

    polowa_boku = bok / 2 # połowa boku sześcianu, czyli 1.5

    zaakceptowane = 0 # licznik punktów, które znajdują się jednocześnie w
    sześcianie i kuli

    for i in range(N): # wykonujemy N losowań
        x = random.uniform(-polowa_boku, polowa_boku) # losujemy x z przedziału
        [-1.5,1.5]
        y = random.uniform(-polowa_boku, polowa_boku) # losujemy y z przedziału
        [-1.5,1.5]
        z = random.uniform(-polowa_boku, polowa_boku) # losujemy z z przedziału
        [-1.5,1.5]

        if x ** 2 + y ** 2 + z ** 2 <= r ** 2: # sprawdzamy, czy punkt należy do
            kuli o promieniu 2
            zaakceptowane += 1 # jeśli tak, punkt należy do części wspólnej

    objetosc_szescianu = bok ** 3 # objętość sześcianu o boku 3 wynosi 27

```

```

    return objetosc_szescianu * zaakceptowane / N # zwracamy przybliżoną objętość
    części wspólnej

N = 200000 # liczba losowanych punktów

wynik_kula = objetosc_kuli_jednostkowej(N) # obliczamy objętość kuli jednostkowej
dokladna_kula = 4 / 3 * math.pi # dokładna objętość kuli jednostkowej

wynik_wspolna = objetosc_wspolna_szescianu_i_kuli(N) # obliczamy objętość części
wspólnej sześcianu i kuli

print("\na) Objętość kuli jednostkowej") # nagłówek podpunktu a)
print("Wynik Monte Carlo:", wynik_kula) # wypisujemy wynik Monte Carlo
print("Wartość dokładna:", dokladna_kula) # wypisujemy wartość dokładną
print("Błąd bezwzględny:", abs(dokladna_kula - wynik_kula)) # wypisujemy błąd
bezwzględny

print("\nb) Objętość części wspólnej sześcianu i kuli") # nagłówek podpunktu b)
print("Stosunek promienia kuli do boku sześcianu: 2:3") # wypisujemy stosunek z
zadania
print("Przyjmujemy r = 2 oraz bok = 3") # wypisujemy przyjęte wartości
print("Wynik Monte Carlo:", wynik_wspolna) # wypisujemy wynik Monte Carlo

```

Wnioski do zadania 2

W zadaniu 2 użyto metody akceptacji–odrzuceń.

Dla kuli jednostkowej losujemy punkty z sześcianu:

$$[-1, 1]^3$$

i sprawdzamy warunek:

$$x^2 + y^2 + z^2 \leq 1$$

Dla części wspólnej sześcianu i kuli losujemy punkty z sześcianu o boku 3 i sprawdzamy, które z nich znajdują się również w kuli o promieniu 2.

W obu przypadkach wynik ma postać:

$$V \approx V_{\text{obszaru}} \cdot \frac{k}{N}$$

gdzie k oznacza liczbę zaakceptowanych punktów.

Metoda jest prosta, ale wynik jest przybliżony i zależy od liczby losowań. Im większe N , tym wynik powinien być dokładniejszy.